



Data Structures and its Applications

V R BADRI PRASAD

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Introduction to TRIE Trees

V R BADRI PRASAD

Department of Computer Science & Engineering

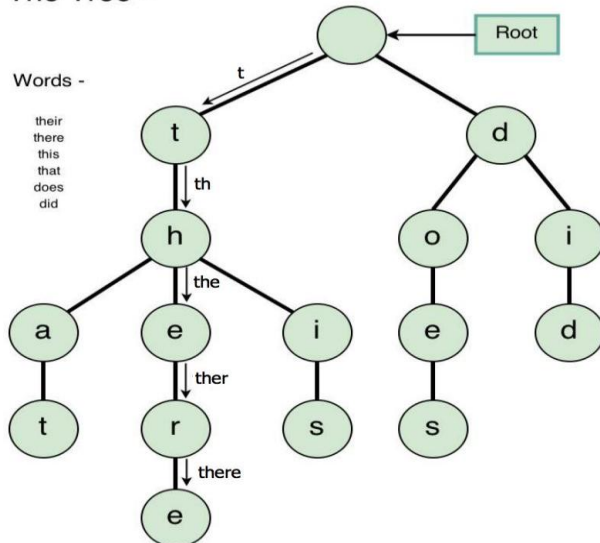
Data Structures and its Applications

TRIE Trees – An Introduction

- **TRIE** tree is a digital search tree, need not be implemented as a binary tree.
- Each node in the tree can contain 'm' pointers – corresponding to 'm' possible symbols in each position of the key.
- Generally used to store strings.

Examples:

Trie Tree -

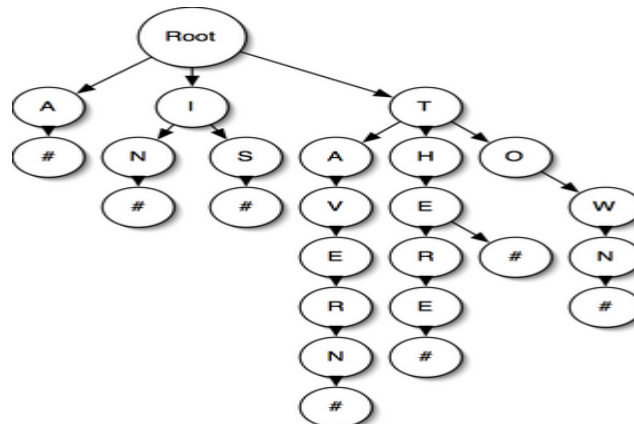


Data Structures and its Applications

TRIE Trees – An Introduction

- A trie, pronounced “try”, is a tree that exploits some structure in the keys
 - e.g. if the keys are strings, a binary search tree would compare the entire strings but a trie would look at their individual characters
 - A trie is a tree where each node stores a bit indicating whether the string spelled out to this point is in the set

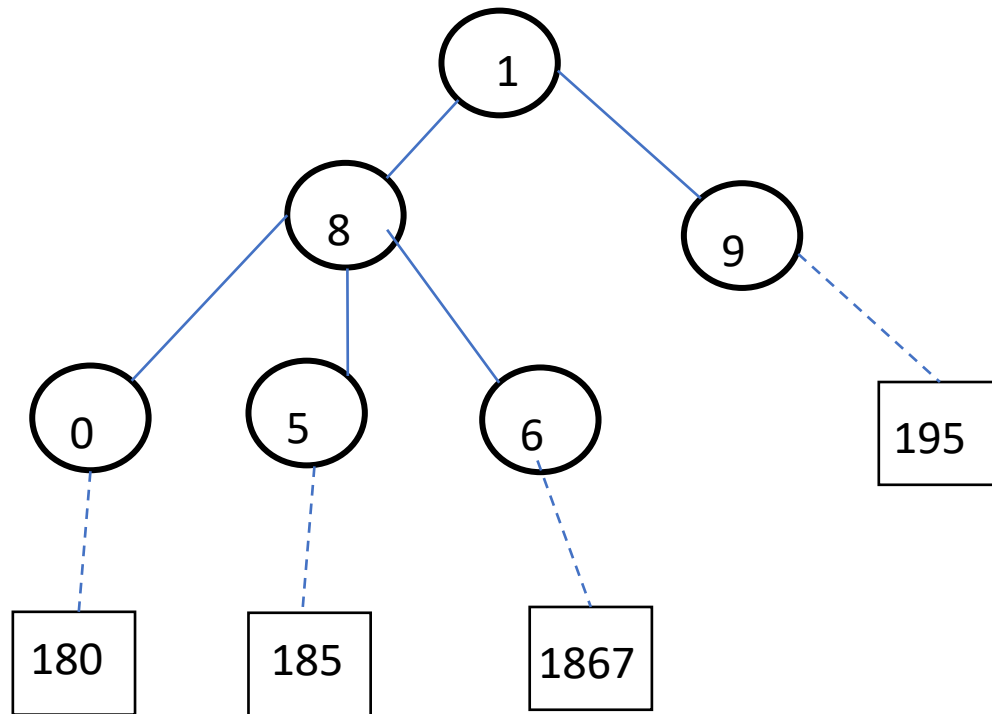
-Examples:



Data Structures and its Applications

TRIE Trees – Numeric Keys : Example2

- If the keys are numeric, there would be 10 pointers in a node.



Data Structures and its Applications

TRIE Trees – Numeric Keys : Example1

- If the keys are numeric, there would be 10 pointers in a node.
- Consider the SSN number as shown.

Name | **Social Security Number (SS#)**

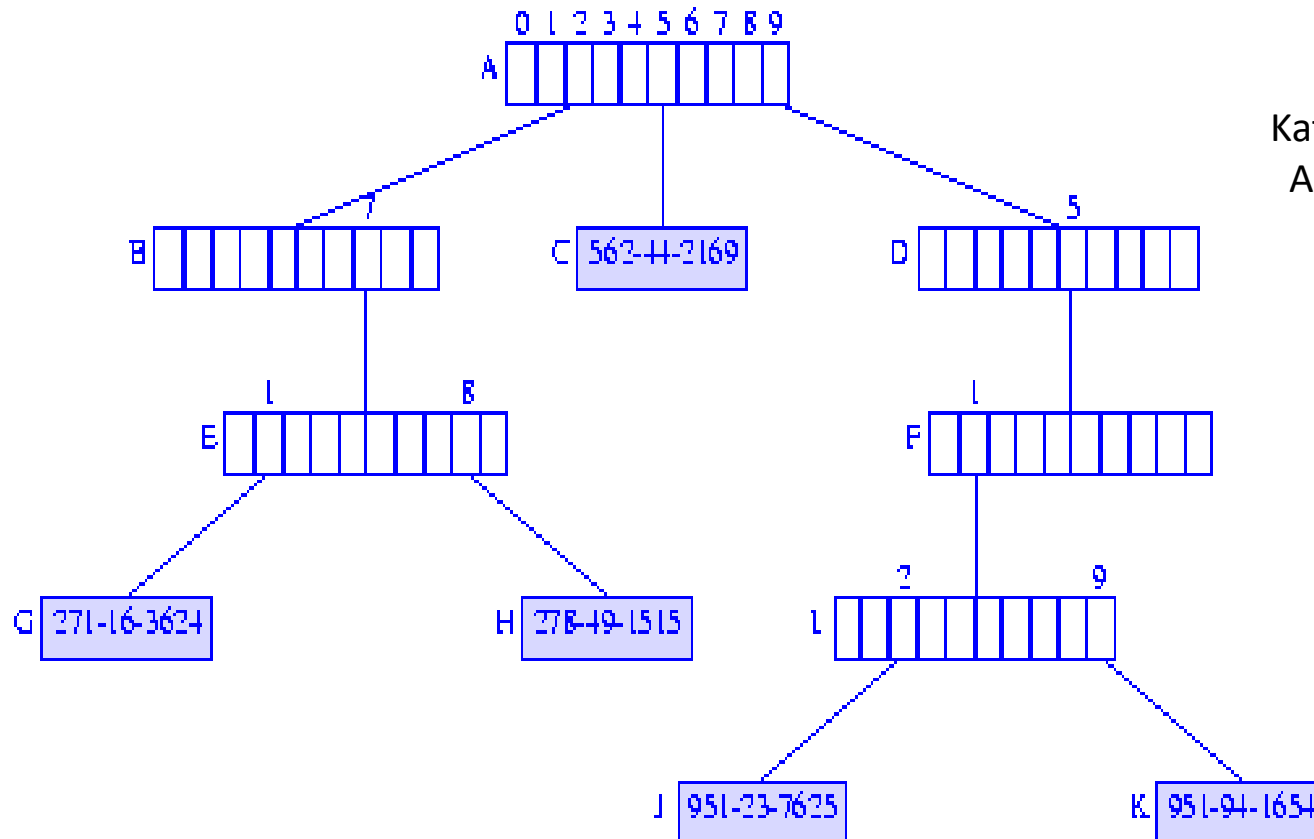
Jack | 951-94-1654

Jill | 562-44-2169

Bill | 271-16-3624

Kathy | 278-49-1515

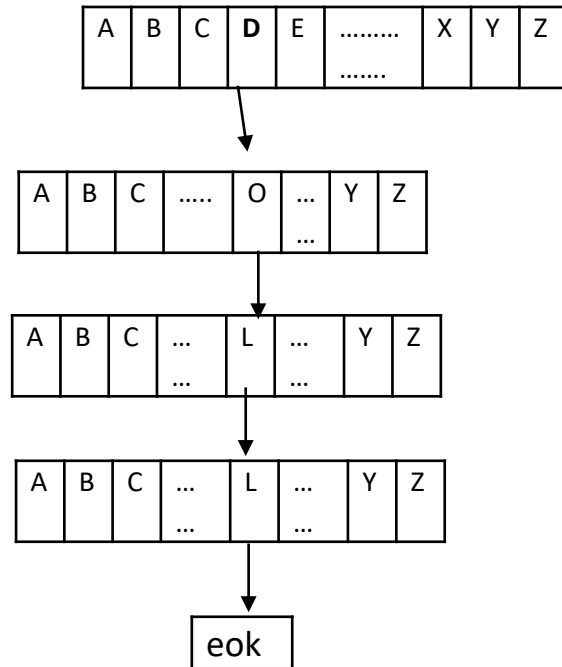
April | 951-23-7625



Data Structures and its Applications

TRIE Trees – An Introduction

- If the keys are Alphabetic, there would be 26 pointers.



Ex: The word DOLL has been stored as shown in the figure.

Data Structures and its Applications

TRIE Trees – An Introduction



- An extra pointer corresponding to eok (end of key) or a flag with each pointer indicating that it point to a record rather than to a tree node. (normally \$ symbol is used).
- A pointer in the node is associated with a particular symbol value based on its position in the node.
 - First pointer corresponds to the lowest value.
 - Second pointer to the second lowest and so forth.
- This way of implementation of a digital search tree is called a **TRIE** tree.
- The word **TRIE** is extracted from re**trie**val word.

- Tries are extremely special and useful data-structure that are based on the *prefix of a string*.
- Strings are stored in a top to bottom manner on the basis of their prefix in a TRIE.
- All prefixes of length 1 are stored at until level 1, all prefixes of length 2 are sorted at until level 2 and so on.

Suffix Trie:

- Suffix Trie is a space-efficient data structure to store a string that allows many kinds of queries to be answered quickly.
- Example:
Text is “**banana**\" where ‘\’ is the string terminating character.

- A Trivial Algorithm for building a suffix tree.
 - Step1 : Generate all suffixes of a given text
 - Step2: Consider all suffixes as individual words and build a compressed trie.

- Example1:

Text is “**banana**\" data-bbox="95 455 418 539"/>

Following are the suffixes of Text

“**banana**\" data-bbox="82 632 190 670"/>

“**anana**\" data-bbox="91 676 188 714"/>

“**nana**\" data-bbox="100 720 188 758"/>

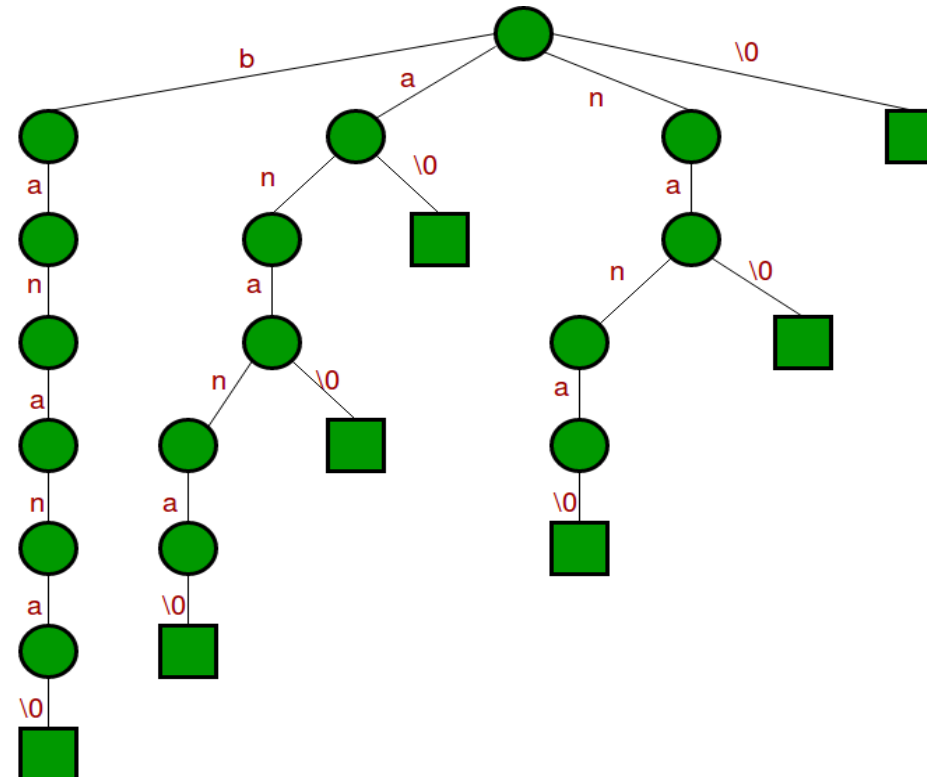
“**ana**\" data-bbox="108 764 186 802"/>

“**na**\" data-bbox="118 808 186 847"/>

“**a**\" data-bbox="128 852 183 890"/>

“**/**\" data-bbox="138 895 183 936"/>

Suffix trie



Data Structures and its Applications

Suffix Trie – Building



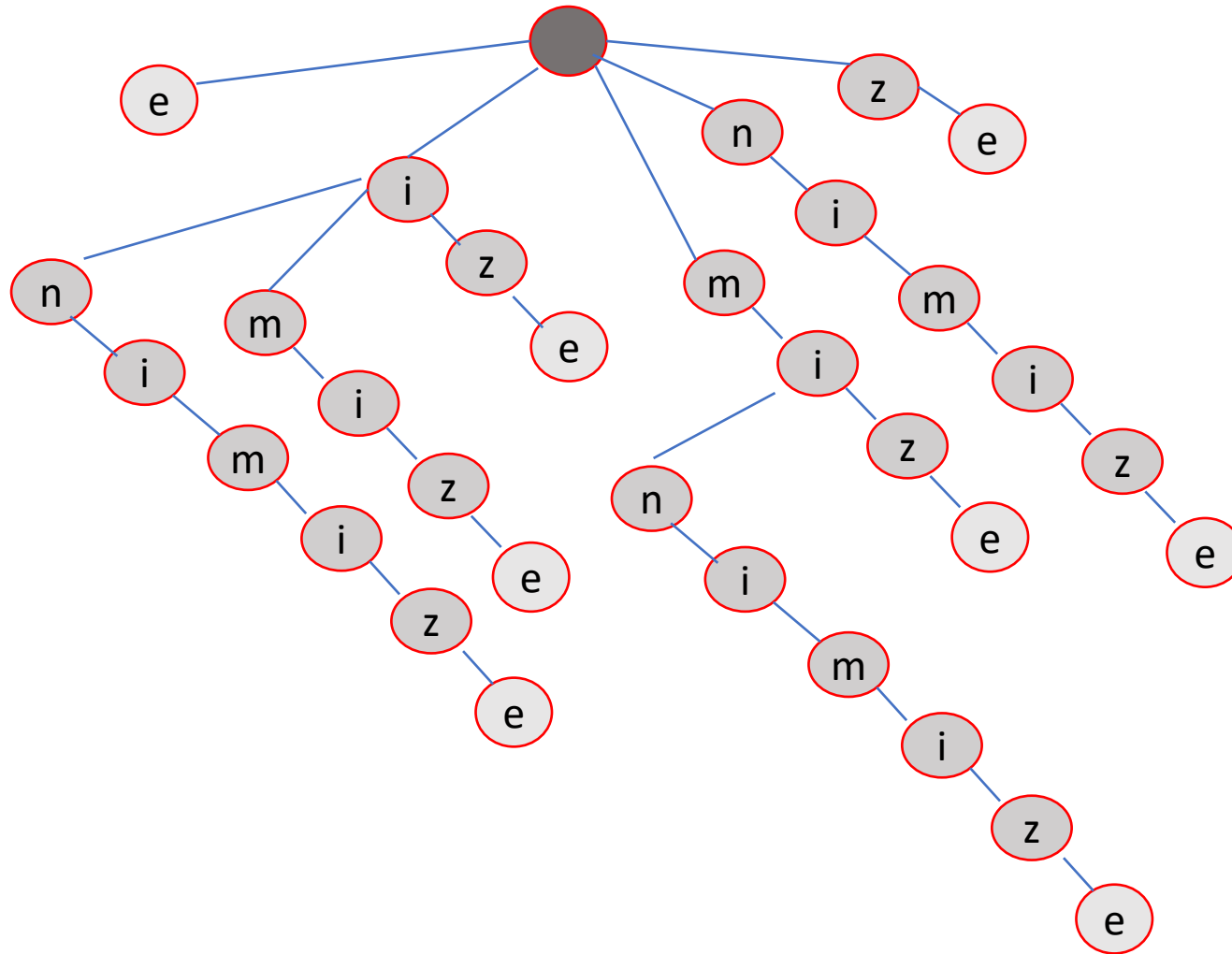
Example 2 : Generate the suffix trie for the word **minimize**

Step1. Generate all the suffixes of the word **minimize**.

 e
ze
ize
mize
imize
nimize
inimize
minimize

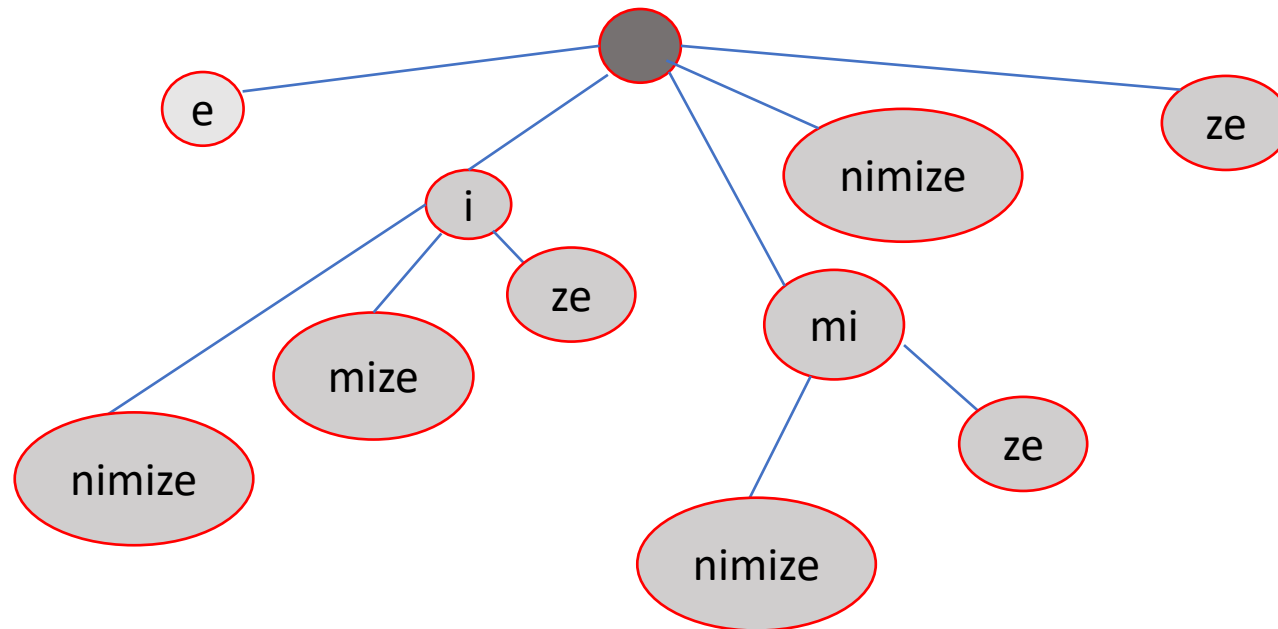
S - set of strings to
include in the suffix trie.

Suffix Trie – Building - for the word minimize



e
ze
ize
mize
imize
nimize
inimize
minimize

S - set of strings to include in the suffix trie

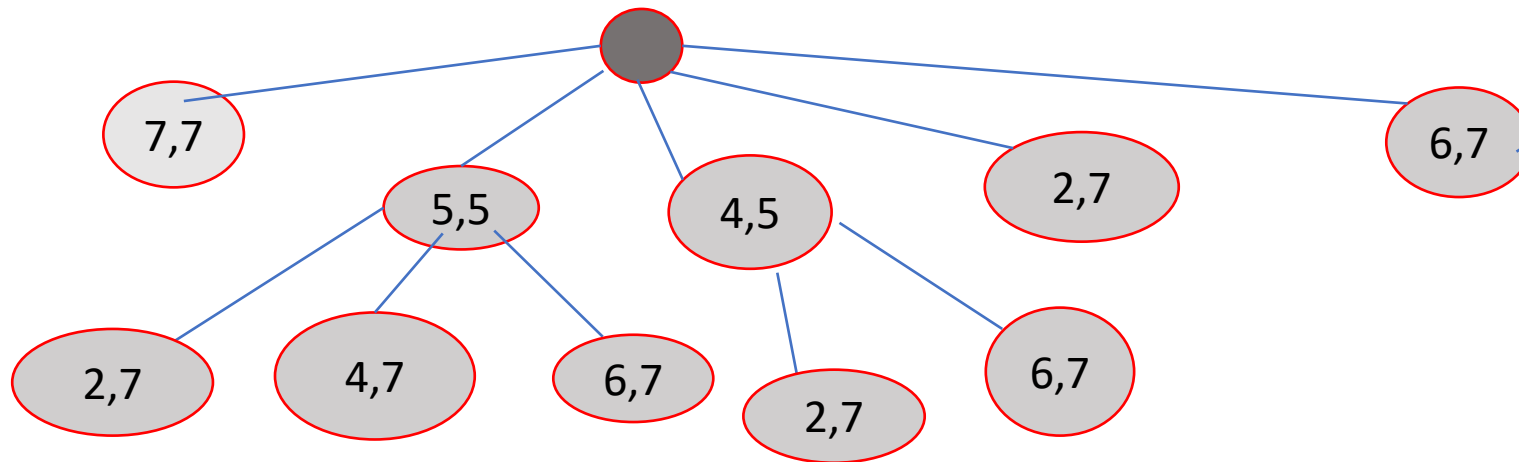


Data Structures and its Applications

Suffix Trie – Compressed Trie – using numbers

- Representation of Compressed trie using numbers - (Indexes)
- The indexes of the word is ...

0	1	2	3	4	5	6	7
m	i	n	i	m	i	z	e



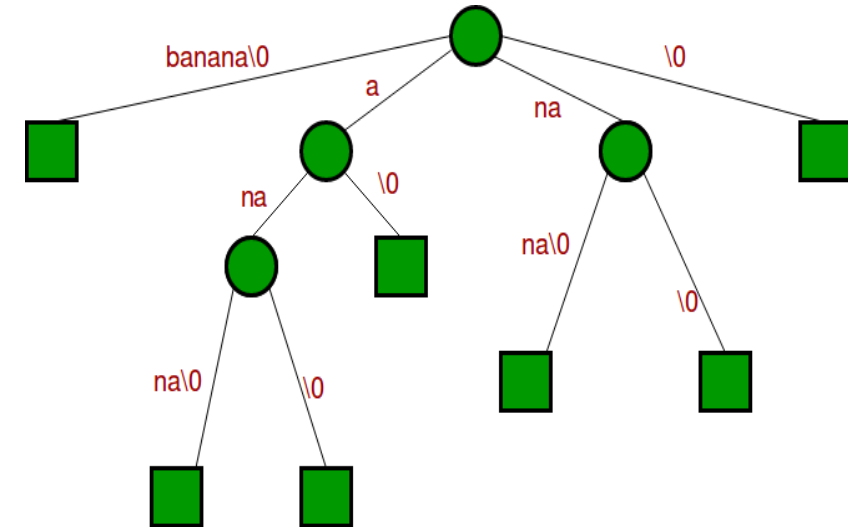
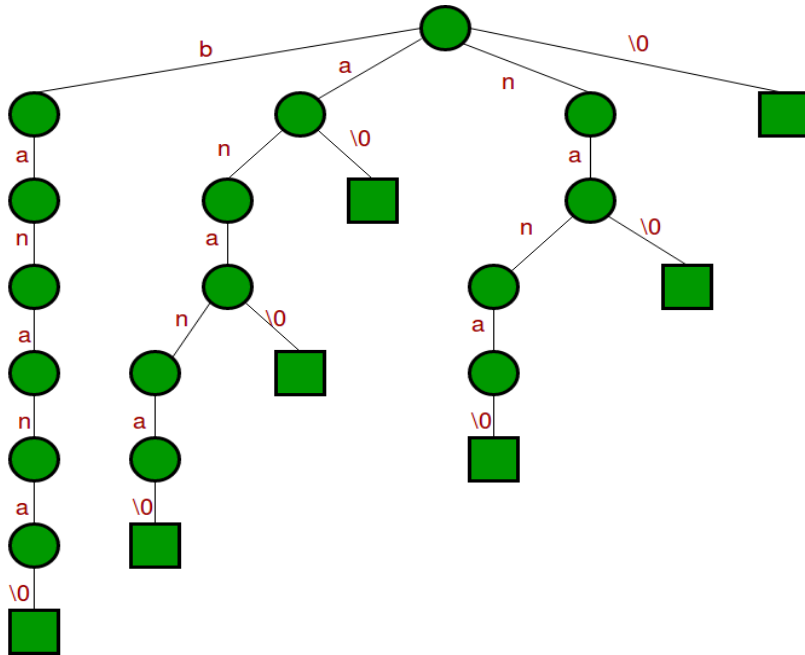
- A simple data structure for string searching
- It's a compressed Trie Tree
- Allow many fast implementations of many important string operations
- Properties of a suffix trees:
 - ✓ A suffix tree for a text X of size n from an alphabet of size d .
 - ✓ Stores all the $n(n-1)$ suffixes of X .
 - ✓ Supports arbitrary pattern matching and prefix matching queries

Example – Banana\$

Data Structures and its Applications

Suffix Trees – Introduction

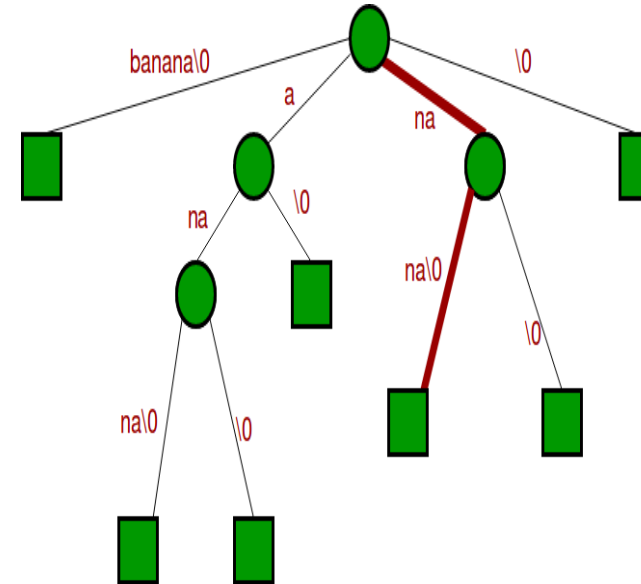
- Join chains of single nodes, to get the following compressed trie, which is the Suffix tree for given text “banana\0”



Data Structures and its Applications

Search for a substring in a Suffix Tree

- 1) Starting from the first character of the pattern and root of Suffix Tree, do following for every character.
 - i) For the current character of pattern, if there is an edge from the current node of suffix tree, follow the edge.
 - ii) If there is no edge, print “pattern doesn’t exist in text” and return.
- 2) If all characters of pattern have been processed, i.e., there is a path from root for characters of the given pattern, then print “Pattern found”.



Data Structures and its Applications

TRIE Trees – Applications, advantages and disadvantages



Applications:

- English dictionary
- Predictive text
- Auto-complete dictionary found on Mobile phones and other gadgets.

Advantages:

- Faster than BST
- Printing of all the strings in the alphabetical order easily.
- Prefix search can be done (Auto complete).

Disadvantages:

- Need for a lot of memory to store the strings,
- Storing of too many node pointers.



THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu



DATA STRUCTURES & ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES & ITS APPLICATIONS

Trie, Suffix Tree

Kusuma K V

Department of Computer Science & Engineering

Trie



- In computer science, a trie, also called digital tree or prefix tree, is a type of search tree, a tree data structure used for locating specific keys from within a set
- These keys are most often strings, with links between nodes defined not by the entire key, but by individual characters
- Common **applications** of tries include storing a predictive text or autocomplete dictionary and implementing approximate matching algorithms, such as those used in spell checking
- Such applications take advantage of a trie's ability to quickly search for, insert, and delete entries

DATA STRUCTURES & ITS APPLICATIONS

Trie



Construct a Trie for the words: algorithm, data, datum, all, self, sea, search

Applications:

- Dictionary
- Auto-complete feature found on Search Engine, Mobile Phone

Advantages:

- Strings can be printed in the alphabetical order easily
- Prefix search can be done (Auto complete)

Disadvantages:

- Need for a lot of memory to store the strings because of storing many node pointers

Suffix Tree

Suffix Trie: A trie containing all the suffixes of the given data

Suffix Tree: A compressed suffix Trie

Suffix Tree

Construct a suffix trie for the word banana and show its suffix tree.



THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution Quadratic Probing, Double Hashing

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Quadratic Probing

Quadratic Probing (Open addressing, closed hashing) resolves collision by using the below formula:

$$h(\text{key}) = (h(\text{key}) + i^2) \% \text{tableSize}$$

where $i = 1, 2, 3, \dots$

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Quadratic Probing



Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7. Use $\text{key} \% \text{tableSize}$ as the hash function & quadratic probing to resolve collision.

$$h(7) = 7 \% 7 = 0$$

$$h(20) = 20 \% 7 = 6$$

$$h(41) = 41 \% 7 = 6 \text{ collision}$$

Resolving collision: $h(41) = ((6 + 1^2) \% 7) = 0$, $h(41) = ((6 + 2^2) \% 7) = 3$

$$h(31) = 31 \% 7 = 3 \text{ collision}$$

Resolving collision: $h(31) = ((3 + 1^2) \% 7) = 4$

$$h(18) = 18 \% 7 = 4 \text{ collision}$$

Resolving collision: $h(18) = ((4 + 1^2) \% 7) = 5$

$$h(8) = 8 \% 7 = 1$$

$$h(9) = 9 \% 7 = 2$$

0	1	2	3	4	5	6
7	8	9	41	31	18	20

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Double Hashing

Double Hashing (Open addressing, closed hashing) resolves collision by using a second hash function whenever there results a collision. The below formula is used:

$$\text{hash}(\text{key}) = (\text{hash1}(\text{key}) + i * \text{hash2}(\text{key})) \% \text{tableSize}$$

Here hash1() and hash2() are hash functions and tableSize is the size of hash table, $i = 1, 2, \dots$

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Double Hashing



Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use **key%tableSize** as the **first hash function** & double hashing (closed hashing) to resolve collision. Use **key%5** as the **second hash function**.

0	1	2	3	4	5	6
7	41		31	18		20

$$h(7) = 7 \% 7 = 0$$

$$h(20) = 20 \% 7 = 6$$

$$h(41) = 41 \% 7 = 6 \text{ collision}$$

Resolving collision: 2nd hash function: $h(41) = 41 \% 5 = 1$

Therefore, $h(41) = (6 + 1*1) \% 7 = 0$, $h(41) = (6 + 2*1) \% 7 = 1$

$$h(31) = 31 \% 7 = 3$$

$$h(18) = 18 \% 7 = 4$$

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Double Hashing



Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use **key%tableSize** as the **first hash function** & double hashing (closed hashing) to resolve collision. Use **key%5** as the **second hash function**.

contd..,

$h(8) = 8 \% 7 = 1$ collision

0	1	2	3	4	5	6
7	41	8	31	18	9	20

Resolving collision: 2nd hash function: $h(8) = 8 \% 5 = 3$

Therefore, $h(8) = (1 + 1*3) \% 7 = 4$, $h(8) = (1 + 2*3) \% 7 = 0$, $h(8) = (1 + 3*3) \% 7 = 3$

$h(8) = (1 + 4*3) \% 7 = 6$, $h(8) = (1 + 5*3) \% 7 = 2$

$h(9) = 9 \% 7 = 2$ collision

Resolving collision: 2nd hash function: $h(9) = 9 \% 5 = 4$

Therefore, $h(9) = (2 + 1*4) \% 7 = 6$, $h(9) = (2 + 2*4) \% 7 = 3$, $h(9) = (2 + 3*4) \% 7 = 0$

$h(9) = (2 + 4*4) \% 7 = 4$, $h(9) = (2 + 5*4) \% 7 = 1$, $h(9) = (2 + 6*4) \% 7 = 5$



THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution Linear Probing

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Linear Probing

Linear Probing (open addressing, closed hashing) resolves collision by finding the next vacant spot in the hash table, in other words, it uses the below formula to resolve collision:

$$h(\text{key}) = (h(\text{key}) + i) \% \text{tableSize}$$

where $i = 1, 2, 3, \dots$

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Linear Probing



Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use $\text{key} \% \text{tableSize}$ as the hash function & linear probing(closed hashing)to resolve collision.

$$h(7) = 7 \% 7 = 0$$

$$h(20) = 20 \% 7 = 6$$

$$h(41) = 41 \% 7 = 6 \text{ collision,}$$

0	1	2	3	4	5	6
7	41	8	31	18	9	20

Resolving collision: $h(41) = ((6+1)\%7)=0$, $h(41) = ((6+2)\%7)=1$

$$h(31) = 31 \% 7 = 3$$

$$h(18) = 18 \% 7 = 4$$

$$h(8) = 8 \% 7 = 1 \text{ collision}$$

Resolving collision: $h(8) = ((1+1)\%7)=2$

$$h(9) = 9 \% 7 = 2 \text{ collision}$$

Resolving collision: $h(9) = ((2+1)\%7)=3$, $h(9) = ((2+2)\%7)=4$, $h(9) = ((2+3)\%7)=5$

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Linear Probing

```
void initTable(NODE ht[],int *n)
{
    for(int i=0;i<SIZE;i++)
        ht[i].mark=-1;

    *n=0;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Linear Probing



```
int insertRecord(NODE ht[],int rNo,char name[],int *n) {  
    if(SIZE==*n)  
        return 0;  
    int index=rNo%SIZE;           //hash function  
    while(ht[index].mark!=-1)  
        index=(index+1)%SIZE;  
    ht[index].rNo=rNo;  
    strcpy(ht[index].name,name);  
    ht[index].mark=1;  
    (*n)++;  
    return 1;  
}
```

```
int deleteRecord(NODE ht[],int rNo,int *n){
    if(*n==0)
        return 0;

    int index=rNo%SIZE;    //hash function
    int i=0;
    while(ht[index].rNo!=rNo && i<*n)
    {
        index=(index+1)%SIZE;
        if(ht[index].mark==1)
            i++;
    }
}
```

```
if(ht[index].rNo==rNo && ht[index].mark==1)
{
    ht[index].mark=-1;
    (*n)--;
    return 1;
}
return 0;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Linear Probing



```
int searchRecord(NODE ht[],int rNo,int n){
    if(n==0)
        return 0;

    int index=rNo%SIZE;    //hash function
    int i=0;
    while(ht[index].rNo!=rNo && i<n)
    {
        index=(index+1)%SIZE;
        if(ht[index].mark==1)
            i++;
    }
```

```
if(ht[index].rNo==rNo && ht[index].mark==1)
{
    strcpy(name,ht[index].name);
    return 1;
}
return 0;
}
```



THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Collision Resolution Separate Chaining

Kusuma K V

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Separate Chaining

Separate Chaining (Open hashing) handles collision by making every hash table cell point to linked lists of records with identical hash function values.



DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Separate Chaining

Insert 7, 20, 41, 31, 18, 8, 9 into a hash table of size 7.

Use $\text{key} \% \text{tableSize}$ as the hash function and separate chaining (open hashing) to resolve collision.

$$h(7) = 7 \% 7 = 0$$

$$h(20) = 20 \% 7 = 6$$

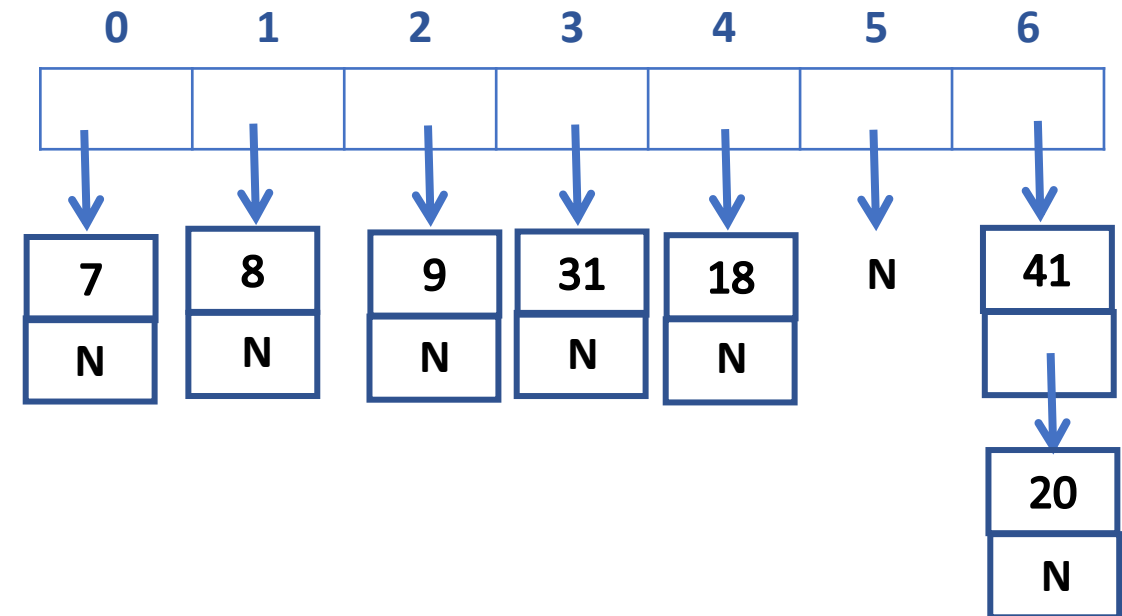
$$h(41) = 41 \% 7 = 6 \text{ collision}$$

$$h(31) = 31 \% 7 = 3$$

$$h(18) = 18 \% 7 = 4$$

$$h(8) = 8 \% 7 = 1$$

$$h(9) = 9 \% 7 = 2$$



DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Separate Chaining

```
typedef struct element
{
    int rNo;
    char name[30];
    struct element *next;
}NODE;
```



DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Separate Chaining

```
void initTable(NODE* ht[SIZE])
{
    for(int i=0;i<SIZE;i++)
        ht[i]=NULL;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Separate Chaining



```
void insert(NODE* ht[SIZE],int rNo,char name[30])
{
    int index=rNo%SIZE;    //hash function

    NODE *newNode=malloc(sizeof(NODE));
    newNode->rNo=rNo;
    strcpy(newNode->name,name);
    newNode->next=ht[index];

    ht[index]=newNode;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Separate Chaining

```
int delete(NODE* ht[SIZE],int rNo)
{
    int index=rNo%SIZE; //hash function

    NODE *p=ht[index];
    NODE *q=NULL;

    while(p!=NULL && p->rNo!=rNo)
    {
        q=p;
        p=p->next;
    }
}
```

```
if(p!=NULL)
{
    if(q==NULL)
        ht[index]=p->next;
    else
        q->next=p->next;

    free(p);
    return 1;
}
return 0;
```


DATA STRUCTURES AND ITS APPLICATIONS

Hashing: Separate Chaining



```
int search(NODE* ht[SIZE],int rNo,char name[30]) {  
    int index=rNo%SIZE;    //hash function  
    NODE *p=ht[index];  
    while(p!=NULL) {  
        if(p->rNo==rNo) {  
            strcpy(name,p->name);  
            return 1;  
        }  
        p=p->next;  
    }  
    return 0;  
}
```



THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Hashing

Kusuma K V

Department of Computer Science & Engineering

Time taken for search

- Linear DS: Array List, Linked List --- $O(n)$
- Non linear DS: Balanced Binary Search Tree --- $O(\log n)$
- Can we search in $O(1)$ time ??
 - Hashing

DATA STRUCTURES AND ITS APPLICATIONS

Hashing



Hashing is the process of transforming any given key into another value.

This is usually represented by a shorter, fixed-length value or key that makes it easier to find the original key.

The most popular use for hashing is the implementation of hash tables.

A hash table stores key and value pairs in a list that is accessible through its index.

DATA STRUCTURES AND ITS APPLICATIONS

Hashing

Direct Mapping

Eg: Employee Records



DATA STRUCTURES AND ITS APPLICATIONS

Hashing

Hash function: A function which maps key value into a hash table range

- Folding
- Truncation
- Modulo

DATA STRUCTURES AND ITS APPLICATIONS

Hashing

Collision: The phenomenon when two or more keys generate the same hash.

Collision resolution:

- Open addressing (Closed Hashing)
- Separate Chaining (Open Hashing)

Collision resolution:

- Open addressing (Closed Hashing)
 - Open addressing handles collisions by storing all data in the hash table itself and then seeking out availability in the next spot created by the algorithm.
 - Linear Probing, Quadratic Probing, Double Hashing
- Separate Chaining (Open Hashing)
 - Separate chaining handles collision by making every hash table cell point to linked lists of records with identical hash function values.

Collision resolution:

- Rehashing (Hash again)
 - Increase the table size when the load factor becomes more than a predefined value (Say 0.75) and hash all the keys present in the table again and store in the new table of increased size.

Load factor = $\frac{\text{No. of filled records}}{\text{Total capacity}}$

DATA STRUCTURES AND ITS APPLICATIONS

References

- Live lecture Videos on Hashing for the course UE19CS202, Data Structures, by Dr. Shylaja S S
- [What is hashing and how does it work? \(techtarget.com\)](https://www.techtarget.com)





THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

**Finding all the paths from a given
source to destination**

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Applications of BFS and DFS

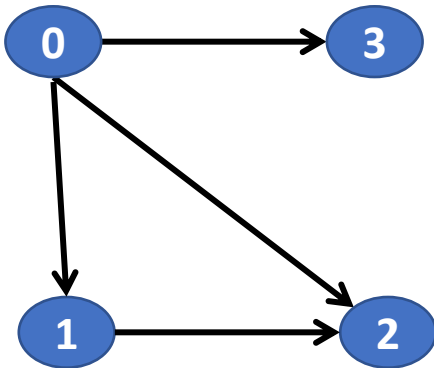
Application of DFS

- Detecting whether a cycle exist in graph.
- Finding a path in a network
- Topological Sorting: Used for job scheduling
- To check whether a graph is strongly connected or not



Path

- Sequence of edges that allows the user to go from vertex A to vertex B



- The paths from vertex 0 to vertex 2:
- 1. 0 → 1 → 2
- 2. 0 → 2

DATA STRUCTURES AND ITS APPLICATIONS

Finding all the paths from the given source to destination



- **Methodology**
- 1.Start from any traversal Method from a given source node
- 2.Store all the visited vertices in an array
- 3.Once the destination vertex is reached, print all the contents of Array.

DATA STRUCTURES AND ITS APPLICATIONS

Function to print all the paths using DFS traversal method



//Function to print all the paths from a given source to destination

```
void path_find(int source,int destination)
```

```
{
```

```
    int visited[10] //An array to store the vertices as visited or not
```

```
    int path[10] //An array to store the path
```

```
    int count=0;
```

```
    for(int i=0;i<n;i++)
```

```
        visited[i]=0 //initilize all the vertices as not visited.
```

```
    printallpaths(source,destination,visited,path);
```

```
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Function to print all the paths using DFS traversal method



```
void printallpaths(int u,int d,int visited[10],int path[10])
{
    visited[u]=1;//Mark the current node and and store it in the array path
    path[count]=u;
    count++;
    if(u==d) //if the current vertex is same as the destination then print the array
    which has stored the path
    {
        for(i=0;i<count;i++)
            printf("%d",path[i]);
    }
    else // if the current vertex is not the destination
    {
        for(NODE temp=a[u];temp!=NULL;temp=temp->link)
```

DATA STRUCTURES AND ITS APPLICATIONS

Function to print all the paths using DFS traversal method



```
if(!visited[temp->data])
{
    printallpaths(temp->data,d,visited,path);
}
}
count-- //Remove the current vertex from the path[] and mark it as
unvisited
Visited[u]=0;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Function to read adjacency list



```
void read_adjacency_list(NODE a[],int n)
{
    int ele,m;
    for(int i=0;i<n;i++)
    {
        printf("enter the number of nodes adjacent to %d:",i);
        scanf("%d",&m);
        if(m==0)
            continue;
        printf("enter the nodes adjacent to %d:",i);
        for(int j=0;j<m;j++)
        {
            scanf("%d",&ele);
            a[i]=insert_rear(ele,a[i]); // insert at rear end
        }
    }
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Function to insert a node at rear end of the list



Function to insert a node at rear end

```
NODE insert_rear(int v,NODE head)
{
    NODE temp;
    NODE cur;
    temp=getnode(); // create a node
    temp->info=v;
    temp->link=NULL;
    if(head==NULL)
        return temp;
    cur=head;
    while(cur->link!=NULL)
    {
        cur=cur->link;
    }
    cur->link=temp;
    return(head); }
```

DATA STRUCTURES AND ITS APPLICATIONS

Connectivity of the graph using Adjacency list



Function to create a node

```
NODE getnode()
{
    NODE temp;
    temp=(NODE)malloc(sizeof(struct node)); //Dynamic allocation
    if(temp==NULL)
    {
        printf("out of memory");
        return;
    }
    return temp;
}
```




THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963



Data Structures and its Applications

V R BADRI PRASAD

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of TRIE Trees

V R BADRI PRASAD

Department of Computer Science & Engineering

Structure of a node in a TRIE Tree :

- A node of a TRIE tree is represented as shown below.
- One field for each alphabet(A – Z), 26 columns.
- Each column is a pointer to another TRIE node or carries NULL and
- One field for end of word (key).

A	B	C	D	E	F		W	X	Y	Z
F1	F2	F3	F4	F5	F6		F23	F24	F25	F26
End of Word / (eok)										

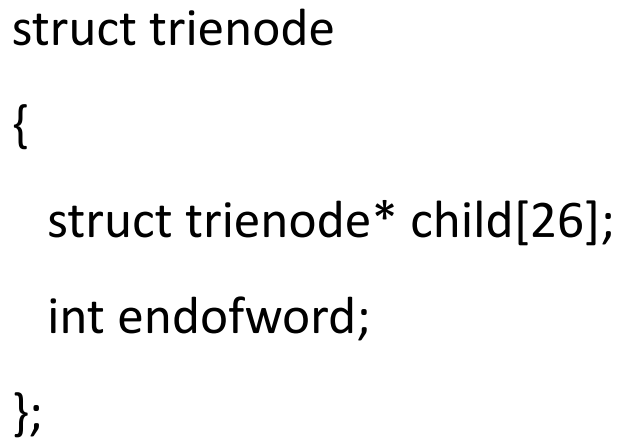
Address of the next node (reference for us)

Field number – for user's reference no field is created , No memory is allocated

End of word / key field

```
struct trienode
{
    struct trienode* child[26];
    int endofword;
};
```

TRIE Trees – Implementation



A	B	C	D	E	F		W	X	Y	Z	Address of the next node (reference for us)
F1	F2	F3	F4	F5	F6		F23	F24	F25	F26	Field number
End of Word / (eok - \$)											End of word / key field

Creation of a node in a TRIE Tree using malloc() function.

```
struct trienode *getnode()
```

```
{
```

```
    int i;
```

```
    struct trienode *temp;
```

```
    temp=(struct trienode*)(malloc(sizeof(struct trienode)));
```

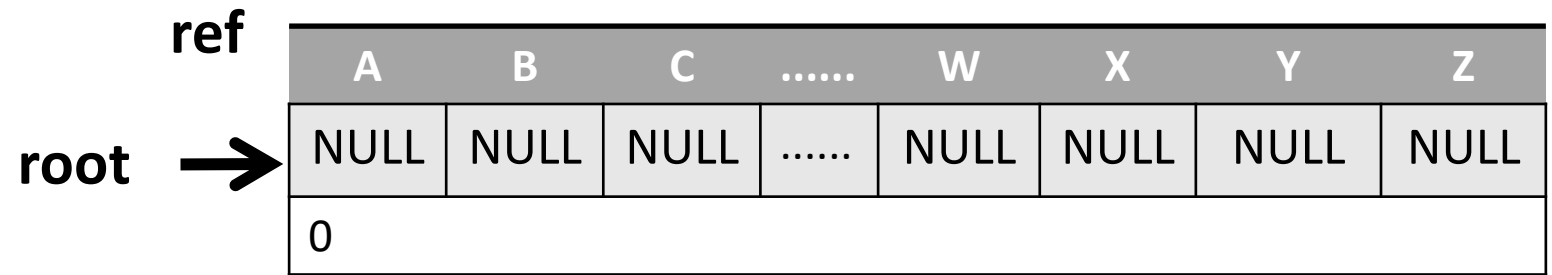
```
    for(i=0;i<26;i++)
```

```
        temp->child[i]=NULL;    // Initialize all the fields to NULL.
```

```
    temp->endofword=0;    // Initialize endofword to 0.
```

```
    return temp;
```

```
}
```



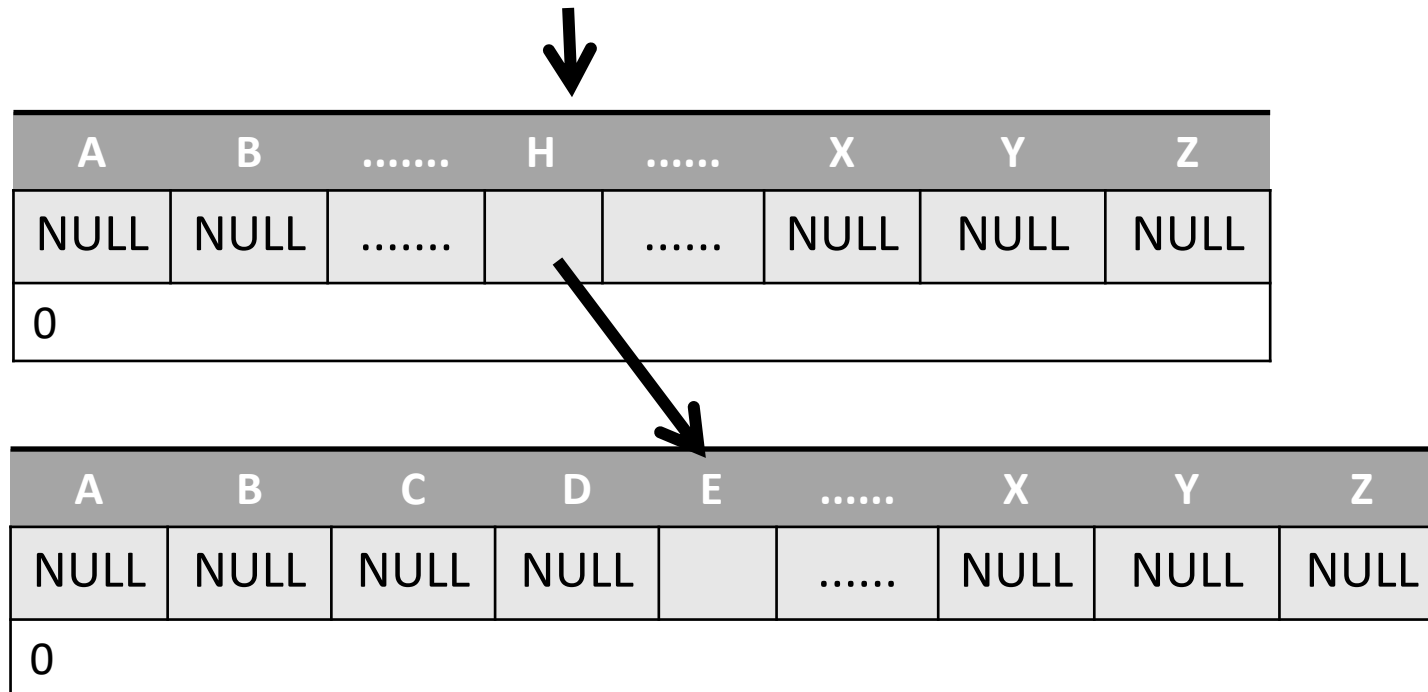
root=getnode();

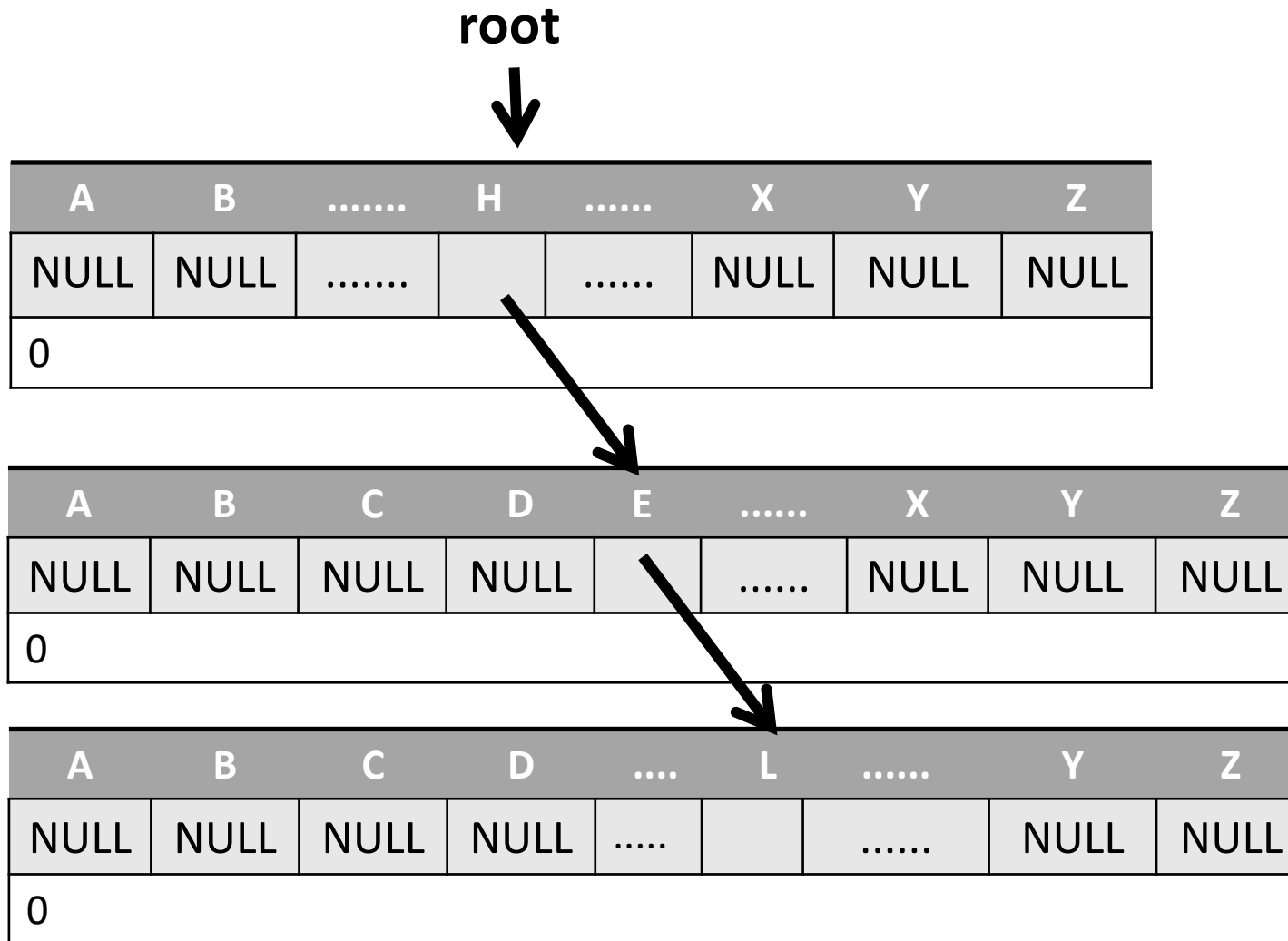
Insertion of a node in a TRIE Tree

```
void insert(struct trienode *root, char *key)
{
    struct trienode *curr;
    int i,index;

    curr=root;
    for(i=0;key[i]!='\0';i++)
    {
        index=key[i]-'a';
        if(curr->child[index]==NULL)
            curr->child[index]=getnode();
        curr=curr->child[index];
    }
    curr->endofword=1;
}
```

```
insert(root,key);    root
```







THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu



Data Structures and its Applications

V R BADRI PRASAD

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of TRIE Trees :

- Display Operation
- Deletion Operation
- Search Operation

V R BADRI PRASAD

Department of Computer Science & Engineering

```
void display(struct trienode *curr)
{
    int i,j;
    for(i=0;i<255;i++)
    {
        if(curr->child[i]!=NULL)
        {
            word[length++]=i;
            if(curr->child[i]->endofword==1)//if end of word
            {
                printf("\n");
                for(j=0;j<length;j++)
                    printf("%c",word[j]);
            }
            display(curr->child[i]);
        }
    }
    length--;
    return ;
}
```

```
void delete_trie(struct trienode *root, char *key)
{
    int i,index,k;
    struct trienode *curr;
    struct stack x;
    curr=root;

    for(i=0;key[i]!='\0';i++)
    {
        index=key[i];
        if(curr->child[index]==NULL)
        {
            printf("The word not found..\n");
            return;
        }
        push(curr,index);
        curr=curr->child[index];
    }
    curr->endofword=0;
    push(curr,-1);
```

```
while(1)
{
    x=pop();
    if(x.index!=-1)
        x.m->child[x.index]=NULL;
    if(x.m==root)//if root
        break;
    k=check(x.m);
    if((k>=1) || (x.m->endofword==1))
        break;
    else
        free(x.m);
}
return;
}
```

```
int search(struct trienode * root,char *key)
{

    int i,index;
    struct trienode *curr;
    curr=root;

    for(i=0;key[i]!='\0';i++)
    {
        index=key[i];

        if(curr->child[index]==NULL)
            return 0;
        curr=curr->child[index];
    }
    if(curr->endofword==1)
        return 1;
    return 0;
}
```




THANK YOU

V R BADRI PRASAD

Department of Computer Science & Engineering

badriprasad@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Connectivity of the graph

Saritha

Department of Computer Science & Engineering

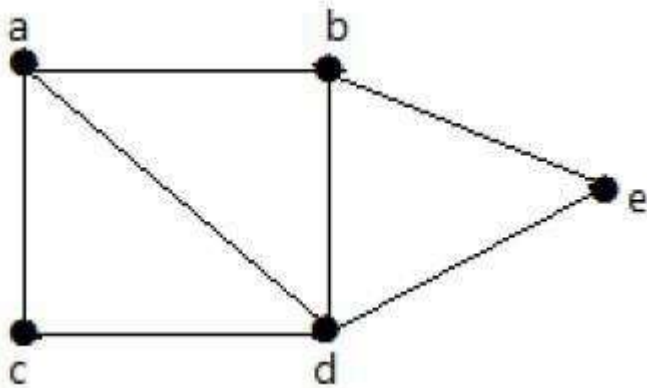
Application of BFS

- Finding the shortest path
- Social Networking websites like twitter, Facebook etc.
- GPS Navigation system
- Web crawlers
- Finding a path in network
- In Networking to broadcast the packets

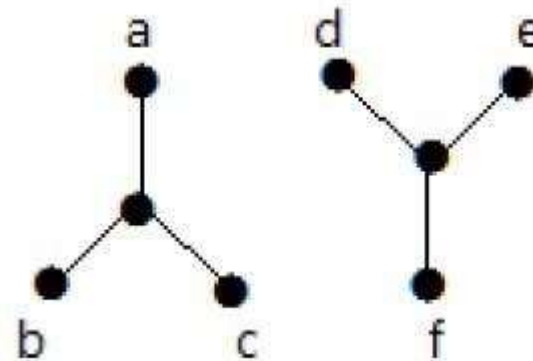
Connectivity of graph

- Connectivity refers to connection between two or more nodes or things

Connected graph



Disconnected graph



DATA STRUCTURES AND ITS APPLICATIONS

Connectivity of the graph

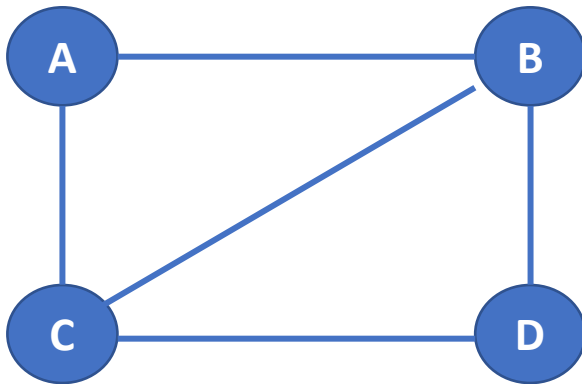


- A graph is connected if there is a path between every pair of vertices.
- The graph that is not connected is called disconnected graph.
- In connected graph there is no unreachable vertex.
- In the area of Information Technology the connectivity refers to internet connectivity through which various devices are connected to global network

DATA STRUCTURES AND ITS APPLICATIONS

Connectedness in the Direct graph using adjacency matrix

- To check whether a graph is connected or not, we traverse the graph using either bfs traversal or dfs traversal method
- After the traversal if there is at-least one node which is not marked as visited then that graph is disconnected graph
- For example: Below graph is connected



	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Procedure to check the whether a graph is connected or not using adjacency matrix

- Read the adjacency matrix .
- Create a visited [] array. Start DFS/BFS traversal method from any arbitrary vertex and mark the visited vertices in the visited[] array.
- Once DFS/BFS is completed check the visited [] array. if there is at-least one vertex which is marked as unvisited then the graph is disconnected otherwise it is connected.

Function to read adjacency matrix:

```
void read_ad_matr(int graph[][],int n)
{
    int i,j;
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d", &graph[i][j]);//reading the values into two dimensional array
        }
    }
}
```

Function to traverse the graph using DFS

```
void traverse(int u, int visited[])
{
    visited[u] = 1; //mark v as visited
    for(int v = 0; v<n; v++)
    {
        if(graph[u][v])
        {
            if(!visited[v])
                traverse(v, visited);
        }
    }
}
```

Function to traverse the graph using bfs:

```
void traverse(int s, int visited[],int n,int graph[][]){
    int f,r,v,q[10];
    f=0,r=-1;
    q[++r]=s;
    visited[s]=1; // mark the nodes as visited
    while(f<=r)
    {
        s=q[f++];
        for(v=0;v<n;v++)
        {
            if(graph[s][v]==1) //adjacent nodes
            {
```

DATA STRUCTURES AND ITS APPLICATIONS

Connectivity of the graph using Adjacency Matrix



```
if(visited[v]==0) //nodes not visited
{
    visited[v]=1; //mark as visited
    q[++r]=v;
}
}
}
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Connectivity of the graph using Adjacency Matrix



Function to check whether the graph is connected or not

```
int isConnected()
{
    int vis[NODE]; //for all vertex u as start point, check whether all nodes are visible or not
    for(int u=0; u < NODE; u++)
    {
        for(int i = 0; i<NODE; i++)
            vis[i] = 0; //initialize as no node is visited
        traverse(u, vis); // any traversal method either bfs or dfs
        for(int i = 0; i<NODE; i++)
        {
            if(!vis[i]) //if there is a node, not visited by traversal, graph is not connected
                return 0;
        }
    }
    return 1;}
}
```



THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963



DATA STRUCTURES AND ITS APPLICATIONS

Balanced Trees

Sandesh B. J

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Balanced Trees

Sandesh B. J

Department of Computer Science & Engineering

In this lecture you will be able to learn:

- Why trees becomes unbalanced?
- Why we need to balance the tree?
- AVL Tree
- How do we balance the unbalanced trees using tree rotation techniques
- Different tree Rotation techniques
 - ✓ Left rotation, right rotation
 - ✓ Left-right rotation and right-left rotations



Why Balanced Trees?

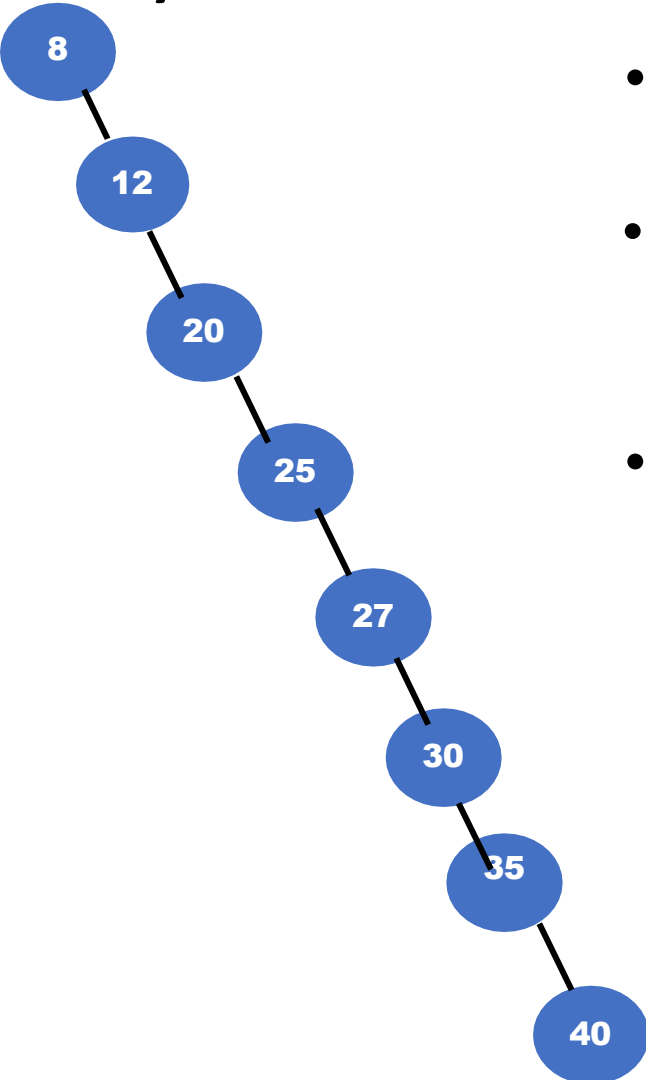
- Binary Search Tree(BST) – Data Structure used to implement the dictionary.
- What do we gain by implementing dictionary using BST instead of array ?

Complexity of Binary Search Tree

Operations	Average	Worst
Insert	$O(\log(n))$	$O(n)$
Delete	$O(\log(n))$	$O(n)$
Search	$O(\log(n))$	$O(n)$



Why Balanced trees?



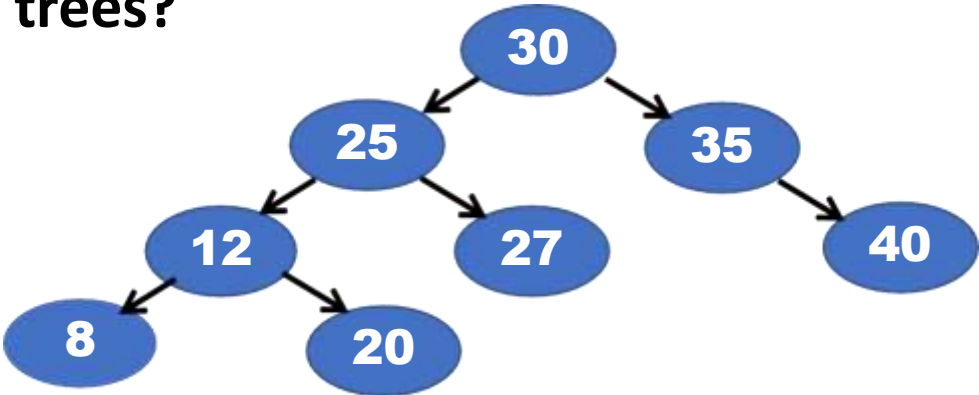
- Height of the BST depends on the order of insertion of elements into tree
- What happens if we insert elements given in increasing order?
 - Insert 8, 12, 20, 25, 27, 30, 35, 40
- Severely unbalanced

Disadvantages of Binary search trees

- The search and insertion algorithm does not ensure that the tree remains balanced
- The degree of balance dependent on the order in which the keys are inserted
- Tree can attain a height which can be as large as $n-1$
- Time taken for most of the operations worst case $O(n)$



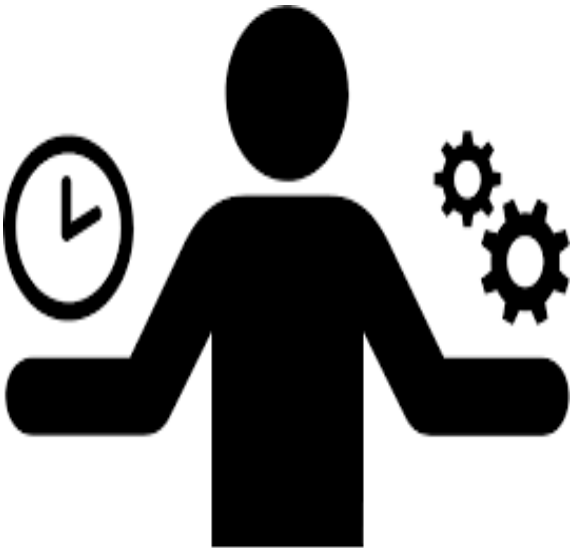
Why Balanced trees?



- we can construct the binary search tree in which the height of the tree is always $\log(n)$

Complexity of Balanced Binary Search Tree

Operations	Average	Worst
Insert	$O(\log(n))$	$O(\log(n))$
Delete	$O(\log(n))$	$O(\log(n))$
Search	$O(\log(n))$	$O(\log(n))$



DATA STRUCTURES AND ITS APPLICATIONS

AVL Tree-Balanced Binary Search Trees

- AVL tree was invented in 1962 by two Russian mathematicians G.M. Adel'son-Vel'skii and E.M. Landis(AVL)
- An AVL tree is a binary search tree in which, for every node, the difference between the heights of the left and right subtrees, called the balance factor is either 0 or +1 or -1

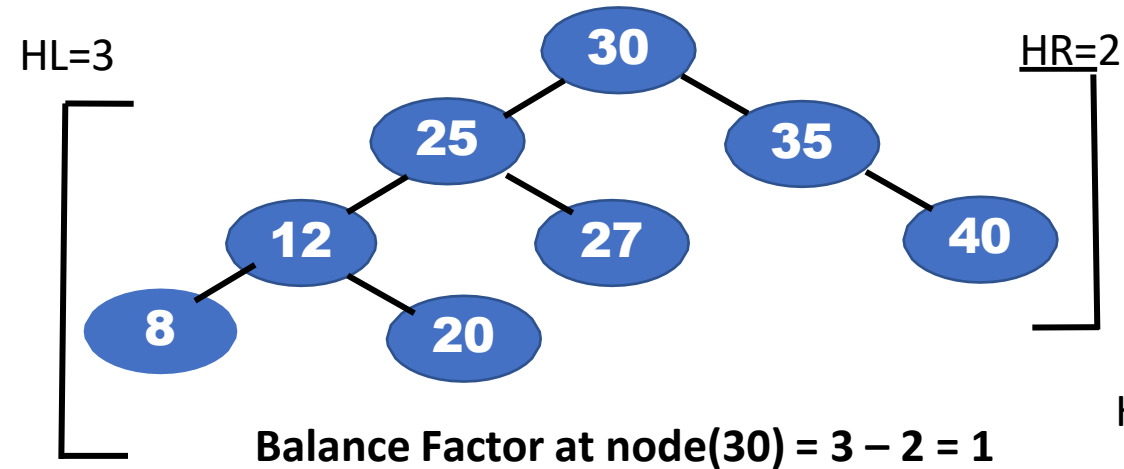
The Balance factor of any node:

Balance Factor = Height(left subtree) – Height(right subtree)

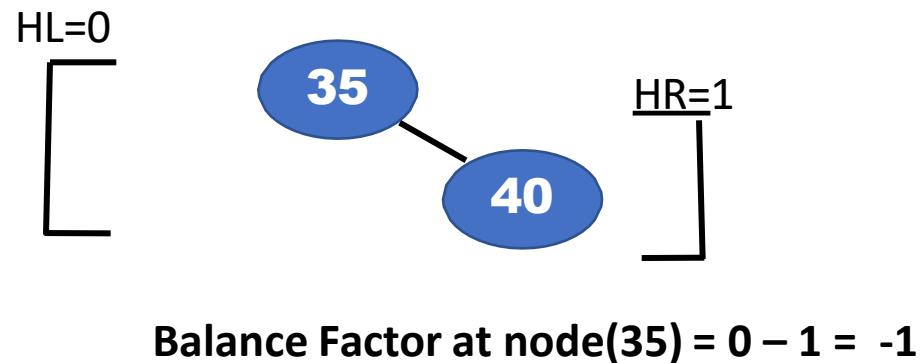
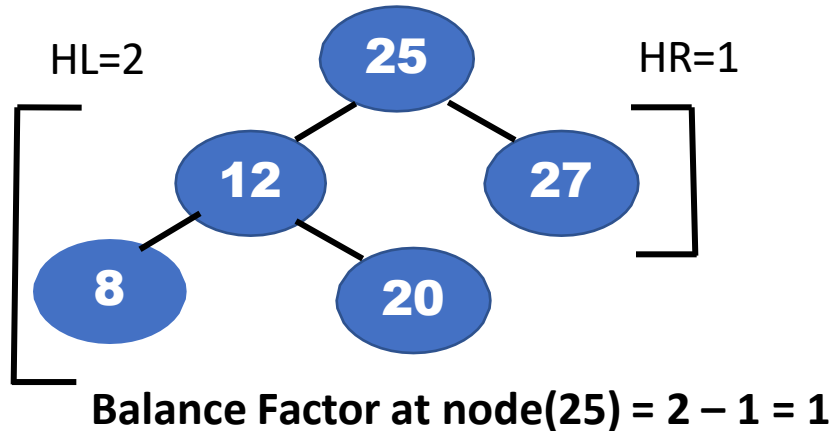
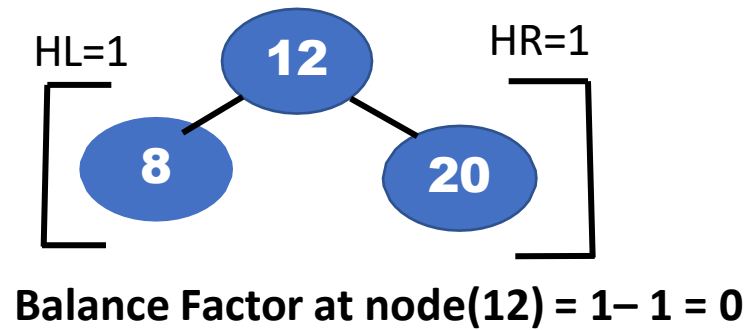


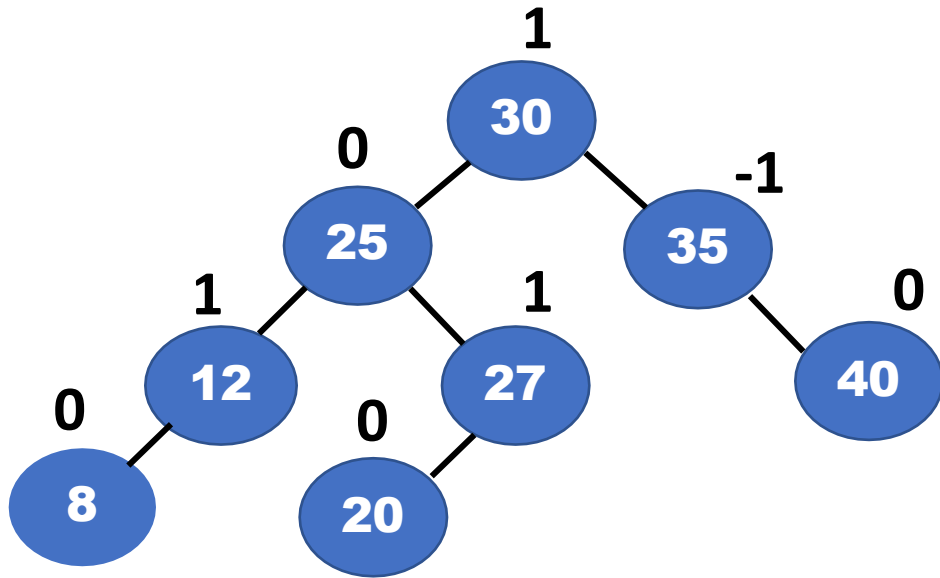
DATA STRUCTURES AND ITS APPLICATIONS

Balanced Binary Search Trees(AVL)

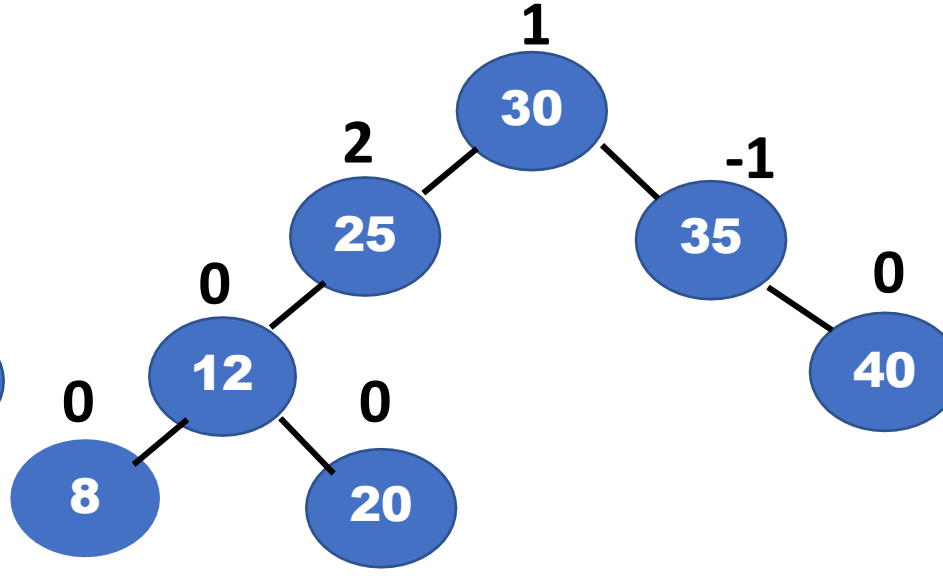


Balance Factor = HL - HR





AVL Tree

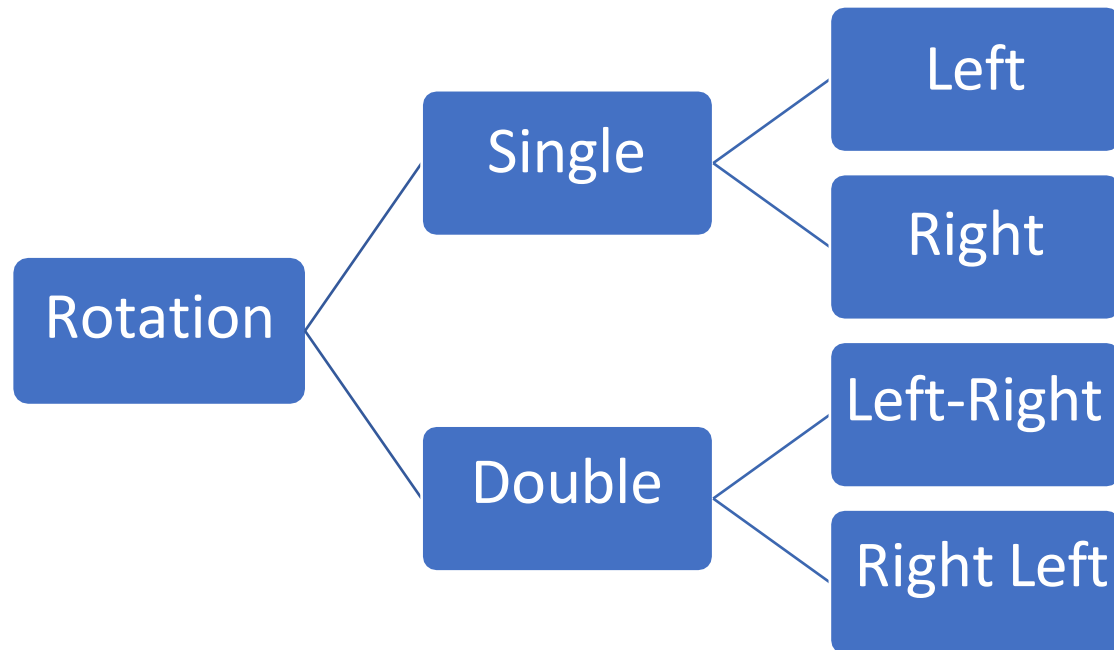


Not an AVL tree

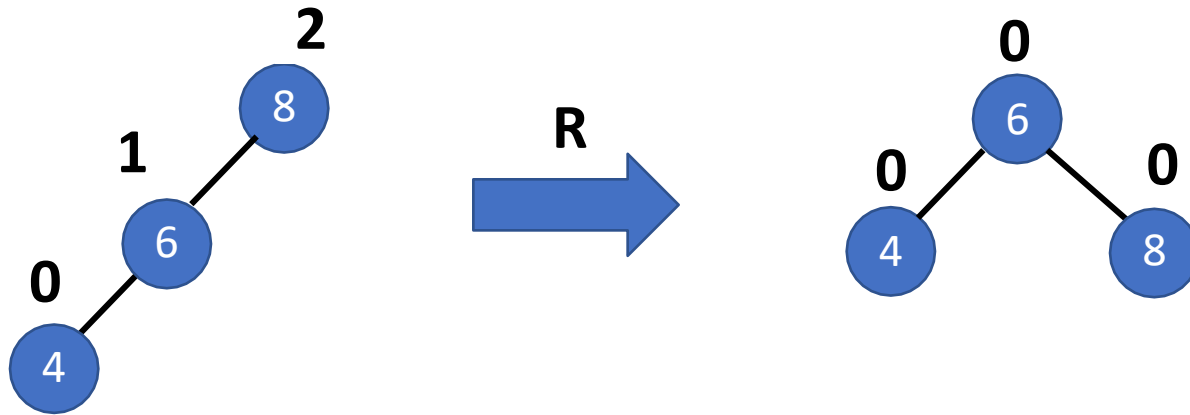
- Node in balanced binary tree has balance of
 - 1 – Height(left subtree) > Height(right subtree)
 - 0 – Height(left subtree) = Height(right subtree)
 - 1 – Height(left subtree) < Height(right subtree)

- The AVL tree may become unbalanced after insert and delete operations
- If a key insertion violates the balance requirement at some node, the subtree rooted at that node is transformed via one of the four *rotations*.
- Rotation in a AVL tree is a local transformation of its subtree rooted at a node whose balance has become either +2 or -2
- In case there are several such nodes, The rotation is always performed for a subtree rooted at “unbalanced” node closest to the newly inserted leaf node.

- Different types of Rotation:

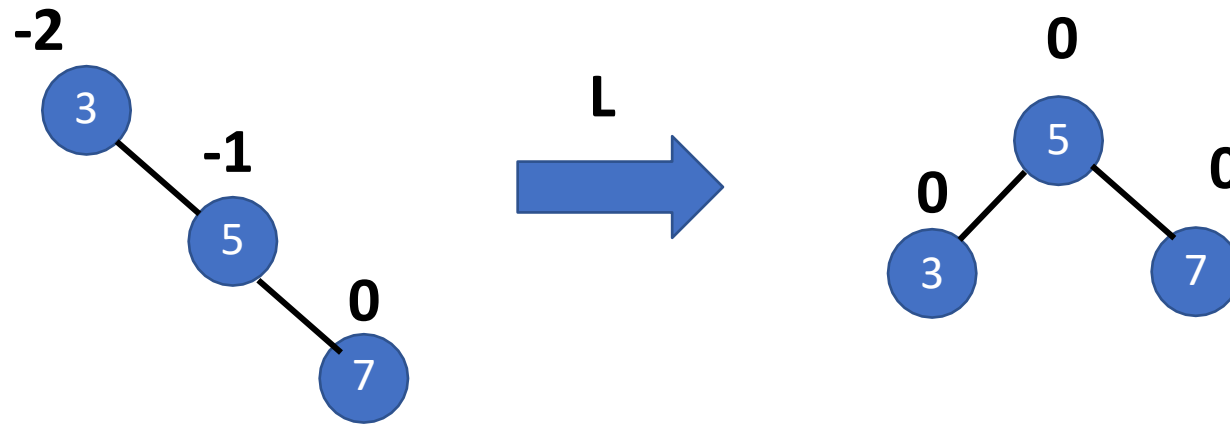


Single Right Rotation (R-Rotation)



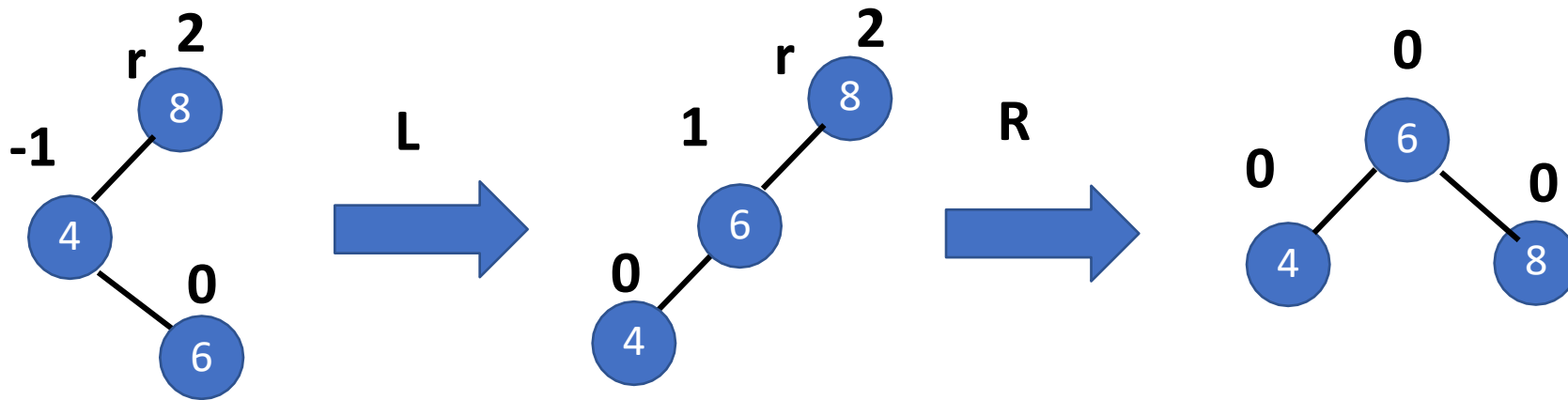
- Root of the tree has balance of +1 before the insertion
- New key is inserted to the left of left child (Left-Left-case)
- To Balance the tree we need to perform the right rotation

Single Left Rotation:



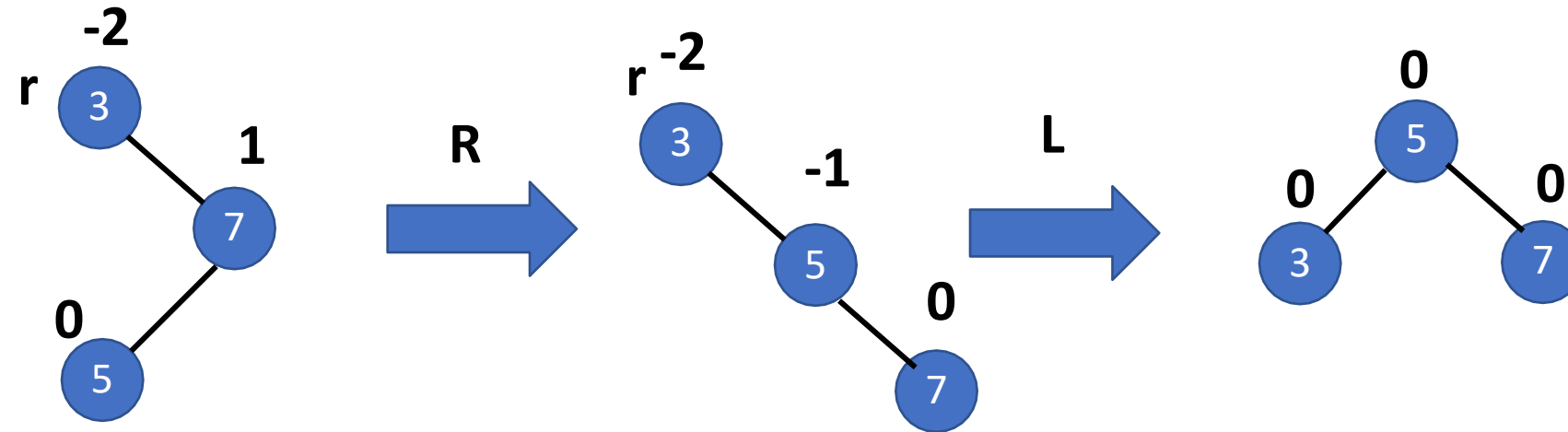
- Root of the tree had balance of -1 before the insertion
- New key is inserted to the right of right child (Right-Right-case)
- To Balance the tree we need to perform Left Rotation

Double Left Right Rotation:



- Root r had the balance of +1 before the insertion
- New key is inserted to the right of left child (Left-Right-case)
- We perform the left rotation of left subtree of root r
- Right rotation of the new tree rooted at r

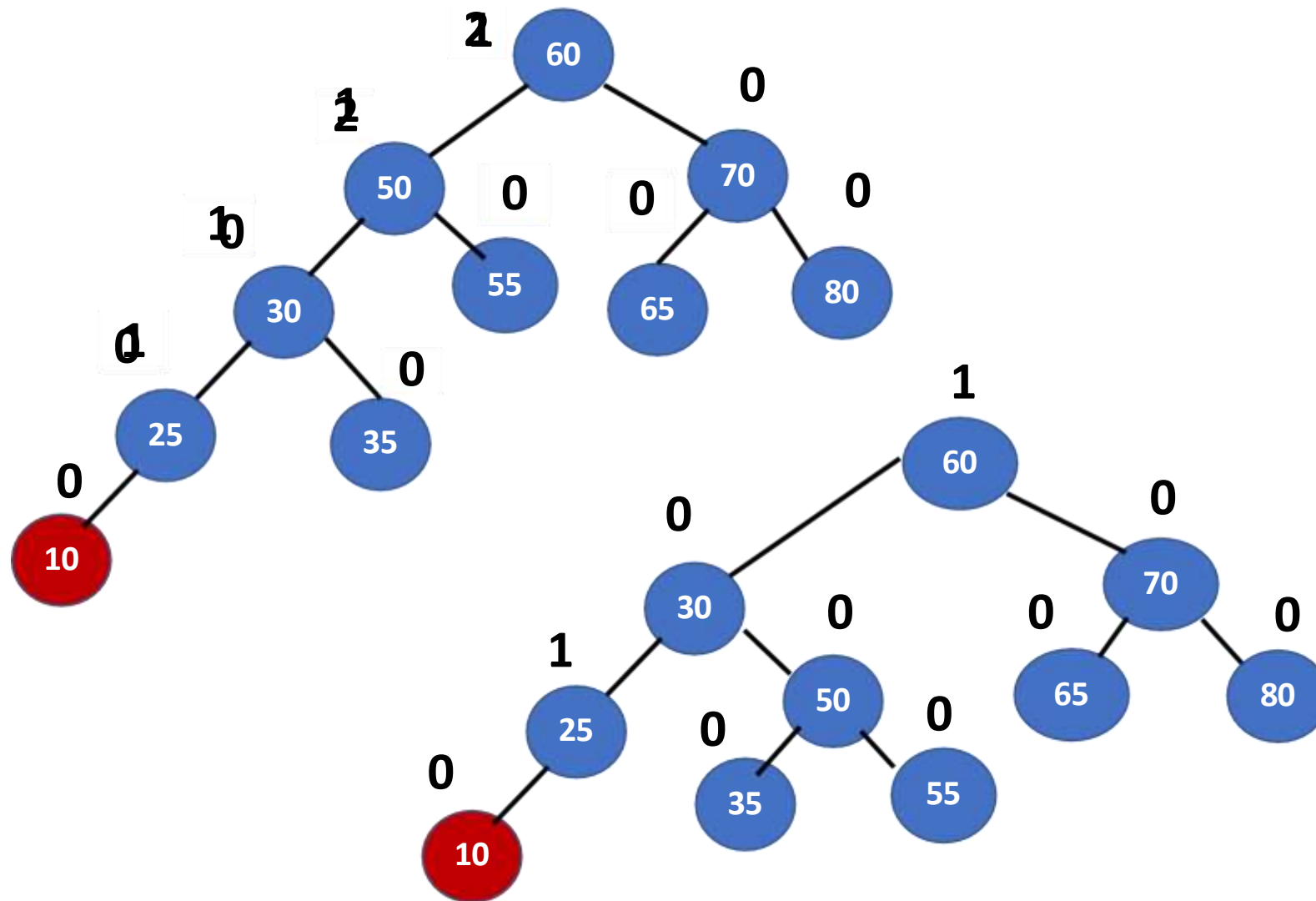
Double Right Left Rotation:



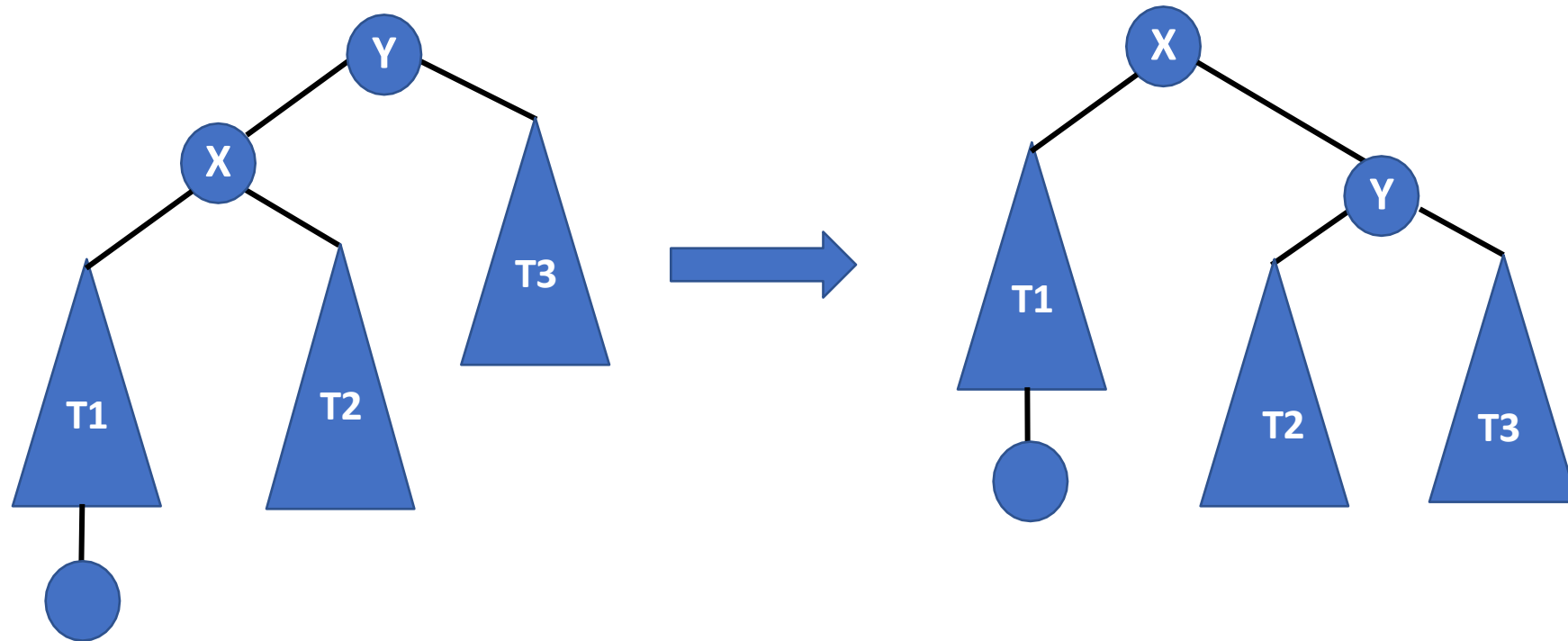
- Root r had the balance of -1 before the insertion
- New key is inserted to the left of right child (Right-Left-case)
- We perform the right rotation of right subtree of root r
- Left rotation of the new tree rooted at r

DATA STRUCTURES AND ITS APPLICATIONS

Example – Rotations in AVL Trees



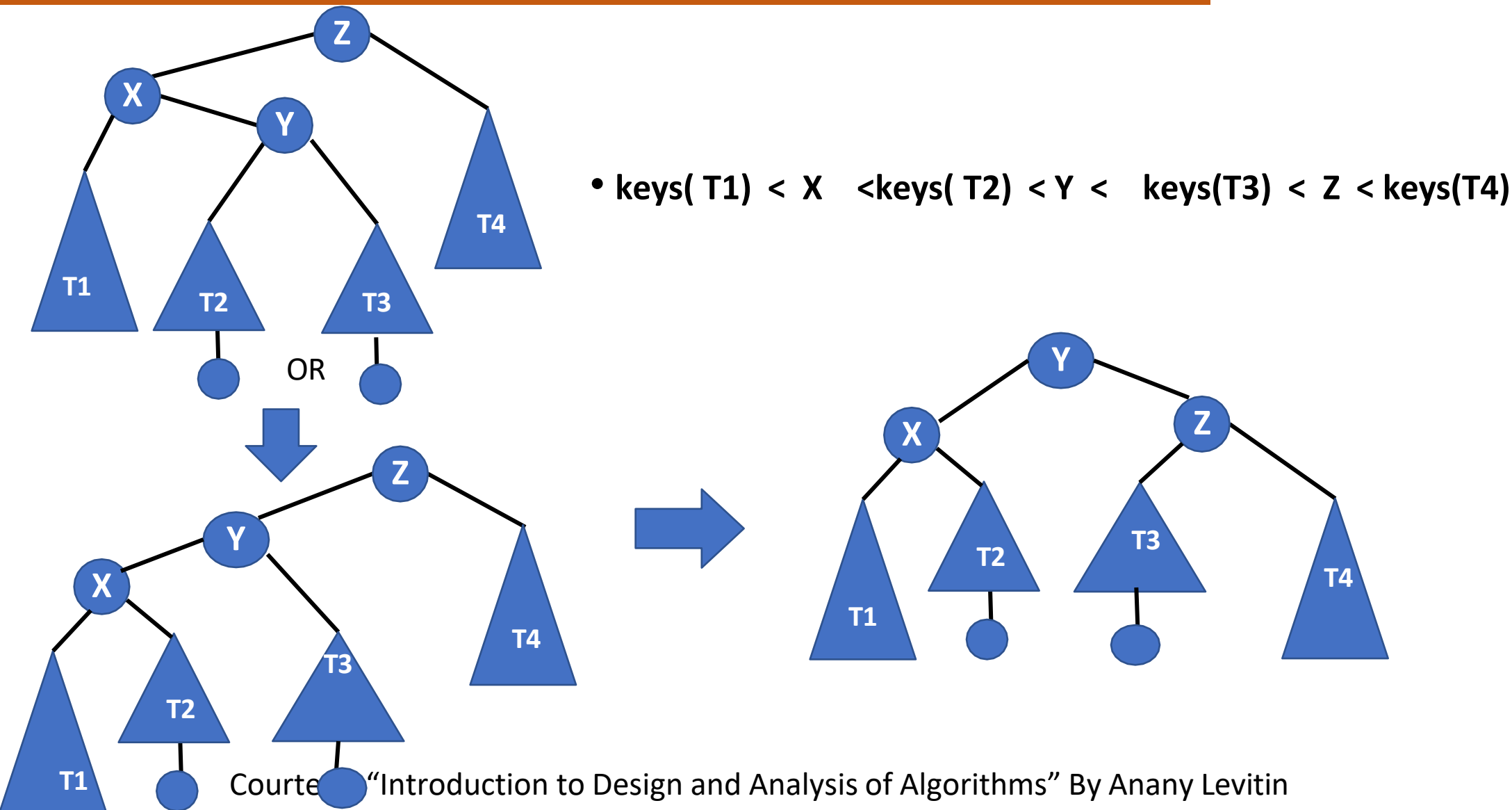
Courtesy: "Introduction to Design and Analysis of Algorithms" By Anany Levitin



- $\text{keys}(T1) < X < \text{keys}(T2) < Y < \text{keys}(T3)$

DATA STRUCTURES AND ITS APPLICATIONS

General form of Left – Right Rotation



Courtesy "Introduction to Design and Analysis of Algorithms" By Anany Levitin



THANK YOU

Sandesh B. J

Department of Computer Science & Engineering

sandesh_bj@pes.edu

+91 80 6618 6623



DATA STRUCTURES AND ITS APPLICATIONS

Graph Representation

Sandesh B. J

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Graph Representation

Sandesh B. J

Department of Computer Science & Engineering

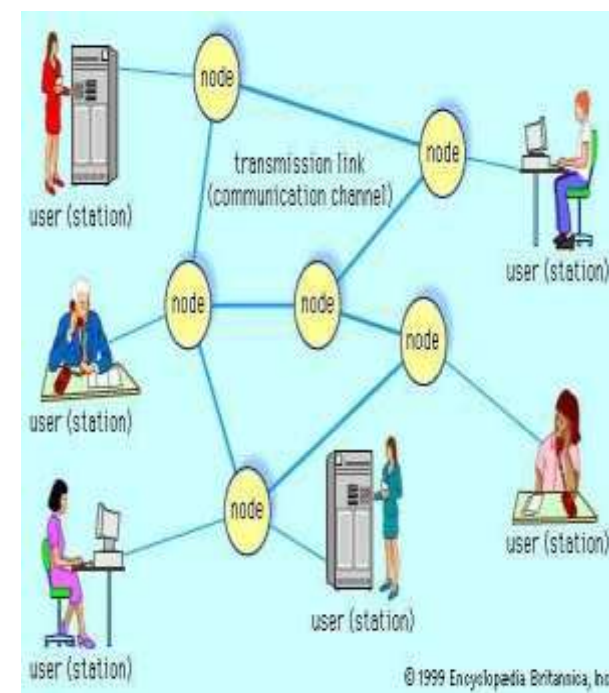
Learning objectives:

- Why we need to represent the structure of the graph in computer memory?
- How do we represent the graph ?
- Data structure used for the representation:
 - ✓ Adjacency Matrix and Adjacency List
- Representation : Directed, Undirected and Weighted graphs
- Implementation details of representation using 'c'
- Multilinked Representation

DATA STRUCTURES AND ITS APPLICATIONS

Graph Representation

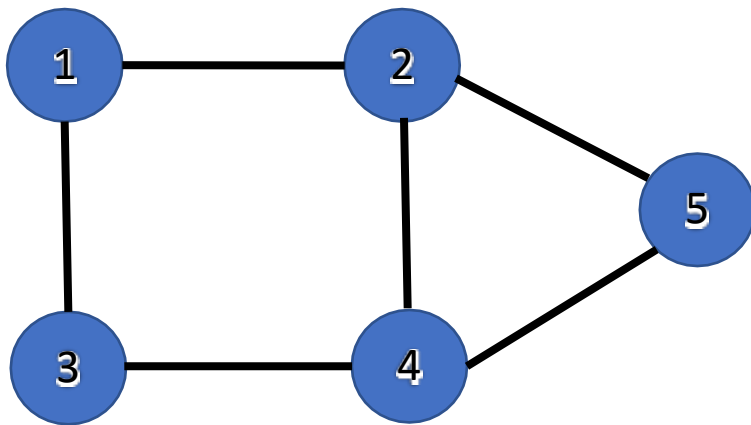
Why Graph Representation?



- How do we store mathematical structure of a graph in the computer memory?
- What types of data structures are used to represent the graph?
- Information Required to represent the graph
 - set of vertices of the graph
 - for each vertex neighbours of the vertex(edge information)

- Depending on the density of edges, ease of use and types of operation performed , Graphs can be represented by
 - ✓ Adjacency Matrix
 - Two Dimensional Array
 - ✓ Adjacency List
 - Linked List

- Adjacency matrix = $n \times n$ matrix M , graph of n vertices/nodes
- $M[i][j] = 1$ if (i, j) is an edge
- $M[i][j] = 0$, no edge between the pair of vertices i and j



Undirected Graph

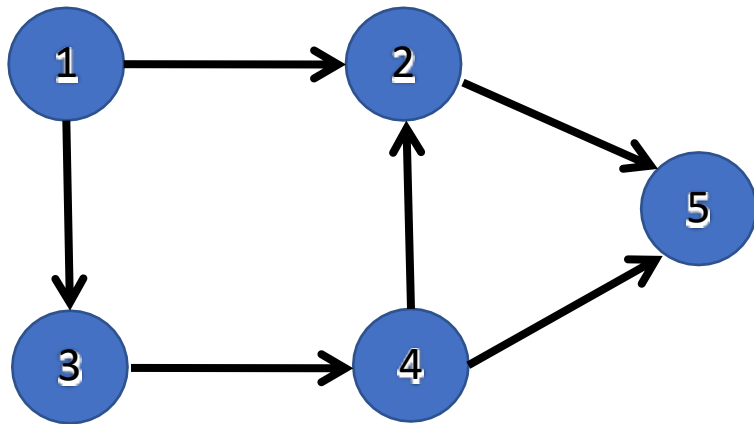
	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	1
3	1	0	0	1	0
4	0	1	1	0	1
5	0	1	0	1	0

Main diagonal

Note:

- For undirected graph, M is symmetric i.e $M[i][j] = M[j][i]$
- Assume no edge from node to itself. So diagonal elements has value 0

- In directed graph edge is directed
- In directed graph $\text{edge}(i,j)$ is not equal to $\text{edge}(j,i)$
- Adjacency matrix is asymmetric

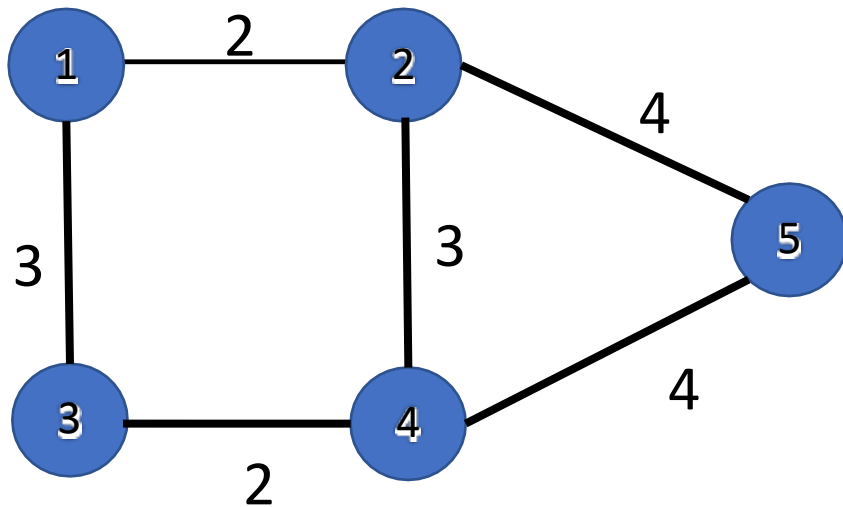


Directed Graph

	1	2	3	4	5
1	0	1	1	0	0
2	0	0	0	0	1
3	0	0	0	1	0
4	0	1	0	0	1
5	0	0	0	0	0

Main diagonal

- In the weighted graph distance or cost between the nodes are represented on the edge
- cost/distance value specified on the edge between adjacent nodes are stored in the adjacency matrix

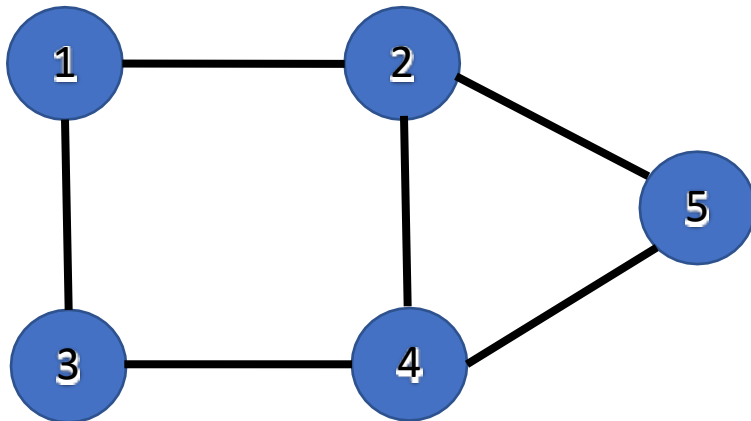


Weighted Graph

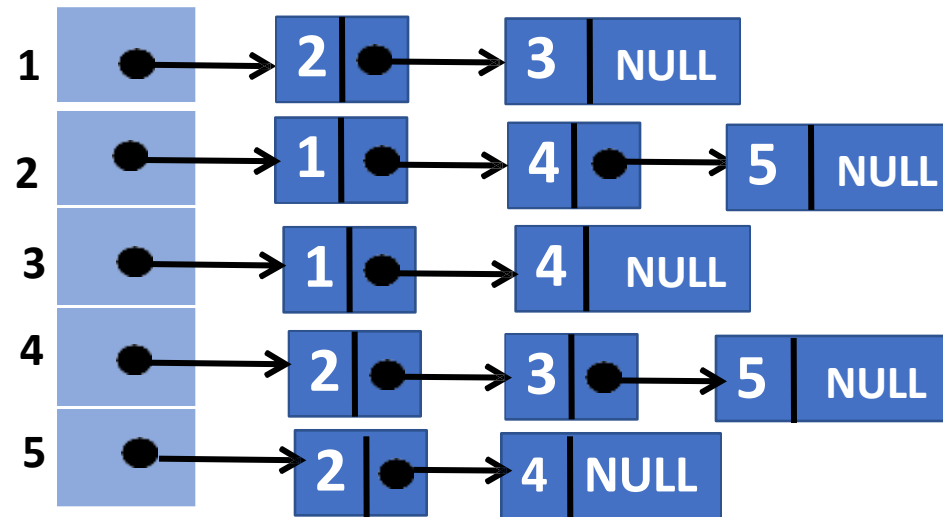
	1	2	3	4	5
1	0	2	3	0	0
2	2	0	0	3	4
3	3	0	0	2	0
4	0	3	2	0	4
5	0	4	0	4	0

- Number of nodes in the graph needs to be known in prior
- In case graph needs to be updated dynamically as the program proceeds new matrix must be created for each addition or deletion
- To detect presence of edge between pair of nodes takes constant time $O(1)$ but it takes $O(v^2)$ to visit all the neighbouring nodes of each node.
- Adjacency matrix becomes sparse in case graph has very few edges.
- space complexity is $O(v^2)$. v^2 locations are required for graph with v nodes

- Each node maintains
 - linked list of its neighbours

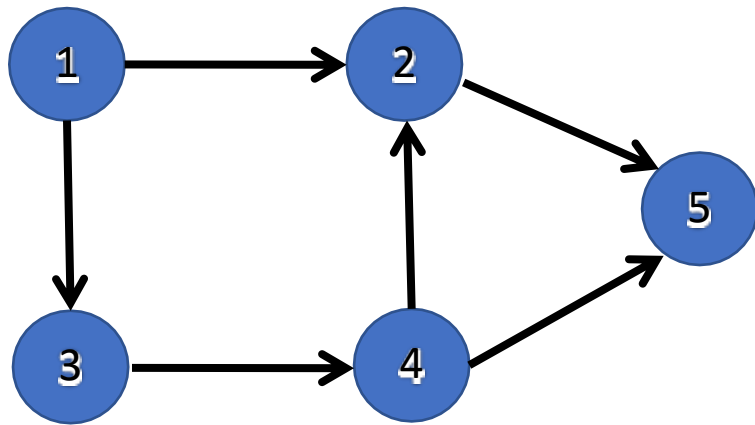


Node 2 → Node 1
Node 2 → Node 4
Node 2 → Node 5



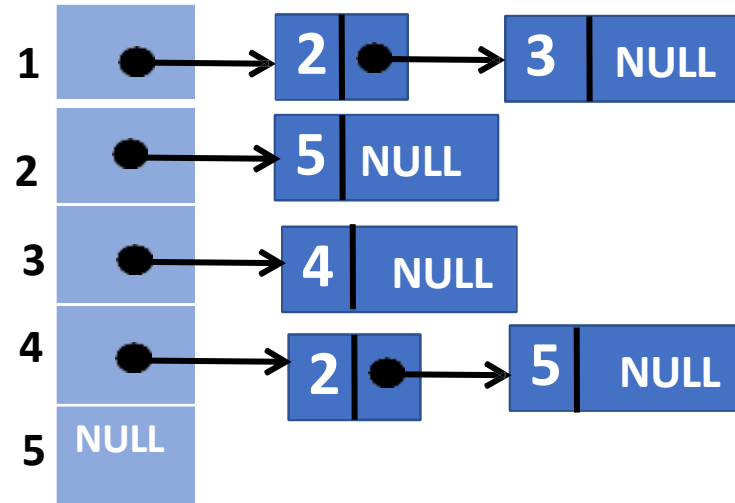
DATA STRUCTURES AND ITS APPLICATIONS

Adjacency list representation – Directed graph



Directed Graph

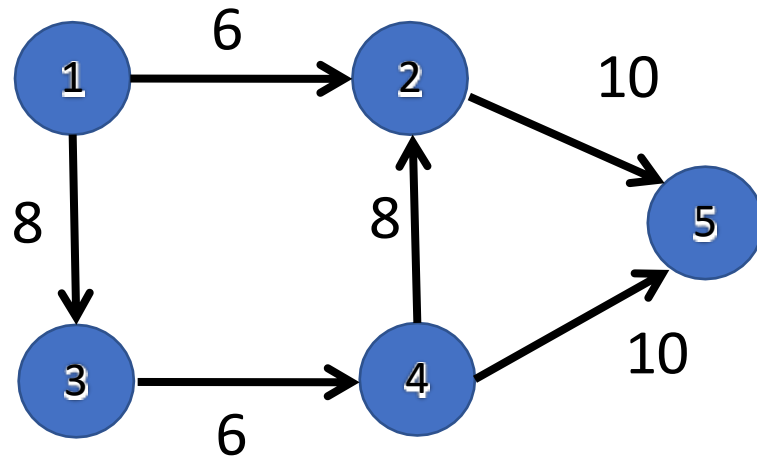
Node 2 → Node 5



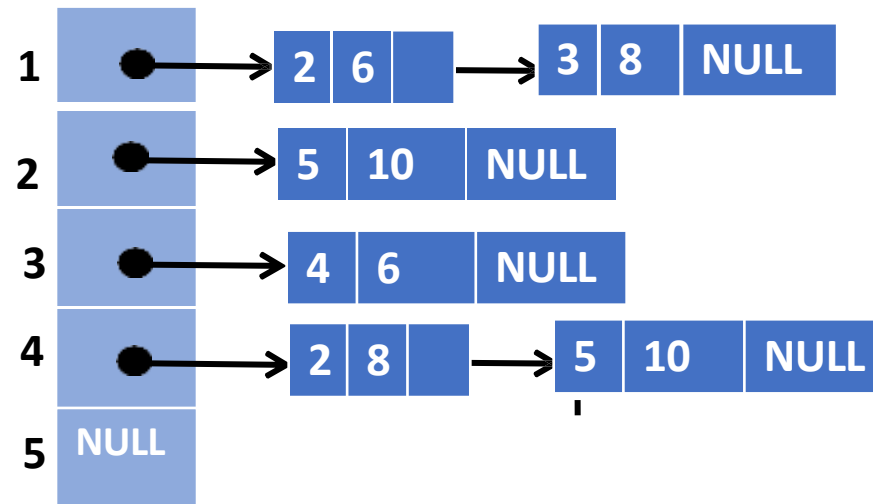
DATA STRUCTURES AND ITS APPLICATIONS

Adjacency matrix representation – weighted graph

- In weighted graph the distance or cost between the nodes are represented on the edge
- Along with the information of adjacent node , cost/distance value is stored in each node of the linked list.



Weighted Graph



- Space Complexity of Adjacency list is $O(V+E)$, because it stores the information of edges that actually exists in the graph
- In case of low density edges, the adjacency matrix becomes sparse using adjacency list is better for representation
- To detect the presence of edge between two nodes, we need to traverse the linked list of the node.

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of graph using adjacency matrix



Data structure to represent the adjacency matrix:

- To represent only the existence of edge between nodes

```
#define maxnodes 50  
adj[maxnodes][maxnodes];
```

```
Typedef Boolean AdjacencyMatrix[maxnodes][maxnodes]
```

```
Typedef struct graph
```

```
{
```

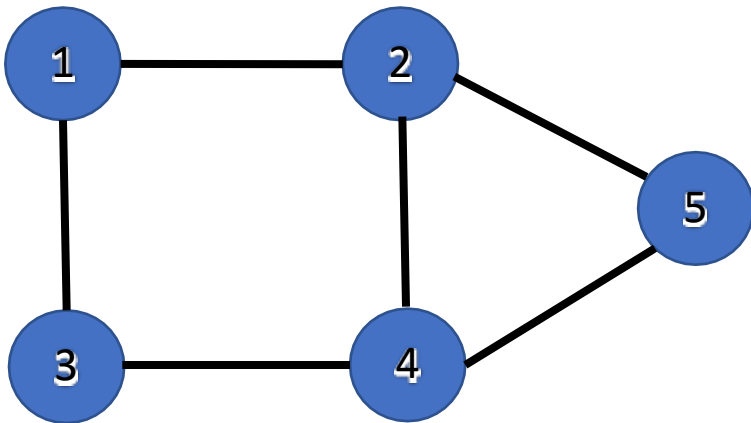
```
    int n;    /* number of vertices in the graph */
```

```
    AdjacencyMatrix adj;
```

```
}Graph;
```

To check the presence of edge between pair of nodes:

```
if ( adj[i][j] == 1)  */ i and j represents the vertex in a graph */  
    then "edge exists between the pair of nodes"  
else  
    " there is no edge between the pair of nodes"
```



	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	1
3	1	0	0	1	0
4	0	1	1	0	1
5	0	1	0	1	0

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of graph using adjacency matrix:



To add an edge from node1 to node2:

```
void join(int adj[][MAXNODES],int node1,int node2)
{
    adj[node1][node2]=TRUE;
}
```

To delete an edge from node1 to node2 if it exists:

```
void remv(int adj[][MAXNODES],int node1,int node2)
{
    adj[node1][node2]=FALSE;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of adjacency matrix:



To check whether arc exists between node1 and node2:

```
int adjacent(int adj[][MAX],int node1,int node2)
{
    return((if(adj[node1][node2])==TRUE)?TRUE:FALSE);
}
```

```
#define MAXNODES 50
struct node
{
    //information associated with each node
};
struct arc
{
    int adj;// information associated with each edge
};
struct graph
{
    struct node nodes[MAXNODES];
    struct arc arcs[MAXNODES][MAXNODES];
}
struct graph g;
```

- Weighted graph with fixed number of nodes is declared as

```
struct arc{
    int adj;
    int weight;
};
struct arc g[maxnodes][maxnodes];
```

To add an edge from node1 to node2:

```
void joinwt(struct arc g[][maxnodes],int node1,int node2, int wt)
{
    g[node1][node2].adj = TRUE;
    g[node1][node2].weight= wt;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of adjacency List Representation:

struct node

{

int node;

struct node *nextedge;

};

NODE info

Nextedge

To create a node list:

struct node *nodelist;

for(i=0; i < no_of_nodes; i++)

nodelist[i]= NULL;

struct node

{

int node;

int weight;

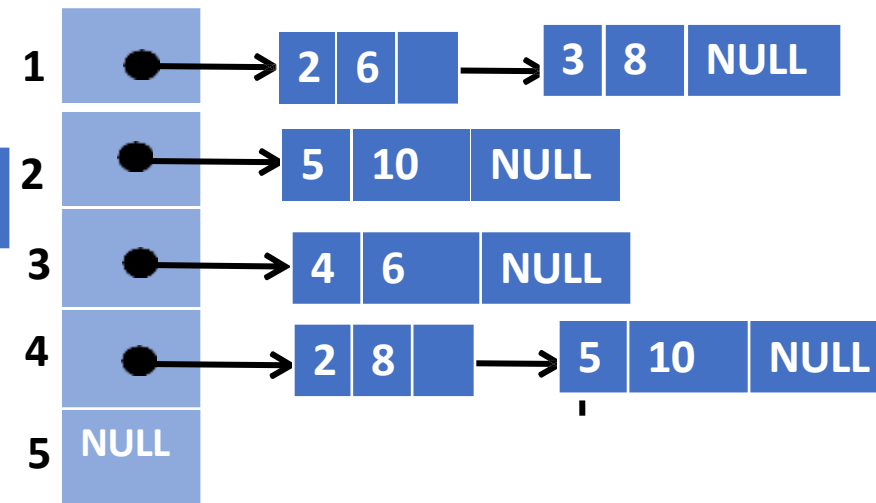
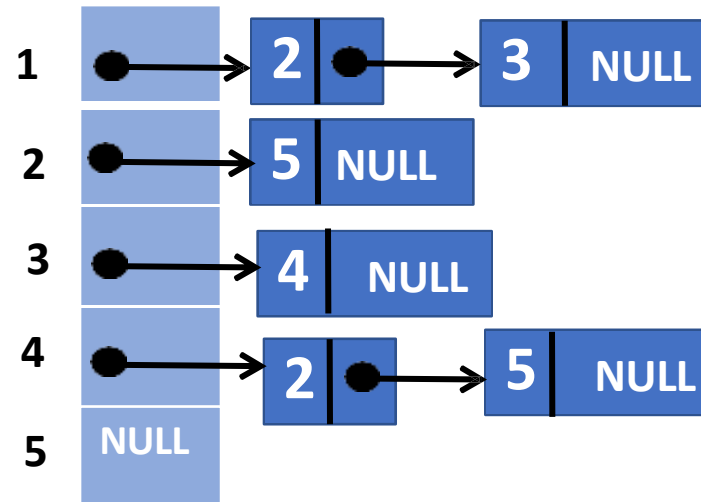
struct node *nextedge;

};

nodeinfo

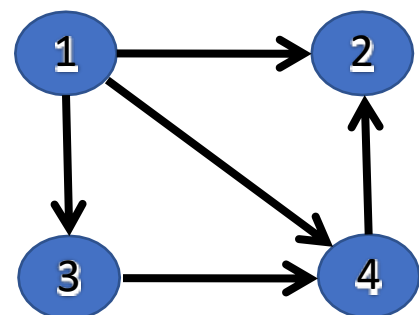
weight

nextedge



DATA STRUCTURES AND ITS APPLICATIONS

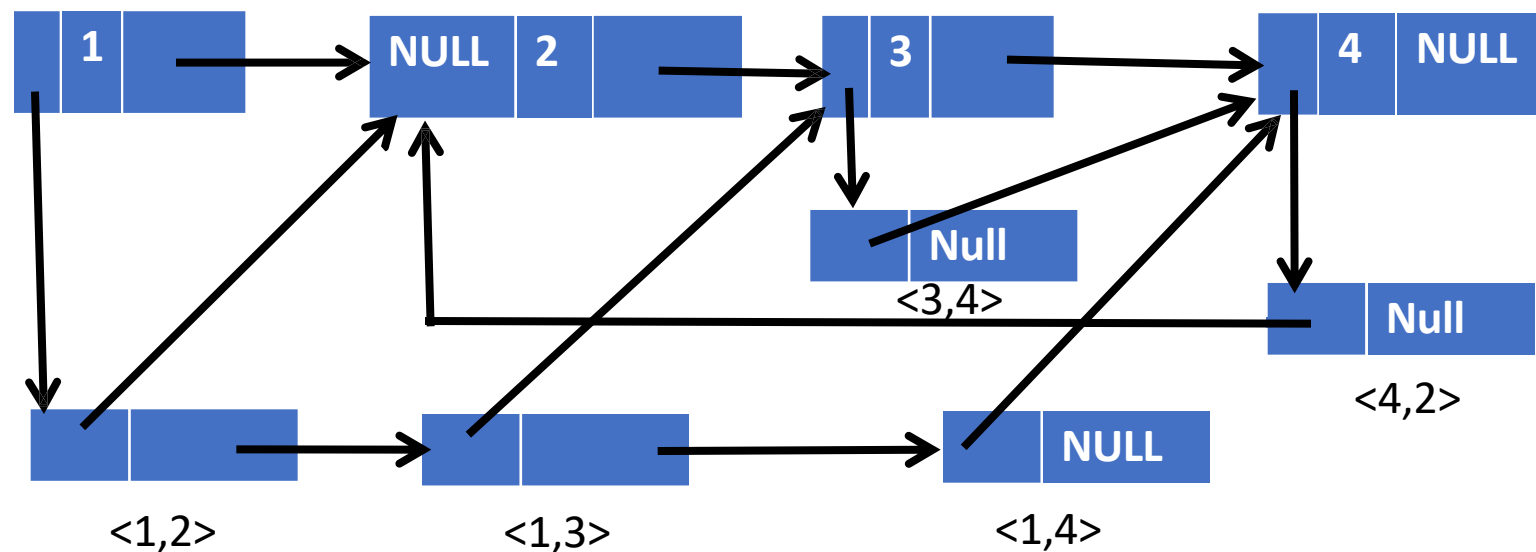
Multilinked structure representation



Sample header node representing the graph node



A sample list node representing edge



Dynamic Implementation:

```
struct nodetype
{
    int info;
    struct nodetype *ptr;
    struct node type *next;
};
struct nodetype *nodeptr;
```

edgeptr	Info	nextnode
---------	------	----------

Sample header node representing the graph node

nodeptr	nextedge
---------	----------

A sample list node representing edge

- Note that header node and list node have different formats and must be represented by different structures.
- In case of weighted graph in which each list node contains info field to store the weight of the edge the same dynamic implementation could be used for both types of nodes.



THANK YOU

Sandesh B. J

Department of Computer Science & Engineering

sandesh_bj@pes.edu

+91 80 6618 6623



DATA STRUCTURES AND ITS APPLICATIONS

Balanced Trees

Sandesh B. J

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

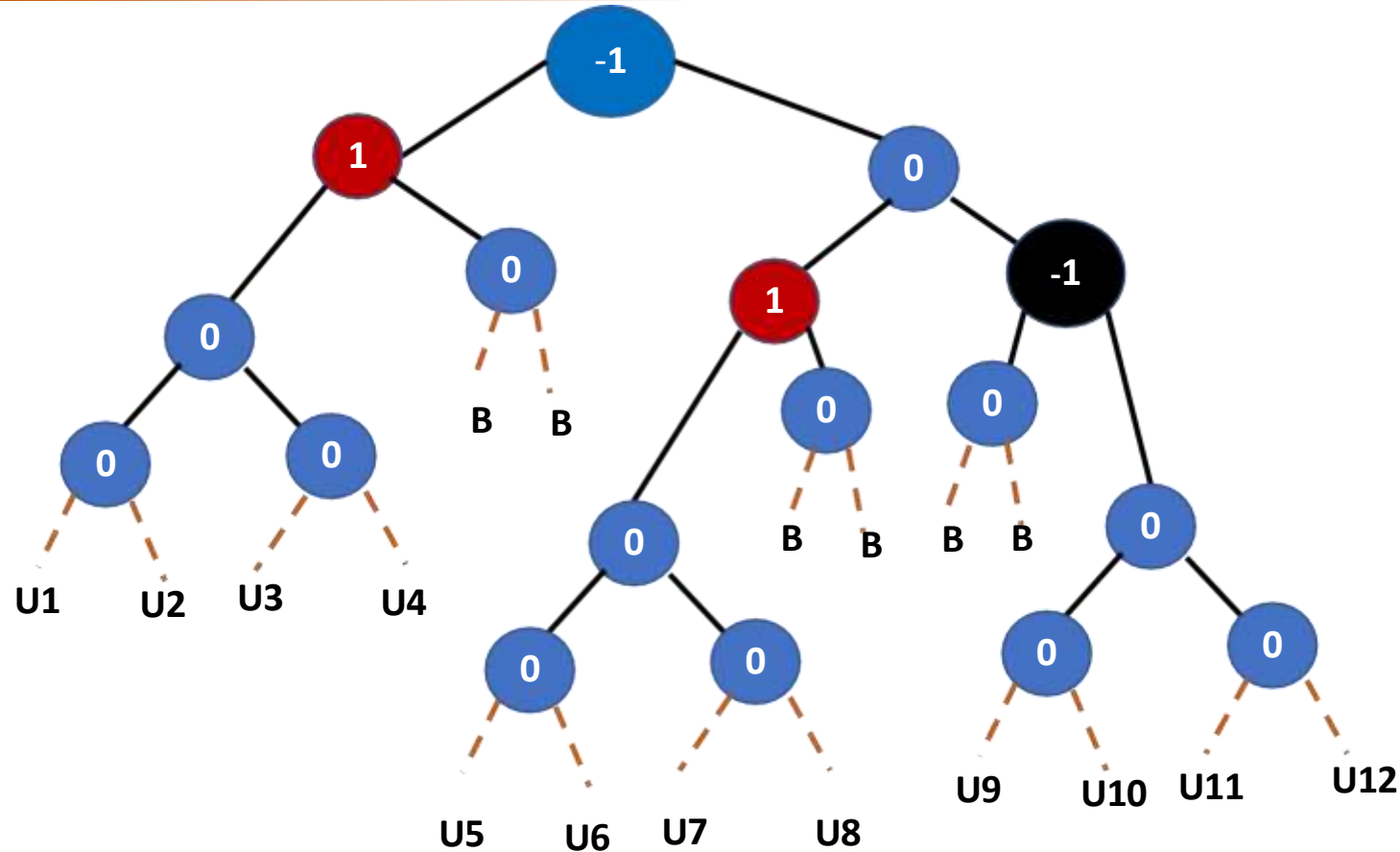
Balanced Trees

Sandesh B. J

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

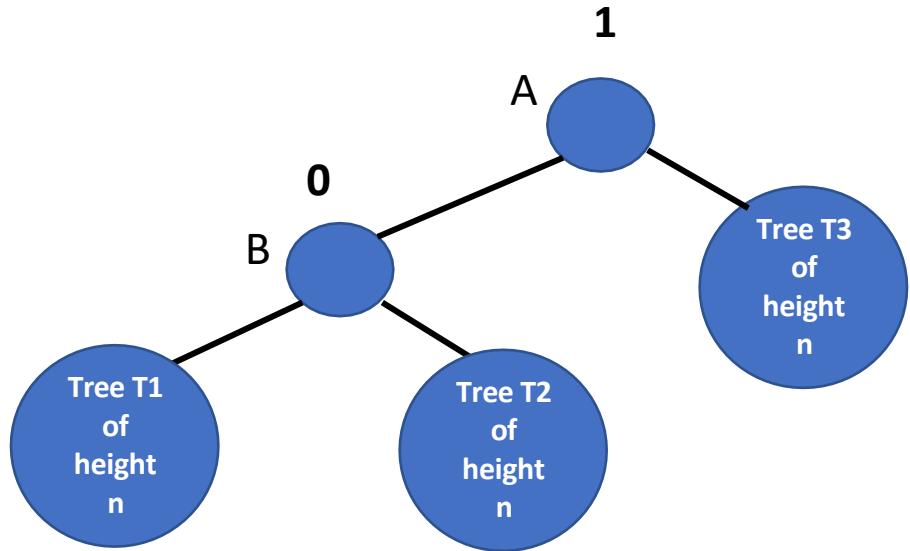
Possible insertion into AVL tree



- Unbalanced insertions are indicated by **U**
- Balanced insertions are indicated by **B**

Courtesy: "Data Structures using c and c++ " By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

AVL tree Insertions

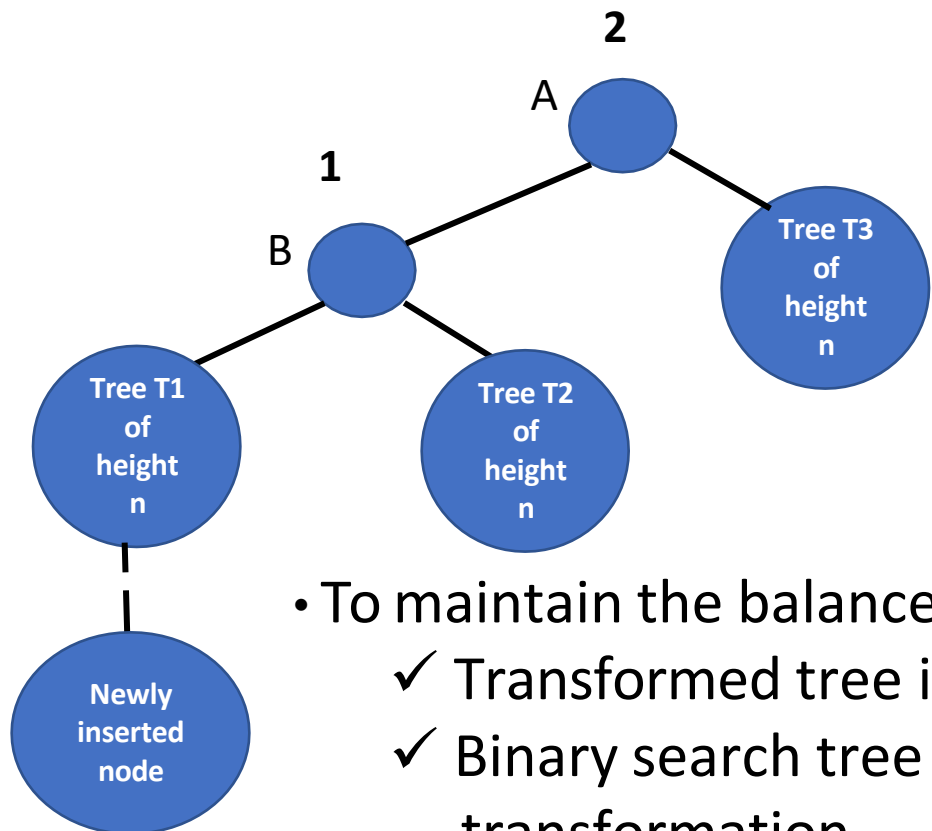


$$\text{Balance factor}(A) = (n+1) - n = 1$$
$$\text{Balance factor}(B) = n - n = 0$$

- Let us consider A is the youngest ancestor which becomes unbalanced
- Balance factor of A should be 1 before insertion
- A should have a left child B with the balance factor of 0

Unbalanced Tree after inserting a node to left subtree

- Newly inserted node is left descendent of node A
- Changing the balance **B to 1** and **A to 2**
- A is the youngest ancestor of the new node to become unbalanced

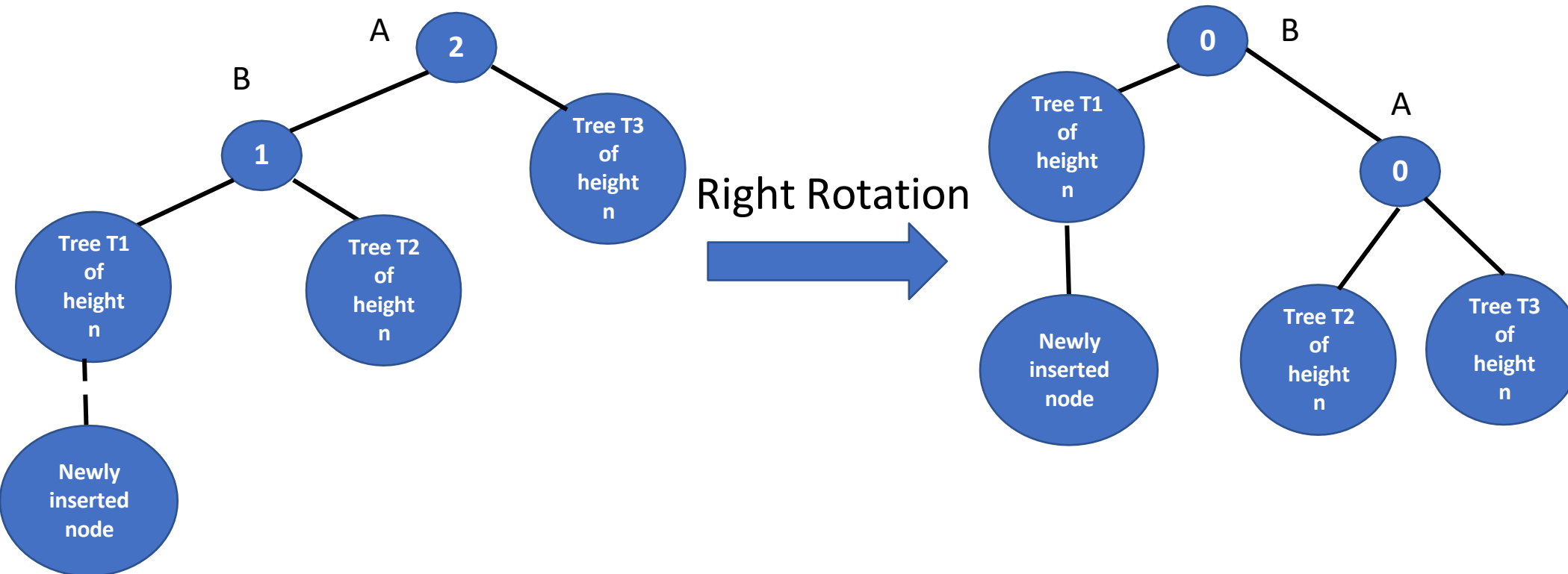


- To maintain the balance : Tree needs to be transformed
 - ✓ Transformed tree is balanced
 - ✓ Binary search tree property is maintained after transformation

Courtesy: "Data Structures using c and c++ " By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

Transformed Balanced Tree after Rotations

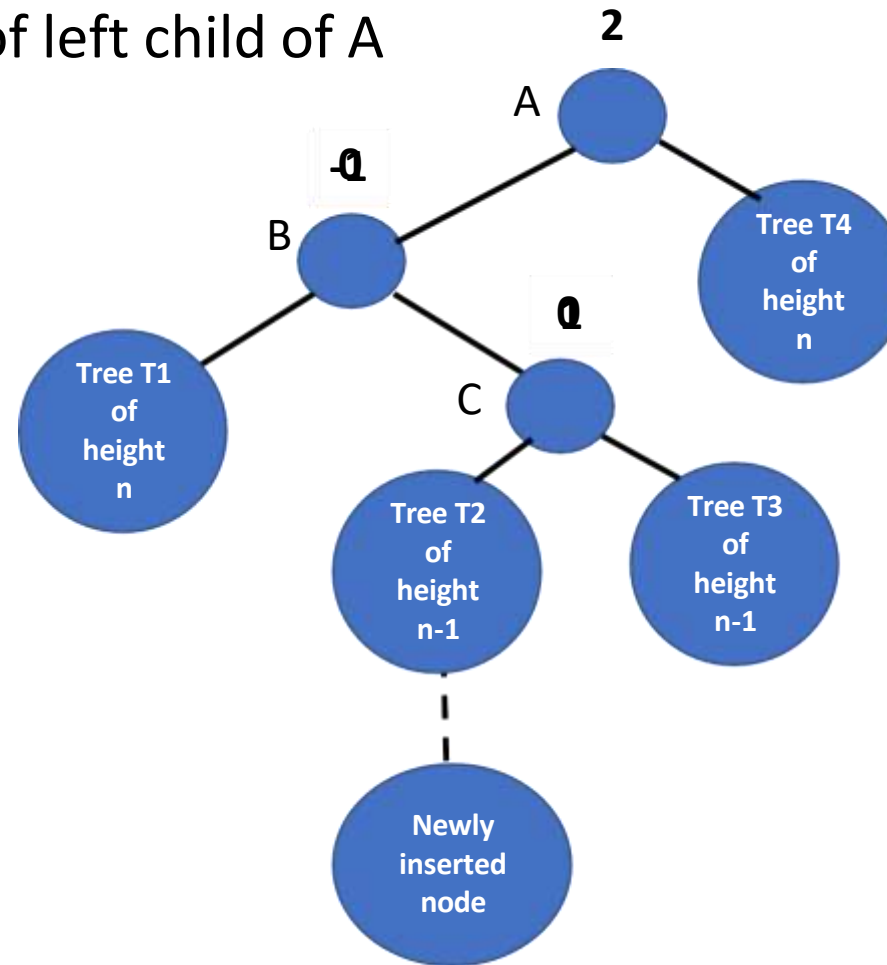
- To maintain a balance we need to rotate sub tree B rooted at node A



Unbalanced Tree after inserting a node to right subtree

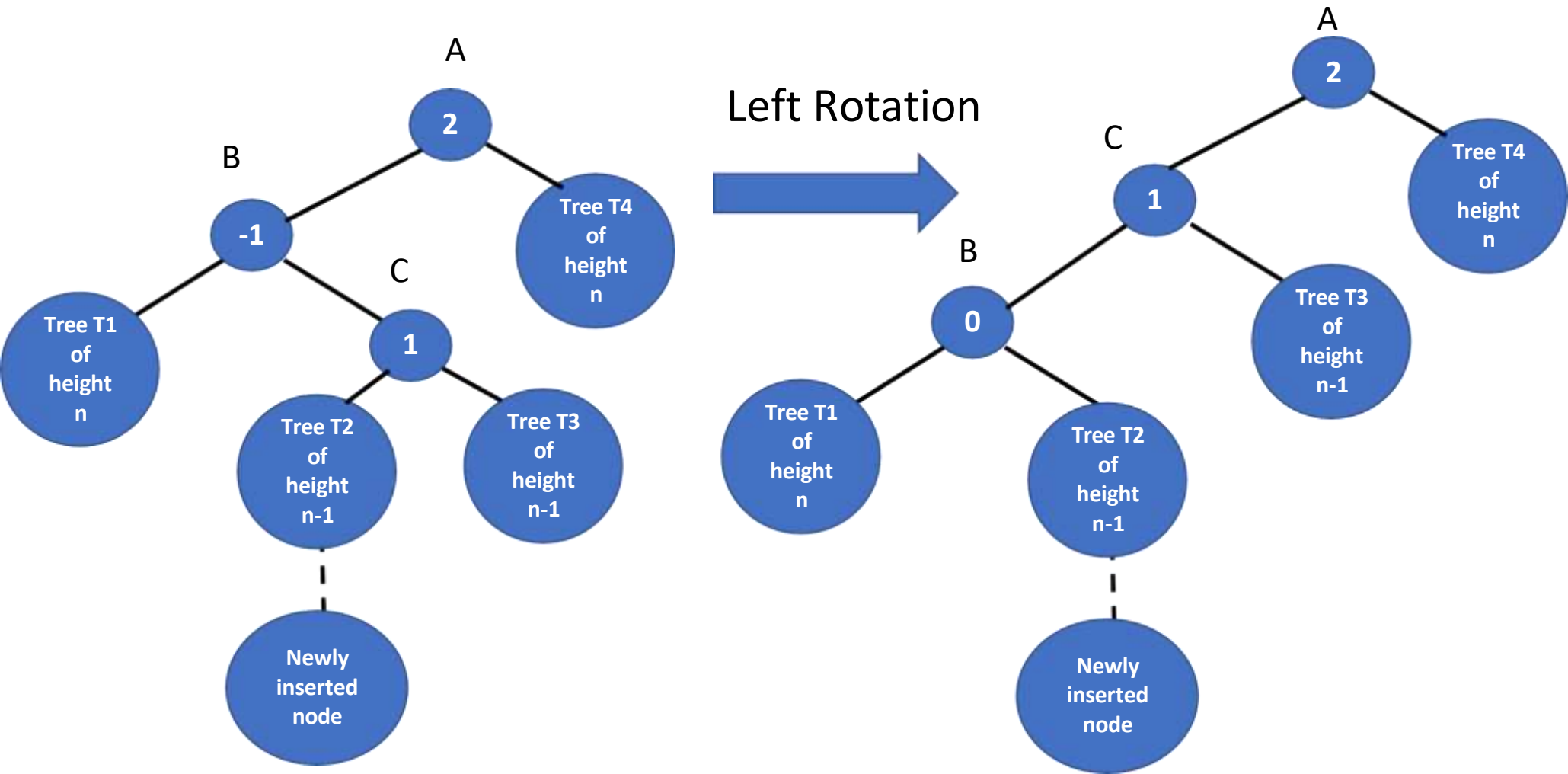
- Newly inserted node is left descendent of the node A
- New node is inserted into right subtree of left child of A

Balance factor(C) = $n - (n - 1) = 1$
Balance factor(B) = $n - (n + 1) = -1$
Balance factor(A) = $n + 2 - n = 2$



DATA STRUCTURES AND ITS APPLICATIONS

Transformed Balanced tree after Rotations

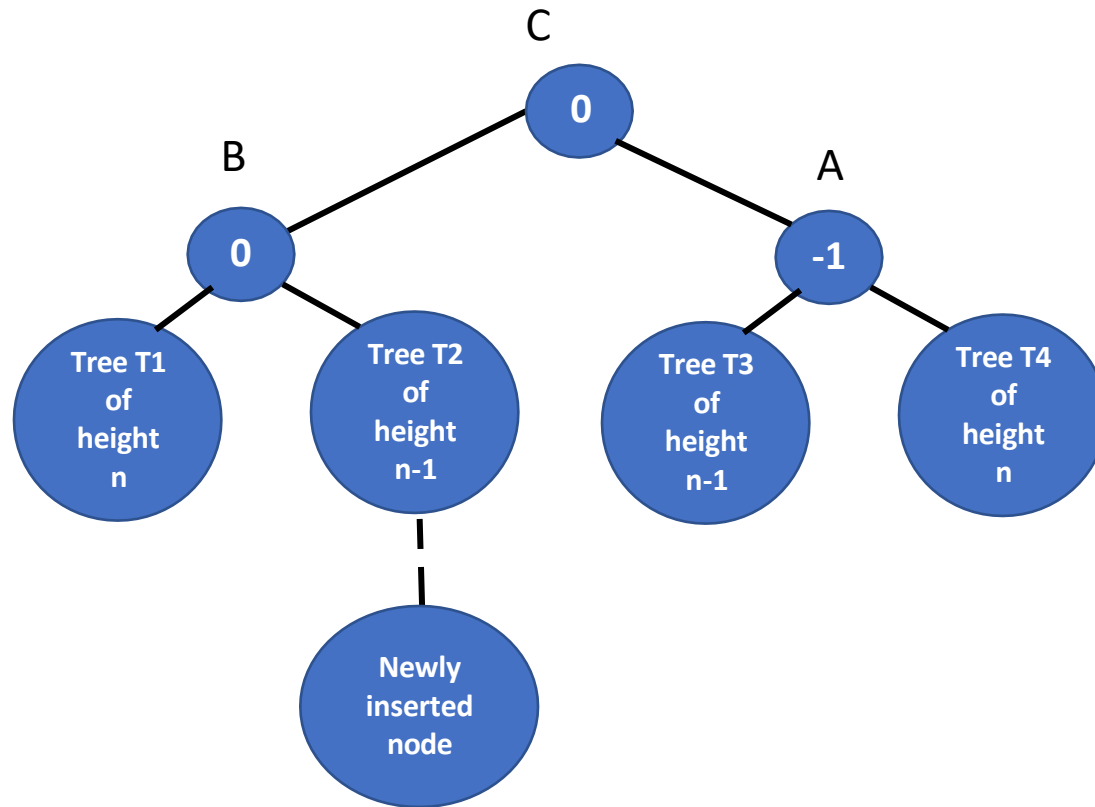


Courtesy: “Data Structures using c and c++ ” By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

DATA STRUCTURES AND ITS APPLICATIONS

Transformed Balanced tree after Rotations

Right Rotation



Courtesy: "Data Structures using c and c++ " By Y Langsam, M. J. Augenstein and Aron M. Tenenbaum

- Insertion in AVL tree is performed using standard BST Insertion
- If tree becomes unbalanced, we rebalance the tree using left or right rotation
- If node X is inserted into balanced BST
- we need to find the youngest ancestor which becomes unbalanced

Four cases:

- IF(Balance factor of node) == 2 – unbalanced node(U)
 - ✓ case-1 : Left-Left case
 - IF((newly inserted key) < (key in the left subtree' root))
 - ✓ case-2: Left-Right case
 - IF((newly inserted key) > (key in the left subtree' root))

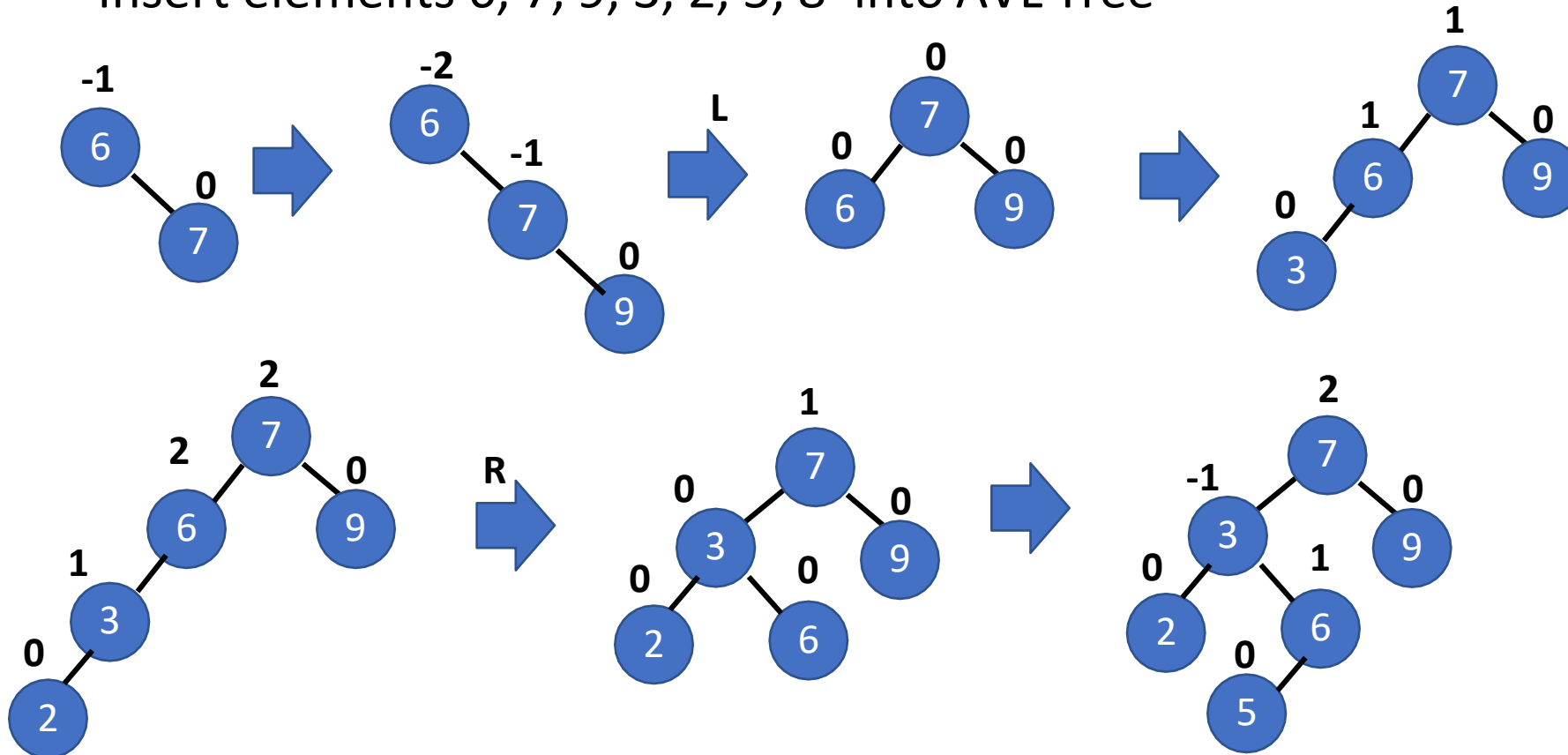
Four cases:

- IF((Balance factor of node)) == -2 – unbalanced node(U)
 - ✓ Case 3: Right-Right case
 - IF((newly inserted key) > (key in the right subtree' root))
 - ✓ Case 4: Right-Left case
 - IF((newly inserted key) < (key in the right subtree' root))

DATA STRUCTURES AND ITS APPLICATIONS

Examples – AVL Tree Insertions

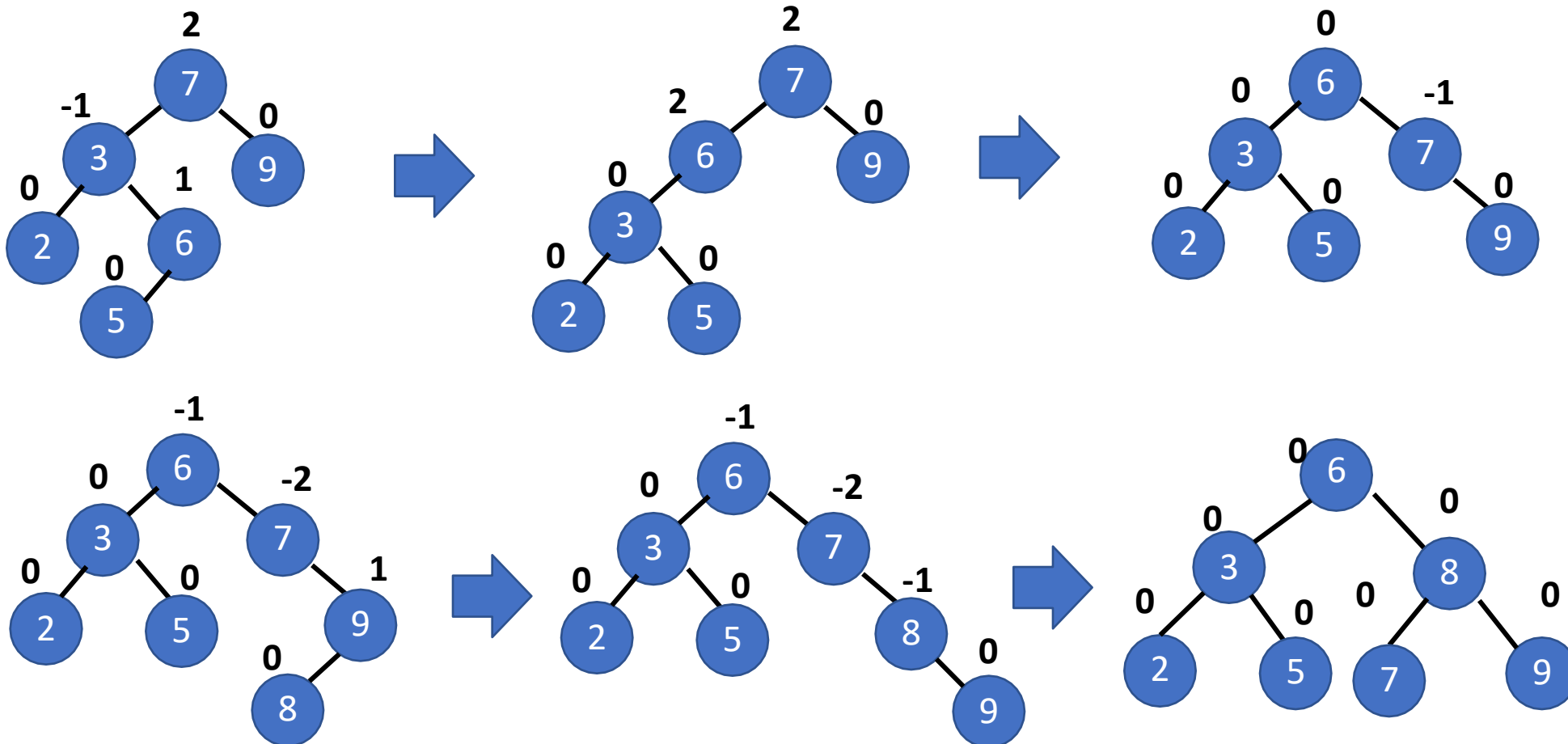
- Insert elements 6, 7, 9, 3, 2, 5, 8 into AVL Tree



DATA STRUCTURES AND ITS APPLICATIONS

Example – AVL Tree Insertions

- Insert elements 6, 7, 9, 3, 2, 5, 8 into AVL Tree



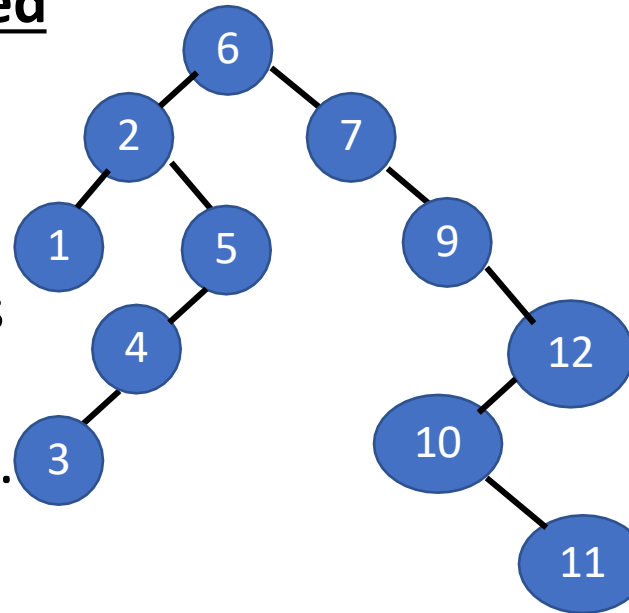
- Deletion in AVL tree is performed using standard BST Deletion
- If tree becomes unbalanced, we rebalance the tree using left or right rotation

BST Deletion: 3- case: Node to be deleted

Case 1: Does not have any children

Case 2: has either left or right subtrees

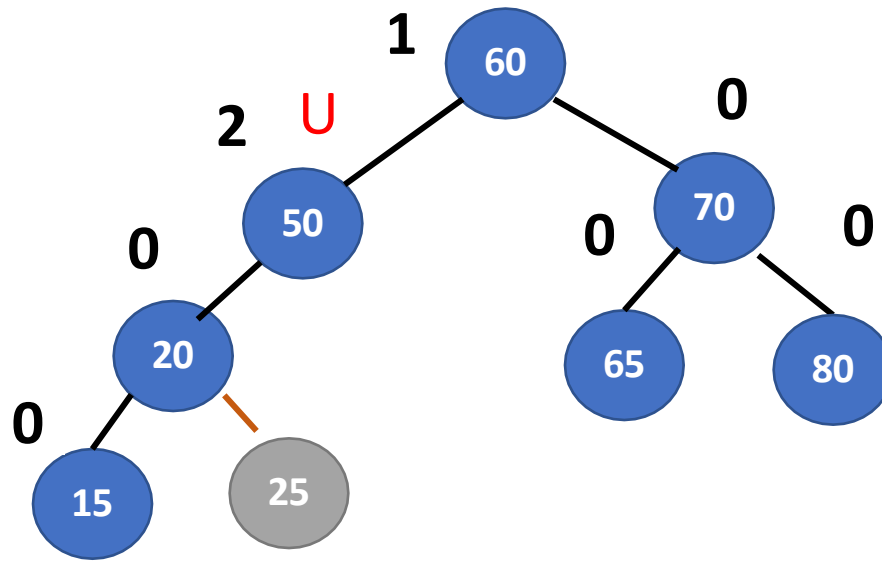
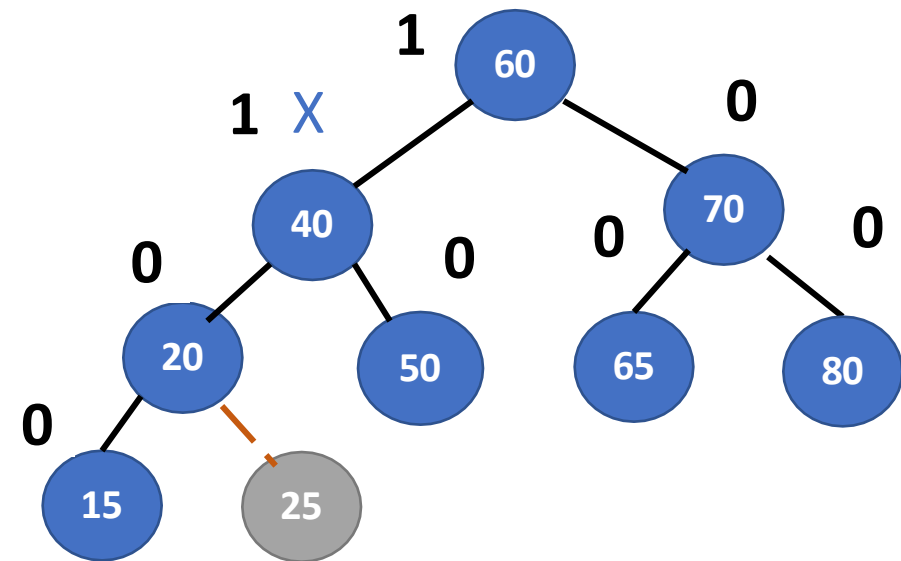
Case 3: has both left and right subtrees.



- If node X is deleted from the BST
- we need to find the youngest ancestor which becomes unbalanced

Four cases: Case-1

- IF((Balance factor of a node) == **2**) - **unbalanced node(U)**
 - ✓ Left-Left case:
 - IF(Balance factor of left subtree's root) $\geq$ **0**

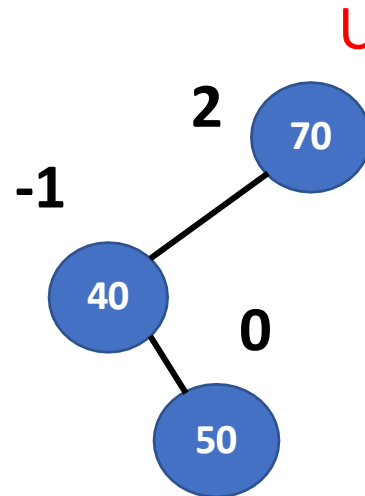
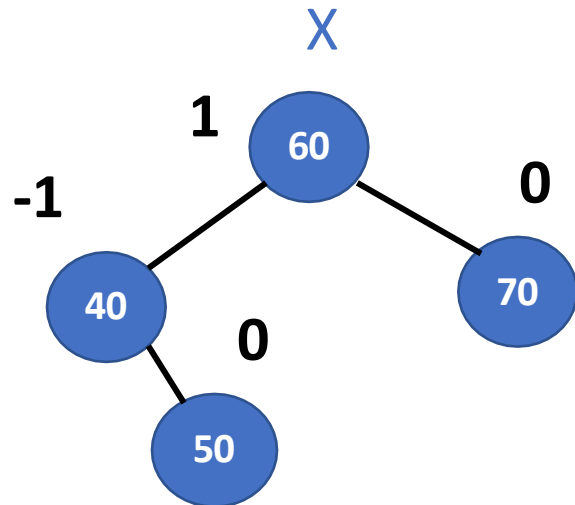


Case-2:

- IF (Balance factor of a node == 2) – unbalanced node(U)

✓ Left-Right case

- IF (Balance factor of left subtree's root) < 0



DATA STRUCTURES AND ITS APPLICATIONS

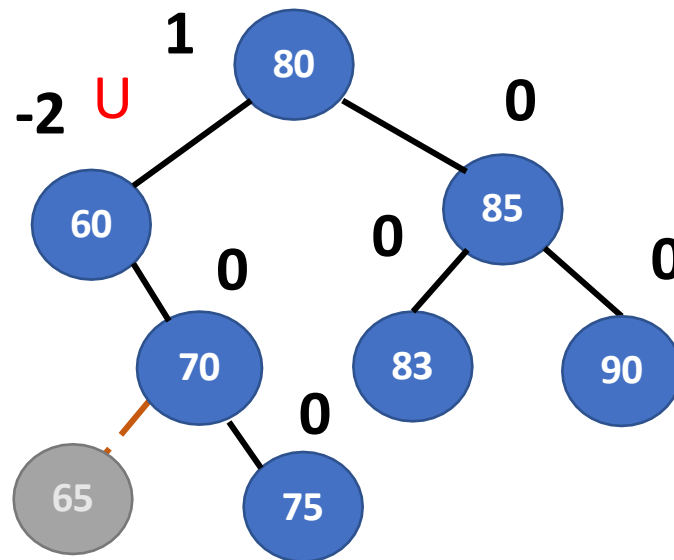
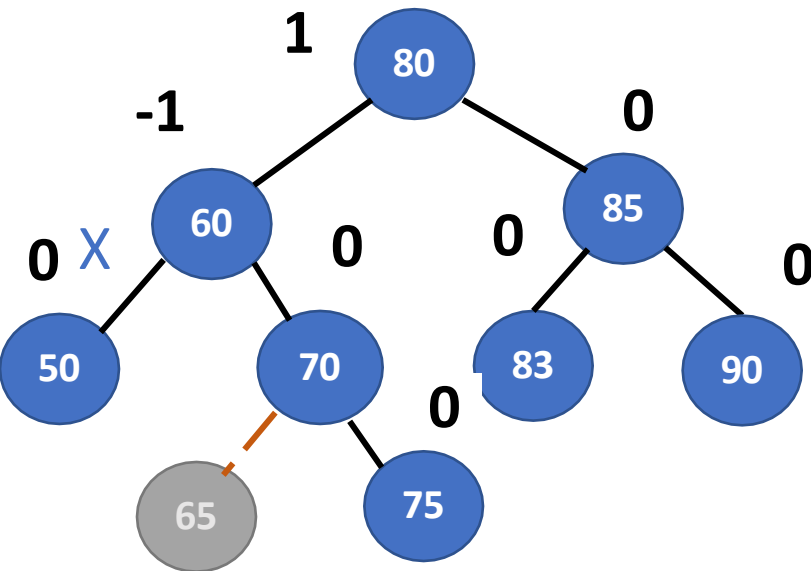
Deletions in AVL tree

Case-3:

- If ((Balance factor of a node) == -2) unbalanced node(U)

✓ Right-Right case

- IF(Balance factor of Right subtree's root) ≤ 0

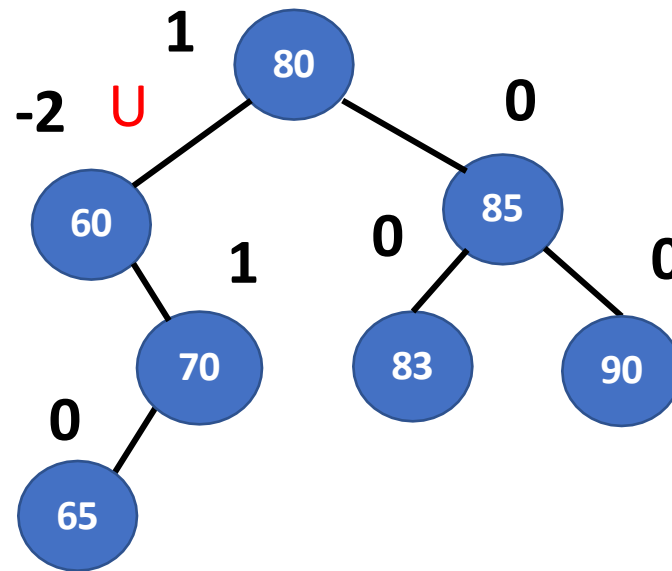
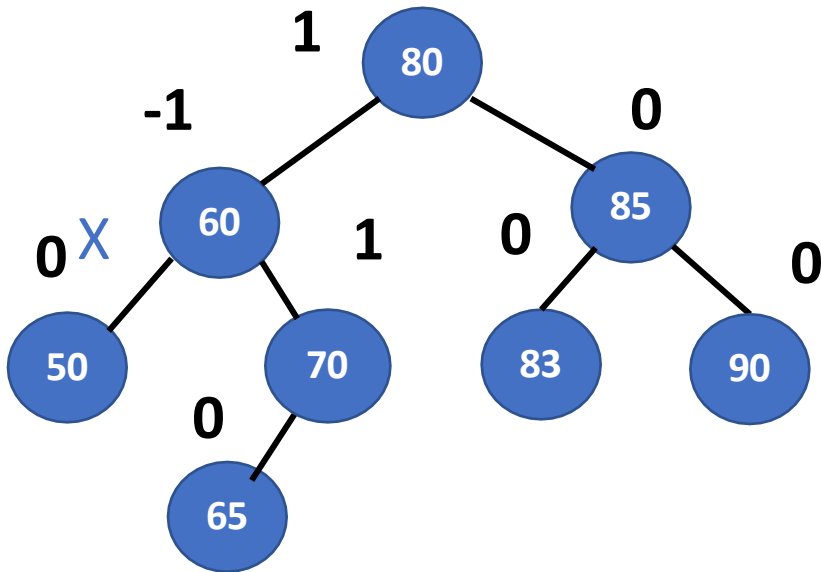


Case-4:

IF((Balance factor of unbalanced node) == **-2**) unbalanced node(U)

✓ Right-Left case

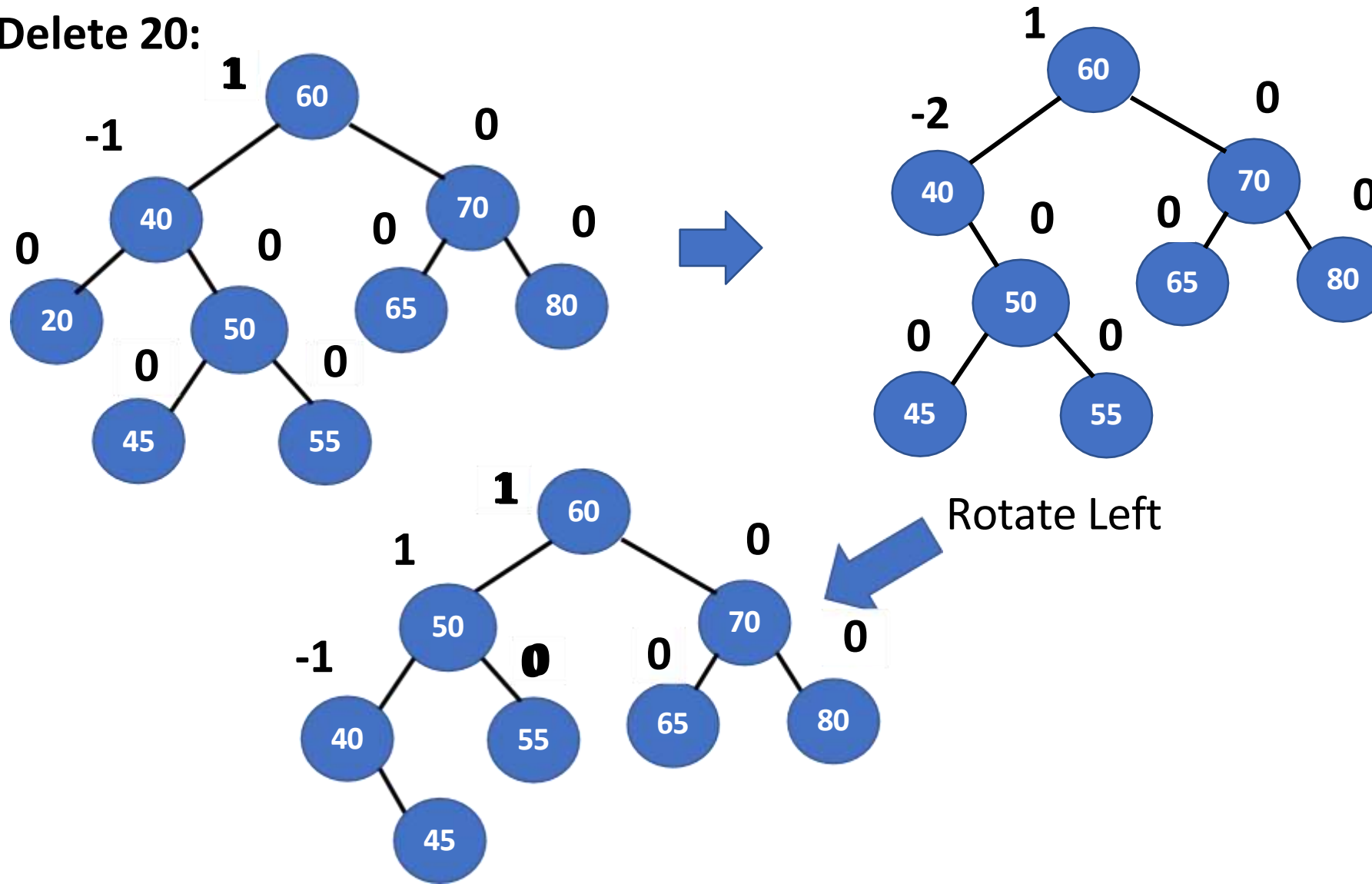
▪ IF(Balance factor of Right sub tree's root) > **0**



DATA STRUCTURES AND ITS APPLICATIONS

Example – Deletions in AVL tree

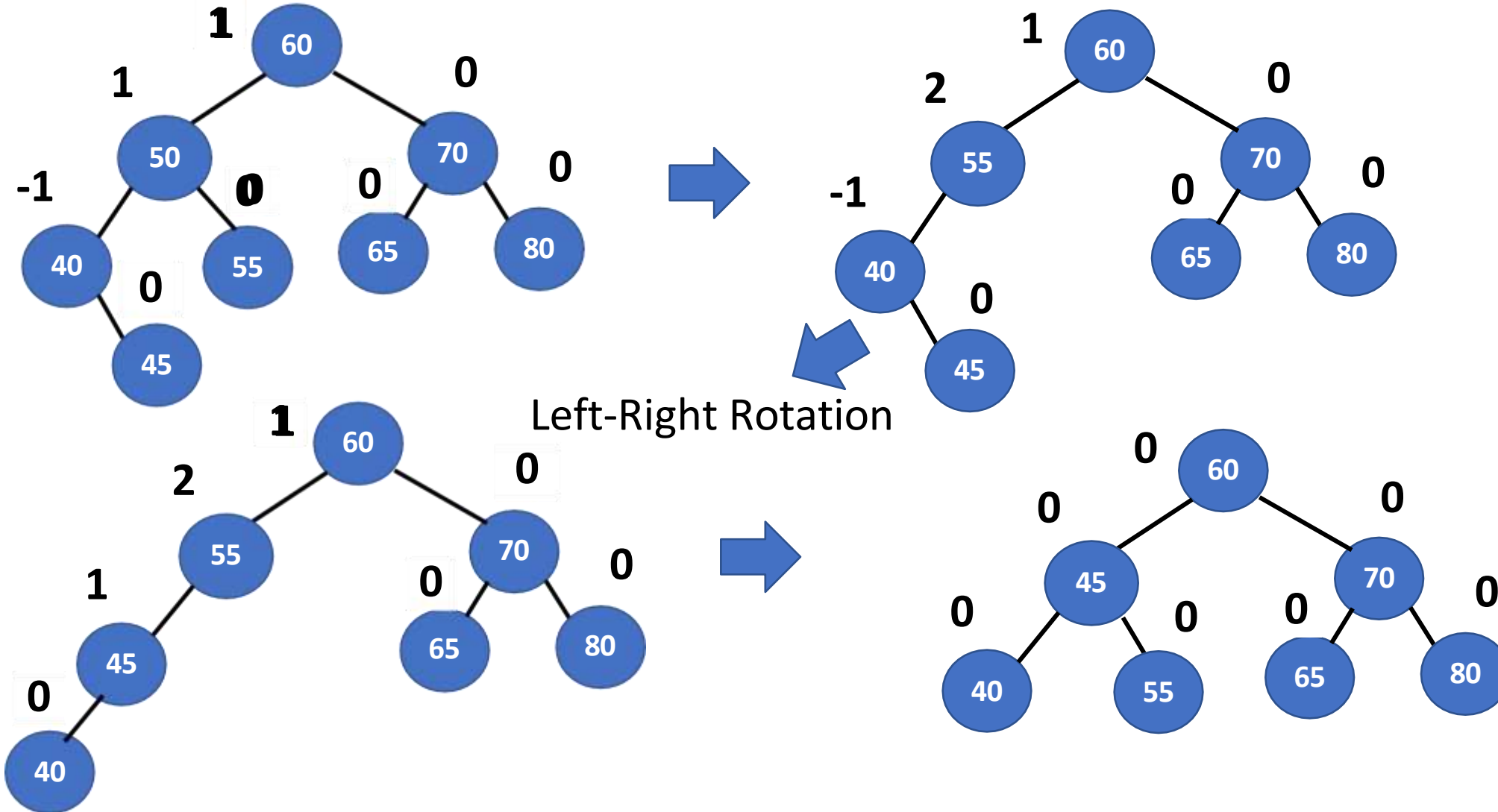
Delete 20:



DATA STRUCTURES AND ITS APPLICATIONS

Example – Deletions in AVL tree

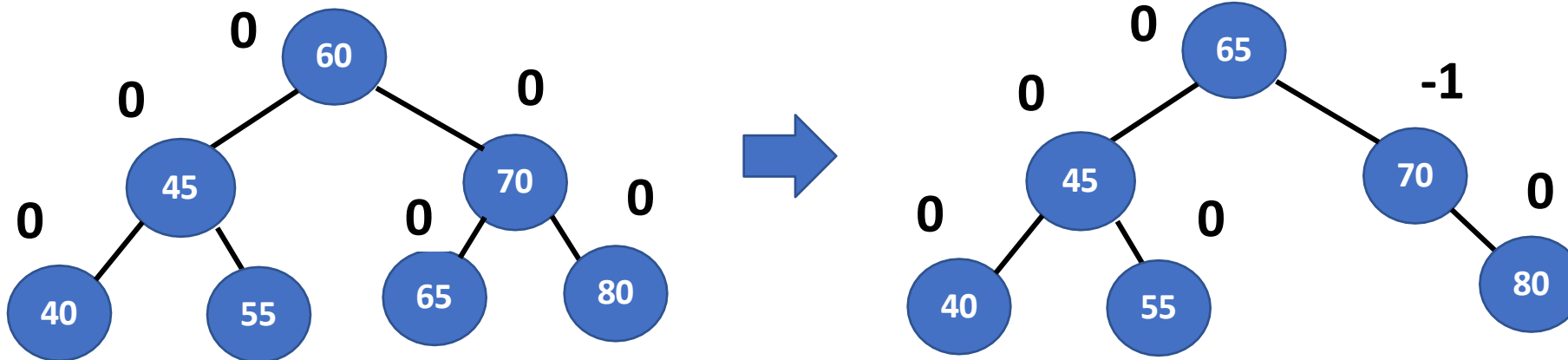
Delete 50:



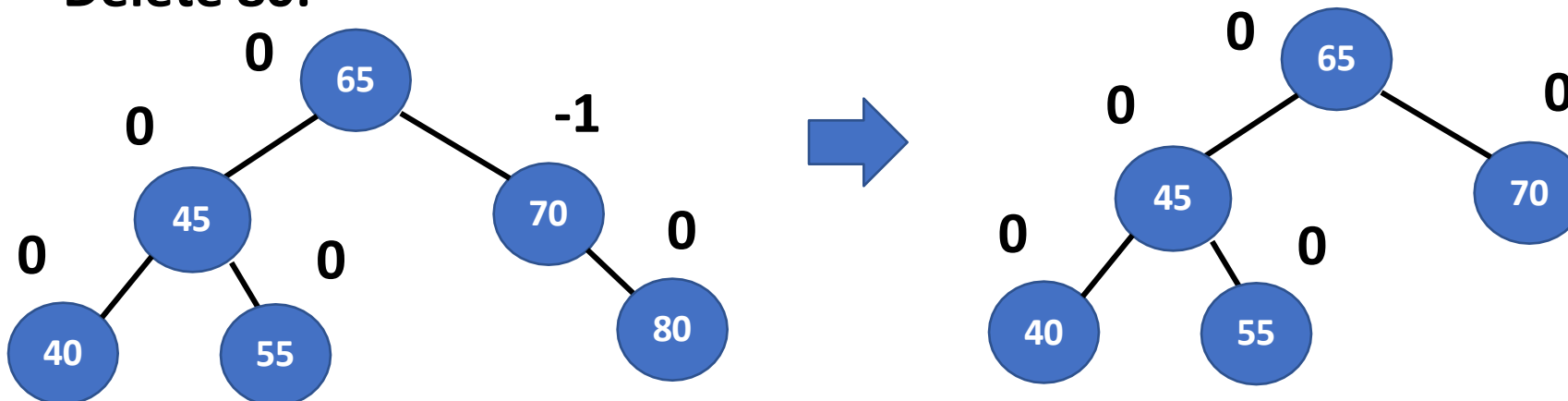
DATA STRUCTURES AND ITS APPLICATIONS

Example – Deletions in AVL tree

Delete 60:



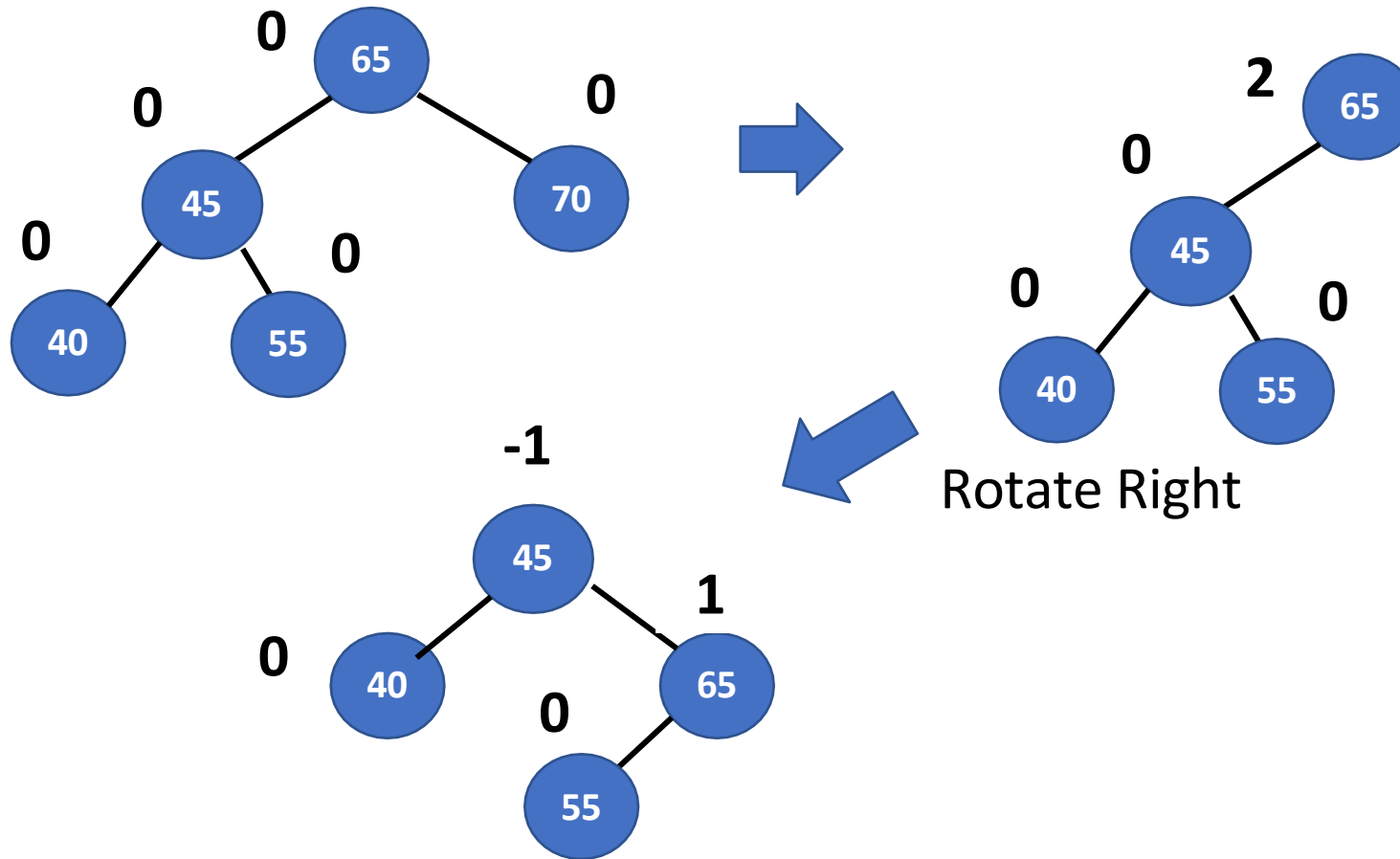
Delete 80:



DATA STRUCTURES AND ITS APPLICATIONS

Example – Deletions in AVL tree

Delete 70:

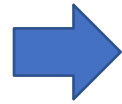
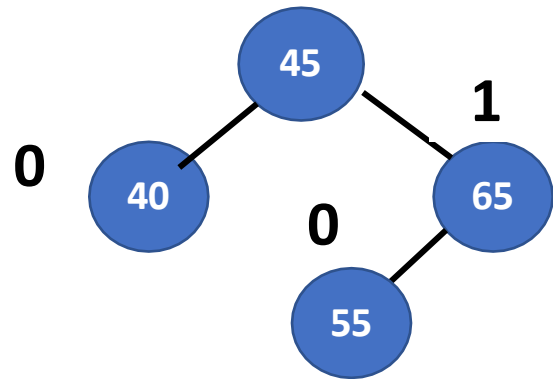


DATA STRUCTURES AND ITS APPLICATIONS

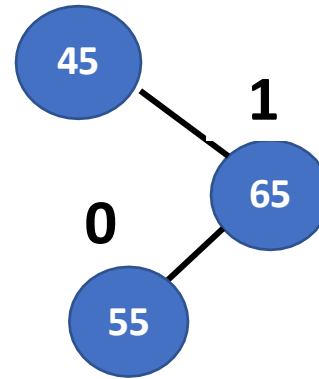
Example – Deletions in AVL tree

Delete 40:

-1

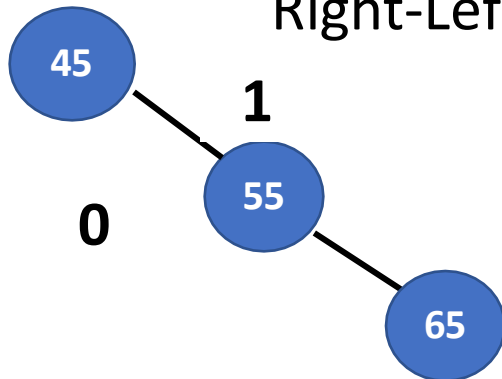


-2

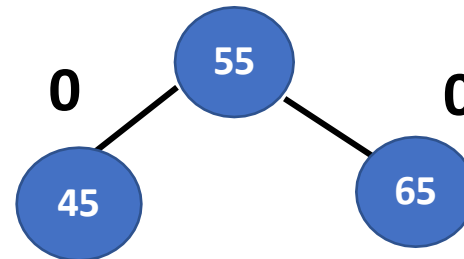


-2

Right-Left Rotation



1





THANK YOU

Sandesh B. J

Department of Computer Science & Engineering

Sandesh_bj@pes.edu

+91 80 6618 6649



DATA STRUCTURES AND ITS APPLICATIONS

UE19CS202

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering



DATA STRUCTURES AND ITS APPLICATIONS

BST Implementation using Dynamic Allocation: Insertion

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree – An Application of Binary Tree



Background

Problem : find a target key in a list of elements

Sequential: Potentially enumerate every key

Ordered List: Searching can be done on $\log n$

Frequent insertions and deletions : Ordered List is much slower

Solution: Binary Trees provide an excellent solution to this by organizing every element in the list as a node in the tree

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree: Definition

A Binary Search Tree is a binary tree which has the following properties:

- all the elements in the left subtree of a node **n** are less than the contents of node **n**
- all the elements in the right subtree of a node **n** are greater than or equal to the contents of node **n**

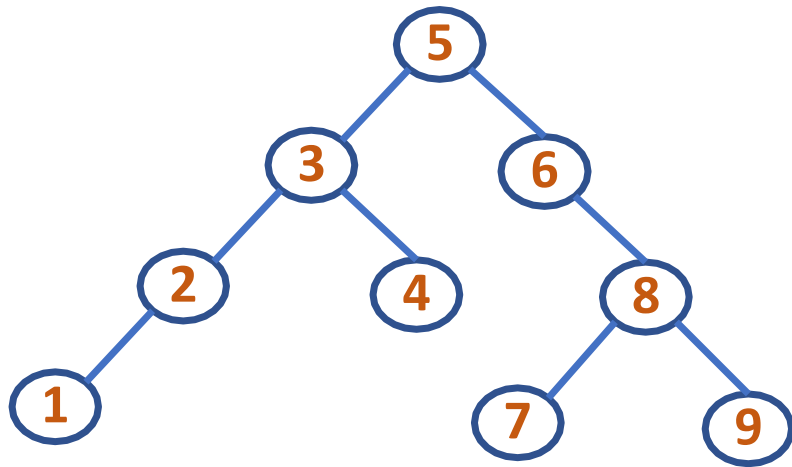


DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree – An Application of Binary Tree



A Binary Search Tree with the nodes inserted in the order:
5, 3, 6, 4, 2, 8, 1, 7, 9



DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation



Linked implementation

Here every node will have its own **info** along with the **links to left child** and **right child**

```
typedef struct tree_linked
{
    int info;
    struct tree_linked *left,*right;
}NODE;
```

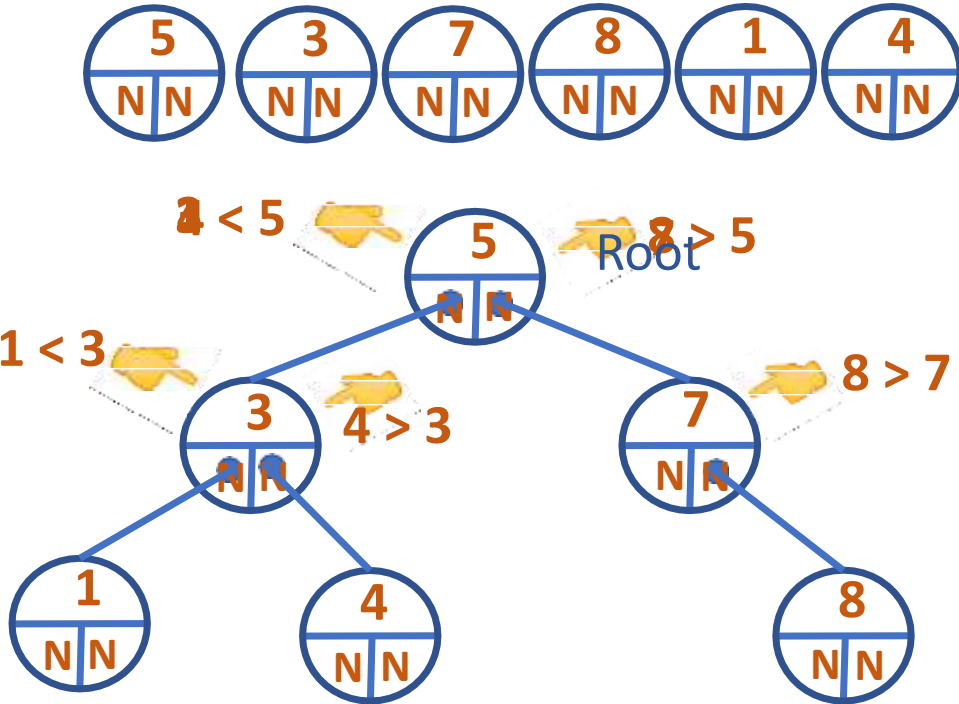
`NODE *root=NULL;` //root points to Root of the tree and initially it is null

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation



Linked implementation: 5, 3, 7, 8, 1, 4





THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

+91 9449867804



DATA STRUCTURES AND ITS APPLICATIONS

UE19CS202

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

BST: Deletion Operations

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion

Deletion of a Node in Binary Search Tree

case1: Node with no child (leaf node)

case2: Node with 1 child

case3: Node with 2 children

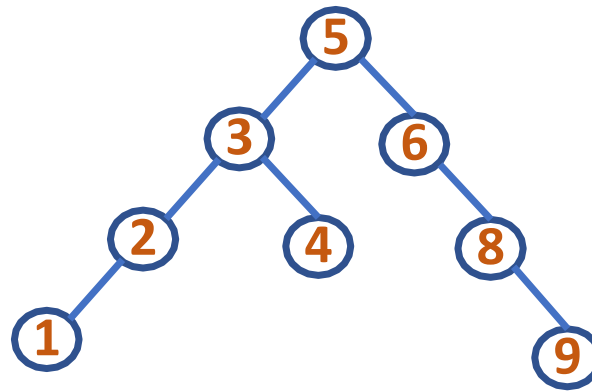
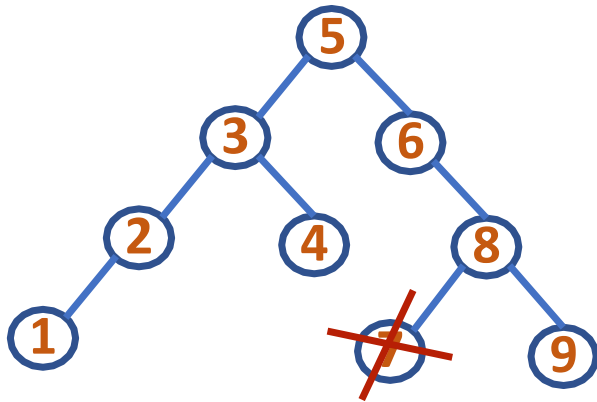


DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion



case1: Node with no child (leaf node)



To delete the node with info 7:

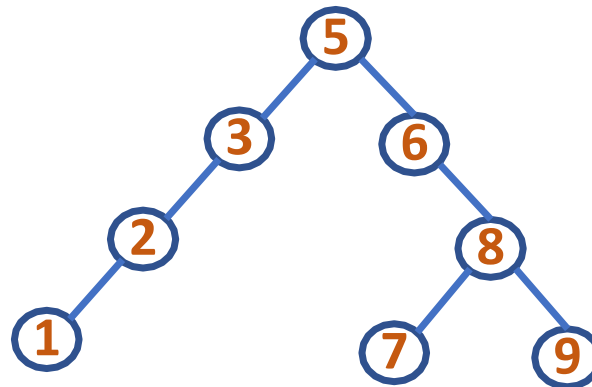
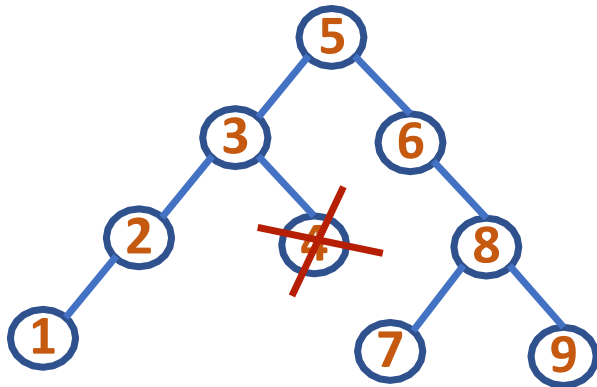
- Set its parent's left child field to point to NULL
- Free memory allocated to node with info 7

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion



case1: Node with no child (leaf node)



To delete the node with info 4:

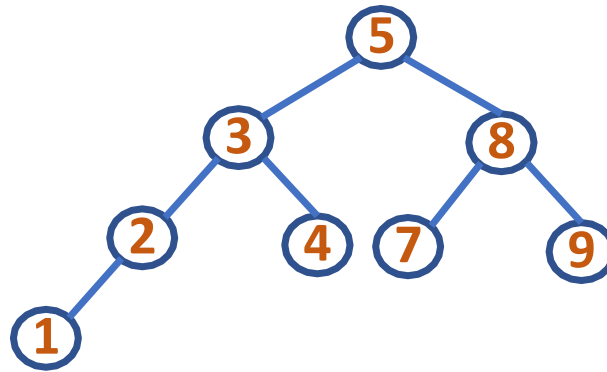
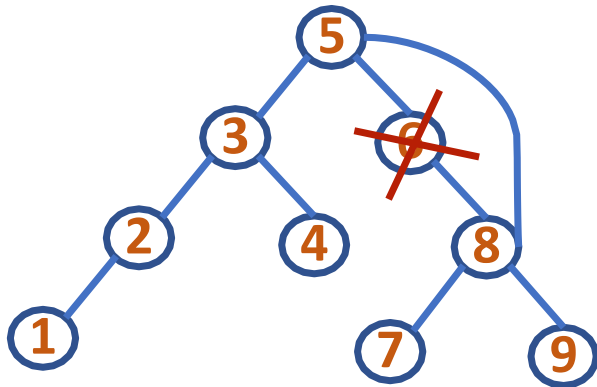
- Set its parent's right child field to point to NULL
- Free memory allocated to node with info 4

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion



case2: Node with 1 child



To delete the node with info 6:

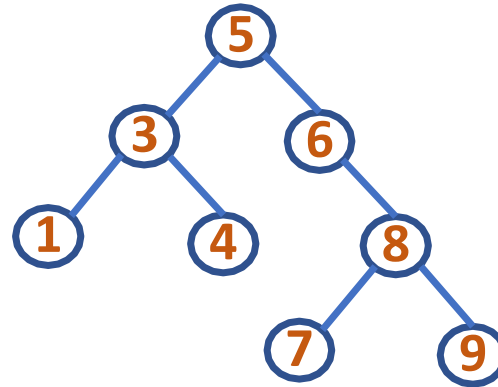
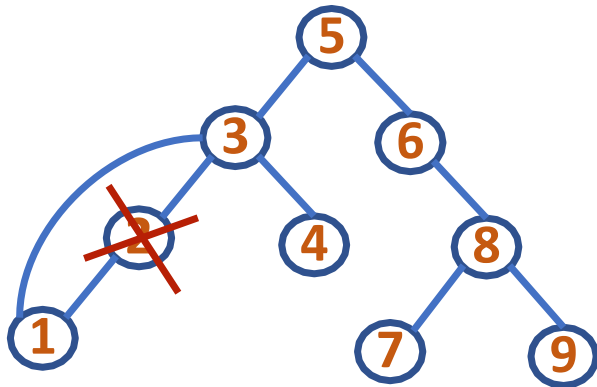
- Set its parent's right child field to point to its only child
- Free memory allocated to node with info 6

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion



case2: Node with 1 child



To delete the node with info 2:

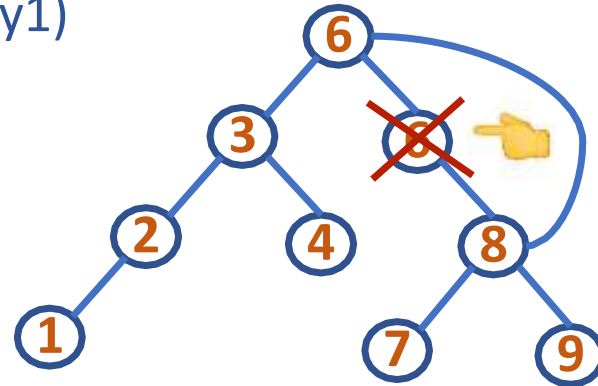
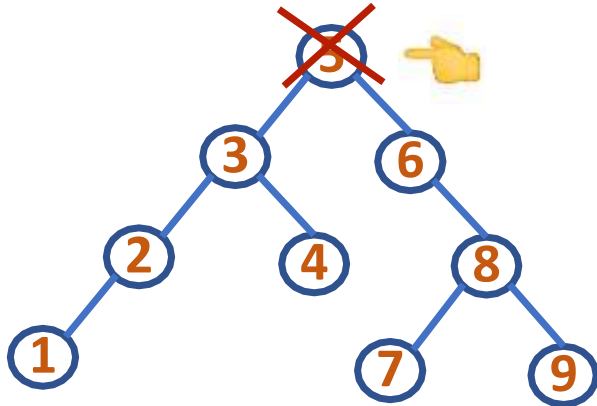
- Set its parent's left child field to point to its only child
- Free memory allocated to node with info 2

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion

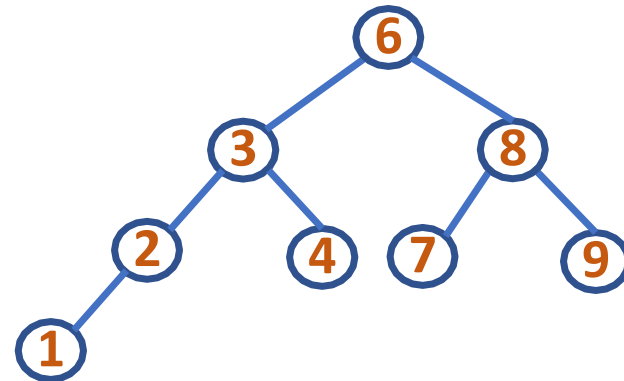


case3: Node with 2 children (Replace with inorder successor)
(Way1)



To delete the node with info 5:

- Replace 5 with its **inorder successor** and delete that inorder successor
- Now case3 has got changed to case2 (In general may change to case2 or case1)

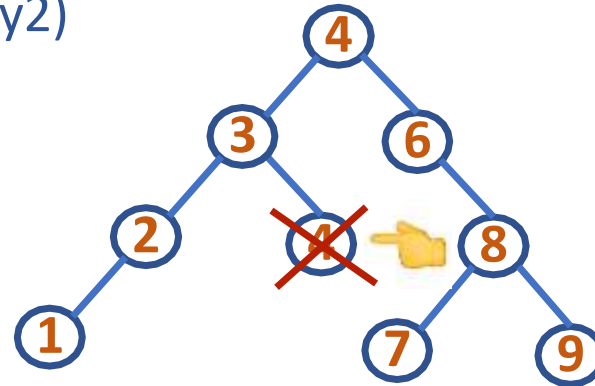
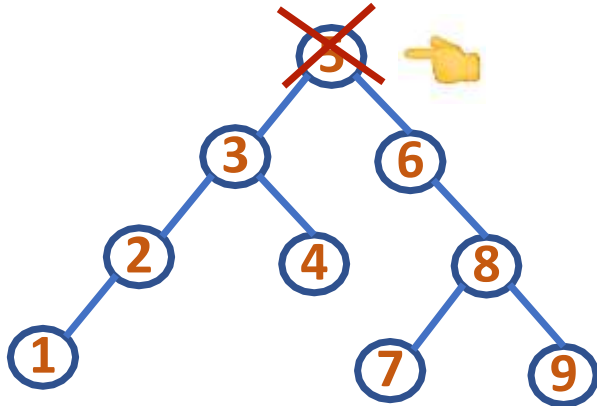


DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Deletion

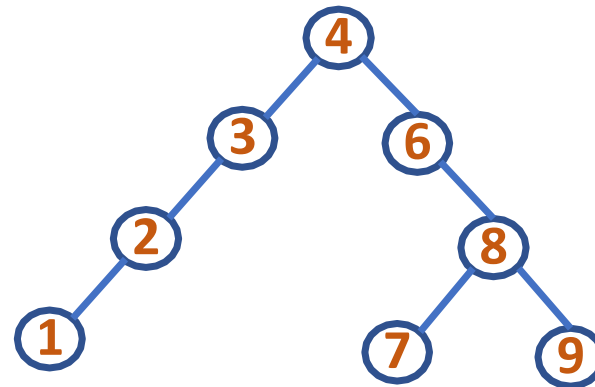


case3: Node with 2 children (Replace with inorder predecessor)
(Way2)



To delete the node with info 5:

- Replace 5 with its **inorder predecessor** and delete that inorder predecessor
- Here case3 has got changed to case1 (In general may change to case2 or case1)





THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

+91 9449867804



DATA STRUCTURES AND ITS APPLICATIONS

UE19CS202

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

BST: Implementation using Arrays

Shylaja S S

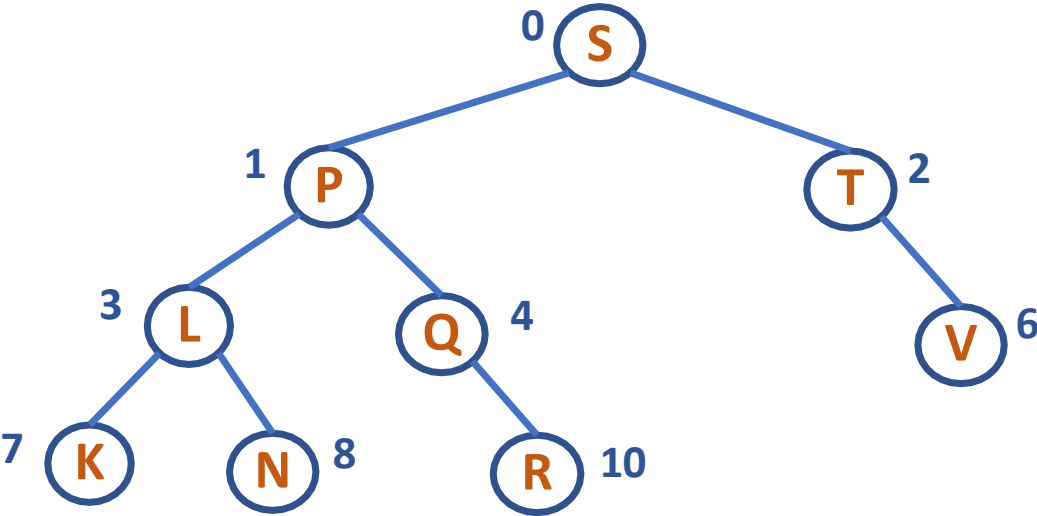
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation



Array Implementation (Implicit implementation)



S	P	T	L	Q	...	V	K	N	...	R
0	1	2	3	4	5	6	7	8	9	10

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation



Array Implementation (Implicit implementation)

```
typedef struct tree_array
{
    int info;
    int used;
}NODE;
```

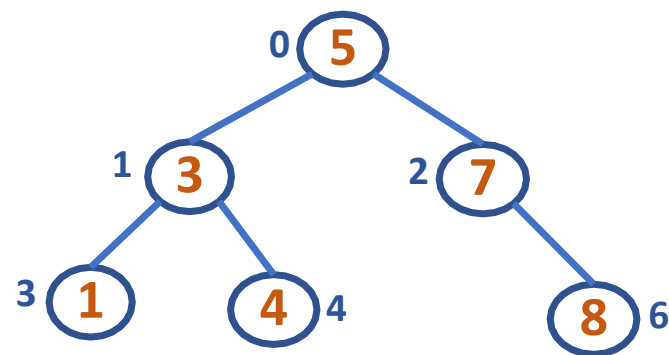
- `NODE bst[MAX];` *//here bst is an array of nodes*
- each node has its **data** and another field by name **used** to contain whether it is a valid node or not
- `used = 1 or 0`

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation



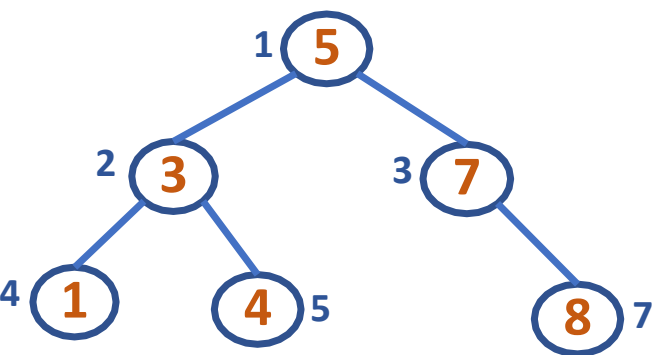
Array Implementation: 5, 3, 7, 8, 1, 4



Root Position: $i = 0$
Left Child Position: $2i + 1$
Right Child Position: $2i + 2$

info	5	3	7	1	4		8
used	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6

OR



Root Position: $i = 1$
Left Child Position: $2i$
Right Child Position: $2i + 1$

info		5	3	7	1	4		8
used	0	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6	7

DATA STRUCTURES AND ITS APPLICATIONS

Binary Search Tree - Implementation



Array Implementation: 5, 3, 7, 8, 1, 4



Root Position: $i = 0$

Left Child Position: $2i + 1$  OR

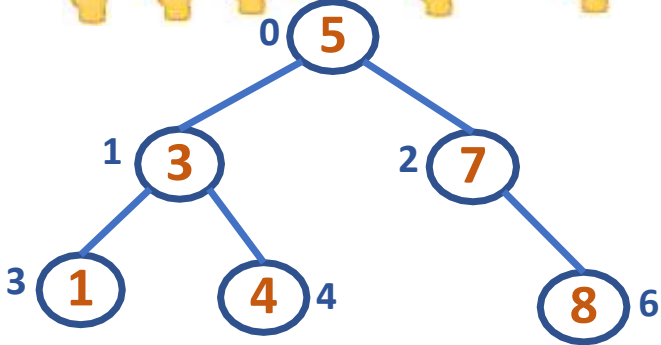
Right Child Position: $2i + 2$ 

Root Position: $i = 1$

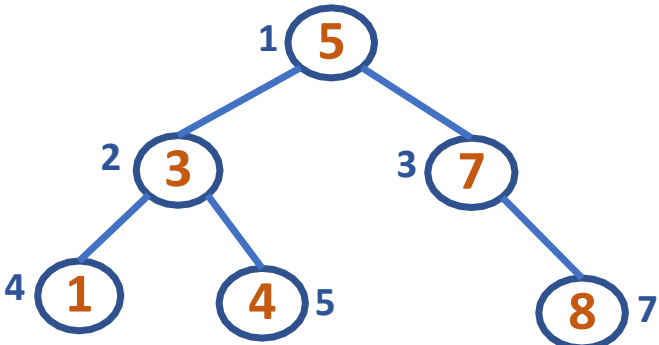
Left Child Position: $2i$

Right Child Position: $2i + 1$

info	5	3	7	1	4		8
used	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6



info		5	3	7	1	4		8
used	0	1	1	1	1	1	0	1
Position: i	0	1	2	3	4	5	6	7





THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

+91 9449867804



DATA STRUCTURES AND ITS APPLICATIONS

UE19CS202

Shylaja S S & Kusuma K V
Department of Computer Science
& Engineering



DATA STRUCTURES AND ITS APPLICATIONS

Threaded BST and its Implementation

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree



Motivation

- Iterative Inorder Traversal requires Explicit stack
- Costly
- Since we loose track of address as and when we navigate, Node addresses were stacked
- If this can be achieved through some other less expensive mechanism, we can eliminate the use of explicit stack
- Small structural modification carried on Binary tree will solve the above problem

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree

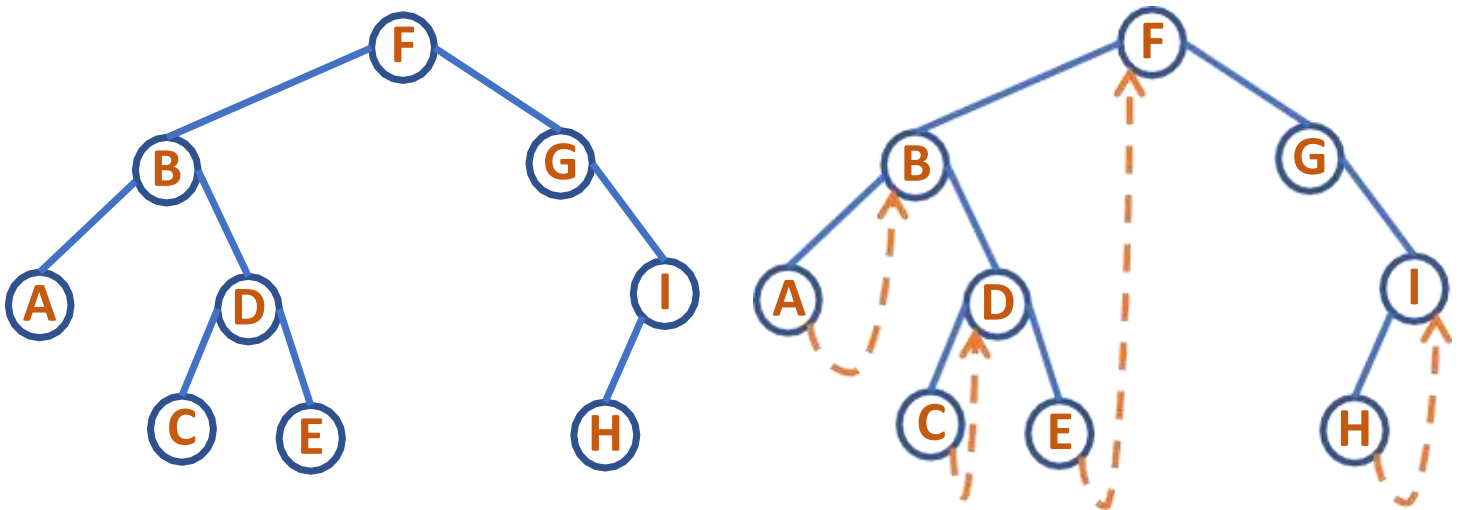
- We can use the right pointer of a node to point to the inorder successor if in case it is not pointing to the child. Such a tree is called **Right-In Threaded** Binary Tree
- If we use the left pointer to store the inorder predecessor, the tree is called **Left-In Threaded** Binary Tree
- If we use both the pointers, the tree is called **In Threaded** Binary Tree



Threaded Binary Search Tree



Right-In Threaded Binary Tree



Binary Tree

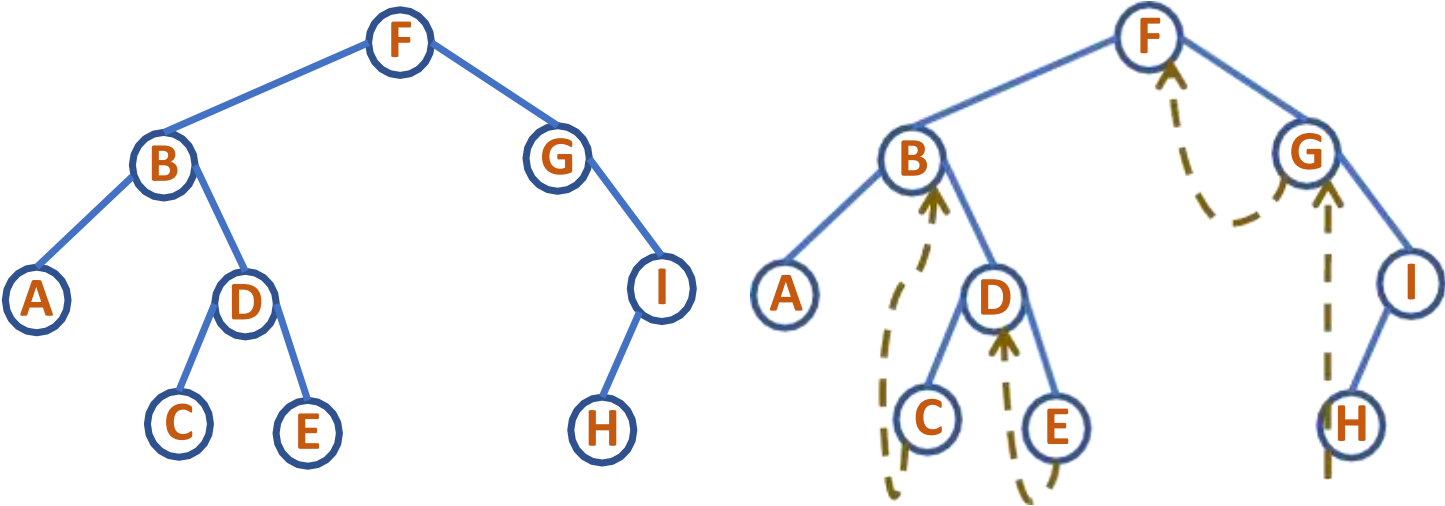
Right-In Threaded Binary Tree

Inorder Traversal:
A B C D E F G H I

Nodes with Right Pointer NULL	A	C	E	H	I
Inorder Successor	B	D	F	I	-



Left-In Threaded Binary Tree



Binary Tree

Left-In Threaded Binary Tree

Inorder Traversal:
A B C D E F G H I

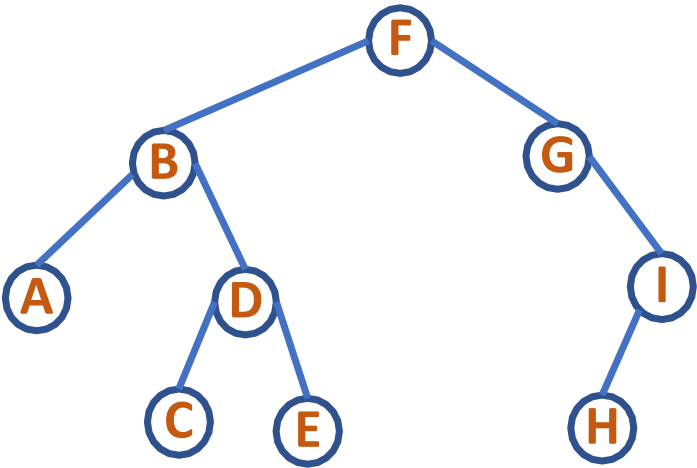
Nodes with Left Pointer NULL	A	C	E	G	H
Inorder Predecessor	-	B	D	F	G

DATA STRUCTURES AND ITS APPLICATIONS

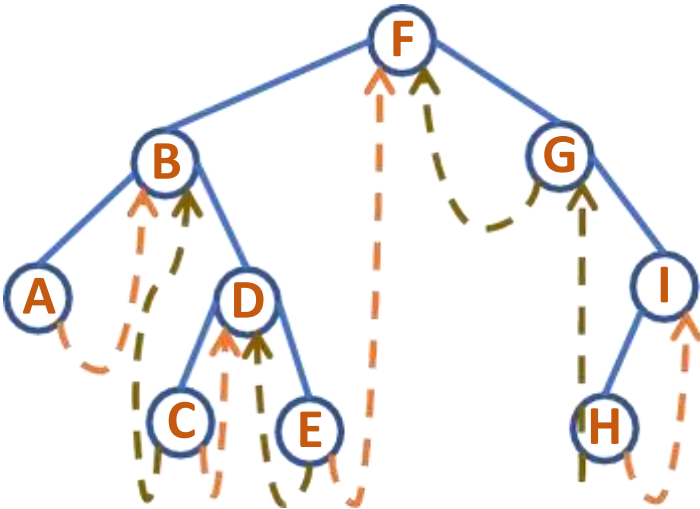
Threaded Binary Search Tree



In Threaded Binary Tree



Binary Tree



In Threaded Binary Tree

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation



Right In Threaded Binary Tree

```
typedef struct node
{
    int info;
    struct node *left;    // pointer to left child
    struct node *right;   // pointer to right child
    int rthread;          // rthread is TRUE if right is NULL
                        // or a non-NULL thread
}NODE;
```

Node Structure

info	left	right	rthread
------	------	-------	---------

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation



```
NODE* createNode(int e) 📌
```

```
{
```

```
    NODE* temp=malloc(sizeof(NODE)); 📌
```

```
    temp->info=e; 📌
```

```
    temp->left=NULL; 📌
```

```
    temp->right=NULL; 📌
```

```
    temp->rthread=1; 📌
```

```
    return temp; 📌
```

// Returns: 2000

```
}
```

info	left	right	rthread
------	------	-------	---------

createNode(57)

temp ->

57	NULL	NULL	1
----	------	------	---

Let Address of this node on Heap: 2000

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation



Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57

Address: 800

57

Node Structure

info	left	right	rthread
57	NULL	NULL	1

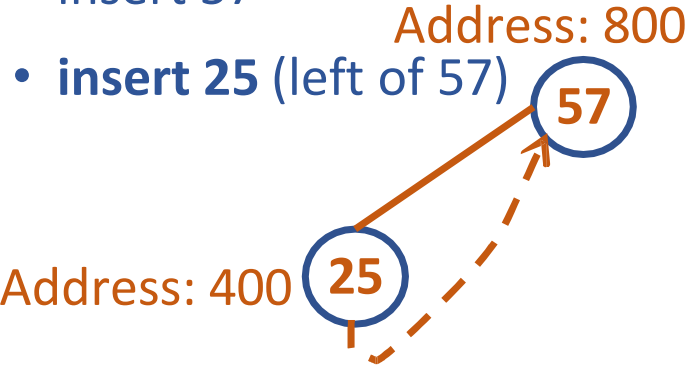
DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation



Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- insert 25 (left of 57)



Node Structure

info	left	right	rthread
57	400	NULL	1
25	NULL	800	1

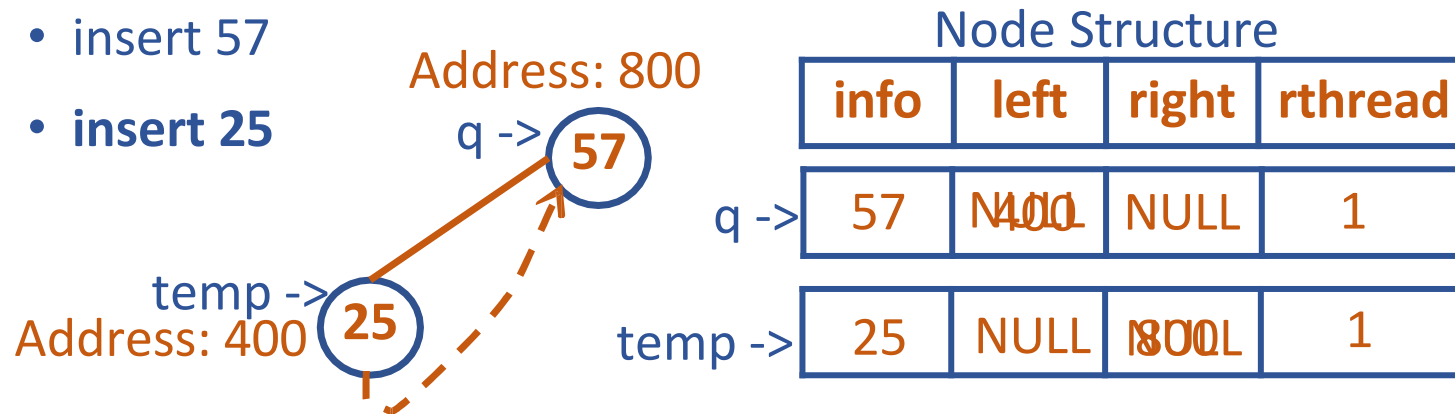
DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation



Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- insert 25



```
void setLeft(NODE* q, int e) { //set node with info e to left of q
    NODE* temp=createNode(e); //e=25
    q->left=temp;
    temp->right=q;
}
```

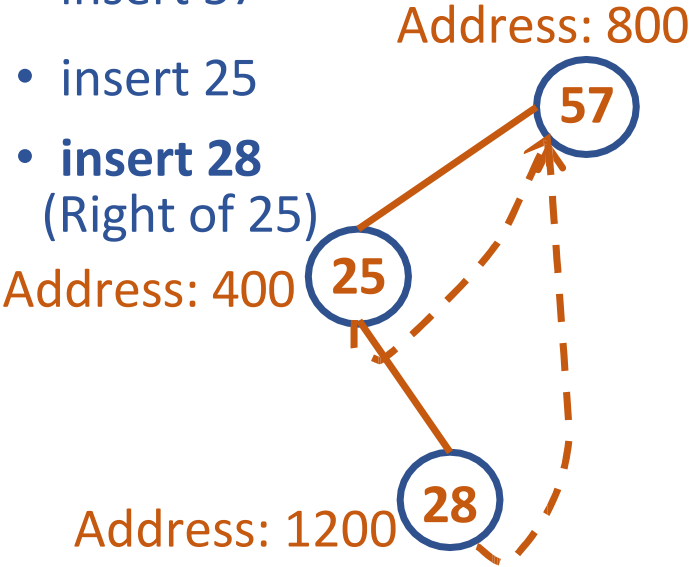
DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation



Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- insert 25
- **insert 28**
(Right of 25)



Node Structure

info	left	right	rthread
57	400	NULL	1
25	NULL	1200	0
28	NULL	800	1

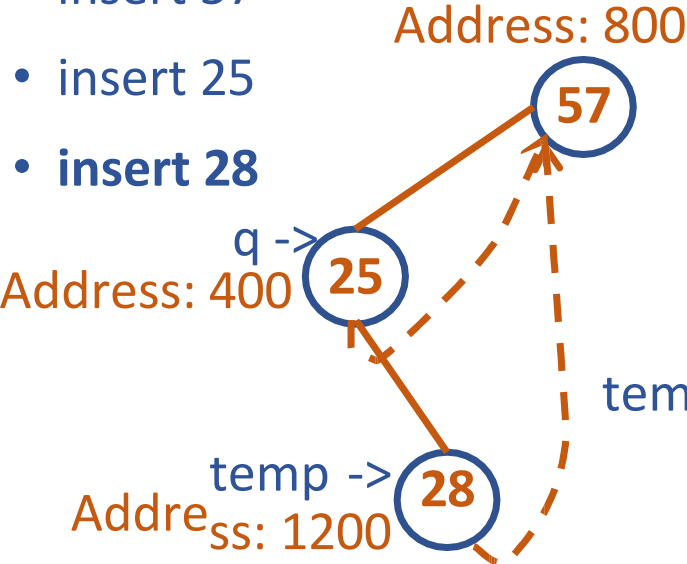
DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Implementation



Right In Threaded Binary Tree: 57, 25, 28

- A node is created with rthread set to TRUE
- insert 57
- insert 25
- **insert 28**



Node Structure

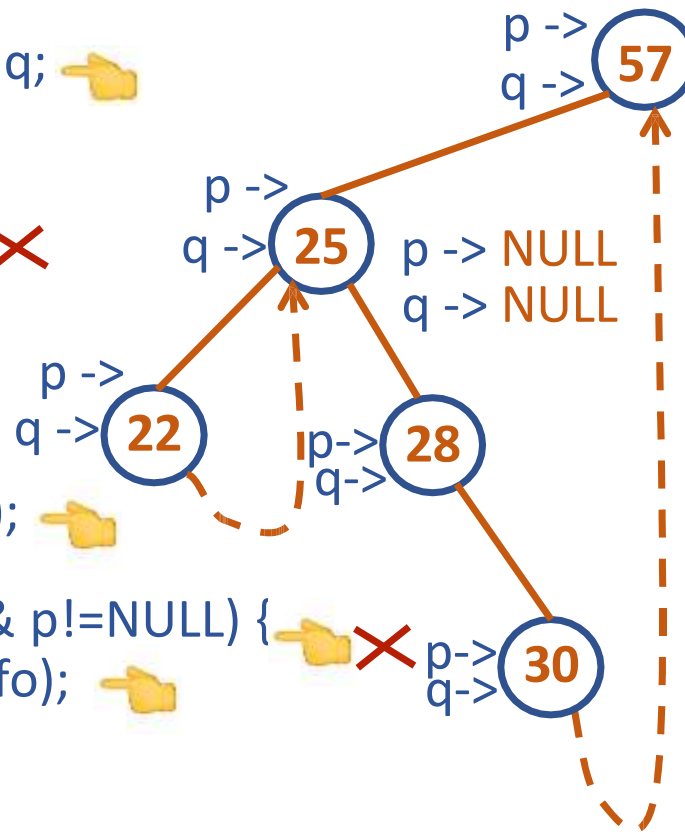
info	left	right	rthread
57	400	NULL	1
25	NULL	1200	0
28	NULL	800	1

```
void setRight(NODE* q,int e) {  
    NODE* temp=createNode(e);  
    temp->right=q->right;  
    q->right=temp;  
    q->rthread=0;  
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Threaded Binary Search Tree: Inorder Traversal

```
void inOrder(NODE *root) {  
    NODE *p=root;  NODE *q;  
    do{  
        q=NULL;  
        while(p!=NULL) {  
            q=p;  
            p=p->left;  
        }  
        if(q!=NULL) {  
            printf("%d ",q->info);  
            p=q->right;  
            while(q->rthread && p!=NULL) {  
                printf("%d ",p->info);  
                q=p;  
                p=p->right;  
            }  
        }  
        while(q!=NULL);  
    }  
}
```



q -> **NULL**

rthread is TRUE for nodes
with info: 22, 30, 57
Inorder Traversal:
22 25 28 30 57





THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

+91 9449867804



DATA STRUCTURES AND ITS APPLICATIONS

UE19CS202

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering



DATA STRUCTURES AND ITS APPLICATIONS

Implementation of Binary Expression Tree

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree

- An expression can be represented using the **Expression Tree** data structure
- Such a tree is built normally for translating the code as data and then analysing and evaluating expressions
- Immutable**: To change the expression another tree has to be constructed



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

- Normally a postfix expression is used in constructing the Expression tree
- When an operand is received, a new node is created which will be a leaf in the expression tree
- If an operator, it connects to two leaves
- Stack DS is used as intermediary storing place of node's address



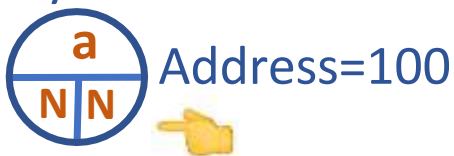
DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction



Postfix Expression: **abc*+**

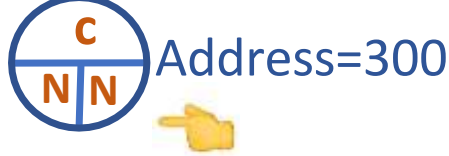
Symbol = a



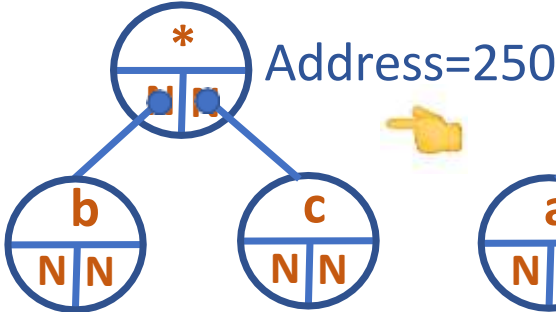
Symbol=b



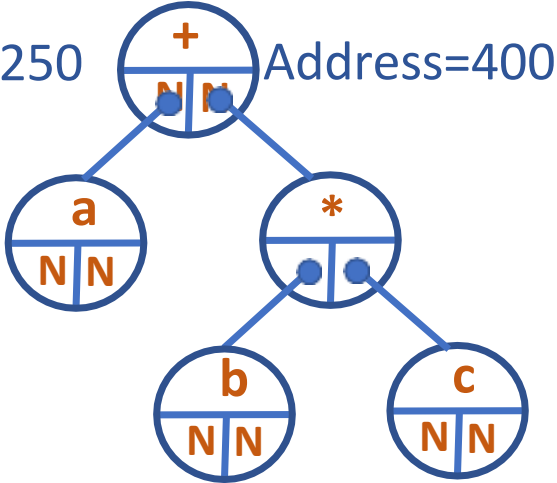
Symbol=c



Symbol = *



Symbol = +



300
150
400

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

- Scan the postfix expression till the end, one symbol at a time
 - Create a new node, with symbol as info and left and right link as NULL
 - If symbol is an operand, push address of node to stack
 - If symbol is an operator
 - Pop address from stack and make it right child of new node
 - Pop address from stack and make it left child of new node
 - Now push address of new node to stack
- Finally, stack has only element which is the address of the root of expression tree



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

Postfix Expression: **a bc * +**

- Scan the postfix expression till the end, one symbol at a time

- Create a new node, with symbol as info and left and right link as NULL

- If symbol is an operand, push address of node to stack

- If symbol is an operator

- Pop the address from stack and make it right child of new node

- Pop the address from stack and make it left child of new node

- Now push address of new node to stack

- Finally, stack has only element which is the address of the root of expression tree

Symbol = a



Address=100



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

Postfix Expression: **a bc * +**

- Scan the postfix expression till the end, one symbol at a time

- Create a new node, with symbol as info and left and right link as NULL

- If symbol is an operand, push address of node to stack

- If symbol is an operator

- Pop the address from stack and make it right child of new node

- Pop the address from stack and make it left child of new node

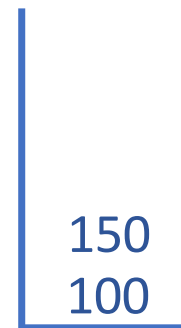
- Now push address of new node to stack

- Finally, stack has only element which is the address of the root of expression tree

Symbol = b



Address=150



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

Postfix Expression: **a b c * +**

- Scan the postfix expression till the end, one symbol at a time

- Create a new node, with symbol as info and left and right link as NULL

- If symbol is an operand, push address of node to stack

- If symbol is an operator

- Pop the address from stack and make it right child of new node

- Pop the address from stack and make it left child of new node

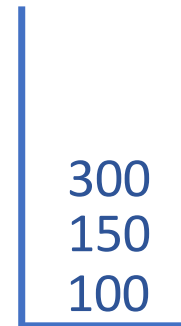
- Now push address of new node to stack

- Finally, stack has only element which is the address of the root of expression tree

Symbol = c



Address=300



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

Postfix Expression: **a b c * +**

- Scan the postfix expression till the end, one symbol at a time

- Create a new node, with symbol as info and left and right link as NULL

- If symbol is an operand, push address of node to stack

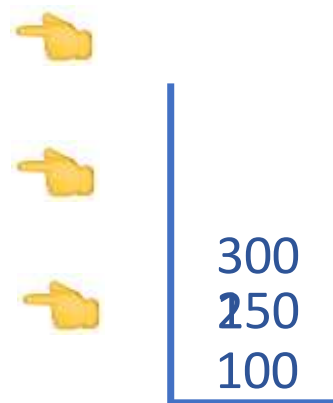
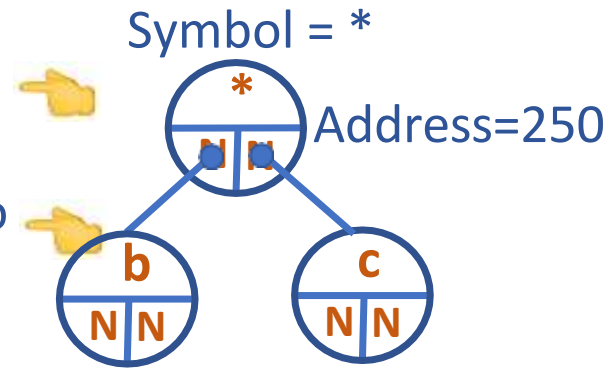
- If symbol is an operator

- Pop the address from stack and make it right child of new node

- Pop the address from stack and make it left child of new node

- Now push address of new node to stack

- Finally, stack has only element which is the address of the root of expression tree



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Construction

Postfix Expression: **a b c * +**

- Scan the postfix expression till the end, one symbol at a time

- Create a new node, with symbol as info and left and right link as NULL

- If symbol is an operand, push address of node to stack

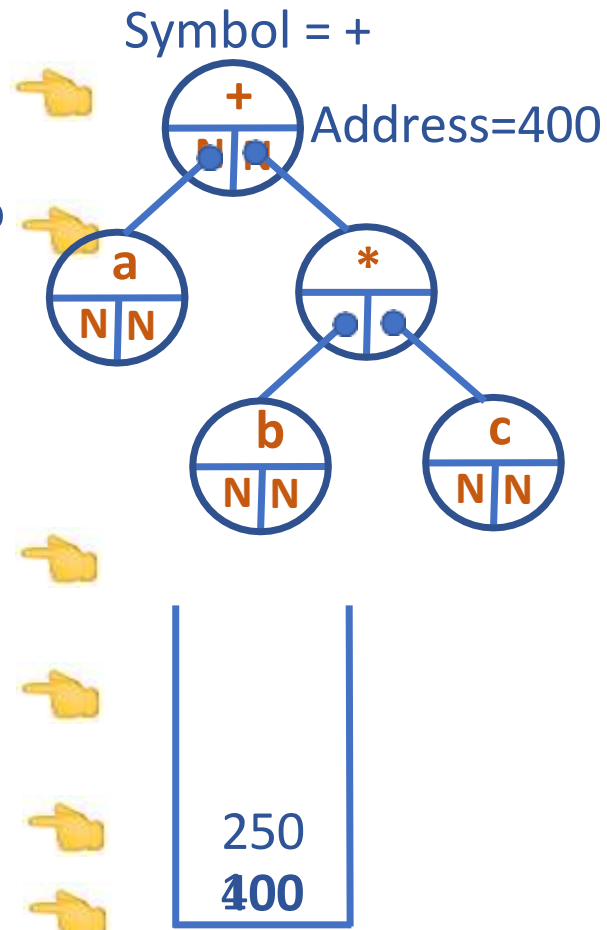
- If symbol is an operator

- Pop the address from stack and make it right child of new node

- Pop the address from stack and make it left child of new node

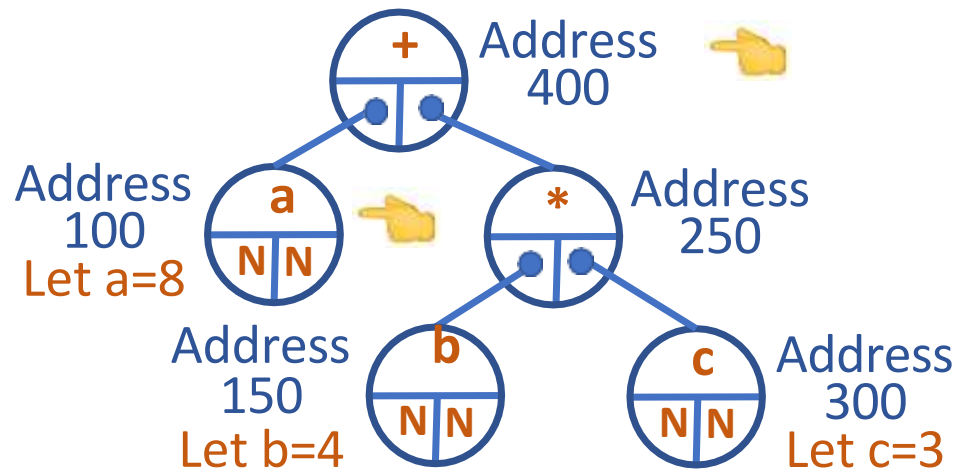
- Now push address of new node to stack

- Finally, stack has only element which is the address of the root of expression tree



DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



eval(400)
return eval(100) + eval(250)

eval(100)
return 8

- Think in terms of recursion

eval(t) // 't' has the address of the root node of expression tree

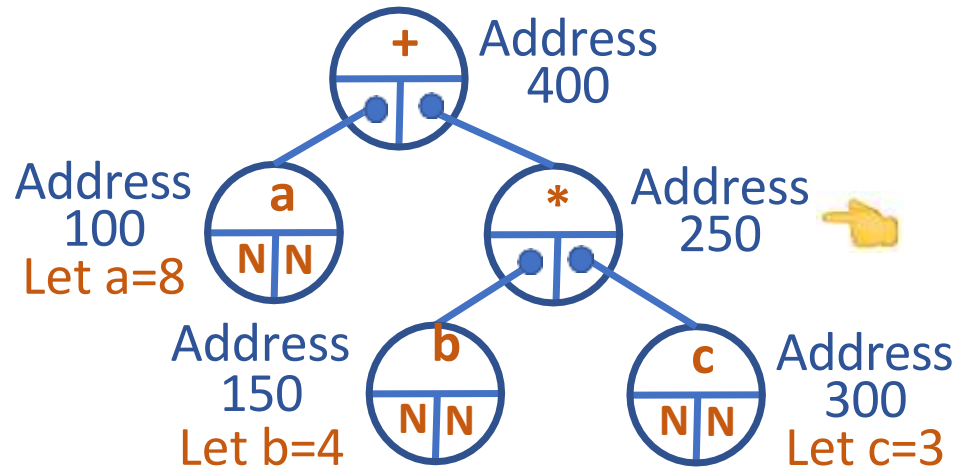
if t->data is an operator

return eval (t->left) t->data eval(t->right)

return t->data

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



eval(400)
return 8 + eval(250)
eval(250)
return eval(150) * eval(300)

- Think in terms of recursion

eval(t) // 't' has the address of the root node of expression tree

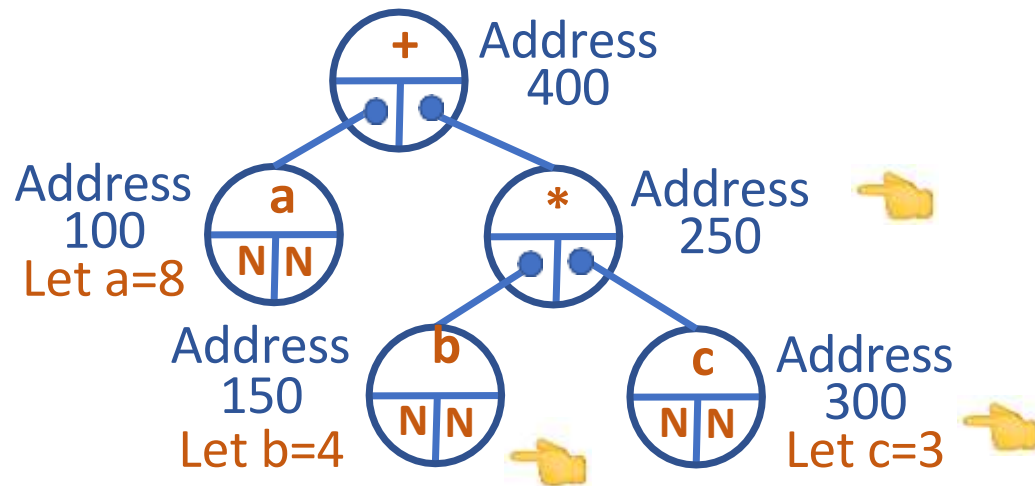
if t->data is an operator

return eval (t->left) t->data eval(t->right)

return t->data

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



```
eval(400)
  return 8 + eval(250)
eval(250)
  return eval(150) * eval(300)
eval(150)
  return 4
eval(300)
  return 3
```

- Think in terms of recursion

`eval(t)` // 't' has the address of the root node of expression tree

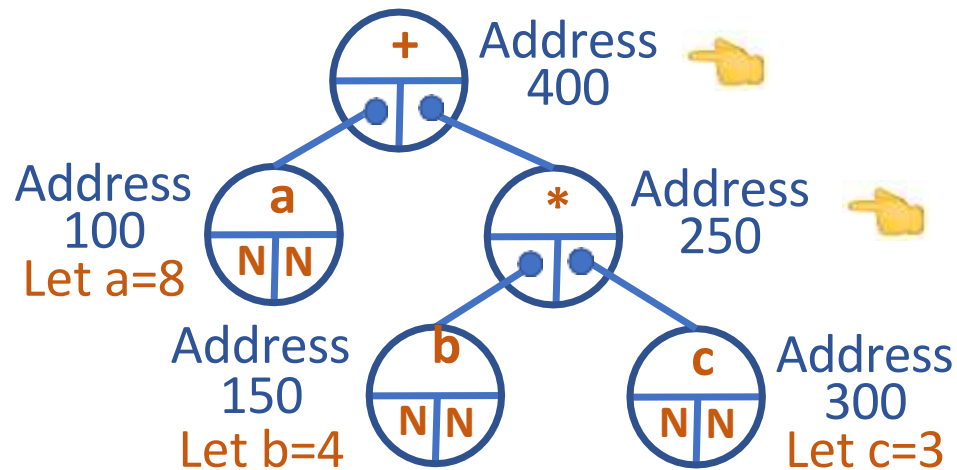
if `t->data` is an operator

return `eval(t->left) t->data eval(t->right)`

return `t->data`

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



eval(400)

return

8

+ eval(250)

eval(250)

return

12



- Think in terms of recursion

eval(t) // 't' has the address of the root node of expression tree

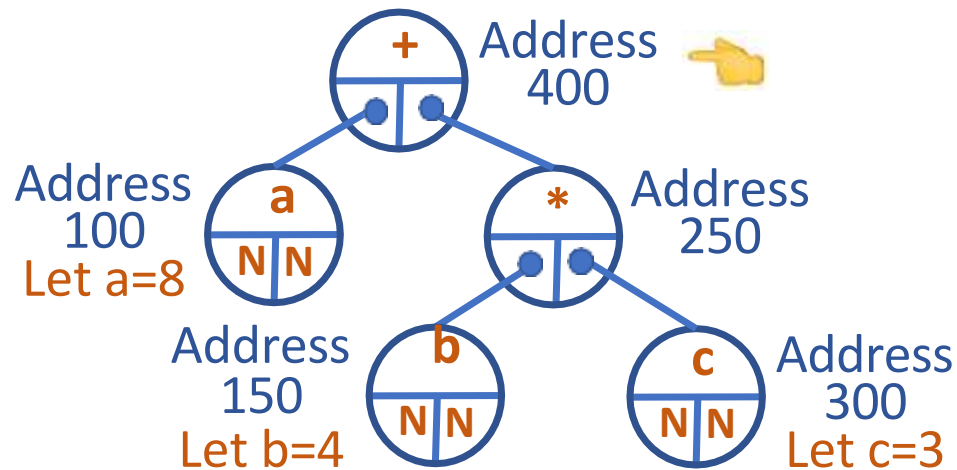
if t->data is an operator

return eval (t->left) t->data eval(t->right)

return t->data

DATA STRUCTURES AND ITS APPLICATIONS

Expression Tree Evaluation



eval(400)
return 208 + 12
Postfix abc*+ : 20



- Think in terms of recursion

eval(t) // 't' has the address of the root node of expression tree

if t->data is an operator

return eval (t->left) t->data eval(t->right)

return t->data

DATA STRUCTURES AND ITS APPLICATIONS

General Expression Tree Evaluation

```
struct treenode
{
    short int utype;
    union{
        char operator[MAX];
        float val;
    }info;
    struct treenode *child;
    struct treenode *sibling;
};
typedef struct treenode TREENODE;
```

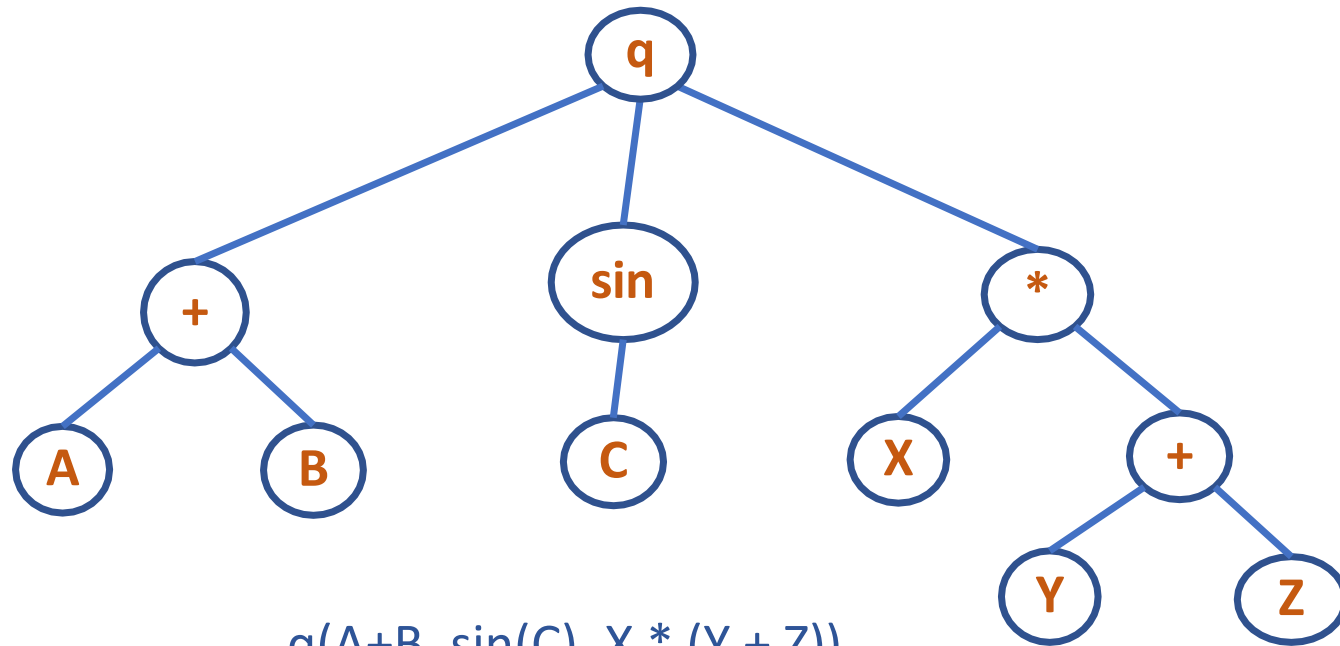


DATA STRUCTURES AND ITS APPLICATIONS

General Expression Tree Evaluation



Here node can be either an operand or an operator



$q(A+B, \sin(C), X * (Y + Z))$

Tree representation of an arithmetic expression

DATA STRUCTURES AND ITS APPLICATIONS

General Expression Tree Evaluation



```
void replace(TREENODE *p)
{
    float val;
    TREENODE *q,*r;
    if(p->utype == operator)
    {
        q = p->child;
        while(q != NULL)
        {
            replace(q);
            q = q->next;}
    }
```

DATA STRUCTURES AND ITS APPLICATIONS

General Expression Tree Evaluation

```
value = apply(p);
p->utype = OPERAND;
p->val = value;
q = p->child;
p->child = NULL;
while(q != NULL)
{
    r = q;
    q = q->next;
    free(r);
}
}
```



DATA STRUCTURES AND ITS APPLICATIONS

General Expression Tree Evaluation

```
float eval(TREENODE *p)
{
    replace(p);
    return(p->val);
    free(p);
}
```



DATA STRUCTURES AND ITS APPLICATIONS

Constructing a Tree

```
void setchildren(TREENODE *p,TREENODE *list)
{
    if(p == NULL) {
        printf("invalid insertion");
        exit(1);
    }
    if(p->child != NULL) {
        printf("invalid insertion");
        exit(1);
    }
    p->child = list;
}
```



DATA STRUCTURES AND ITS APPLICATIONS

Constructing a Tree

```
void addchild(TREENODE *p,int x)
{
    TREENODE *q;
    if(p==NULL)
    {
        printf("void insertion");
        exit(1);
    }
}
```



DATA STRUCTURES AND ITS APPLICATIONS

Constructing a Tree

```
r = NULL;
q = p->child;
while(q != NULL)
{
    r = q;
    q = q->next;
}
q = getnode();
q->info = x;
q->next = NULL;
```



DATA STRUCTURES AND ITS APPLICATIONS

Constructing a Tree

```
if(r==NULL)
    p->child=q;
else
    r->next=q;
}
```





THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

+91 9449867804



DATA STRUCTURES AND ITS APPLICATIONS

UE19CS202

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Implementation of Priority Queue using min heap/max heap

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Ascending and Descending Heap

- Ascending Heap: Root will have the lowest element. Each node's data is greater than or equal to its parent's data. It is also called min heap.
- Descending Heap: Root will have the highest element. Each node's data is lesser than or equal to its parent's data. It is also called max heap.



- Priority Queue is a Data Structure in which intrinsic ordering of the elements does determine the results of its basic operations
- Ascending Priority Queue: is a collection of items into which items can be inserted arbitrarily and from which only the smallest item can be removed
- Descending Priority Queue: is a collection of items into which items can be inserted arbitrarily and from which only the largest item can be removed

Consider the properties of a heap

- The entry with largest key is on the top(Descending heap) and can be removed immediately. But $O(\log n)$ time is required to readjust the heap with remaining keys
- If another entry need to be done, it requires $O(\log n)$
- Therefore, heap is advantageous to implement a Priority Queue

DATA STRUCTURES AND ITS APPLICATIONS

Priority Queue using Heap: Implementation

- dpq: Array that implements descending heap of size k (position from 0 to $k-1$)
- pqinsert(dpq,k,elt): insert element into the heap dpq of size k . Size increases to $k+1$
- This insertion is done using siftup operation



Algorithm for siftup

$c = k;$

$p = (c-1)/2;$

while($c > 0$ && $dpq[p] < elt$) {

$dpq[c] = dpq[p];$

$c = p;$

$p = (c-1)/2;$

}

$dpq[c] = elt;$

```
pqmaxdelete(dpq,k) //for a descending heap of size k
```

```
p = dpq[0];
```

```
adjustheap(0,k-1)
```

```
return p;
```

Algorithm largechild(p,m)

$c = 2 * p + 1;$

if($c + 1 \leq m$ && $x[c] < x[c + 1]$)

$c = c + 1;$

if($c > m$)

return -1;

else

return (c);

Algorithm `adjustheap(root,k)` `//recursive`

```
p = root;
```

```
c = largechild(p,k-1);
```

```
if(c >= 0 && dpq[k] < dpq[c]){
```

```
    dpq[p] = dpq[c];
```

```
    adjustheap(c,k);
```

```
}
```

```
else
```

```
    dpq[p] = dpq[k];
```

Iterative version

```
p = root;

kvalue = dpq[k];

c = largechild(p,k-1);
while(c >= 0 && kvalue < dpq[c]){
    dpq[p] = dpq[c];

    p = c;
    c = largechild(p,k-1);
}

dpq[p] = kvalue;
```




THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

+91 9449867804



DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Sandesh B. J & Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha

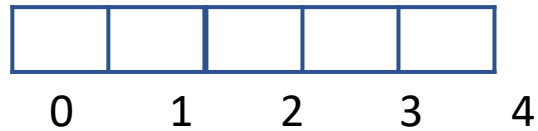
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

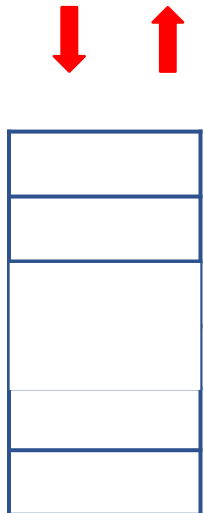
Introduction to graphs

Linear data structures

Array



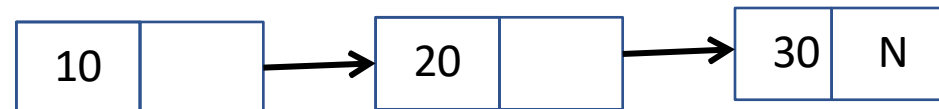
Stack



Queue



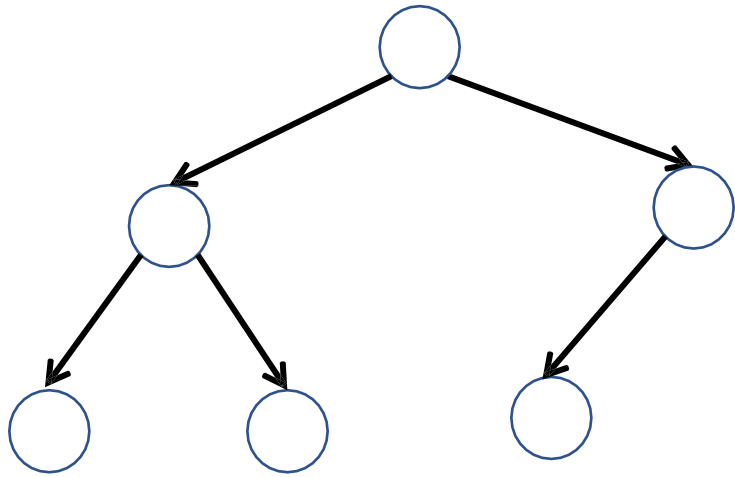
Linked List



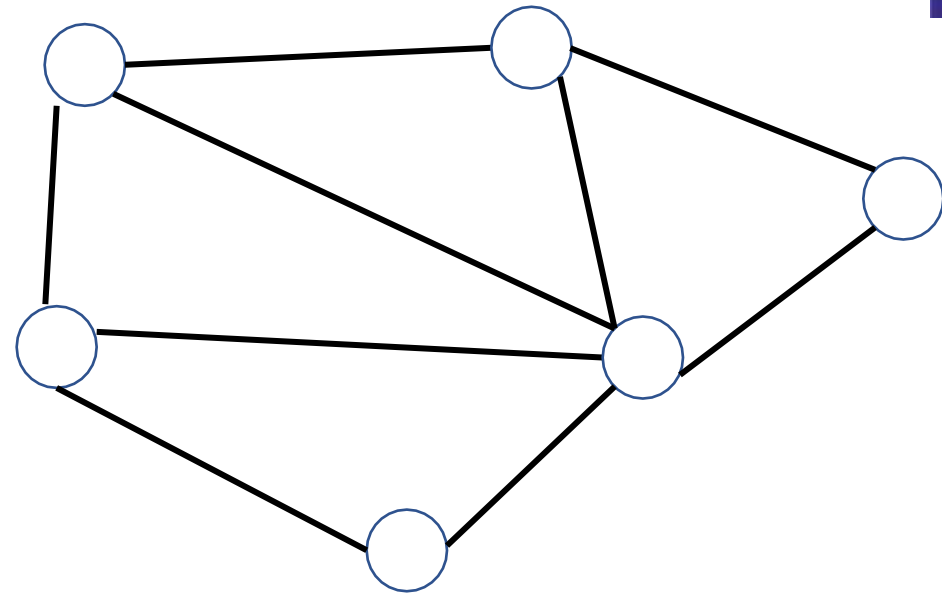
DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Graphs

Non-Linear Data Structure



Tree



Graph

In a tree with N nodes there are $N-1$ edges



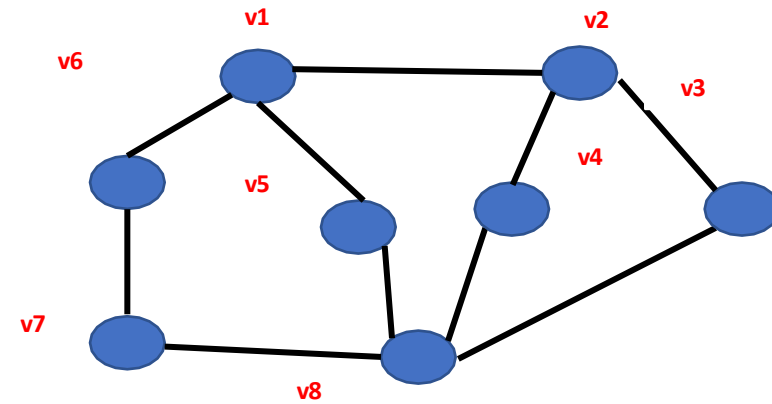
- A Graph is a data structure that consists of set of vertices and a set of edges that relate the node to each other.
- The set of edges represents the relationship among the vertices.
- A graph G is defined as

$$G=(V,E)$$

V: finite nonempty set of vertices

E: a set of edges

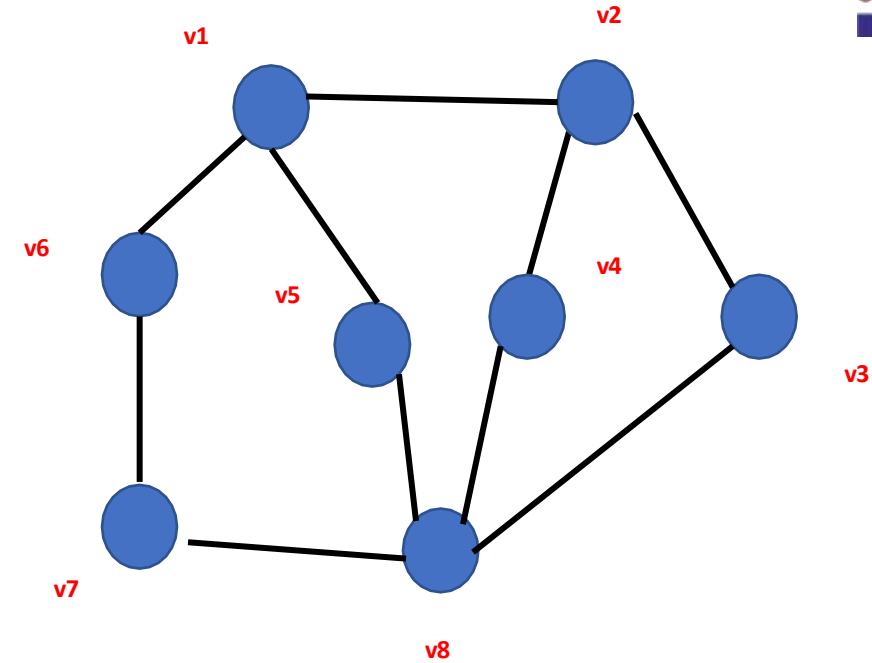
$$V = \{ v1,v2,v3,v4,v5,v6,v7,v8 \}$$



DATA STRUCTURES AND ITS APPLICATIONS

Representation of Edge

$V = \{ v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8 \}$
 $E = \{ (v_1, v_2), (v_2, v_3), (v_2, v_4), (v_4, v_8), (v_1, v_5), (v_1, v_6), (v_6, v_7), (v_5, v_8) \}$



Undirected Graph:

- A graph is undirected, when the pair of vertices representing any edge is unordered.



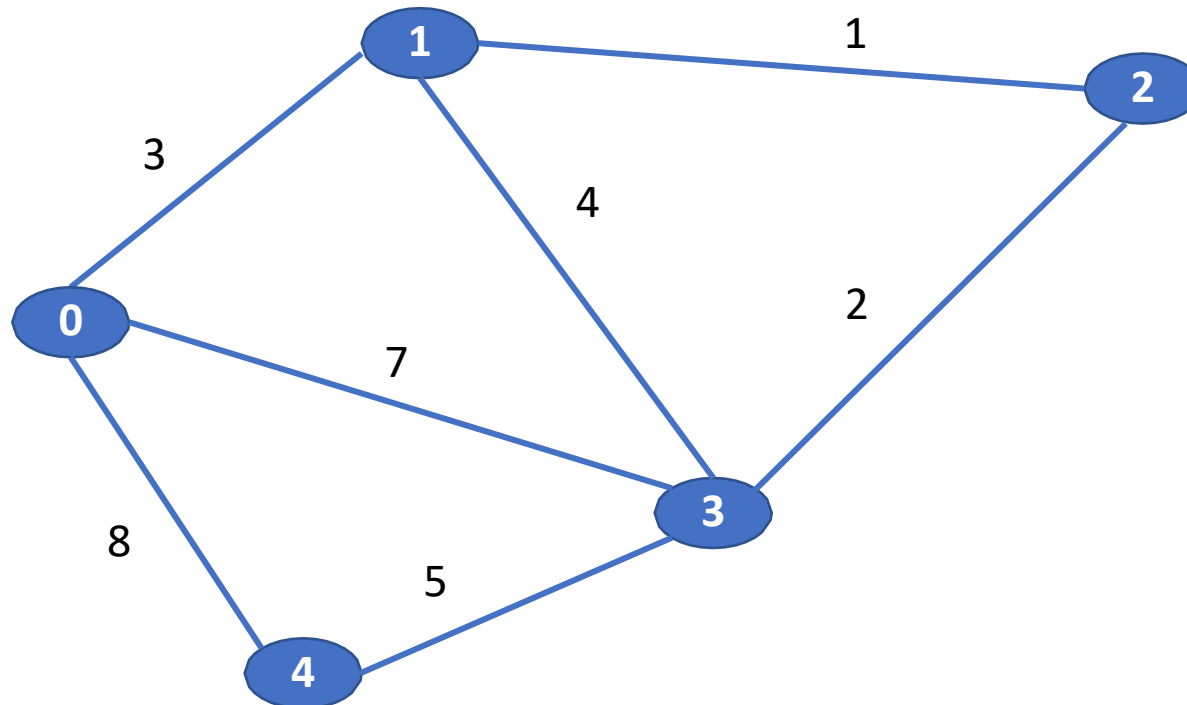
Directed Graph:

- A graph with all directed edges is called diagraph or directed graph.



Weighted Graph:

- A weighted graph is a graph where each edge has a numerical value called weight.



Adjacent Nodes :

- A node n is adjacent to node m if there is an edge from m to n.
- if n is adjacent to m, then n is called the **successor** of m and m is called the **predecessor** of n.



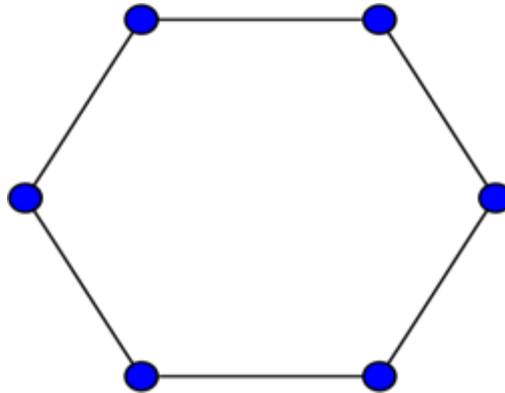
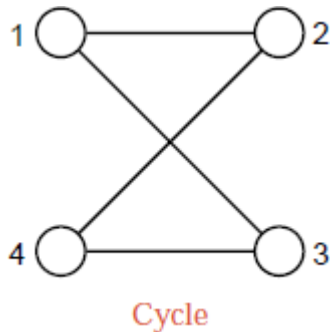
For example : a is adjacent to b

Path:

Path is a sequence of vertices that connect two nodes in a graph.

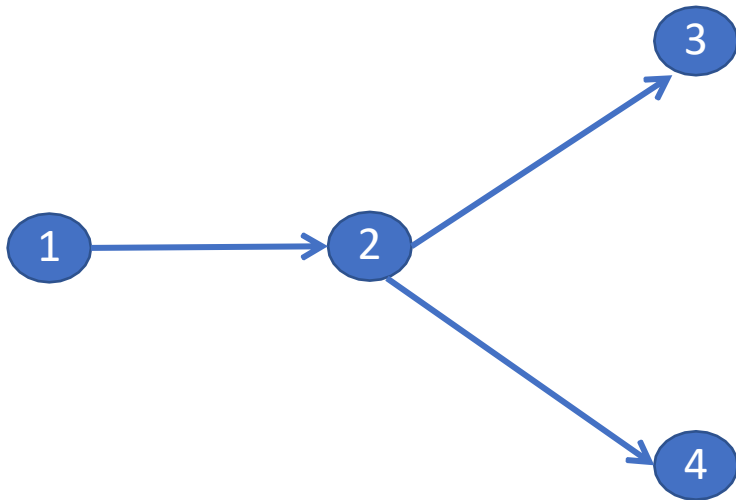
Cycle :

- A path from node to itself is called a cycle or cycle is path in which first and last vertices are same. A graph with at-least one cycle is called cyclic graph. For example the below graph are cyclic graphs



Acyclic :

- A graph with no cycles is called acyclic graph. A directed acyclic graph is called dag. For example below graph is a directed acyclic graph

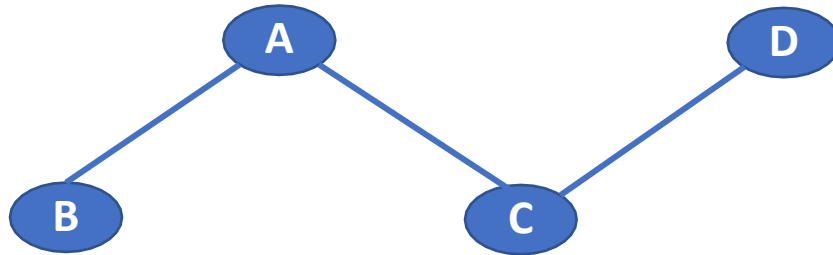


Incident:

A node n is incident to an edge x , if node is one of the two nodes the edge connects.

Degree:

The degree of vertex i is the number of edges incident on vertex i .



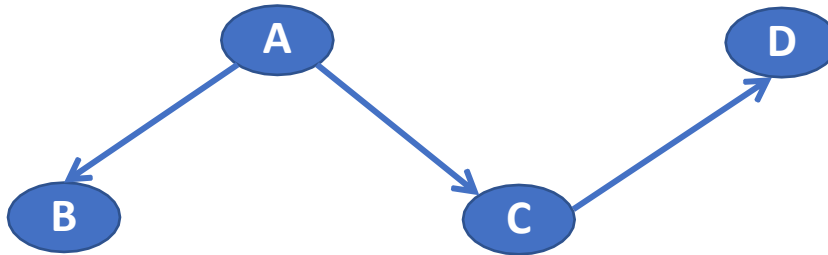
$\text{degree}(A)=2, \text{degree of}(D)=1$

In-degree:

- In-degree of vertex i is the number of edges incident to i .

Out-degree:

- Out-degree of vertex i is the number of edges incident from i .



Out-degree(A)=2, in-degree of(A)=0

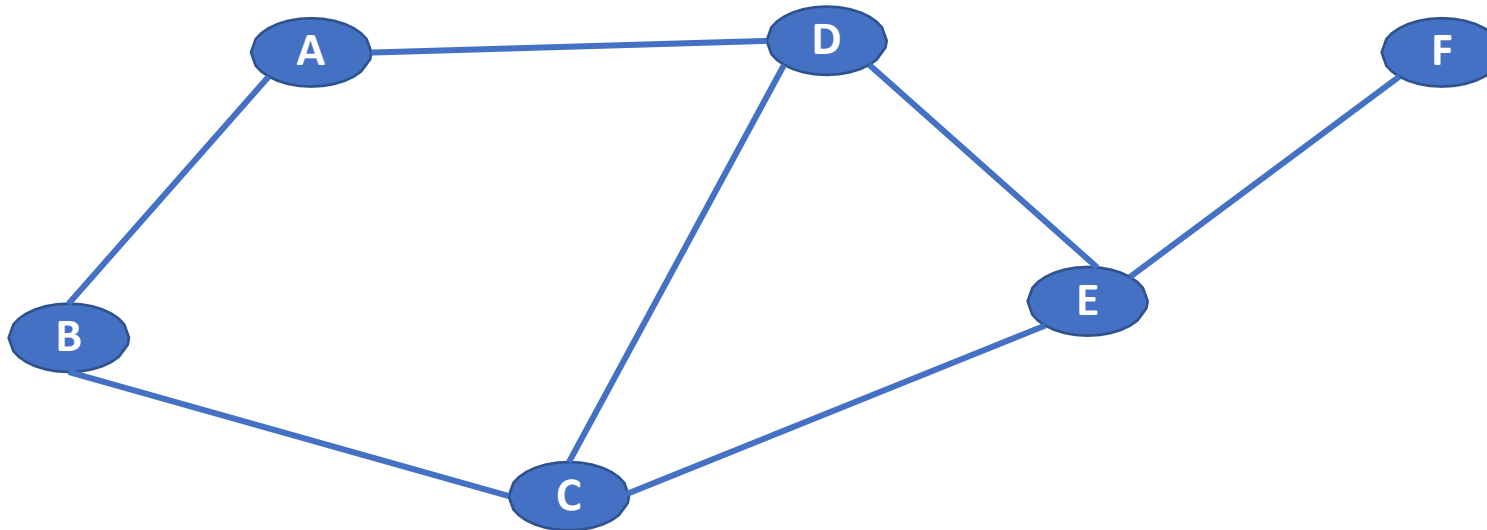
Out-degree(c)=1, in-degree of (c)=1

Directed graph:

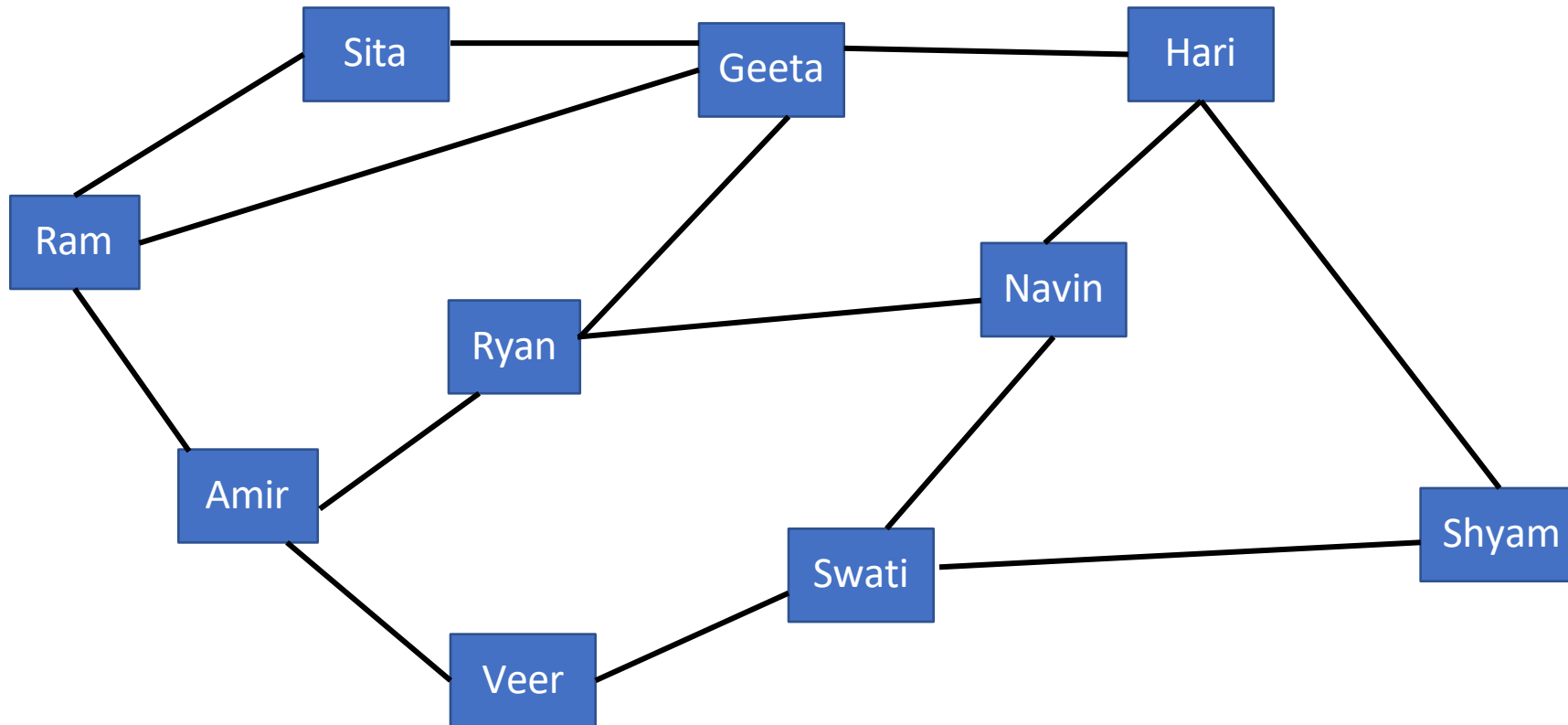
- The number of possible pairs in an m vertex graph is $m*(m-1)$
- The number of edges in an directed graph is $m*(m-1)$ since the edge(u, v) is not the same as the edge(v, u)
- The number of edges in an directed graph is $\leq m*(m-1)$

Undirected graph:

- The number of possible pairs in an m vertex graph is $m*(m-1)$
- The number of edges in an undirected graph is $m*(m-1)/2$ since the edge(u, v) is same as the edge(v, u)



Social Networking sites





THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963



DATA STRUCTURES AND ITS APPLICATIONS

Graph Traversal Methods

Sandesh B. J

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Graph Traversals Methods

Sandesh B. J

Department of Computer Science & Engineering

- Why we need to traverse the graph?
- Traversing the graph using different techniques
 - ✓ Depth First Search and Breadth First Search Traversal
- Algorithms for Depth First and Breadth First Search Traversal

- **Graph traversal**(also known as graph search) refers to the process of visiting/investigating (checking or updating) each vertex of the graph in some systematic order -Wiki
- Traversal can start in any arbitrary vertex
- Traversals are classified by the order in which the vertices are visited

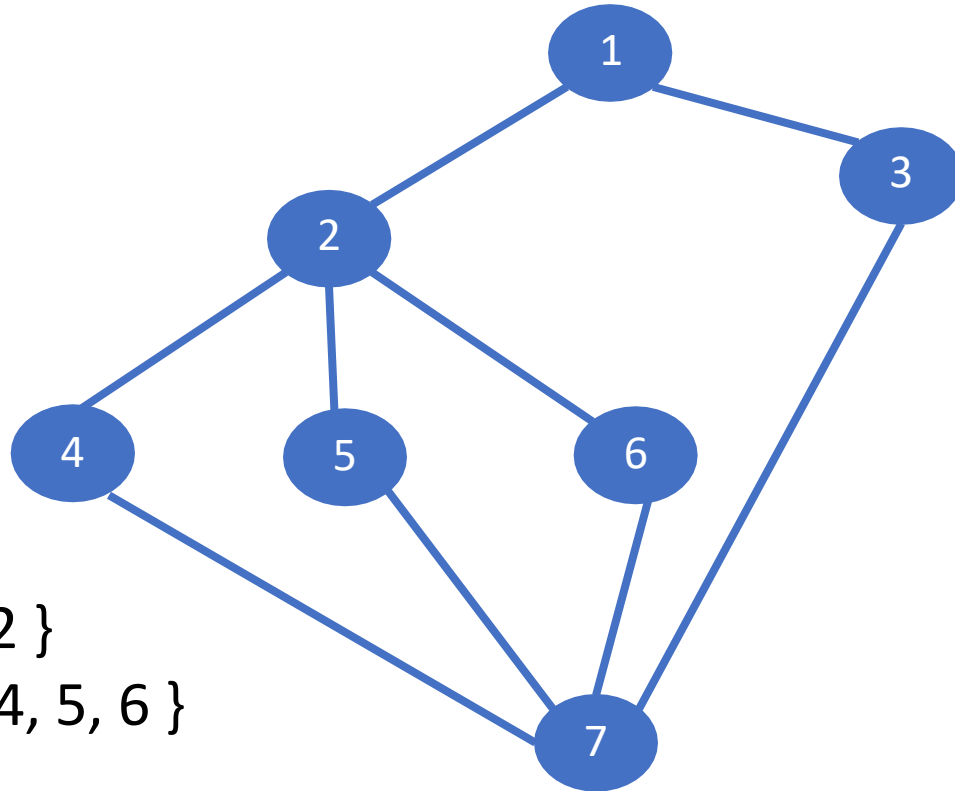
Methods to Traverse the Graph

- Depth First Search
- Breadth First Search

- Visits all the nodes related to one neighbour before visiting the other neighbours and its related nodes. Once it reaches dead end, it backtracks along previously visited node and continue traversing from there.
- Analogues to pre-order traversal of an ordered tree
- Uses stack behaviour, hence implemented using recursive algorithm

Depth First Search Traversal

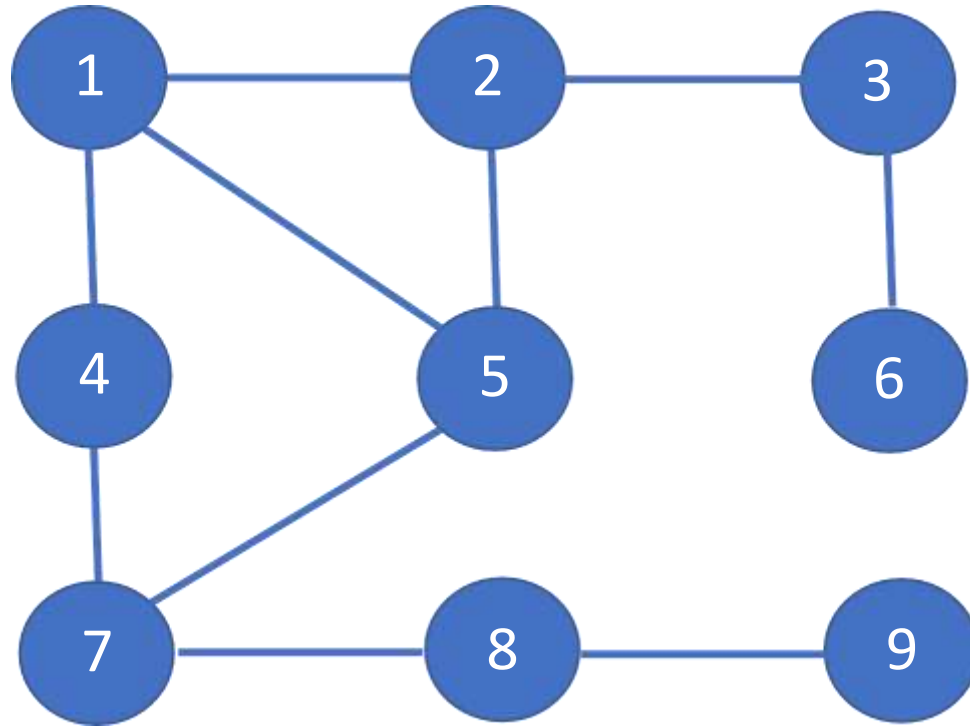
- DFS traverse the depth of the graph, until it cannot go any further at which point it backtracks and continues



$V = \{ 2 \}$

$W = \{ 4, 5, 6 \}$

Traversal order : 1 , 2, 4 , 7, 3, 5, 6



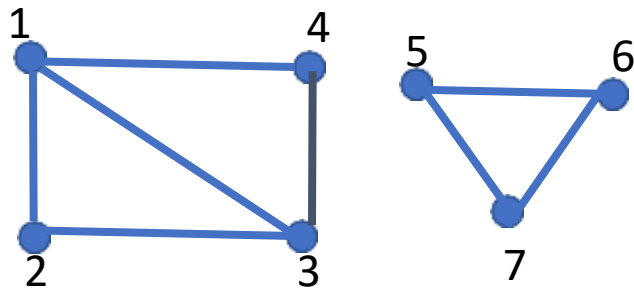
1 2 3 6 5 7 4 8 9

- **Graph may contain cycles**

- Traversal algorithm may reach the same vertex second time.
- To prevent the infinite recursion , we introduce Boolean valued array **visited**
- Set the **visited[v] to true** once node v is visited
- Check the **visited[w]** before processing any node w

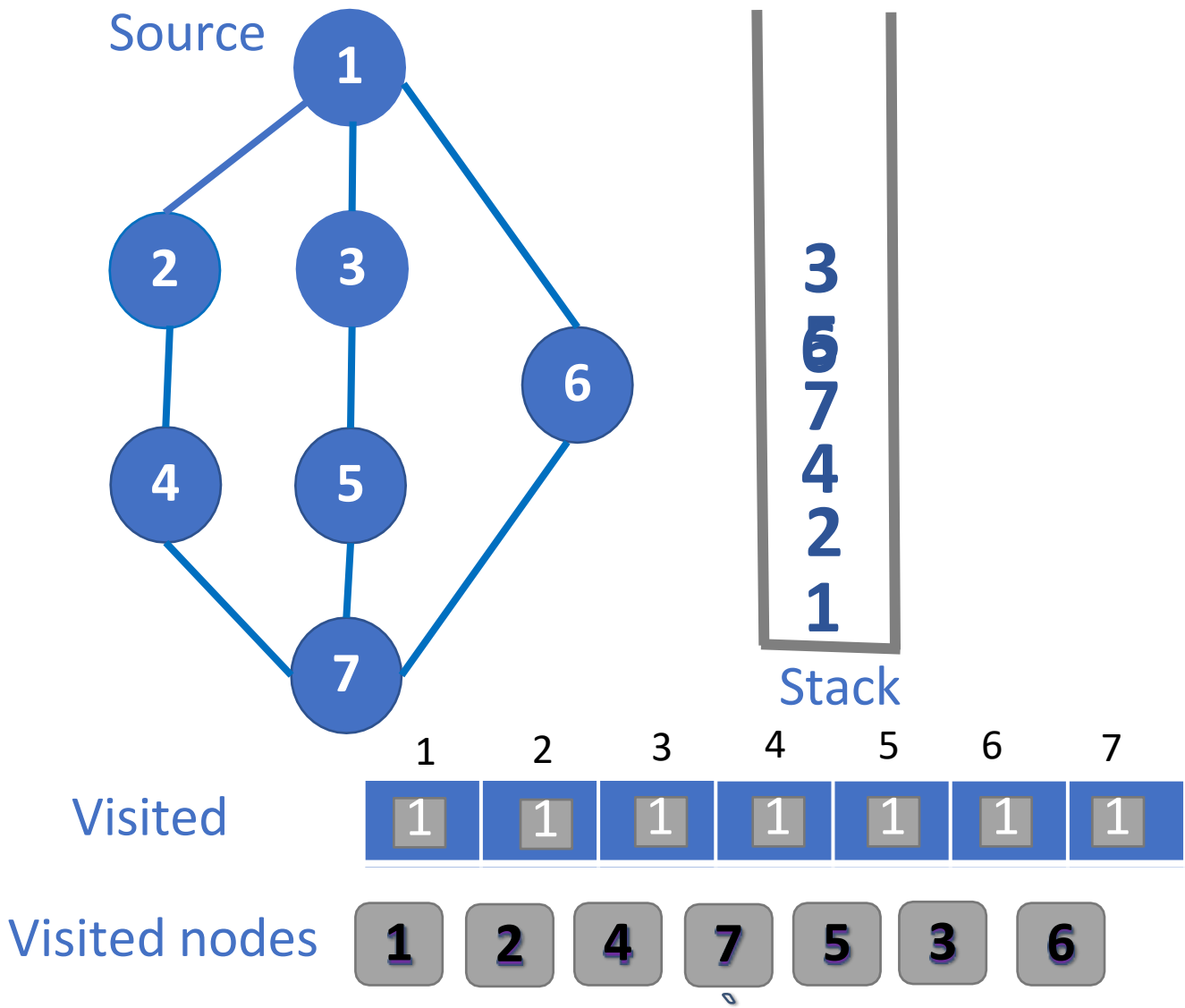
- **Graph may not be connected:**

- Traversal algorithm may fail to reach all the nodes from a single starting point



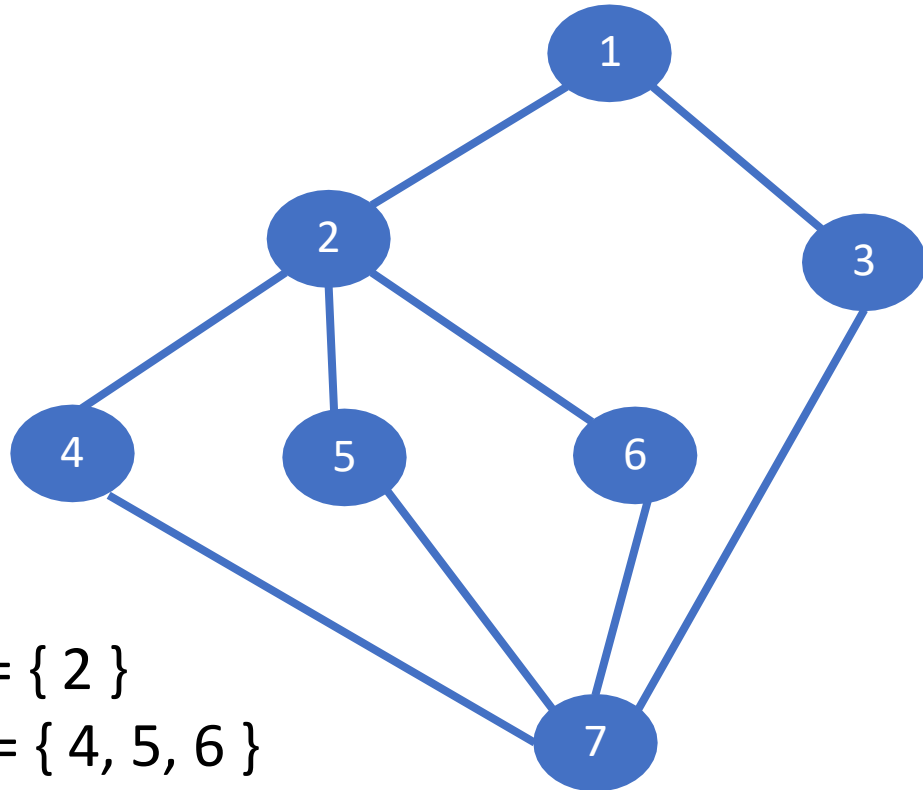
DATA STRUCTURES AND ITS APPLICATIONS

DFS Traversal – Using Stack



- Explores all the neighbour nodes in first level from an arbitrary node, before moving to next level of neighbouring nodes.
- Uses Queue behaviour

- Analogous to level-by-level traversal of ordered tree



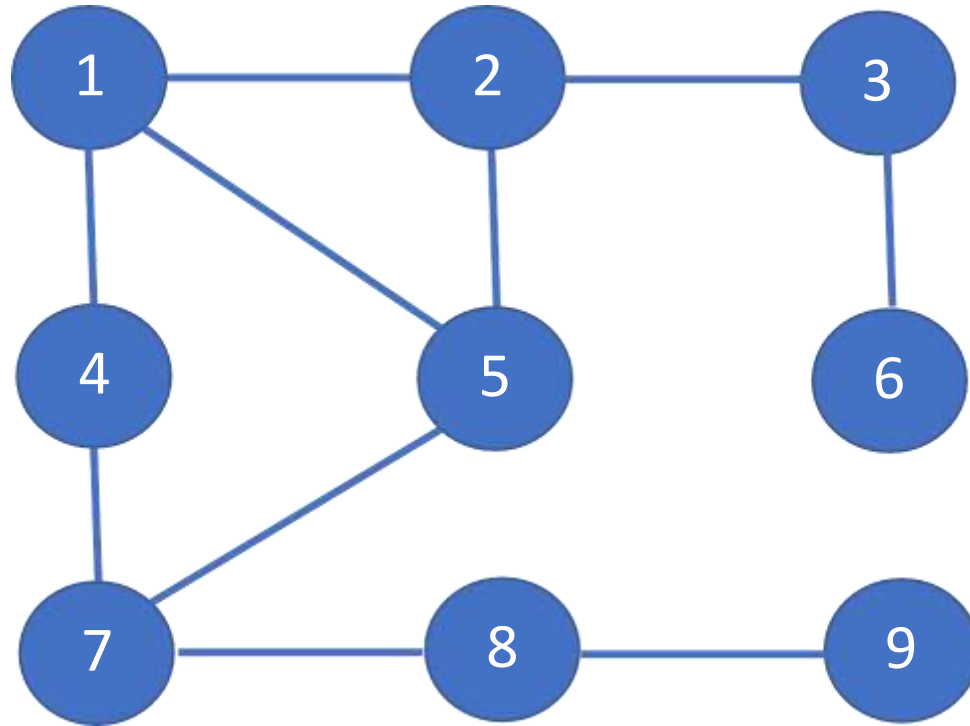
$V = \{ 2 \}$

$W = \{ 4, 5, 6 \}$

Traversal order : 1 , 2, 3, 4, 5, 6 , 7

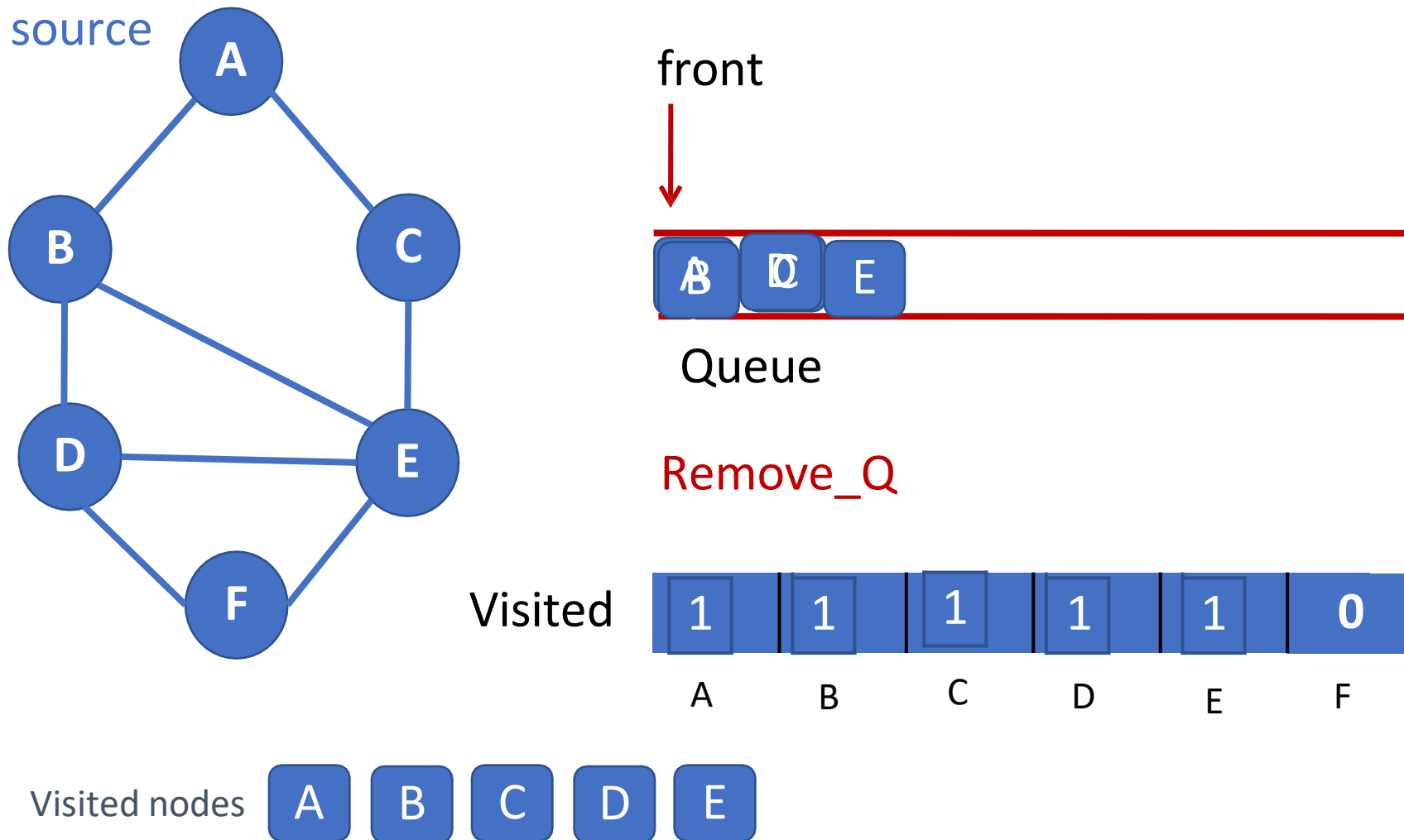
DATA STRUCTURES AND ITS APPLICATIONS

Breadth first search Traversal



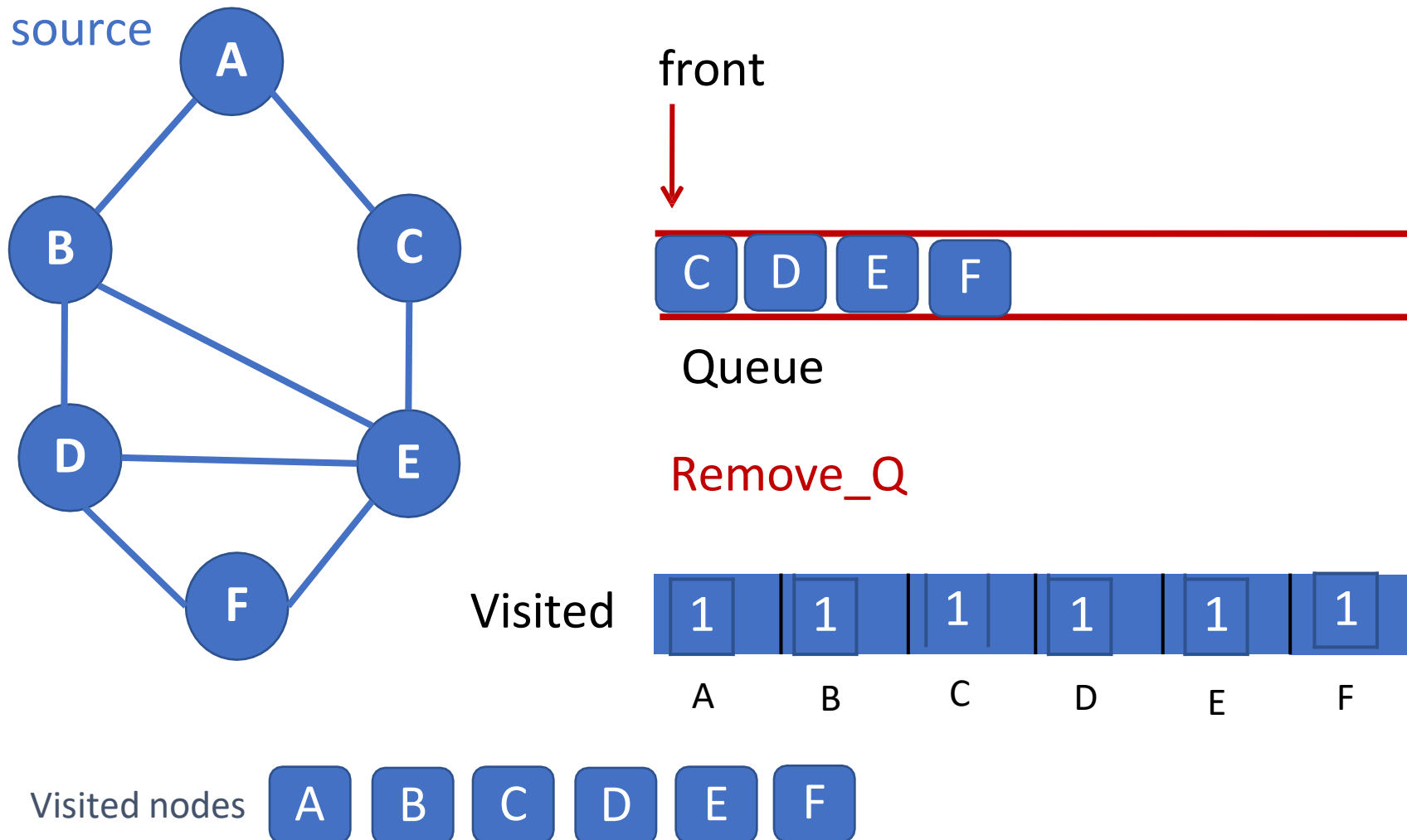
DATA STRUCTURES AND ITS APPLICATIONS

BFS Traversal – Using Queue



DATA STRUCTURES AND ITS APPLICATIONS

BFS – Traversal – Using Queue



DATA STRUCTURES AND ITS APPLICATIONS

DFS - Algorithm



```
Algorithm DFS (Graph G )  
//Implements DFS traversal for given graph  
// Input Graph G = (V, E)  
//Output Graph G with vertices marked as visited  
mark each vertex in V with 0 as a mark of being “unvisited”  
for each vertex v in V do  
    if v is marked with 0  
        dfs(v)
```

```
dfs(v)  
// visits recursively all the unvisited vertices connected to  
vertex v by a path  
For each vertex w in V adjacent to v do  
    if w is marked with 0  
        mark w as visited  
        dfs(w)
```

Courtesy: “Introduction to Design and Analysis of Algorithms” By Amany Levitin

```
Algorithm BFS (Graph G )  
//Implements BFS traversal for given graph  
// Input Graph G = (V, E)  
//Output Graph G with vertices marked as visited  
mark each vertex in V with 0 as a mark of being “unvisited”  
  for each vertex v in V do  
    if v is marked with 0  
      bfs(v)
```

```
bfs(v)
// visits recursively all the unvisited vertices connected to
// vertex v by a path
While the queue is not empty
  for each vertex w in V adjacent to front vertex do
    if w is marked with 0
      mark w as visited
      add w to queue
  remove the front vertex from the queue
```



THANK YOU

Sandesh B. J

Department of Computer Science & Engineering

sandesh_bj@pes.edu

+91 80 6618 6623



DATA STRUCTURES AND ITS APPLICATIONS

Graphs

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Representation of Network Topology

Saritha

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology

Graph data structure is mainly in Computer Networks, Telecommunication, Electronic Circuits and Transport Networks.

Networking uses the Notation $G(N,L)$ instead of $G(V,E)$ for a graph where N is the set of nodes and L is the set of links.



DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology



- Topology is the order in which nodes and edges are arranged in the network.
- How the computers are connected or related to one another in a computer.
- There are 2 types of Topology
 1. Physical
 2. Logical

DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology

- 1. Ring Topology
- 2. Star Topology
- 3. Mesh Topology
- 4. Bus Topology

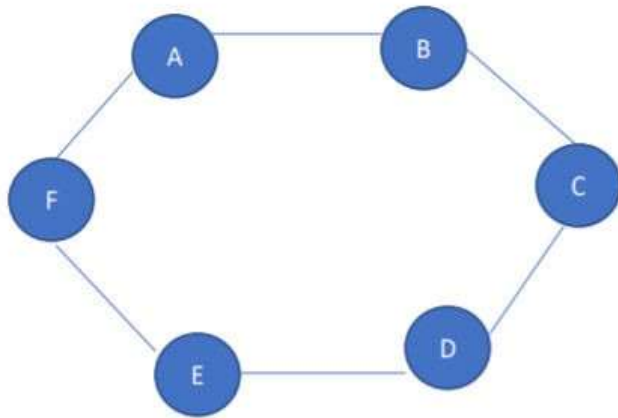


Representation of Graph

1. Adjacency Matrix

2. Adjacency List

1. Ring topology (cycle): A cycle graph is a simple graph which has two degrees of vertices.

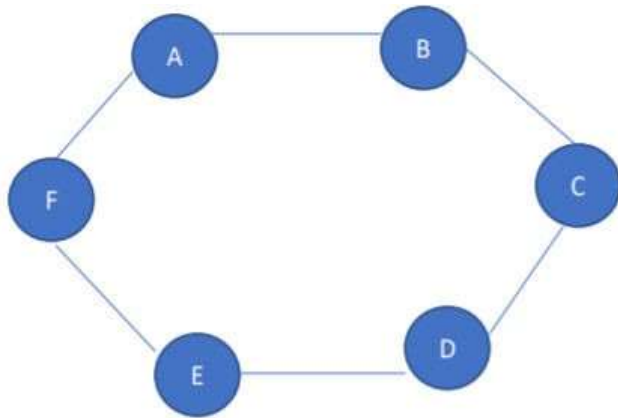


Ring topology

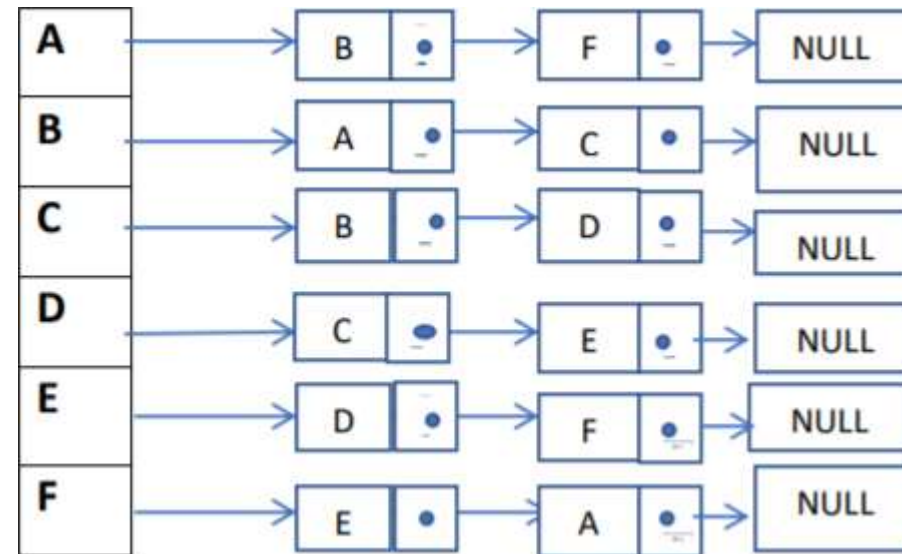
	A	B	C	D	E	F
A	0	1	0	0	0	1
B	1	0	1	0	0	0
C	0	1	0	1	0	0
D	0	0	1	0	1	0
E	0	0	0	1	0	1
F	1	0	0	0	1	0

DATA STRUCTURES AND ITS APPLICATIONS

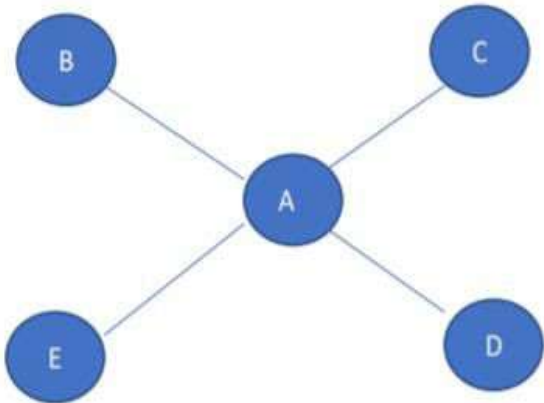
Applications: Network Topology



Ring topology

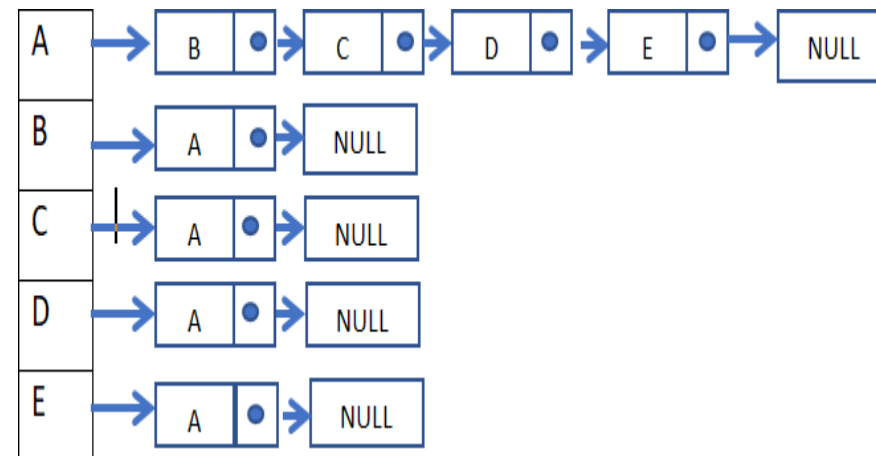


2.Star topology: Star topology is a network topology in the form of merging from the central vertex to each vertex .

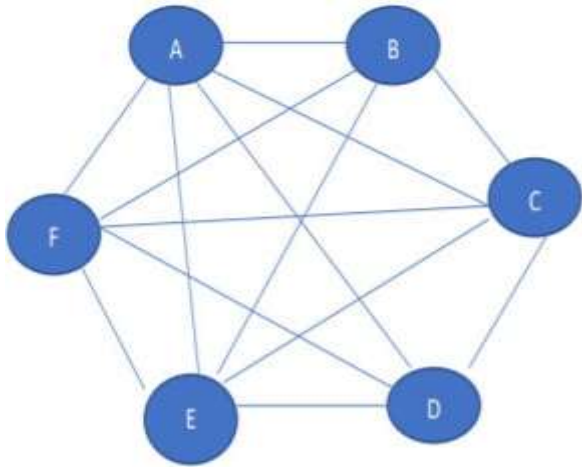


Star topology

	A	B	C	D	E
A	0	1	1	1	1
B	1	0	0	0	0
C	1	0	0	0	0
D	1	0	0	0	0
E	1	0	0	0	0



3.Mesh topology: Mesh Topology is a complete graph in which all the vertex is connected to all other vertices

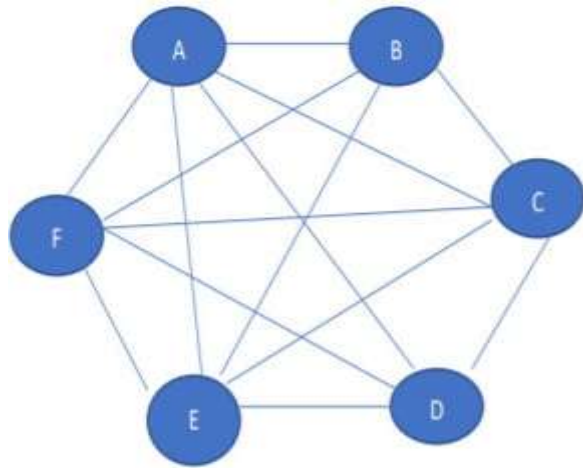


Mesh Topology

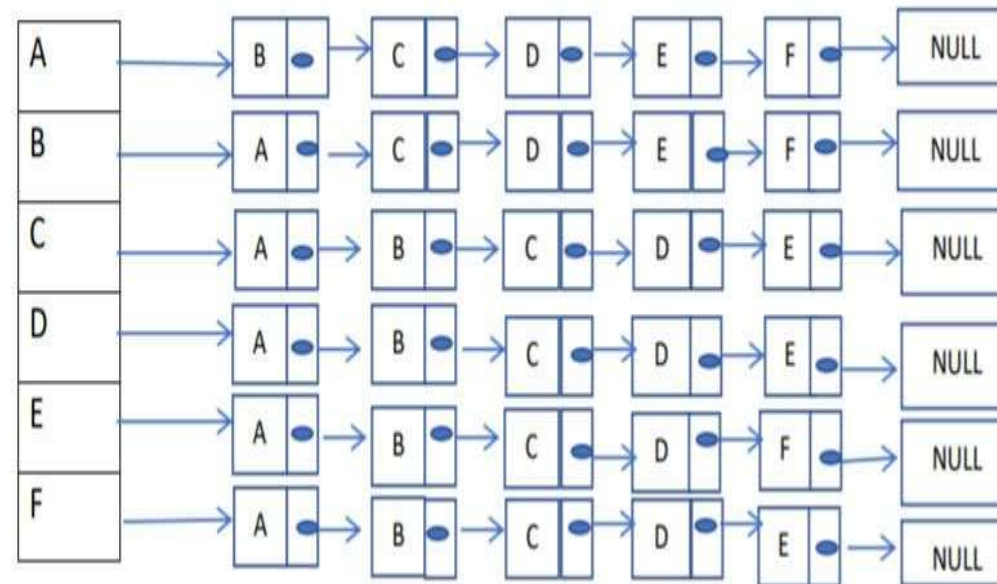
	A	B	C	D	E	F
A	0	1	1	1	1	1
B	1	0	1	1	1	1
C	1	1	0	1	1	1
D	1	1	1	0	1	1
E	1	1	1	1	0	1
F	1	1	1	1	1	0

DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology

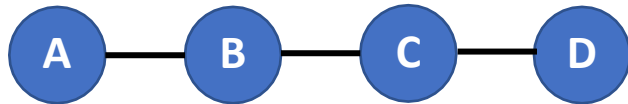


Mesh Topology



A Graph G with V vertices is said to represent a bus topology if

1. Every node except the starting and ending node has degree 2 and starting and ending node have degree 1.
2. Number of edges = Number of vertices - 1

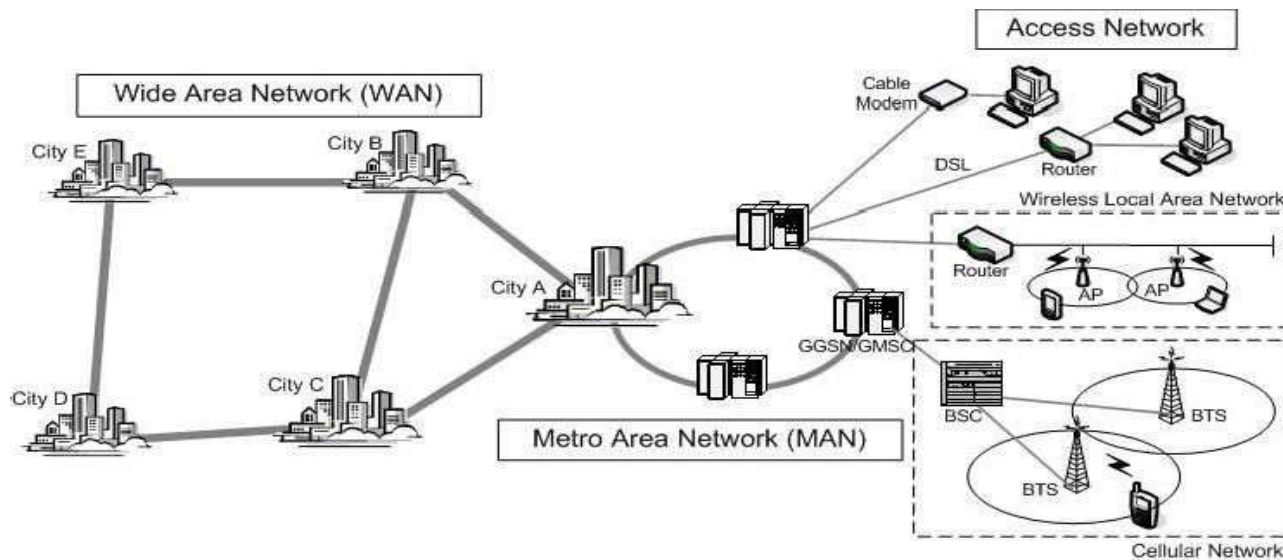


	A	B	C	D
A	0	1	0	0
B	1	0	1	0
C	0	1	0	1
D	0	0	1	0

DATA STRUCTURES AND ITS APPLICATIONS

Applications: Network Topology

Most networks a mix of rings, mesh – depending on network type, cost/traffic/reliability





THANK YOU

Saritha

Department of Computer Science & Engineering

Saritha.k@pes.edu

9844668963



DATA STRUCTURES AND ITS APPLICATIONS

UE22CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Trees

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Tree



//Returns the smallest element in Binary Search Tree

```
int minimum(struct tnode *t)
{
    while(t->left!=NULL)
        t=t->left;
    return(t->data);
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Tree



//Returns the largest element in Binary Search Tree

```
int maximum(struct tnode *t)
{
    while(t->right!=NULL)
        t=t->right;
    return(t->data);
}
```

//Computes the height of a Binary Tree

```
int height(struct tnode *t)
{
    if(t==NULL)
        return -1;
    if((t->left==NULL)&&(t->right==NULL))
        return 0;
    return (1+max(height(t->left),height(t->right)));
}
```


//Count the number of leaf nodes in a Binary Tree

```
int leafcount(struct tnode *t)
{
    if(t==NULL)
        return 0;
    if((t->left==NULL)&&(t->right==NULL))
        return 1;
    int l=leafcount(t->left);
    int r=leafcount(t->right);
    return(l+r);
}
```



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE22CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal

Shylaja S S

Department of Computer Science & Engineering

Important operation: Traversal

Traversal: Moving through all the nodes in a binary tree and visiting each one in turn

Trees: There are many orders possible since it is a nonlinear DS

Tasks: 1. Visiting a node denoted by V

2. Traversing the left subtree denoted by L

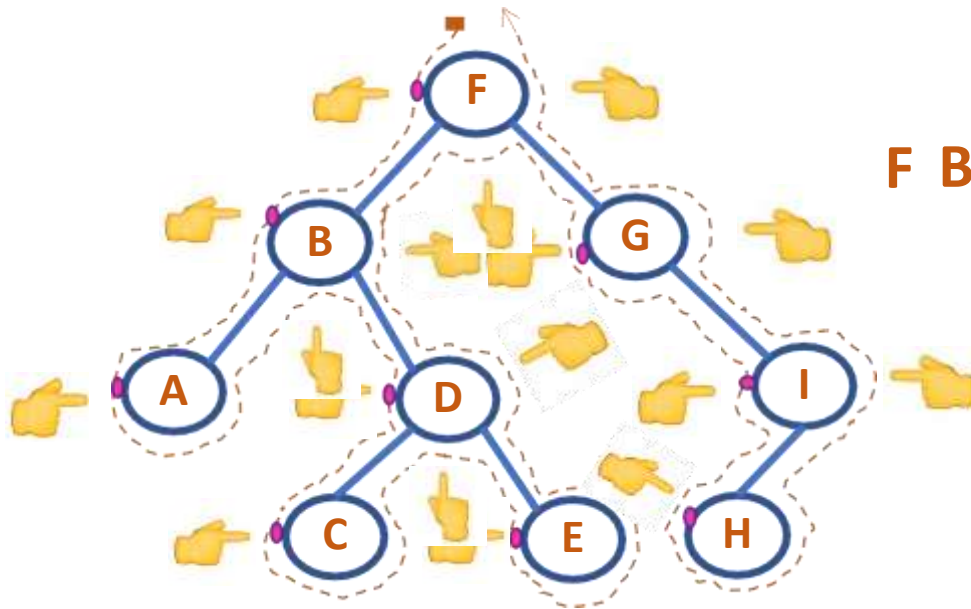
3. Traversing the right subtree denoted by R

Six ways to arrange them: VLR, LVR, LRV, VRL, RVL, RLV

Standard Traversals include: VLR-Preorder, LVR-Inorder,
LRV-Postorder

Steps:

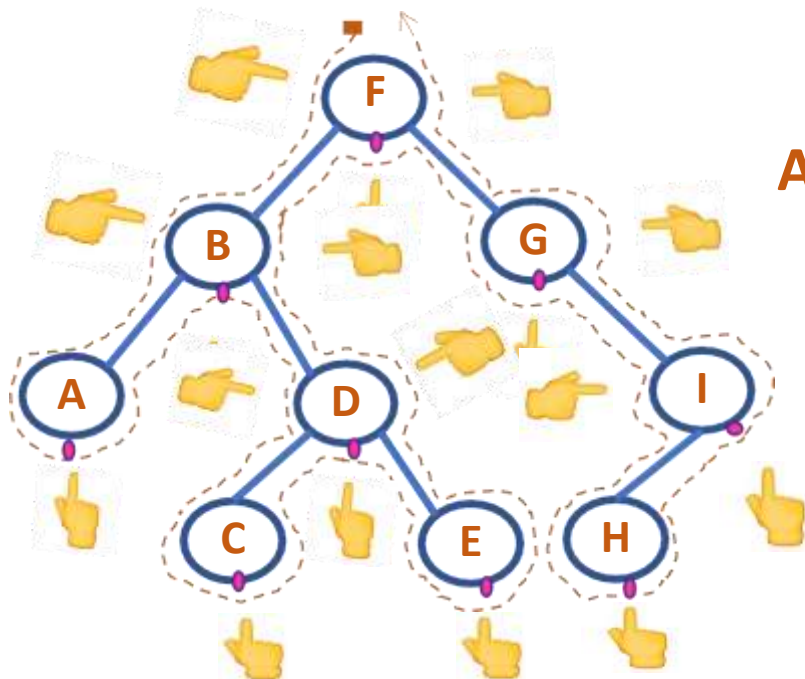
- Root Node is visited before the subtrees
- Left subtree is traversed in preorder
- Right subtree is traversed in preorder



F B A D C E G I H

Steps:

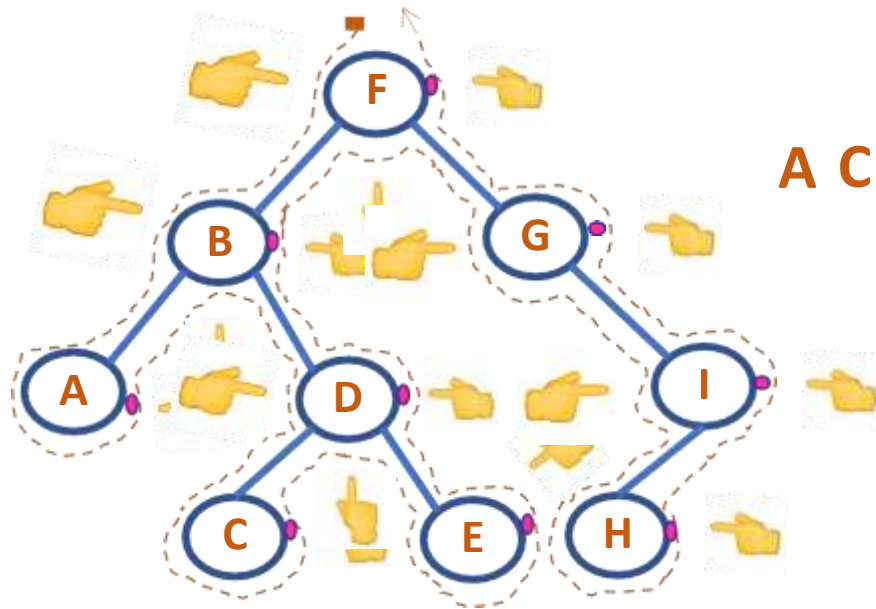
- Left subtree is traversed in Inorder
- Root Node is visited
- Right subtree is traversed in Inorder



A B C D E F G H I

Steps:

- Left subtree is traversed in postorder
- Right subtree is traversed in postorder
- Root Node is visited



A C E D B H I G F

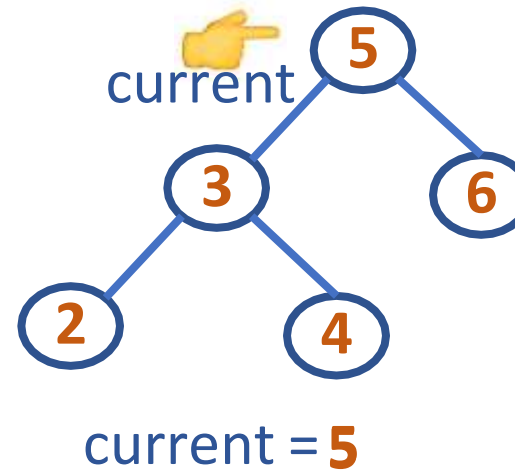

```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null)
    {
        /* Travel down left branches as far as possible
           saving pointers to nodes passed in the stack*/
        push(s, current)
        current = current->left
    } //At this point, the left subtree is empty
    poppedNode = pop(s)
    print poppedNode ->info      //visit the node
    current = poppedNode ->right //traverse right subtree
} while(!isEmpty(s) or current != null)
```

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null)
    {
        push(s, current)
        current = current->left
    }
    poppedNode = pop(s)
    print poppedNode ->info
    current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```

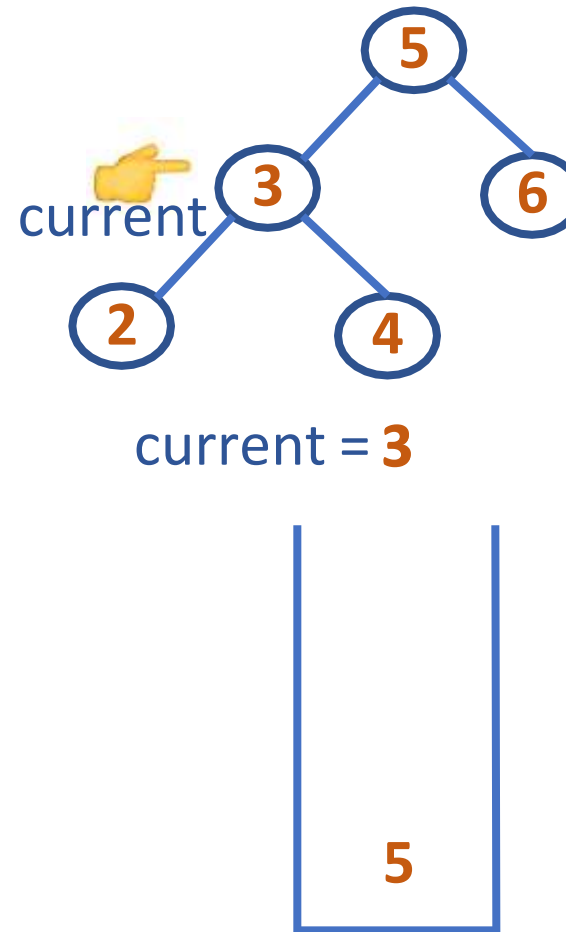
Note: Stack has Address of Nodes Pushed In



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

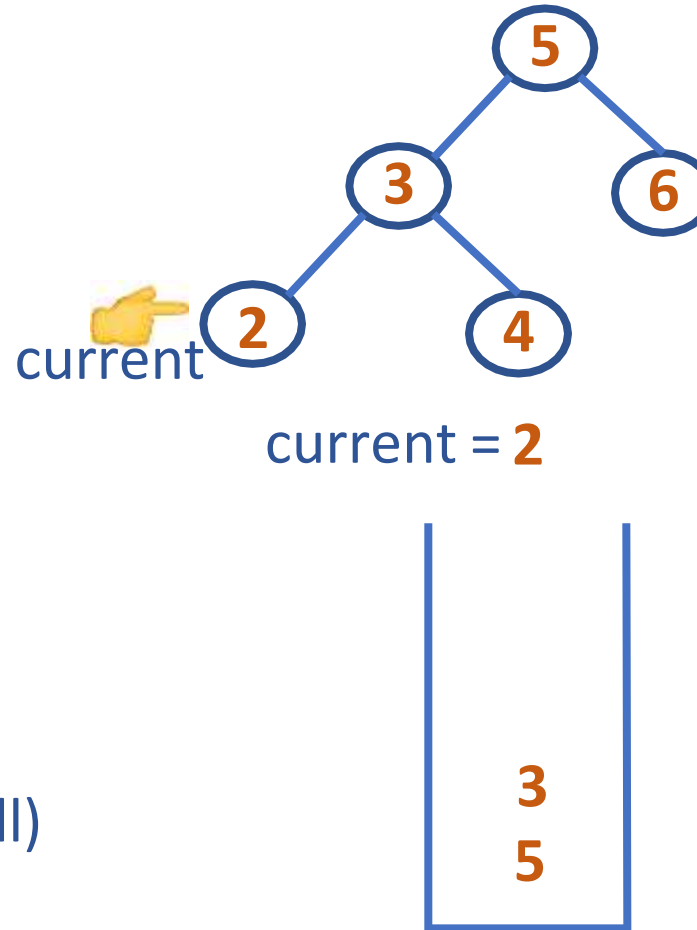
```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null)
    {
        push(s, current)
        current = current->left
    }
    poppedNode = pop(s)
    print poppedNode ->info
    current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

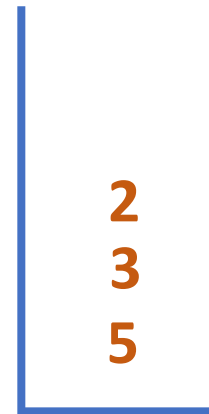
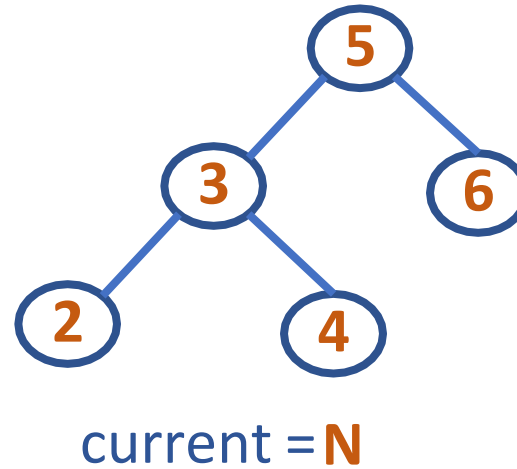
```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null)
    {
        push(s, current)
        current = current->left
    }
    poppedNode = pop(s)
    print poppedNode ->info
    current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

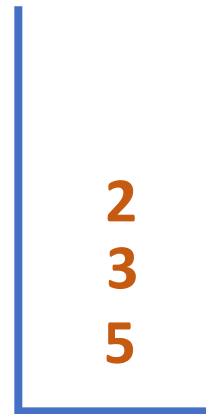
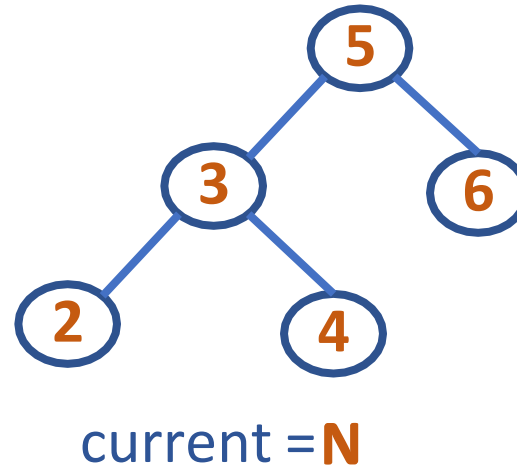
```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null) 📌
    {
        push(s, current)
        current = current->left 📌
    }
    poppedNode = pop(s)
    print poppedNode ->info
    current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null)
    {
        push(s, current)
        current = current->left
    }
    poppedNode = pop(s) ➡ poppedNode =
    print poppedNode ->info ➡
    current = poppedNode ->right ➡
} while(!isEmpty(s) or current != null) ➡
```



Inorder Traversal:

2

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
```

```
s = emptyStack
```

```
current = root
```

```
do {
```

```
    while(current != null)
```

```
    {
```

```
        push(s, current)
```

```
        current = current->left
```

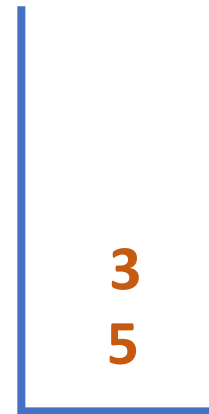
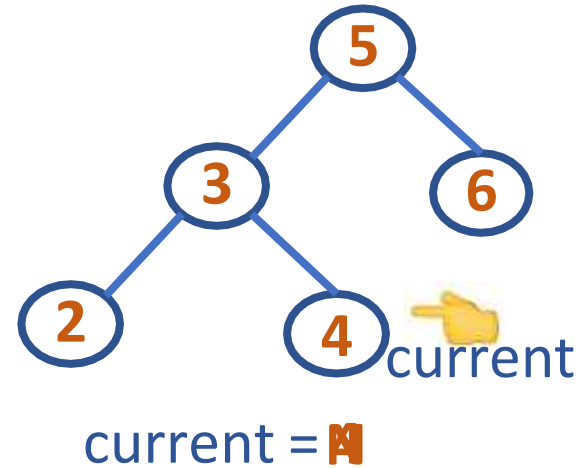
```
    }
```

```
    poppedNode = pop(s)
```

```
    print poppedNode ->info
```

```
    current = poppedNode ->right
```

```
} while(!isEmpty(s) or current != null)
```



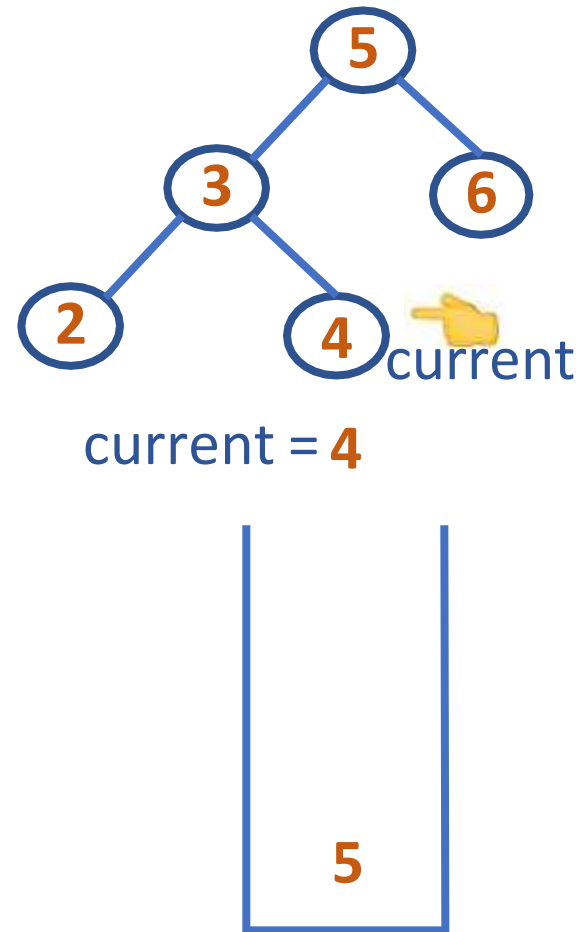
Inorder Traversal:

2 3

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do { 
    while(current != null) 
    {
        push(s, current) 
        current = current->left
    }
    poppedNode = pop(s)
    print poppedNode ->info
    current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



Inorder Traversal:

2 3

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
```

```
s = emptyStack
```

```
current = root
```

```
do {
```

```
    while(current != null) 🖱️
```

```
    {
```

```
        push(s, current)
```

```
        current = current->left 🖱️
```

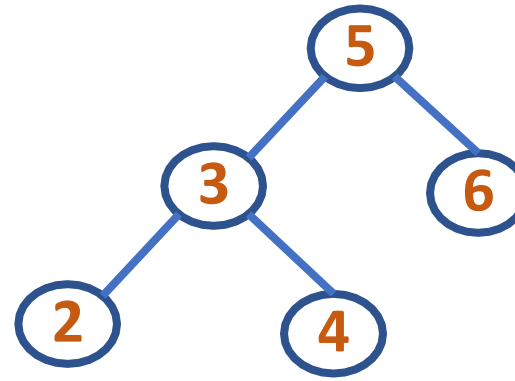
```
    }
```

```
    poppedNode = pop(s) 🖱️
```

```
    print poppedNode ->info 🖱️
```

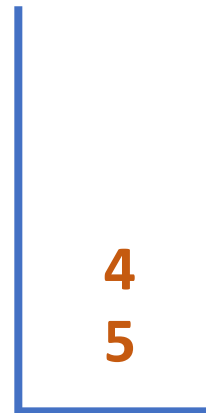
```
    current = poppedNode ->right 🖱️
```

```
} while(!isEmpty(s) or current != null) 🖱️
```



current = **N**

poppedNode = **3**



Inorder Traversal:

2 3 4

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
```

```
s = emptyStack
```

```
current = root
```

```
do {
```

```
    while(current != null)
```

```
    {
```

```
        push(s, current)
```

```
        current = current->left
```

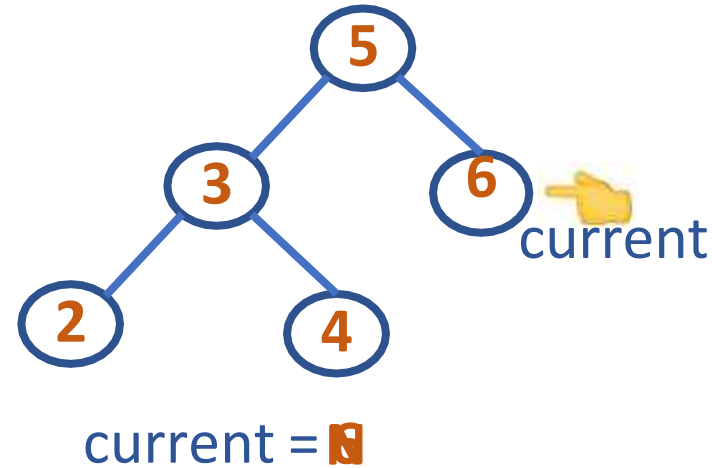
```
    }
```

```
    poppedNode = pop(s)
```

```
    print poppedNode ->info
```

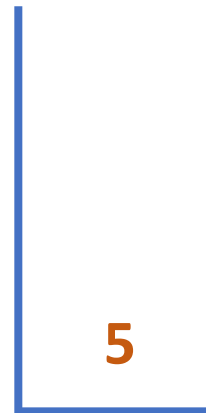
```
    current = poppedNode ->right
```

```
} while(!isEmpty(s) or current != null)
```



Inorder Traversal:

2 3 4 5

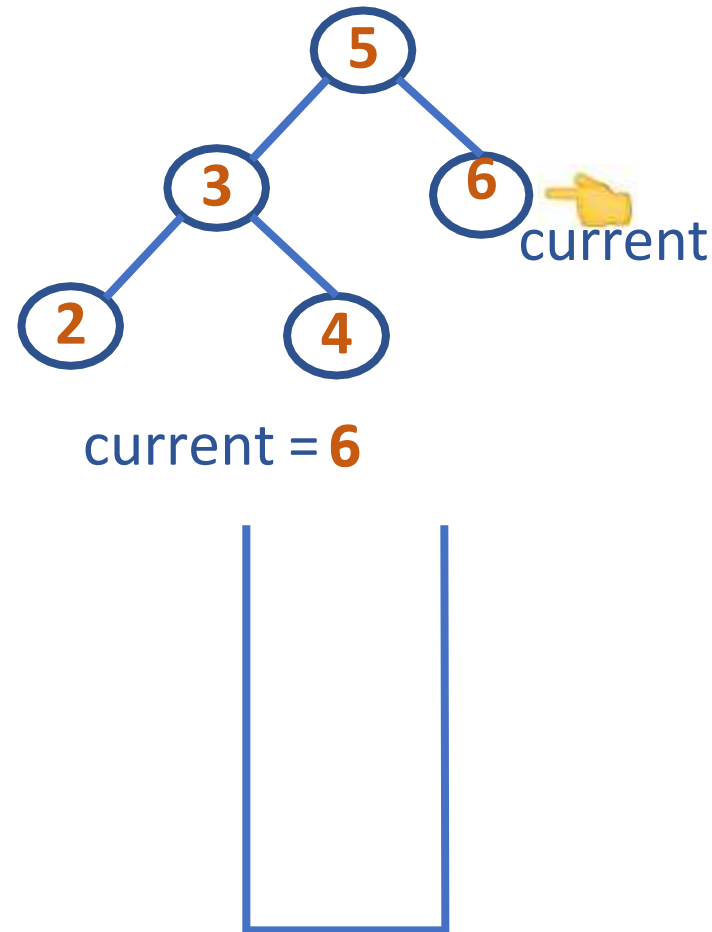


PES
UNIVERSITY
ONLINE

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null)
    {
        push(s, current)
        current = current->left
    }
    poppedNode = pop(s)
    print poppedNode ->info
    current = poppedNode ->right
} while(!isEmpty(s) or current != null)
```



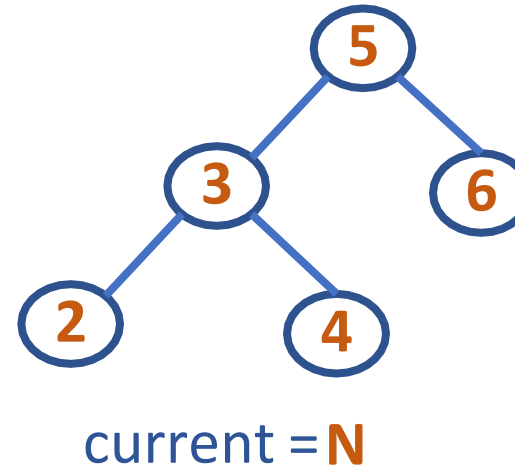
Inorder Traversal:

2 3 4 5

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Inorder Traversal

```
iterativeInorder(root)
s = emptyStack
current = root
do {
    while(current != null) 📌
    {
        push(s, current)
        current = current->left 📌
    }
    poppedNode = pop(s) 📌
    print poppedNode ->info 📌
    current = poppedNode ->right 📌
} while(!isEmpty(s) or current != null) 📌
```



Inorder Traversal:

2 3 4 5 6

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal

```
iterativePreorder(root)
current=root
if (current == null)
    return
s = emptyStack
push(s, current)
while(!isEmpty(s)) {
    current = pop(s)
    print current->info
    //right child is pushed first so that left is processed first
    if(current->right !=NULL)
        push(s, current->right)
    if(current->left !=NULL)
        push(s, current->left)
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal

```
iterativePreorder(root)
```

```
current=root
```

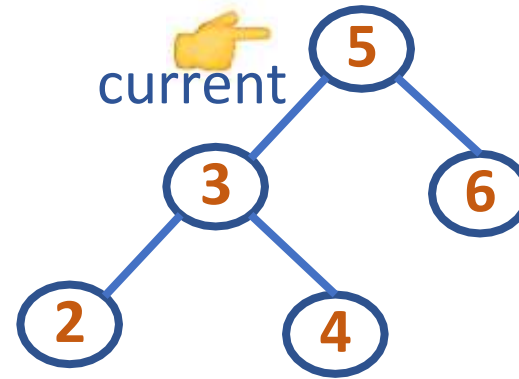
```
if (current == null)
```

```
    return
```

```
s = emptyStack
```

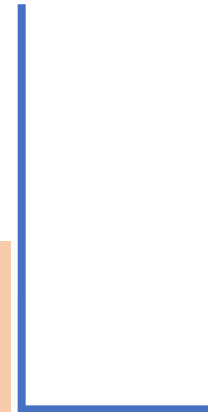
```
push(s, current)
```

```
    ...
```



current = 5

Note: Stack has Address
of Nodes Pushed In



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal

iterativePreorder(root)

⋮

while (!isEmpty(s))



{

current = pop(s)



print current ->info



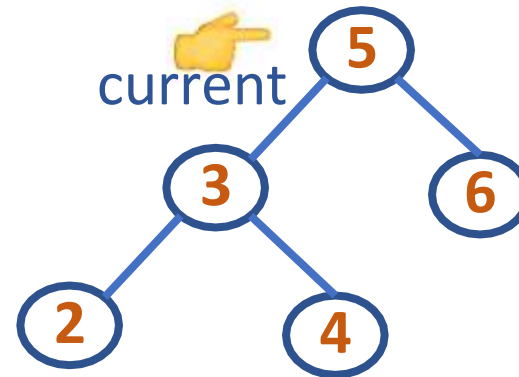
if(current->right != null)

push(s, current->right)

if(current->left != null)

push(s, current->left)

}



current = 5



Preorder Traversal:

5

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

```
while (!isEmpty(s))
```

```
{
```

```
    current = pop(s)
```

```
    print current ->info
```

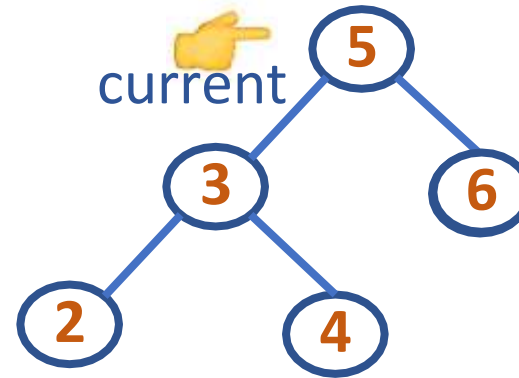
```
    if(current->right != null) ➡
```

```
        push(s, current->right) ➡
```

```
    if(current->left != null)
```

```
        push(s, current->left)
```

```
}
```



current = 5

current->right = 6



Preorder Traversal:

5

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

```
while (!isEmpty(s))
```

```
{
```

```
    current = pop(s)
```

```
    print current ->info
```

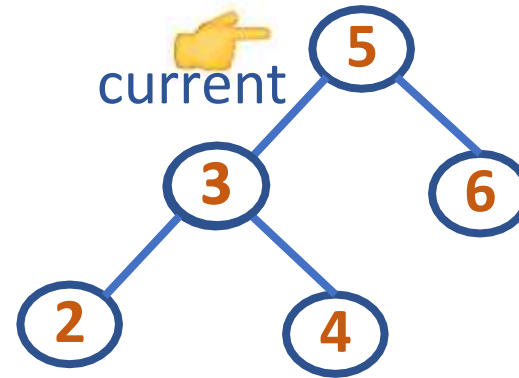
```
    if(current->right != null)
```

```
        push(s, current->right)
```

```
    if(current->left != null) ➡
```

```
        push(s, current->left) ➡
```

```
}
```



current = 5

current->left = 3



Preorder Traversal:

5

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

while (!isEmpty(s))

{

current = pop(s)

print current ->info

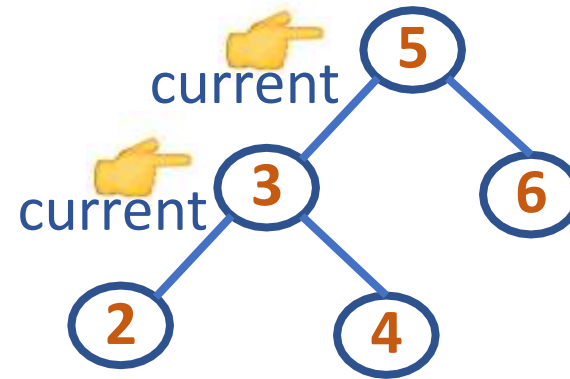
if(current->right != null)

push(s, current->right)

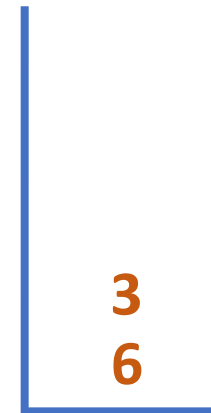
if(current->left != null)

push(s, current->left)

}



current = 3



Preorder Traversal:

5 3

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

```
while (!isEmpty(s))
```

```
{
```

```
    current = pop(s)
```

```
    print current ->info
```

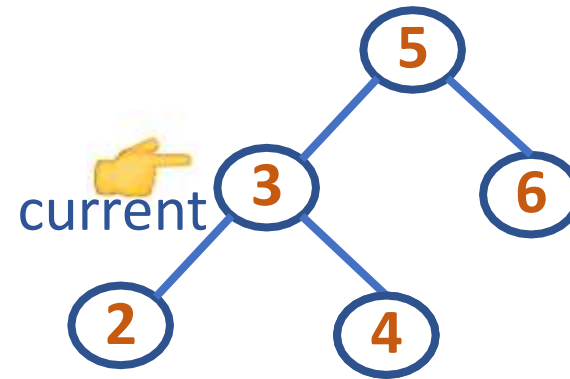
```
    if(current->right != null) ➡
```

```
        push(s, current->right) ➡
```

```
    if(current->left != null)
```

```
        push(s, current->left)
```

```
}
```



current = 3

current->right = 4



Preorder Traversal:

5 3

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

```
while (!isEmpty(s))
```

```
{
```

```
    current = pop(s)
```

```
    print current ->info
```

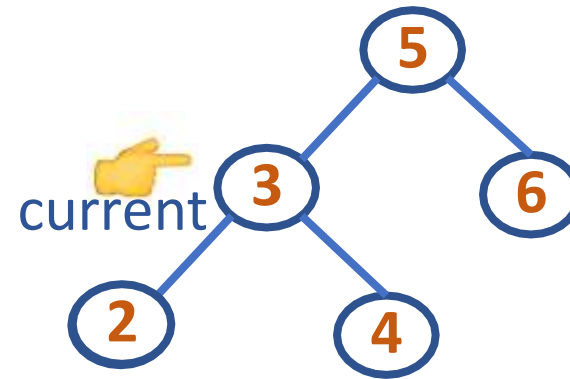
```
    if(current->right != null)
```

```
        push(s, current->right)
```

```
    if(current->left != null) ➡
```

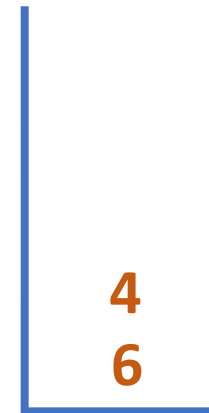
```
        push(s, current->left) ➡
```

```
}
```



current = 3

current->left = 2



Preorder Traversal:

5 3

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

while (!isEmpty(s)) 🖱️

{

current = pop(s) 🖱️

print current ->info 🖱️

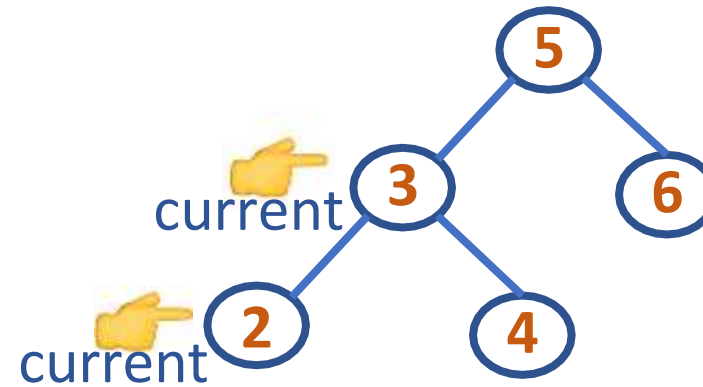
if(current->right != null)

push(s, current->right)

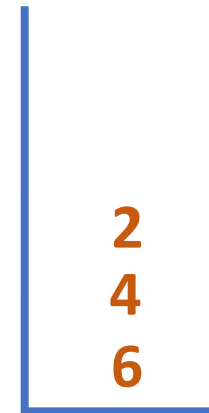
if(current->left != null)

push(s, current->left)

}



current = 3



Preorder Traversal:

5 3 2

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

```
while (!isEmpty(s))
```

```
{
```

```
    current = pop(s)
```

```
    print current ->info
```

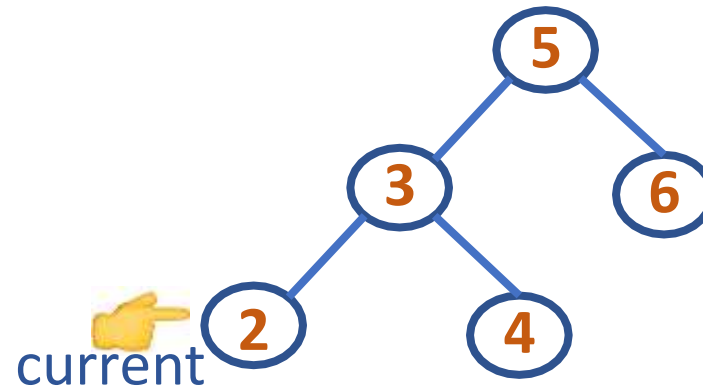
```
    if(current->right != null) ➡
```

```
        push(s, current->right)
```

```
    if(current->left != null)
```

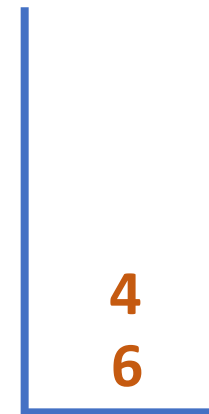
```
        push(s, current->left)
```

```
}
```



current = 2

current->right = N



Preorder Traversal:

5 3 2

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

```
while (!isEmpty(s))
```

```
{
```

```
    current = pop(s)
```

```
    print current ->info
```

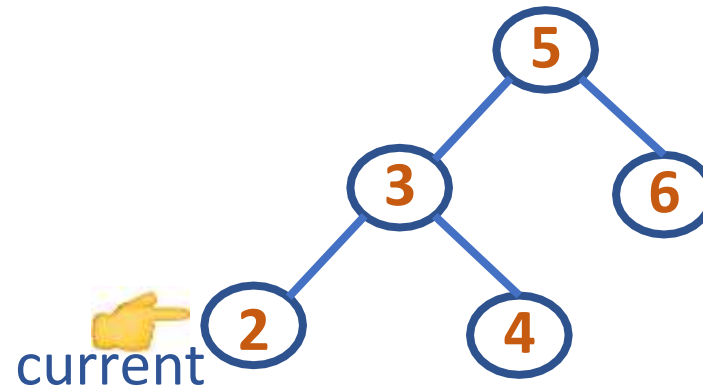
```
    if(current->right != null)
```

```
        push(s, current->right)
```

```
    if(current->left != null) ➡
```

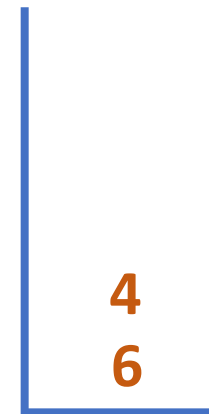
```
        push(s, current->left)
```

```
}
```



current = 2

current->left = N



Preorder Traversal:

5 3 2

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

while (!isEmpty(s)) 🖱️

{

current = pop(s) 🖱️

print current ->info 🖱️

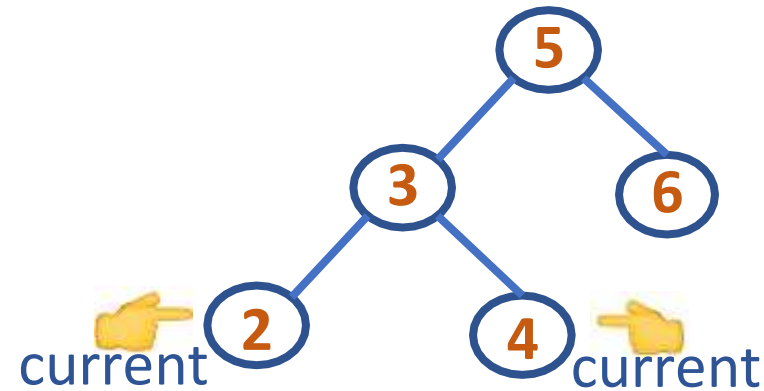
if(current->right != null)

push(s, current->right)

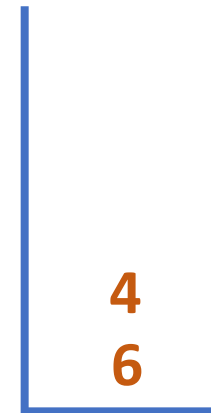
if(current->left != null)

push(s, current->left)

}



current = 2



Preorder Traversal:

5 3 2 4

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal



iterativePreorder(root)

⋮

```
while (!isEmpty(s))
```

```
{
```

```
    current = pop(s)
```

```
    print current ->info
```

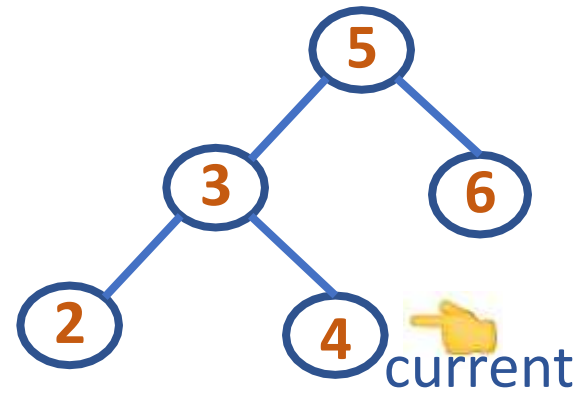
```
    if(current->right != null) ➡
```

```
        push(s, current->right)
```

```
    if(current->left != null) ➡
```

```
        push(s, current->left)
```

```
}
```



current = 4

current->right = N

current->left = N



Preorder Traversal:

5 3 2 4

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Preorder Traversal

iterativePreorder(root)

⋮

while (!isEmpty(s))



{

current = pop(s)



print current ->info



if(current->right != null)



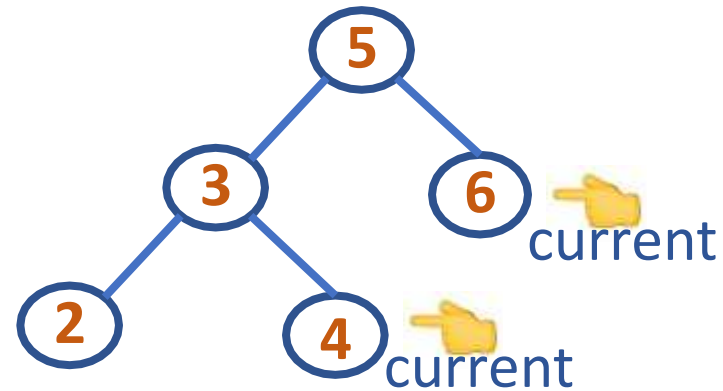
push(s, current->right)

if(current->left != null)



push(s, current->left)

}



Preorder Traversal:

5 3 2 4 6

current = **4**

current->right = **N**

current->left = **N**



```
iterativePostorder(root)
s1 = emptyStack ; s2 = emptyStack ; push(s1, root)
while(!isEmpty(s1)) {
    current = pop(s1)
    push(s2,current)
    if(current->left !=NULL)
        push(s1, current->left)
    if(current->right !=NULL)
        push(s1, current->right)
}
while(!isEmpty(s2)) { //Print all the elements of stack2
    current = pop(s2)
    print current->info
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

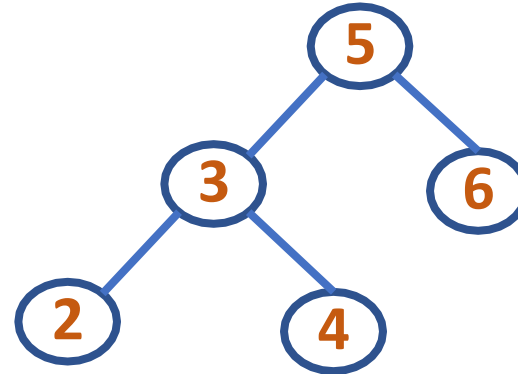
iterativePostorder(root)

s1 = emptyStack ➡

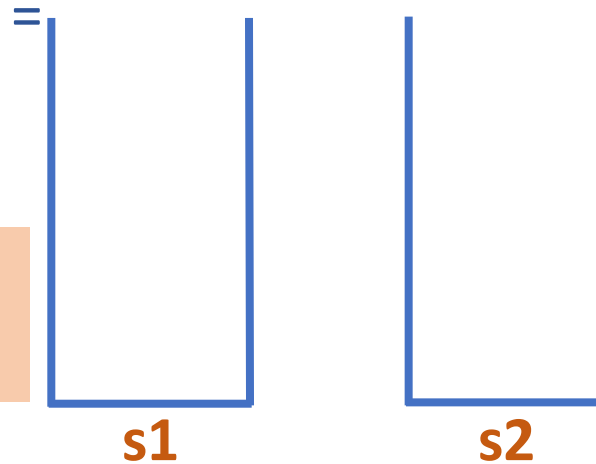
s2 = emptyStack ➡

push(s1, root) ➡

⋮



root 5



Note: Stacks have Address
of Nodes Pushed In

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```



```
{
```

```
    current = pop(s1)
```



```
    push(s2, current)
```

```
    if(current->left != NULL)
```

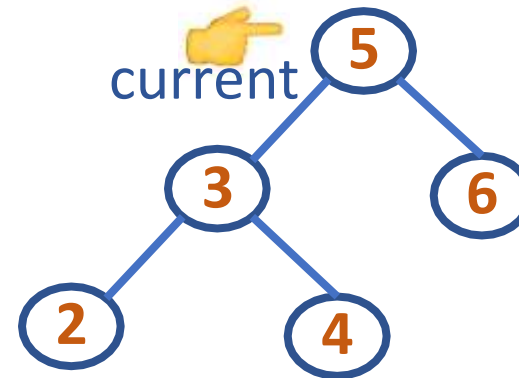
```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

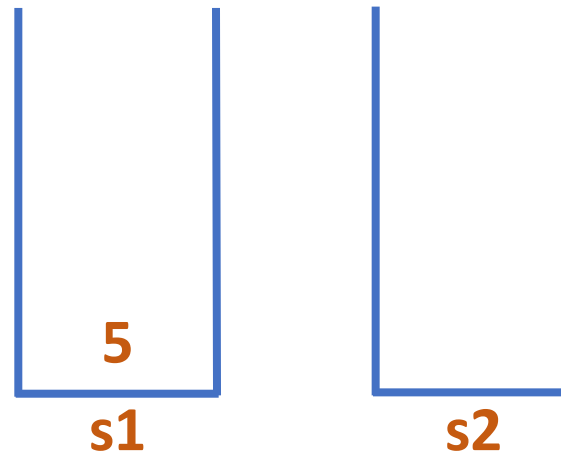
```
        push(s1, current->right)
```

```
}
```

...



current =



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

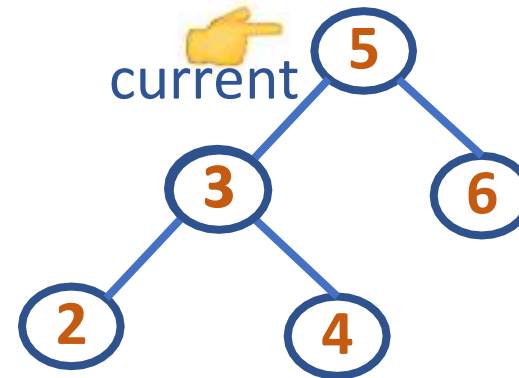
```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

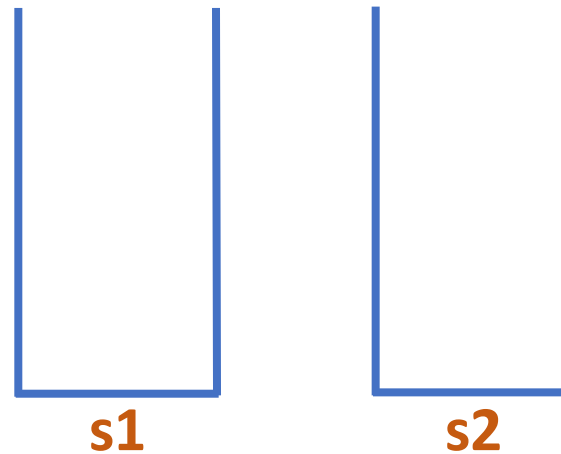
```
}
```

...

current->left = 3



current = 5



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

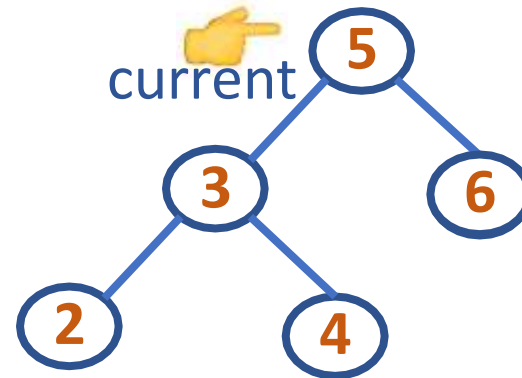
```
    if(current->right != NULL) ➡
```

```
        push(s1, current->right) ➡
```

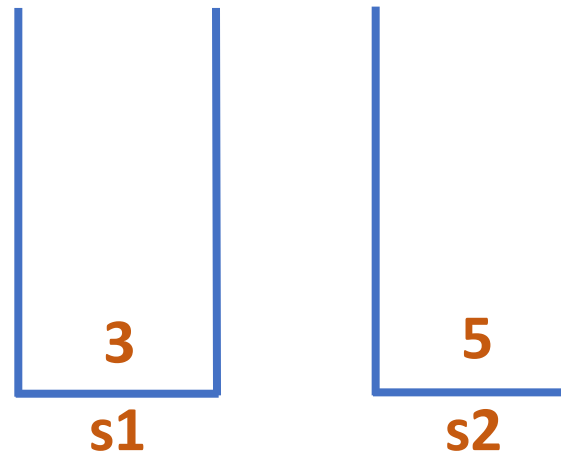
```
}
```

...

current->right = 6



current = 5



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```



```
{
```

```
    current = pop(s1)
```



```
    push(s2, current)
```

```
    if(current->left != NULL)
```

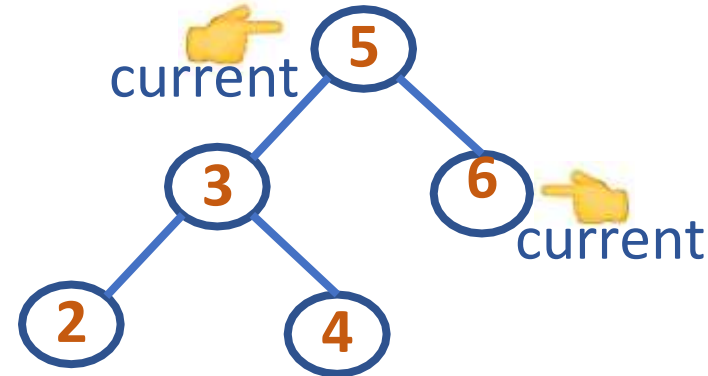
```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

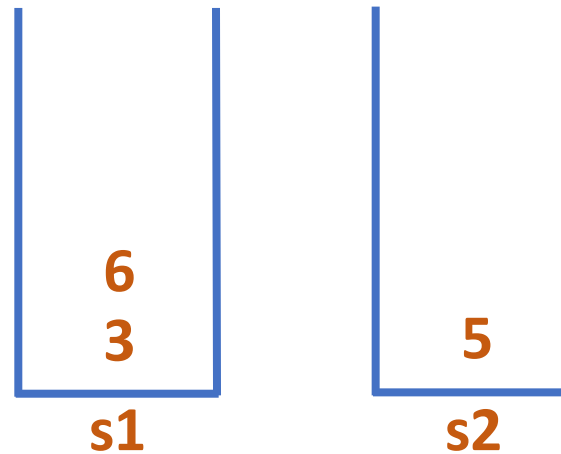
```
        push(s1, current->right)
```

```
}
```

...



current = 5



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

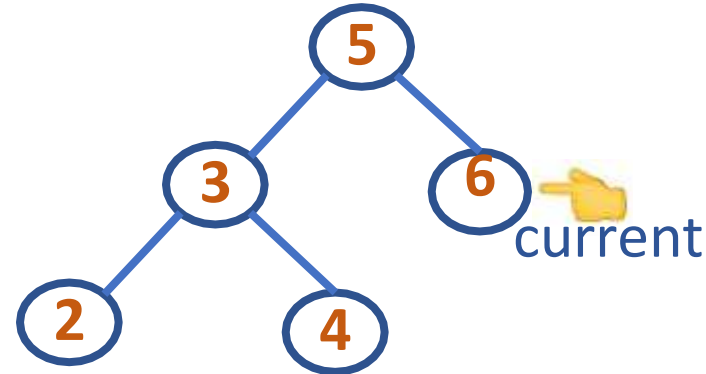
```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

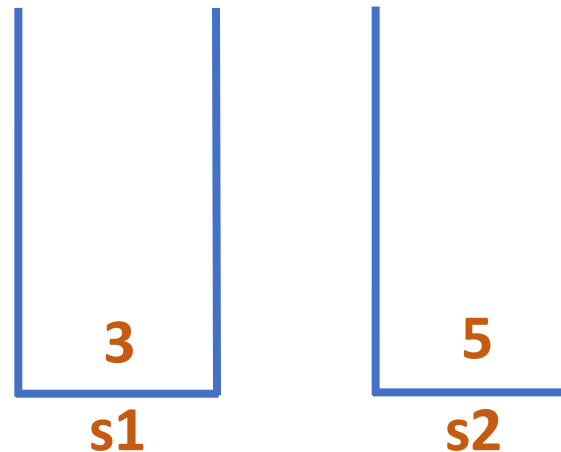
```
}
```

...

```
    current->left = N  
    current->right = N
```



current = 6



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```



```
{
```

```
    current = pop(s1)
```



```
    push(s2, current)
```

```
    if(current->left != NULL)
```

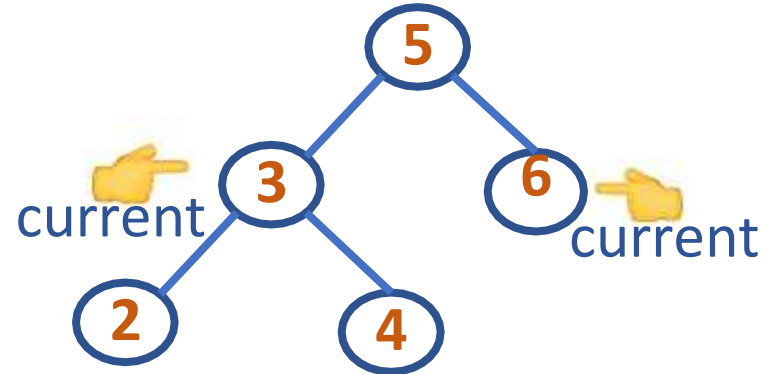
```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

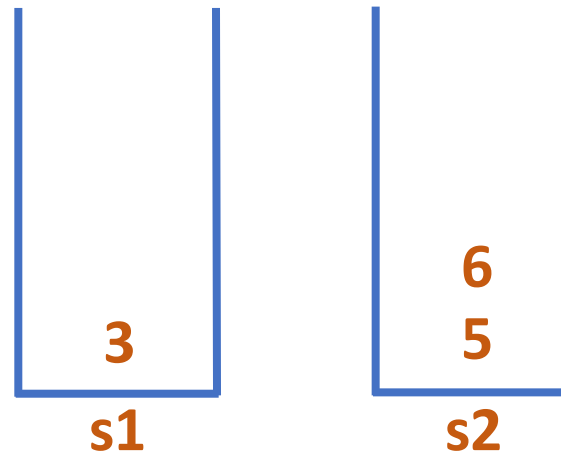
```
        push(s1, current->right)
```

```
}
```

...



current = 6



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

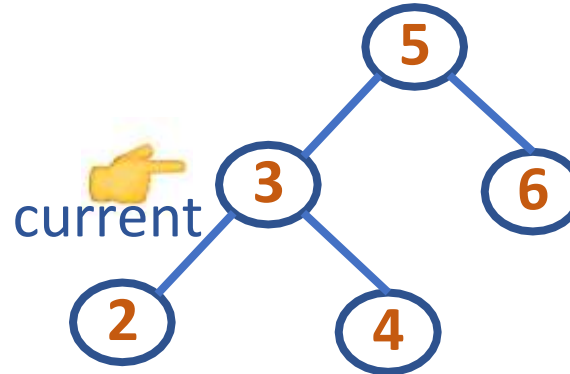
```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

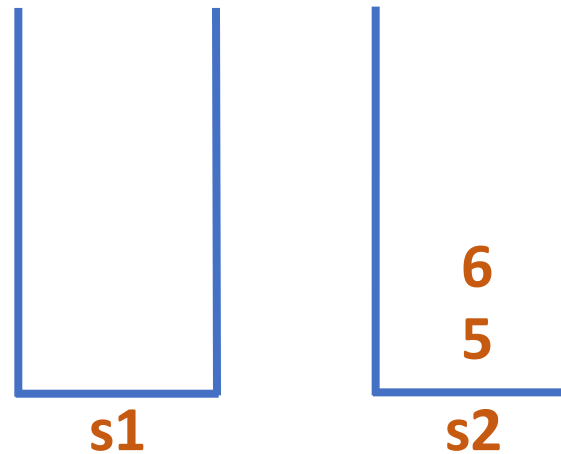
```
        push(s1, current->right)
```

```
}
```

...



current = 3



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL) ➡
```

```
        push(s1, current->left) ➡
```

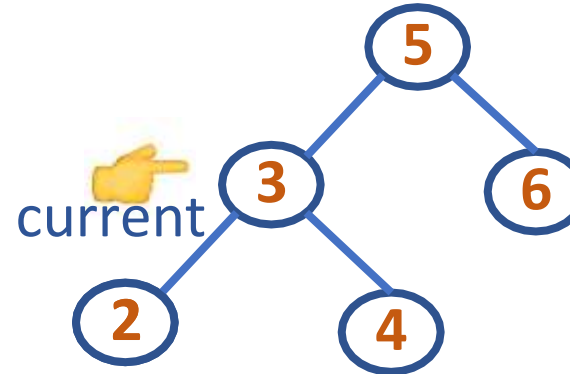
```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

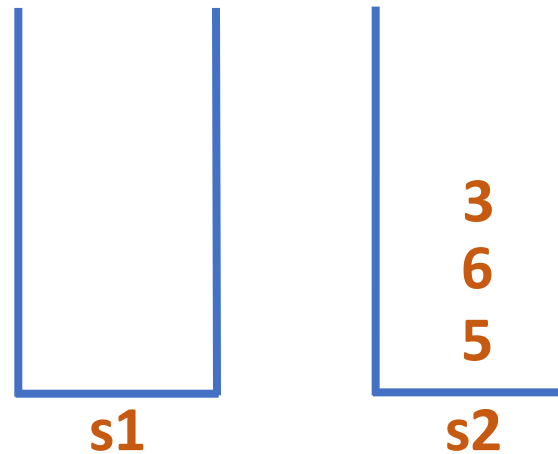
```
}
```

...

current->left = 2



current = 3



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

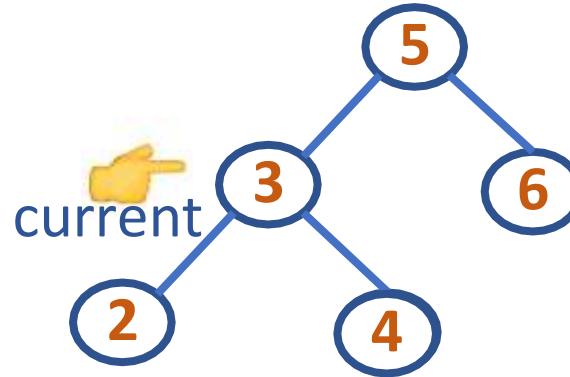
```
    if(current->right != NULL) 
```

```
        push(s1, current->right) 
```

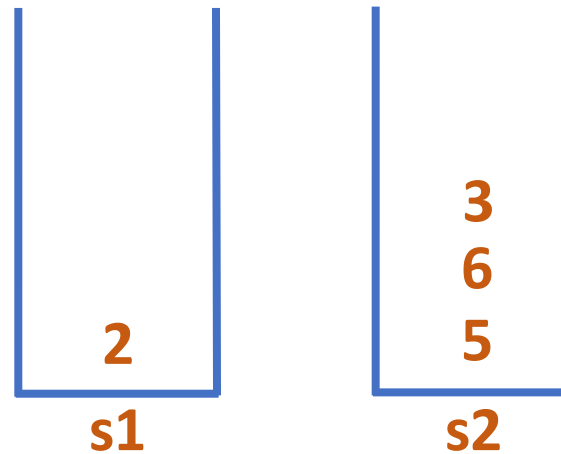
```
}
```

...

current->right = 4



current = 3



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```



```
{
```

```
    current = pop(s1)
```



```
    push(s2, current)
```

```
    if(current->left != NULL)
```

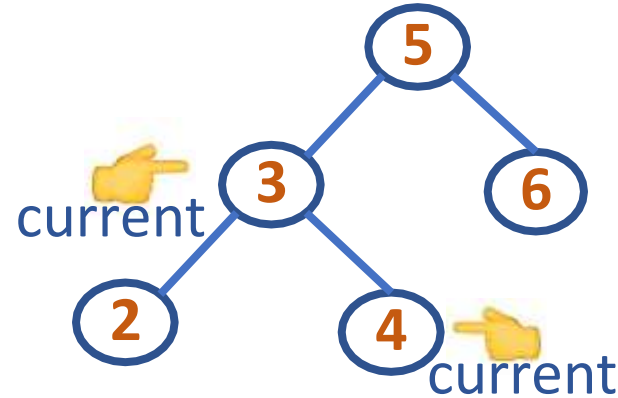
```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

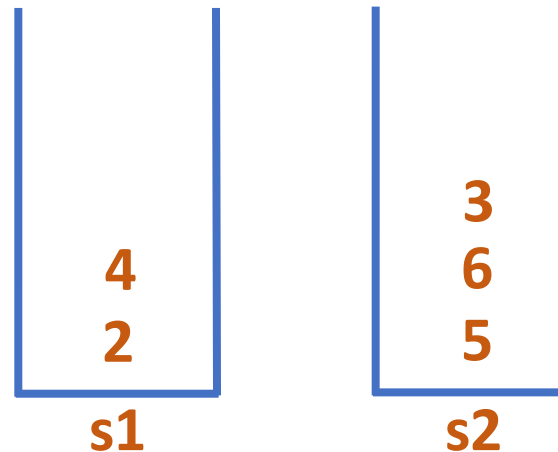
```
        push(s1, current->right)
```

```
}
```

...



current = 3



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

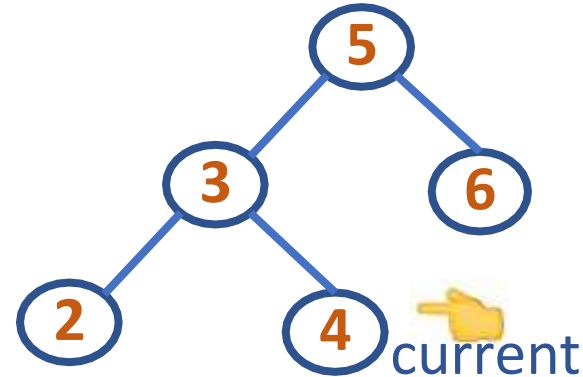
```
        push(s1, current->right)
```

```
}
```

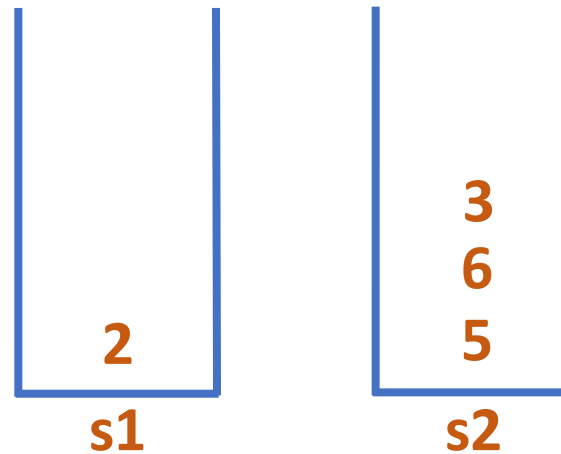
...

```
    current->left = N
```

```
    current->right = N
```



current = 4



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

...

```
while(!isEmpty(s1))
```



```
{
```

```
    current = pop(s1)
```



```
    push(s2, current)
```

```
    if(current->left != NULL)
```

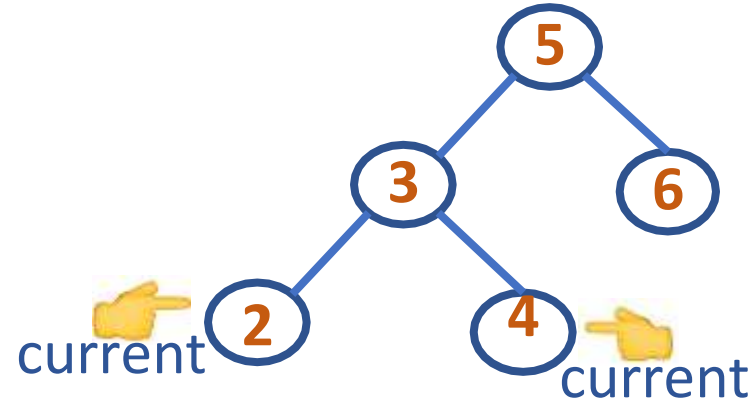
```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

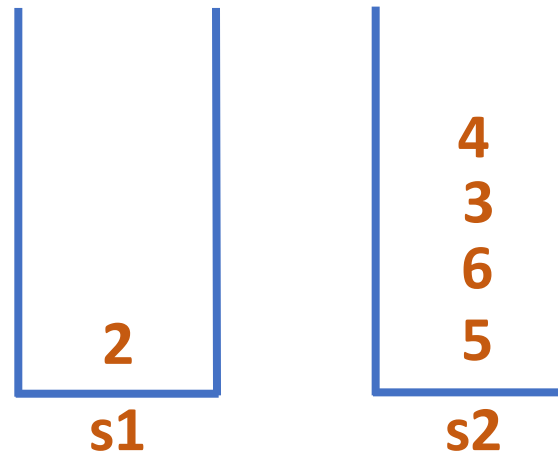
```
        push(s1, current->right)
```

```
}
```

...



current = 4



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

```
...
```

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

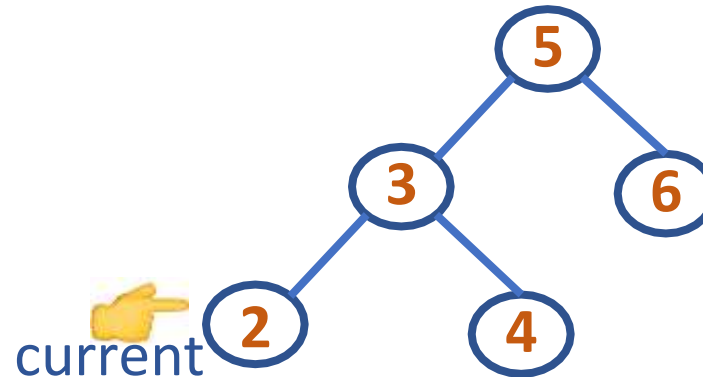
```
        push(s1, current->right)
```

```
}
```

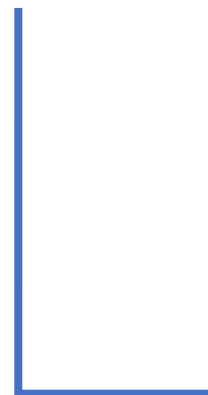
```
...
```

```
    current->left = N
```

```
    current->right = N
```



current = 2



s1



s2

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal

```
iterativePostorder(root)
```

```
...
```

```
while(!isEmpty(s1))
```



```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

```
}
```

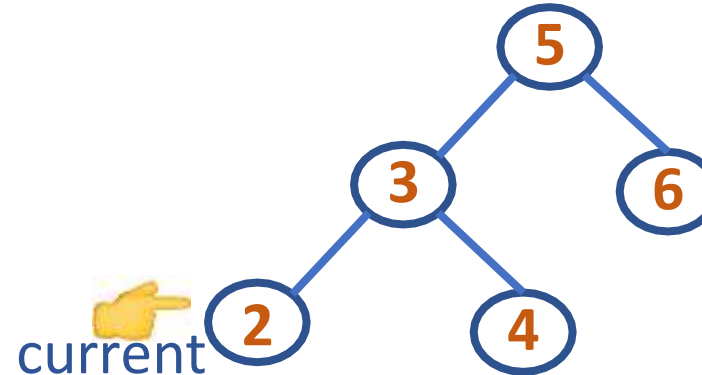
```
while(!isEmpty(s2))
```



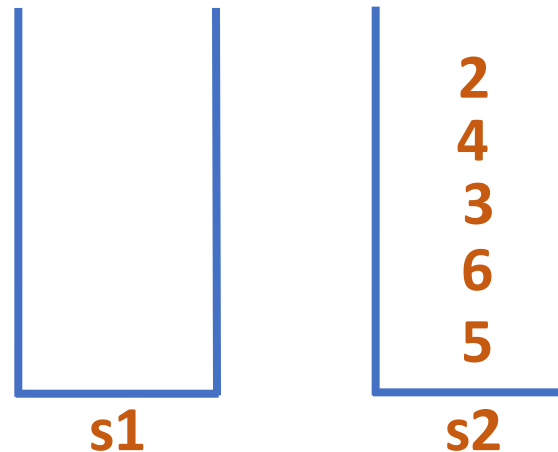
```
    current = pop(s2)
```

```
    print current->info
```

```
}
```



current = 2



DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal



```
iterativePostorder(root)
```

```
...
```

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

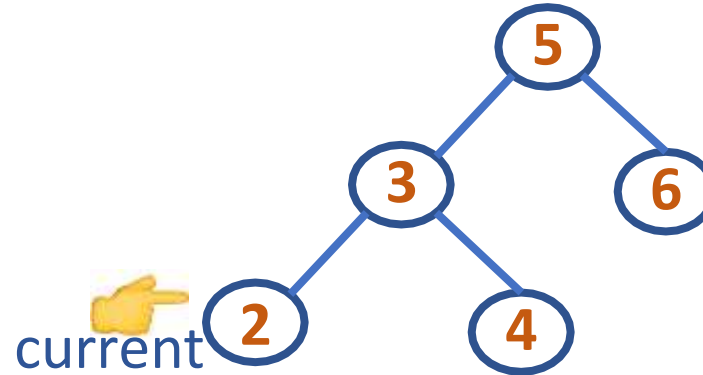
```
}
```

```
while(!isEmpty(s2)) {
```

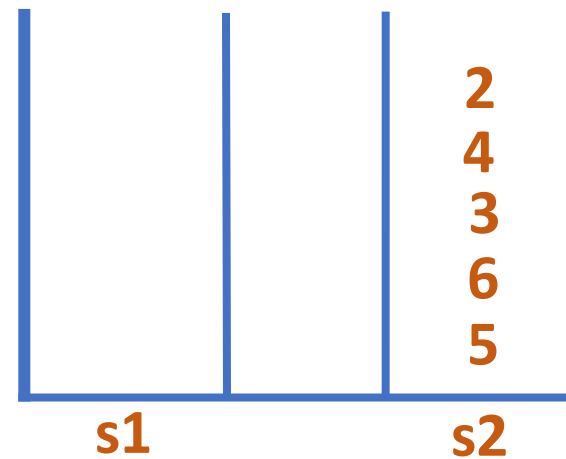
```
    current = pop(s2) ➡
```

```
    print current->info ➡
```

```
}
```



current = 2



Postorder Traversal:

2

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal



```
iterativePostorder(root)
```

```
...
```

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

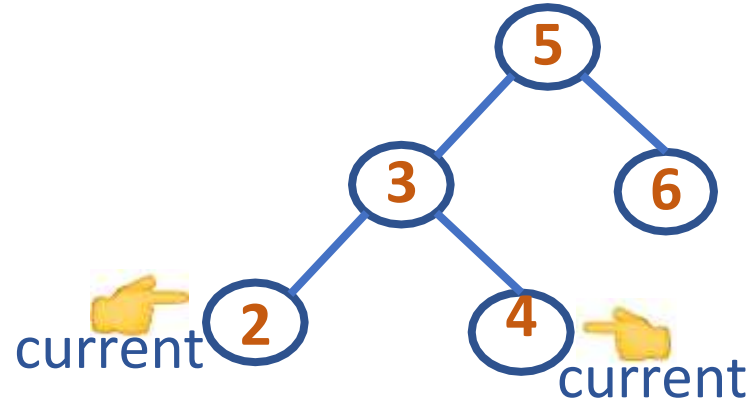
```
}
```

```
while(!isEmpty(s2))
```

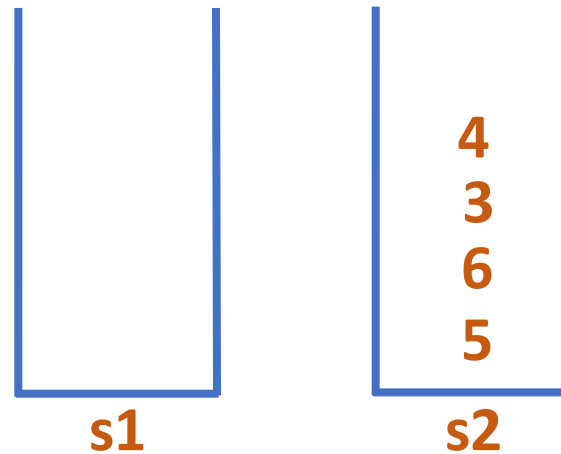
```
    current = pop(s2)
```

```
    print current->info
```

```
}
```



current = 2



Postorder Traversal:

2 4

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal



```
iterativePostorder(root)
```

```
...
```

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

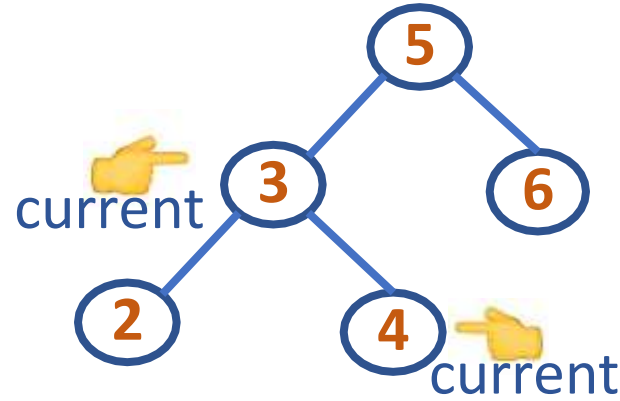
```
}
```

```
while(!isEmpty(s2))
```

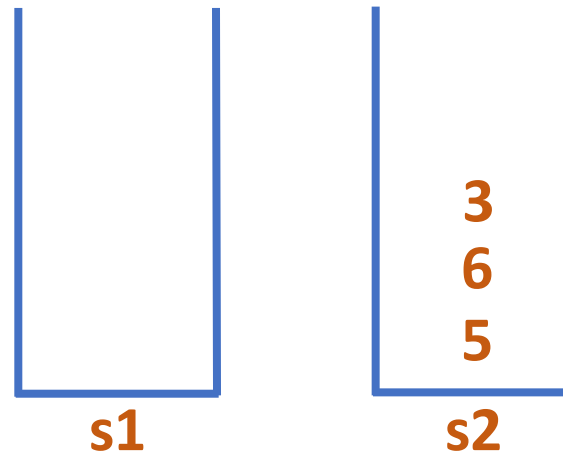
```
    current = pop(s2)
```

```
    print current->info
```

```
}
```



current = 4



Postorder Traversal:

2 4 3

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal



```
iterativePostorder(root)
```

```
...
```

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

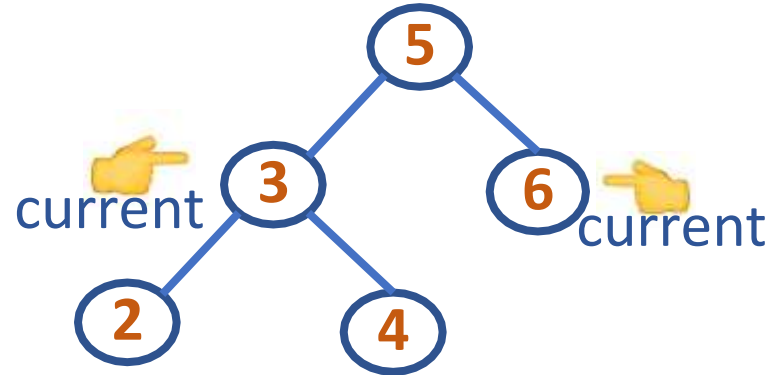
```
}
```

```
while(!isEmpty(s2))
```

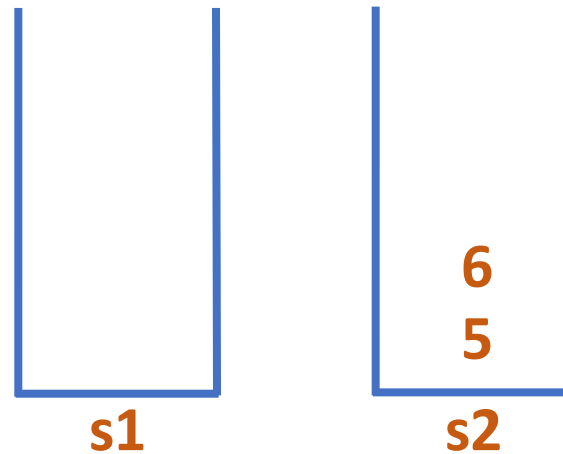
```
    current = pop(s2)
```

```
    print current->info
```

```
}
```



current = 3



Postorder Traversal:

2 4 3 6

DATA STRUCTURES AND ITS APPLICATIONS

Iterative Postorder Traversal



```
iterativePostorder(root)
```

```
...
```

```
while(!isEmpty(s1))
```

```
{
```

```
    current = pop(s1)
```

```
    push(s2, current)
```

```
    if(current->left != NULL)
```

```
        push(s1, current->left)
```

```
    if(current->right != NULL)
```

```
        push(s1, current->right)
```

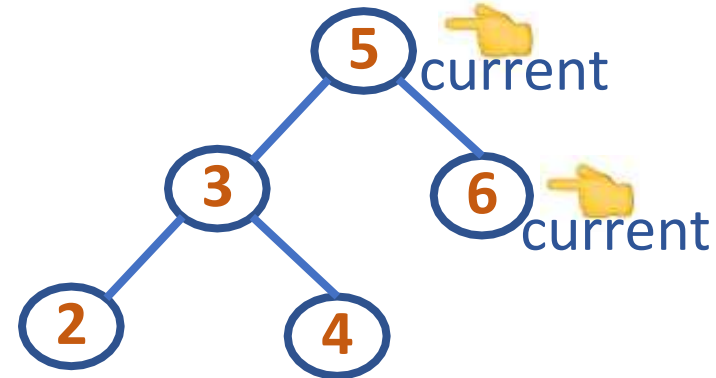
```
}
```

```
while(!isEmpty(s2))
```

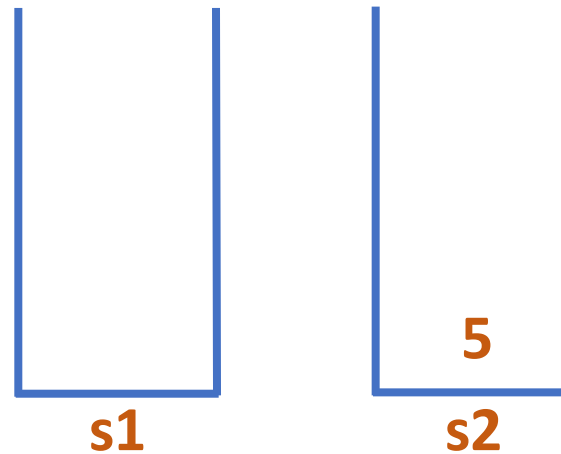
```
    current = pop(s2)
```

```
    print current->info
```

```
}
```



current = 6



Postorder Traversal:

2 4 3 6 5



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE22CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

n-ary Tree Traversal

Shylaja S S

Department of Computer Science & Engineering

Structure of a treenode revisited

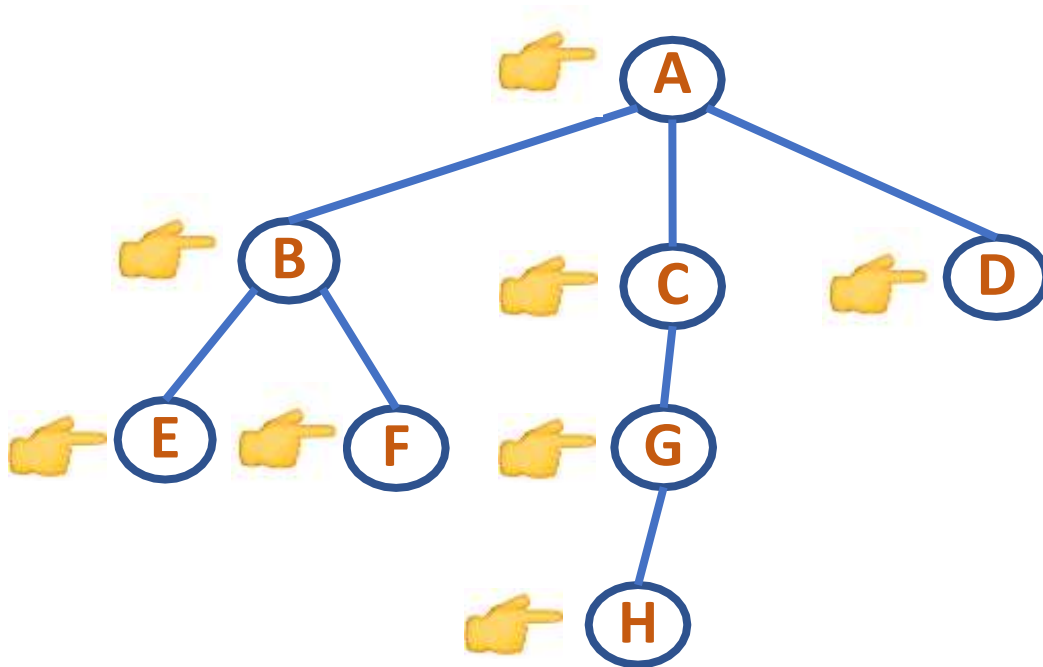
```
struct treenode{  
    int info;  
    struct treenode *child;  
    struct treenode *sibling;  
};
```

With the treenode implemented as having pointers to first child and immediate sibling, the traversal preorder, inorder and postorder for a tree are defined as follows:

Preorder:

1. Visit the root of the first tree in the forest
2. Traverse in preorder the forest formed by the subtrees of the first tree, if any
3. Traverse in preorder the forest formed by the remaining trees in the forest, if any

Preorder Tree Traversal



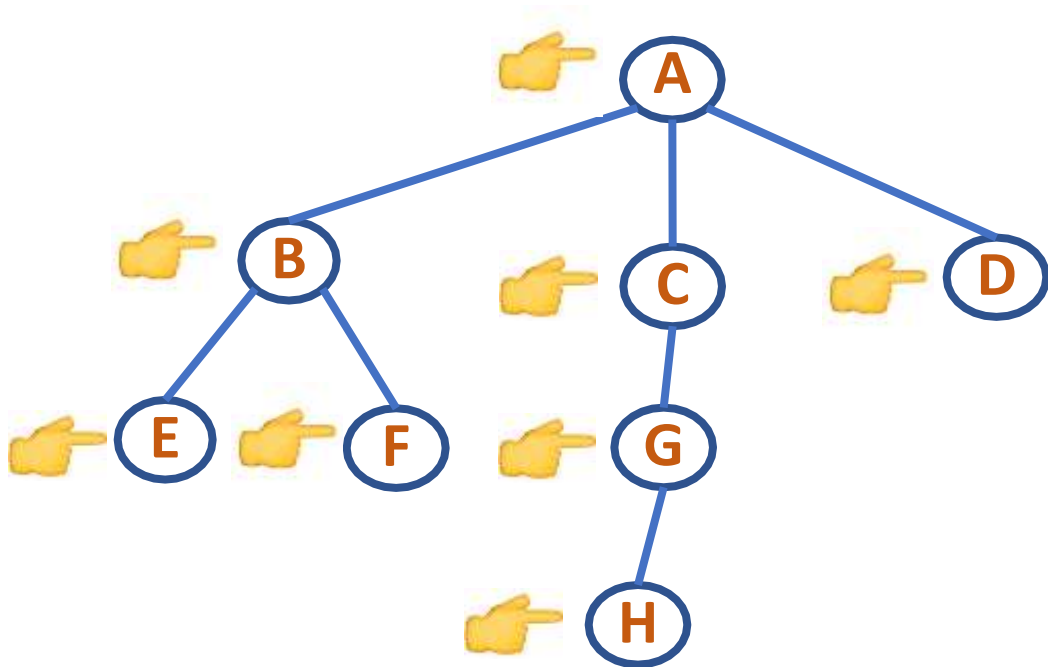
ABEFCGHD

```
void preorder(TREE *root)
{
    if(root!=NULL)
    {
        printf(" %d ",root->info);
        preorder(root->child);
        preorder(root->sibling);
    }
}
```

Inorder

1. Traverse in inorder the forest formed by the subtrees of the first tree, if any
2. Visit the root of the first tree in the forest
3. Traverse in inorder the forest formed by the remaining trees in the forest, if any

Inorder Tree Traversal



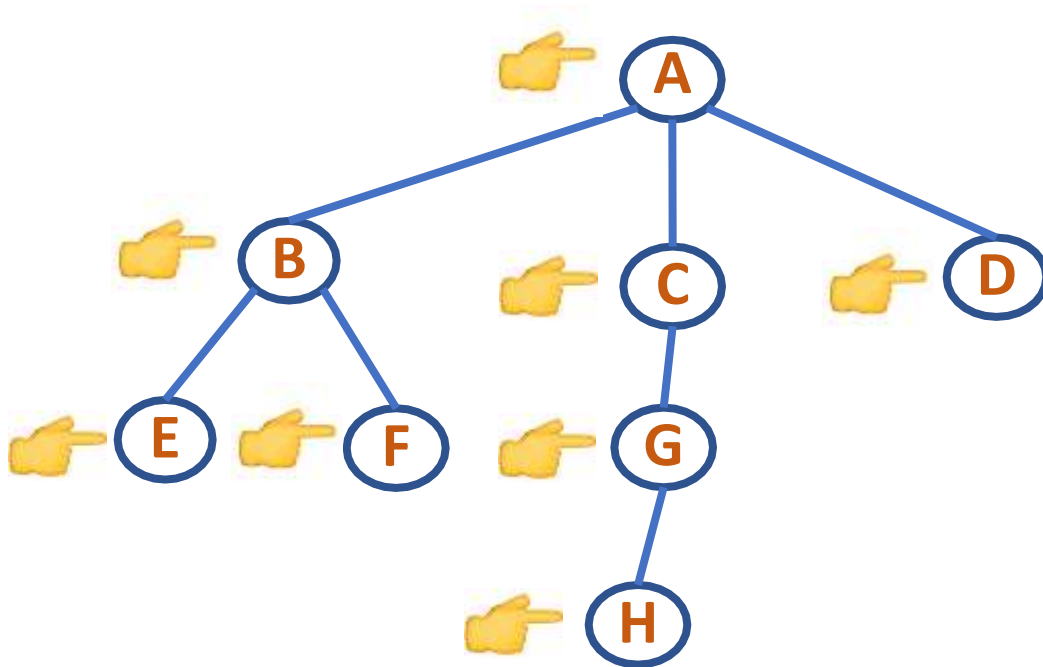
E F B H G C D A


```
void inorder(TREE *root)
{
    if(root!=NULL)
    {
        inorder(root->child);
        printf(" %d ",root->info);
        inorder(root->sibling);
    }
}
```

Postorder

1. Traverse in postorder the forest formed by the subtrees of the first tree, if any
2. Traverse in postorder the forest formed by the remaining trees in the forest, if any
3. Visit the root of the first tree in the forest

Postorder Tree Traversal

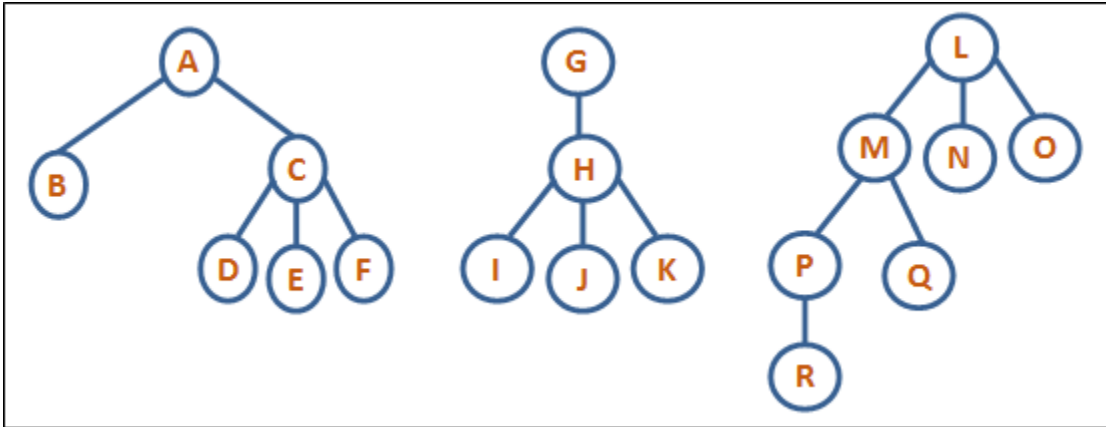


F E H G D C B A

```
void postorder(TREE *root)
{
    if(root!=NULL)
    {
        postorder(root->child);
        postorder(root->sibling);
        printf(" %d ", root->info);
    }
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Forest traversal



Preorder: ABCDEFGHIJKLMRQNO

Inorder: BDEFCAIJKHGRPQMNL

Postorder: FEDCBKJIHRQPONMLGA



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Heap: Definition and Implementation

Shylaja S S

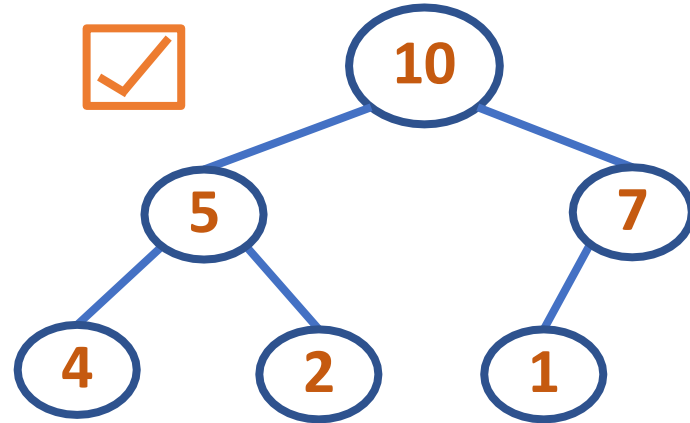
Department of Computer Science & Engineering

Definition: A heap can be defined as a binary tree with keys assigned to its nodes (one key per node) provided the following two conditions are met:

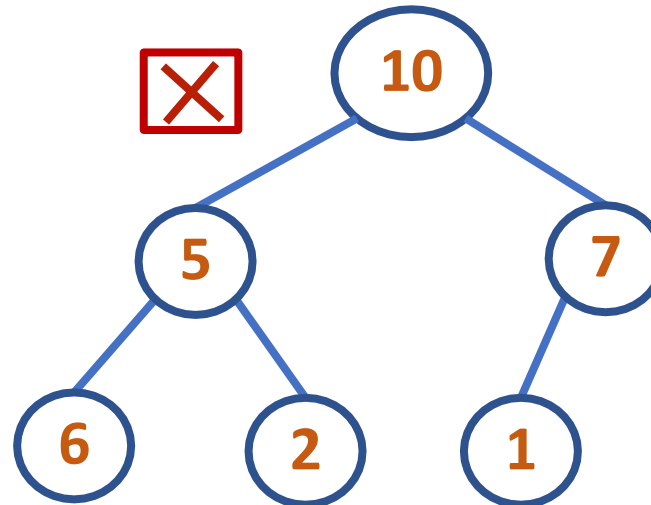
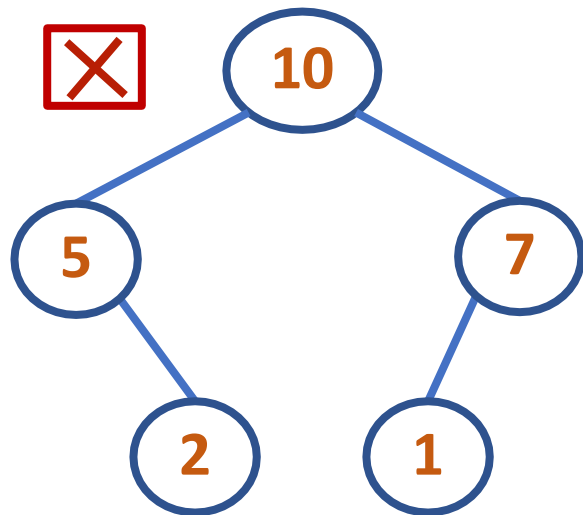
1.The tree's shape requirement - The binary tree is essentially complete, that is, all its levels are full except possibly the last level, where only some rightmost leaves may be missing

2. The parental dominance requirement - The key at each node is greater than or equal to the keys at its children. (This condition is considered automatically satisfied for all leaves.)

Heap Tree



Shape Requirement
not satisfied
Parental dominance
not satisfied



Only the topmost Binary Tree is a heap. Why?

Properties of Heap

1. There exists exactly one essentially complete binary tree with n nodes. Its height is equal to $\lfloor \log_2 n \rfloor$
2. The root of a heap always contains its largest element
3. A node of a heap considered with all its descendants is also a heap
4. A heap can be implemented as an array by recording its elements in the top-down, left-to-right fashion. It is convenient to store the heap's elements in positions 1 through n of such an array, leaving $H[0]$ either unused or putting there a sentinel whose value is greater than every element in the heap.

...

In such a representation,

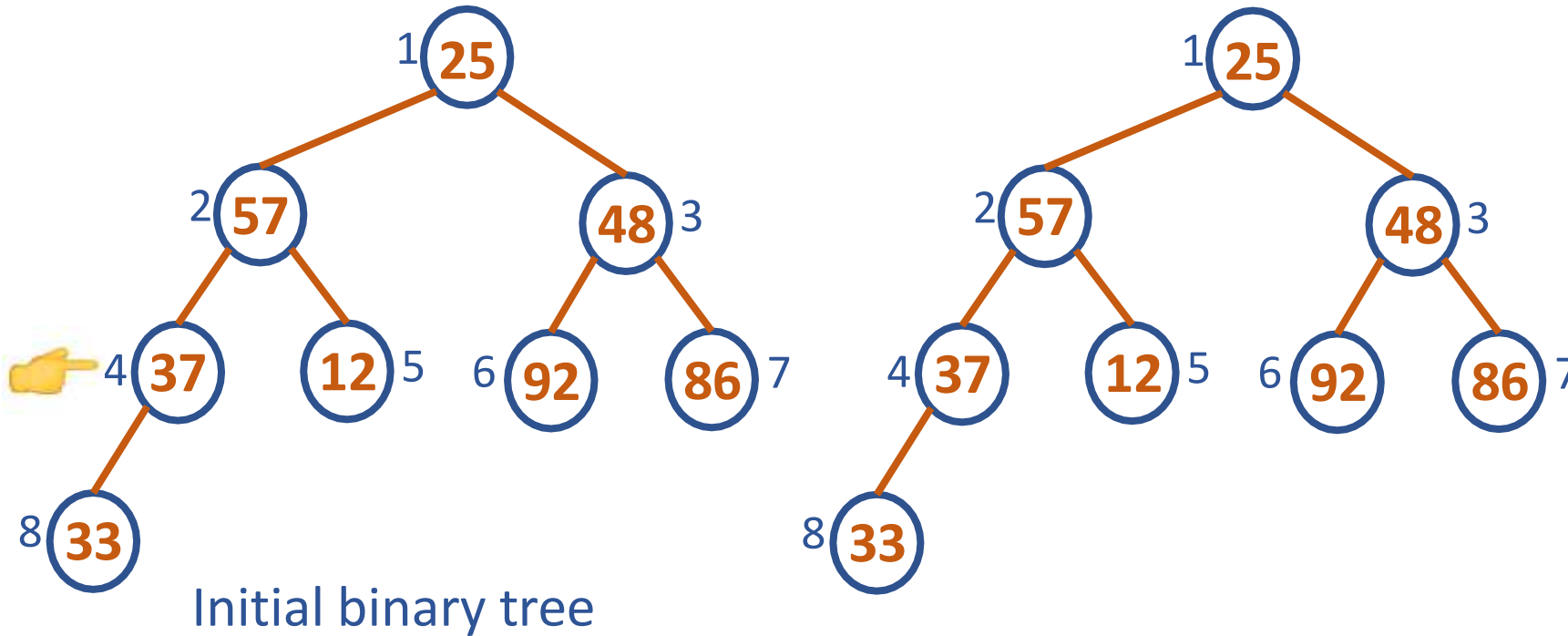
- a) The parental node keys will be in the first $\lfloor n/2 \rfloor$ positions of the array, while the leaf keys will occupy the last $\lfloor n/2 \rfloor$ positions
- b) The children of a key in the array's parental position i ($1 \leq i \leq \lfloor n/2 \rfloor$) will be in positions $2i$ and $2i + 1$, and, correspondingly, the parent of a key in position i ($2 \leq i \leq n$) will be in position $\lfloor i/2 \rfloor$

DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, $n=8$



At $k = 4$, $v = 37$

Compare 37 with its only child 33

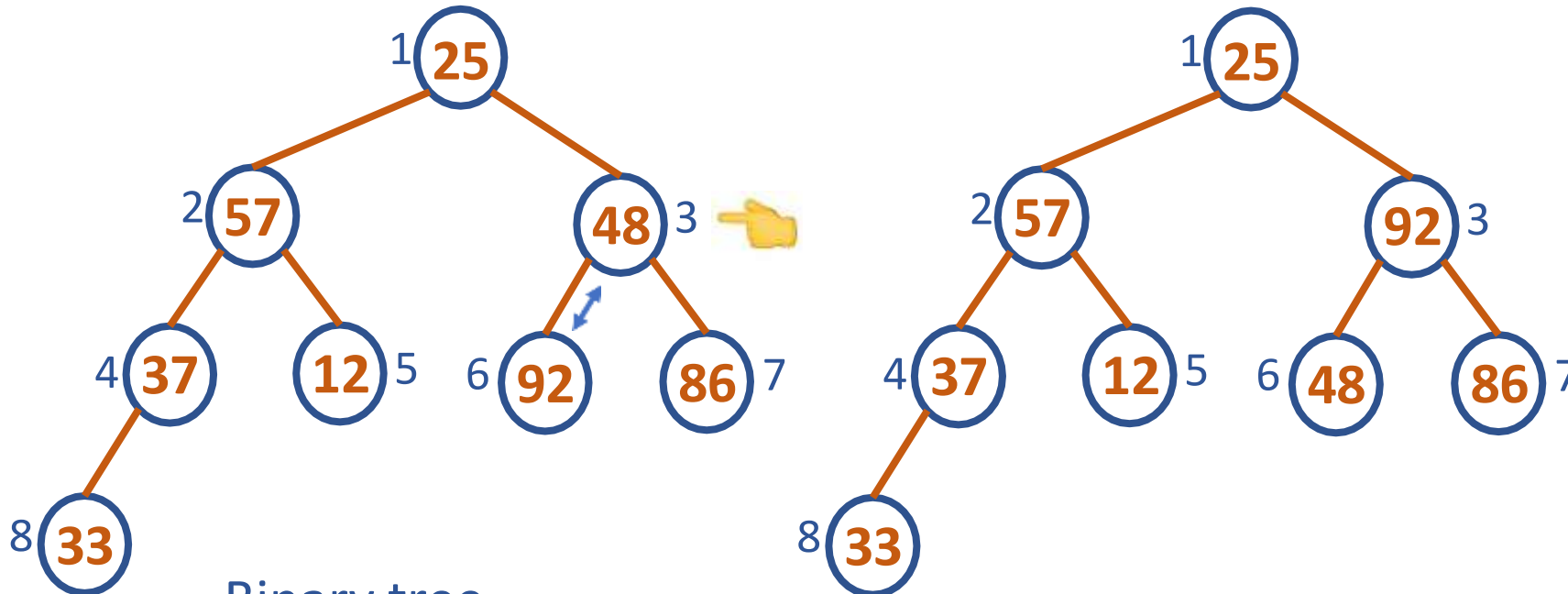
$37 > 33$, it's a heap at $k=4$

DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, $n=8$



Binary tree
after one iteration at $k=4$

At $k = 3$, $v = 48$

Largest child: 92

Compare 48 with its largest child

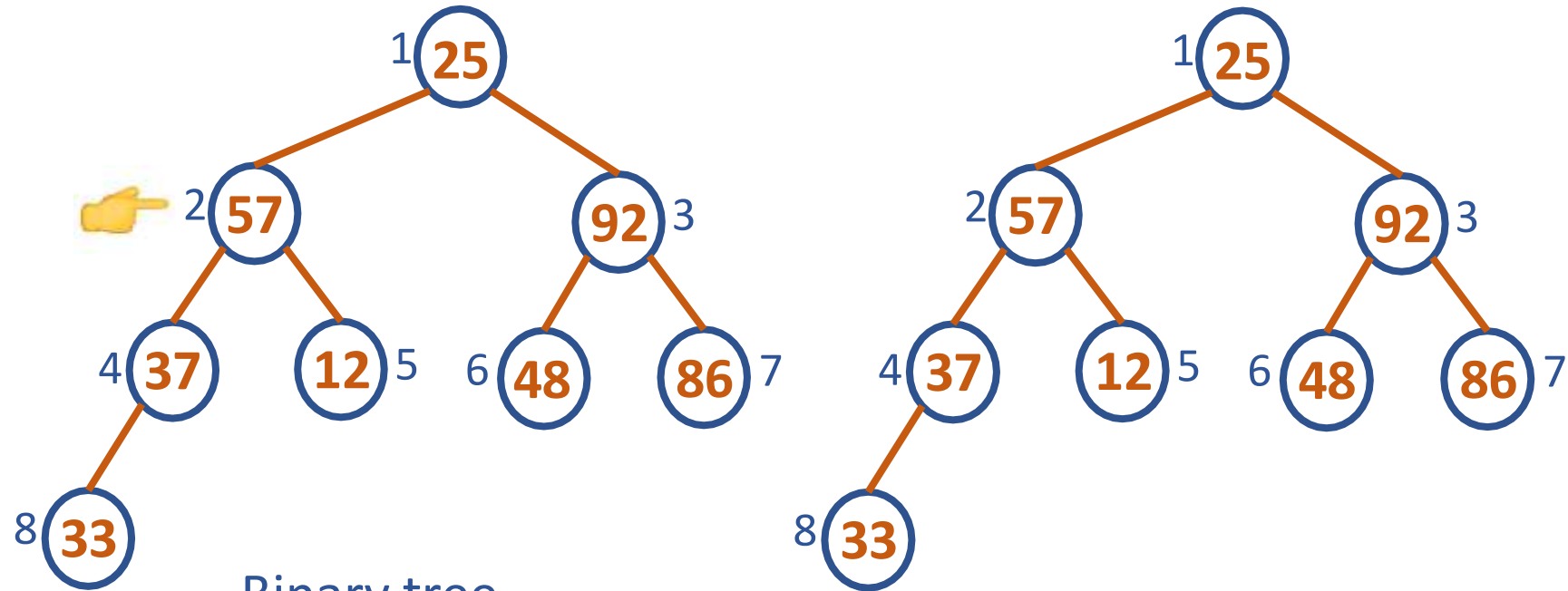
$48 < 92$, Heapify

DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, $n=8$



Binary tree

after two iterations at $k=4$, $k=3$

At $k = 2$, $v = 57$

Largest child: 37

Compare 57 with its largest child

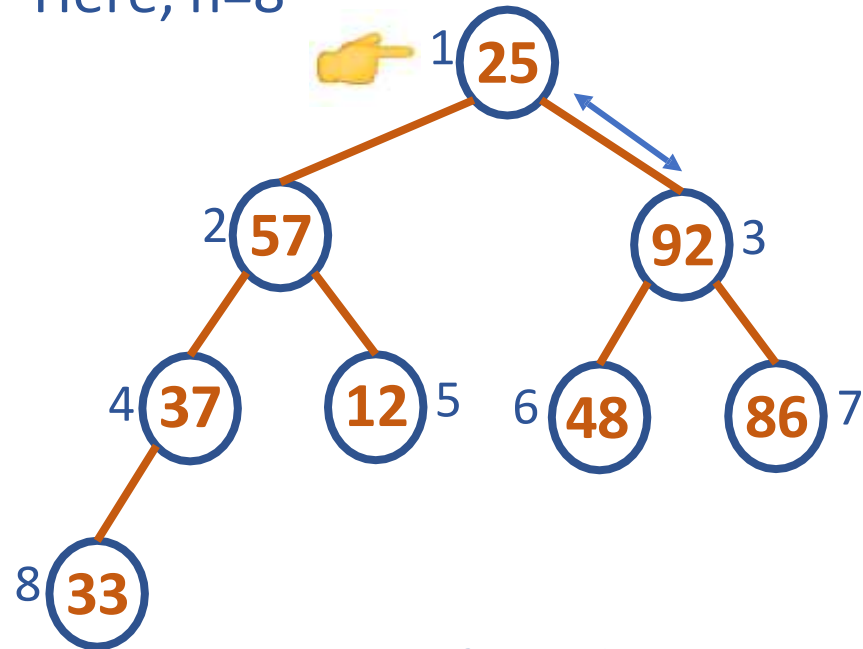
$57 > 37$, it's a heap at $k=2$

DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, $n=8$



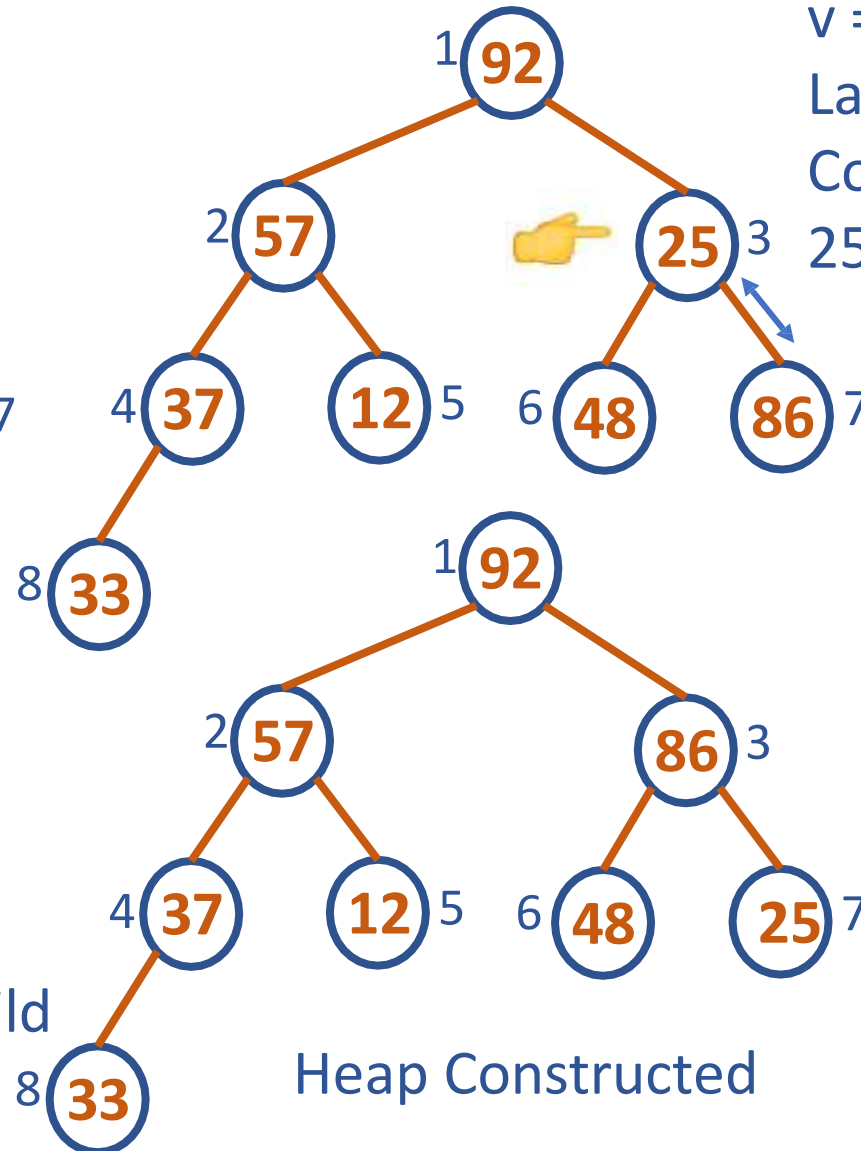
Binary tree after three iterations at $k=4$, $k=3$, $k=2$

At $k = 1$, $v = 25$

Largest child: 92

Compare 25 with its largest child

$25 < 92$, Heapify



Heap Constructed

$v = 25$, Now at $k = 3$,

Largest child: 86

Compare 25 with its largest child

$25 < 86$, Heapify



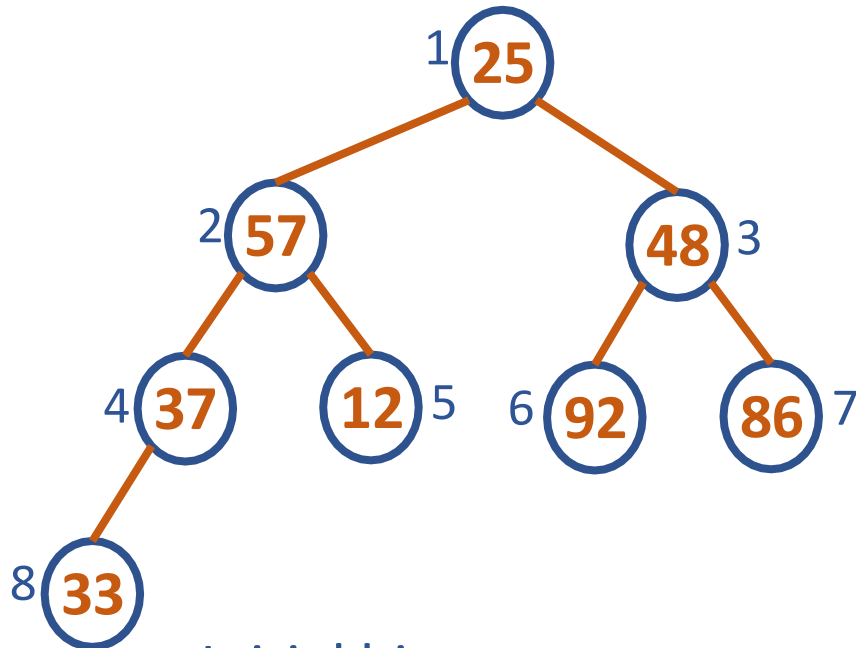
PES
UNIVERSITY
ONLINE

DATA STRUCTURES AND ITS APPLICATIONS

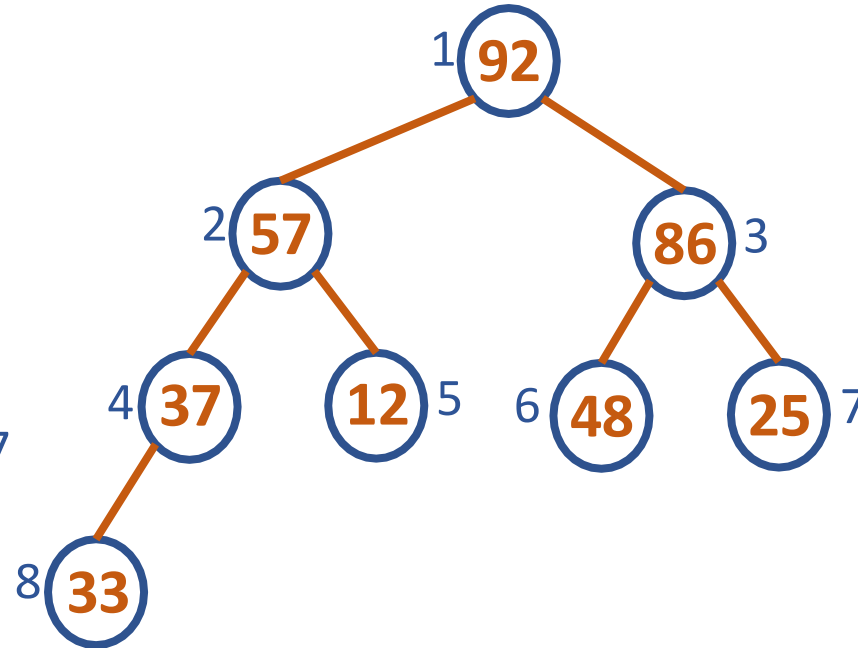
Heap Construction – Bottom Up

Bottom Up Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, $n=8$



Initial binary tree



Bottom Up Heap Constructed

DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Bottom Up

ALGORITHM HeapBottomUp($H[1...n]$)

//Constructs a heap from the elements of a given array by bottom-up algorithm

//Input: An array $H[1...n]$ of orderable items

//Output: A heap $H[1...n]$

for $i \leftarrow \lfloor n/2 \rfloor$ downto 1 {

$k \leftarrow i$

$v \leftarrow H[k]$

 heap $\leftarrow$ false

 while not heap and $2*k \leq n$ {

$j \leftarrow 2*k$

 if $j < n$

 //if there are two children

 if $H[j] < H[j+1]$

$j \leftarrow j+1$

 //find position of largest child

 if $v \geq H[j]$

 //if key of parent node $\geq$ key of largest child

 heap $\leftarrow$ true

 //it's a heap

 else {

 //heapify

$H[k] \leftarrow H[j]$

$k \leftarrow j$

 }

 //end of else

 } //end of while

$H[k] \leftarrow v$

} //end of for

```
for(i=n/2-1;i>=0;i--)
{
    k=i;
    v=h[k];
    heap=0;
    while(!heap && 2*k+1<=n-1)
    {
        j=2*k+1;
        if(j+1<=n-1)
            if(h[j+1]>h[j]) j=j+1;
        if(v>h[j])
            heap=1;
        else
        {
            h[k]=h[j];
            k=j;
        }
    }
    h[k]=v;
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Top Down

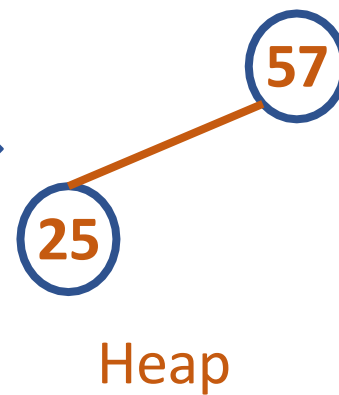
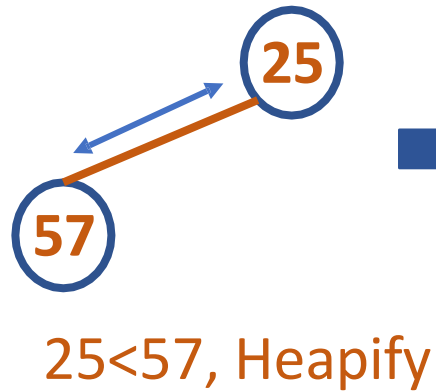
Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, $n=8$

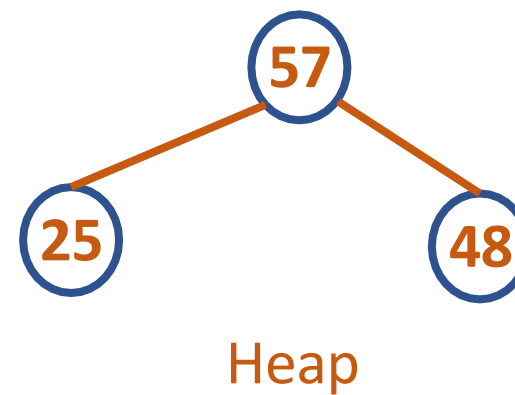
Insert 25



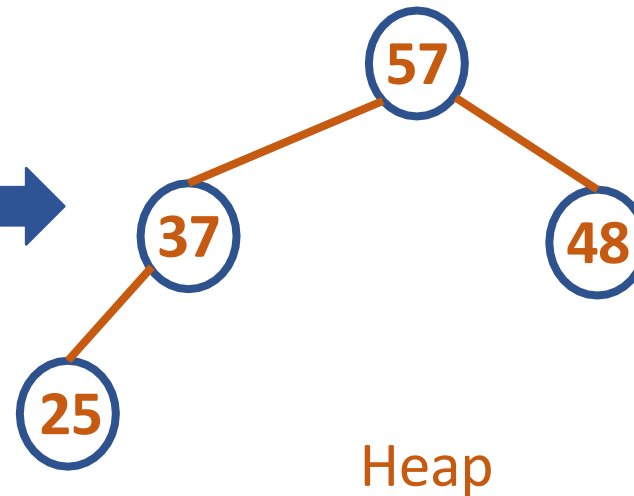
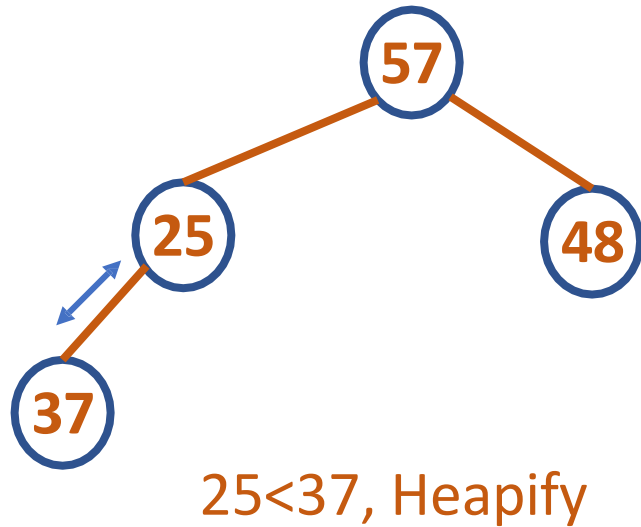
Insert 57



Insert 48



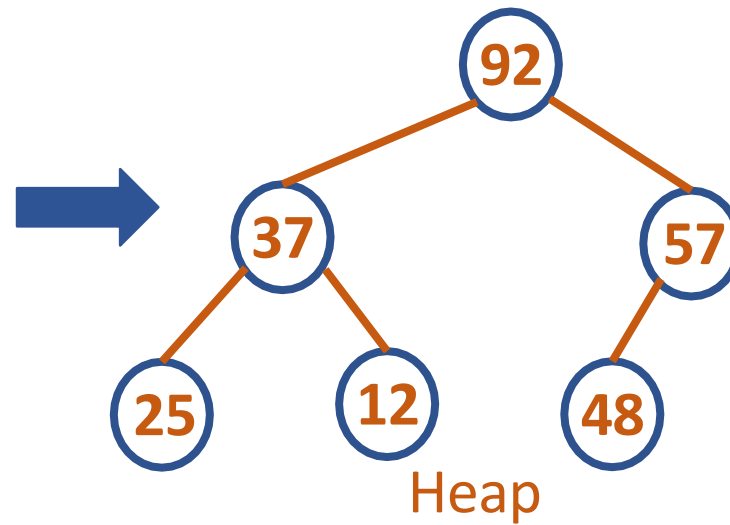
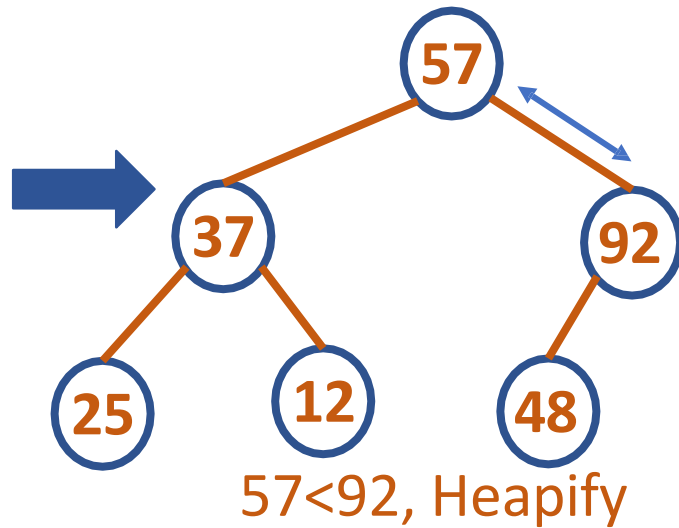
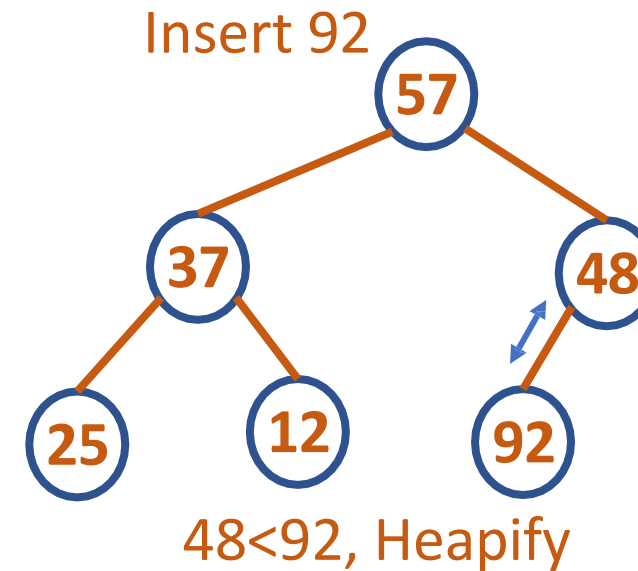
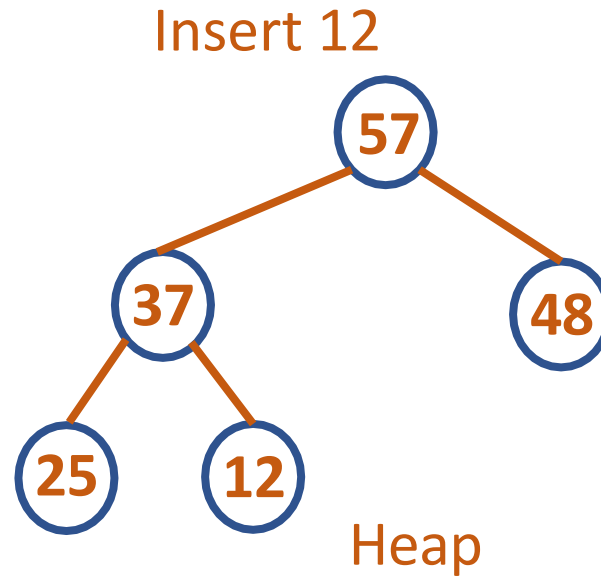
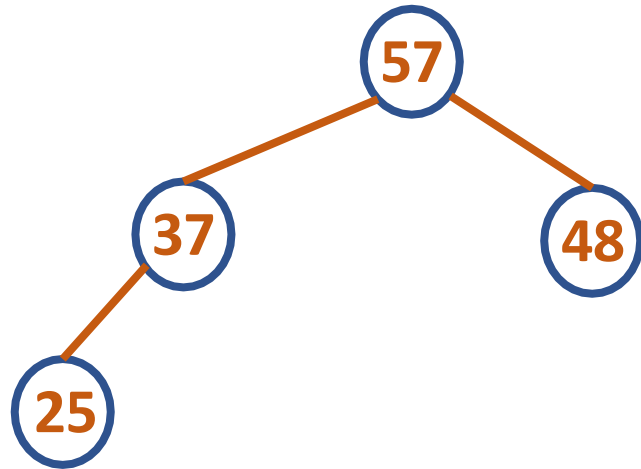
Insert 37



DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Top Down

Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33
Here, $n=8$

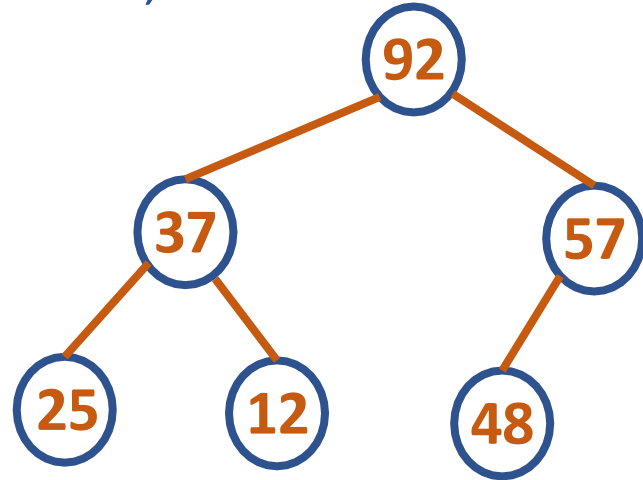


DATA STRUCTURES AND ITS APPLICATIONS

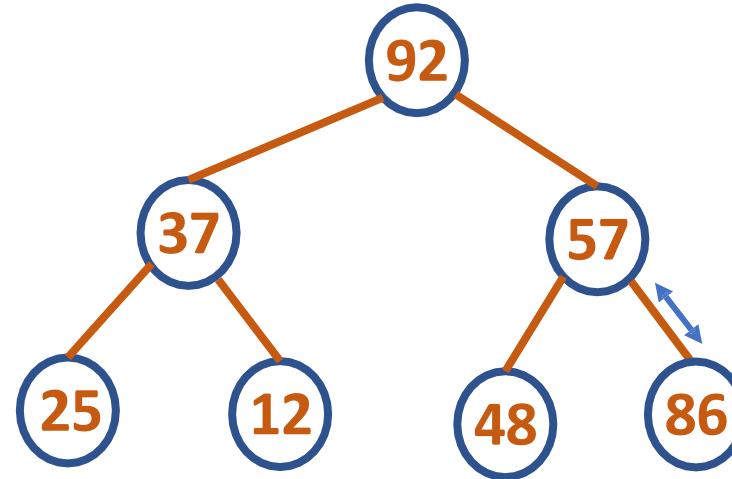
Heap Construction – Top Down

Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

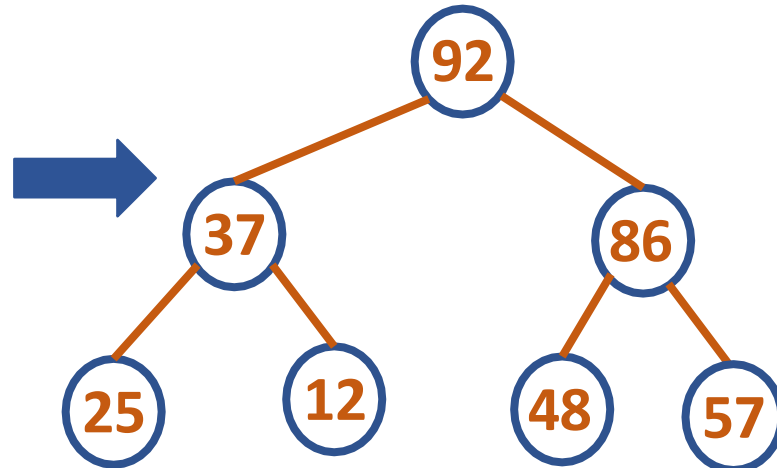
Here, $n=8$



Insert 86



$57 < 86$, Heapify



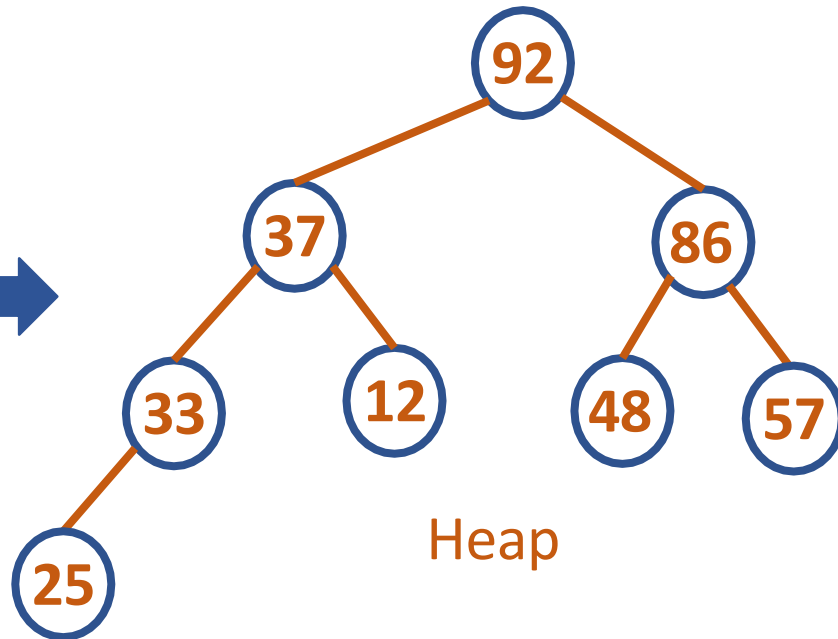
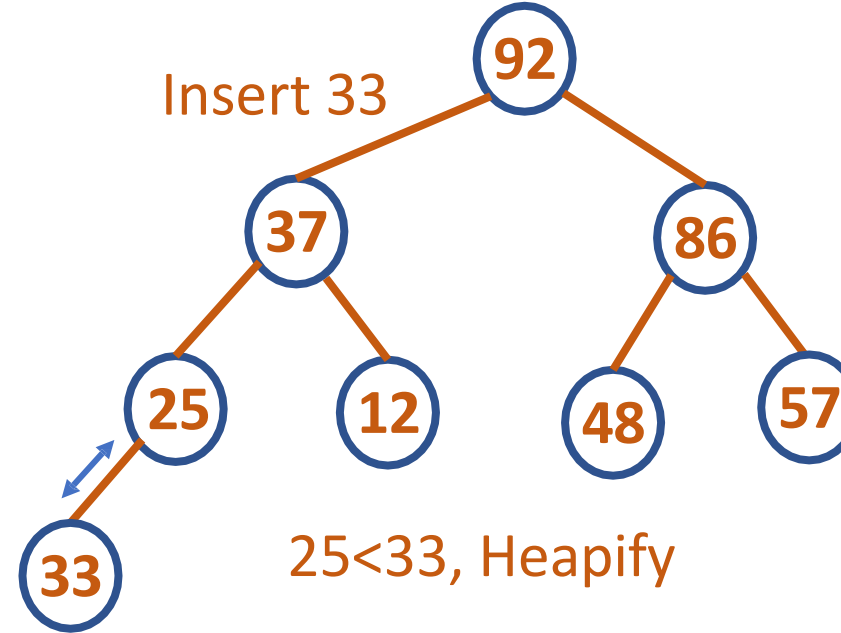
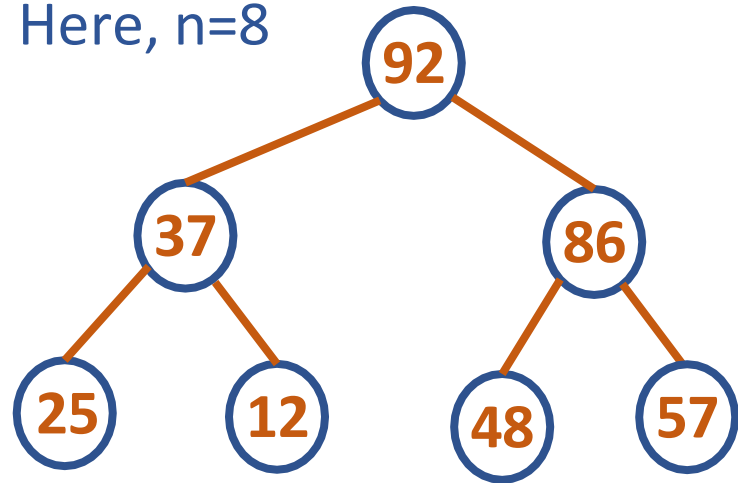
Heap

DATA STRUCTURES AND ITS APPLICATIONS

Heap Construction – Top Down

Top Down Heap Construction: 25, 57, 48, 37, 12, 92, 86, 33

Here, $n=8$



1. First, attach a new node with key K in it after the last leaf of the existing heap
2. Then sift K up to its appropriate place in the new heap as follows
3. Compare K with its parent's key: if the latter is greater than or equal to K , stop (the structure is a heap);
4. otherwise, swap these two keys and compare K with its new parent
5. This swapping continues until K is not greater than its last parent or it reaches the root



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

Department of Computer science and Engineering

PES UNIVERSITY

UE19CS202: Data Structures and its Applications (4-0-0-4-4)

GRAPHS

Abstract

Implementation of Graph using Adjacency List.

Dr.Sandesh and Saritha

Sandesh_bj@pes.edu

Saritha.k@pes.edu

Implementation of adjacency List:

C representation of adjacency list

1.using array implementation

```
#define MAX 50

struct node
{
    int info;
    int point;
    int next;
};

struct node NODE[MAX];
```

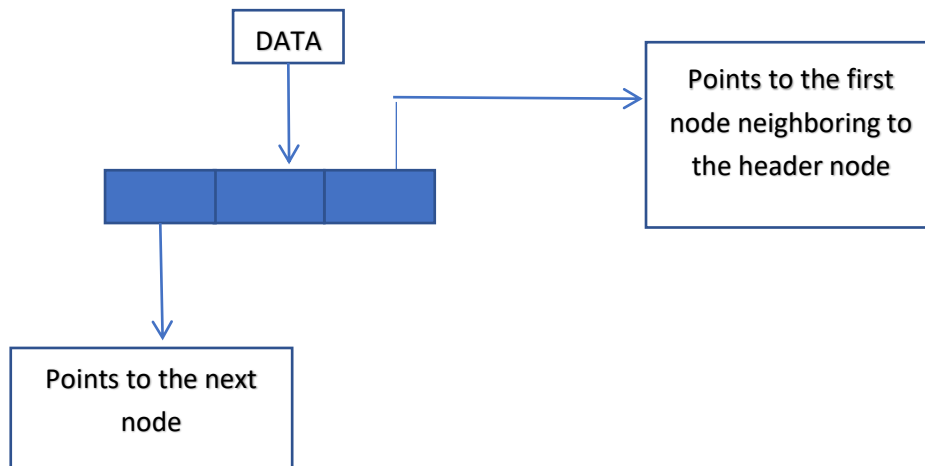
2.Using dynamic implementation

```
struct node
{
    int info;
    struct node *pointer;
    struct node *next;
};

struct node *nodeptr;
```

Two lists are maintained for adjacency of list. The first list is used to store information of all the nodes. The second list is used to store the information of adjacent nodes for each node of a graph.

Header node is used to represent each list, which is assumed to be the first node of a list as shown below.



The different operations performed on adjacency list are

1. To create an arc between two nodes

```
void jointwt(int p,int q,int wt)
{
    int r,r2;
    r2=-1;
    r=node[p].point;
    While(r>=0 && node[r].point!=q)
    {
        r2=r;
        r=node[r].next;
    }
    If(r>=0)
    {
        node[r].info=wt;
    }
}
```

```
        return;
    }

    R=getnode();
    node[r].point=q;
    node[r].next=-1;
    node[r].info=wt;
    (R2<0)?(node[p].point=r):(node[r2].next=r);
}
```

2. To remove an arc between two nodes if it exists:

Void remv(int p,int q)

```
{
    int r,r2;
    r2=-1;
    r=node[p].point;
    while(r>=0 ** node[r].point !=q)
    {
        r2=r;
        r=node[r].next;
    }
    If(r>=0)
    {
        (r2<0)?(node[p].point=node[r].next):(node[r2].next=node[r].next);
        freenode(r);
        return;
    }
}
```

```
}
```

3. Checks whether a node is adjacent to another node

```
int adjacent(int p,int q)
{
    int r;
    r=node[p].point;
    while(r>=0)
        If(node[r].point==q)
            return(TRUE)
        else
            r=node[r].next;
            return(FALSE);
}
```

4. Find a node with a specific information in a graph:

```
int findnode(int graph,int x)
{
    int p;
    p=graph;
    while(p>=0)
        if (node[p].info==x)
            return(p);
        else
            p=node[p].next;
```

```
    return(-1);
```

```
}
```

5. Add a node with specific information to a graph.

```
int addnode(int *graph,int x)
```

```
{
```

```
    int p;
```

```
    p=getnode();
```

```
    node[p].info=x;
```

```
    node[p].point=-1;
```

```
    node[p].next=*graph;
```

```
    *graph=p;
```

```
    return(p);
```

```
}
```



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree

Kusuma K V

Department of Computer Science & Engineering

- A splay tree is a binary search tree with the additional property that recently accessed elements are quick to access again.
- Example: Hospital records
- Like self-balancing binary search trees, a splay tree performs basic operations such as insertion, look-up and removal in $O(\log n)$ amortized time.
- All normal operations on a binary search tree are combined with one basic operation, called splaying.

- Splaying the tree for a certain element rearranges the tree so that the element is placed at the root of the tree.
- One way to do this with the basic search operation is to first perform a standard binary tree search for the element in question, and then use tree rotations in a specific fashion to bring the element to the top.

Splaying

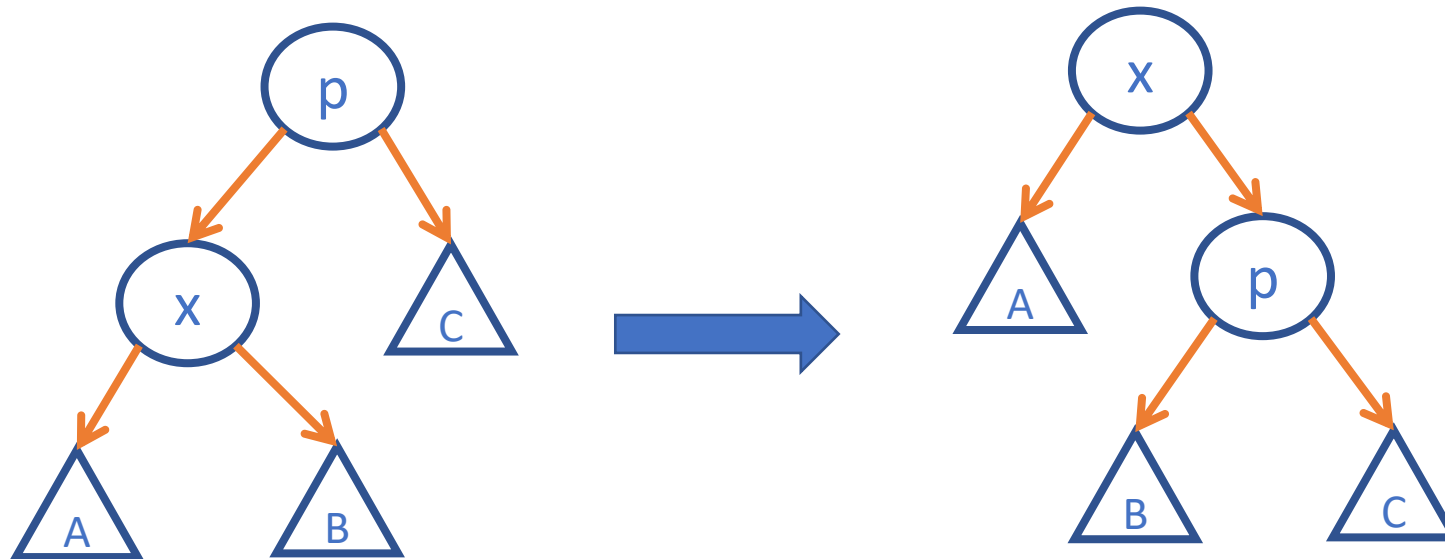
- Each splay step depends on three factors:
 - whether x is the left or right child of its parent node, p ,
 - whether p is the root or not, and if not
 - whether p is the left or right child of its parent, g (the grandparent of x).

Splaying

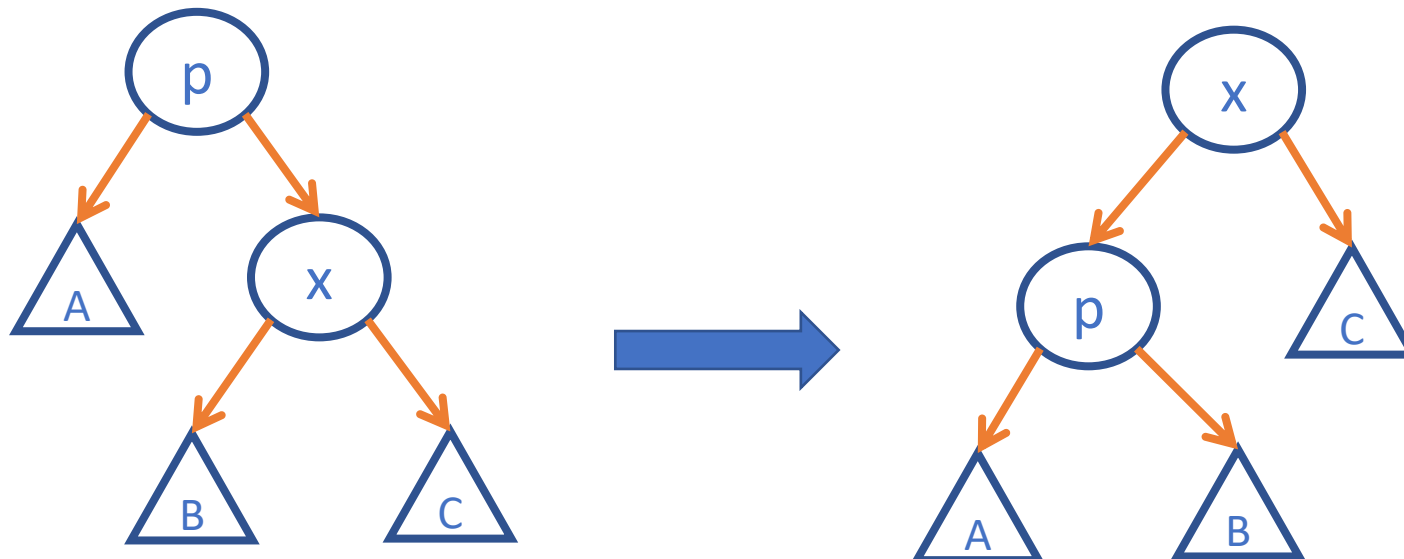
- There are three types of splay steps, each of which has two symmetric variants: left- and right-handed.
- **Zig step**: this step is done when p is the root.
- **Zig - Zig step**: this step is done when p is not the root and x and p are either both right children or are both left children.
- **Zig - Zag step**: this step is done when p is not the root and x is a right child and p is a left child or vice versa (x is left, p is right).

Splay Tree

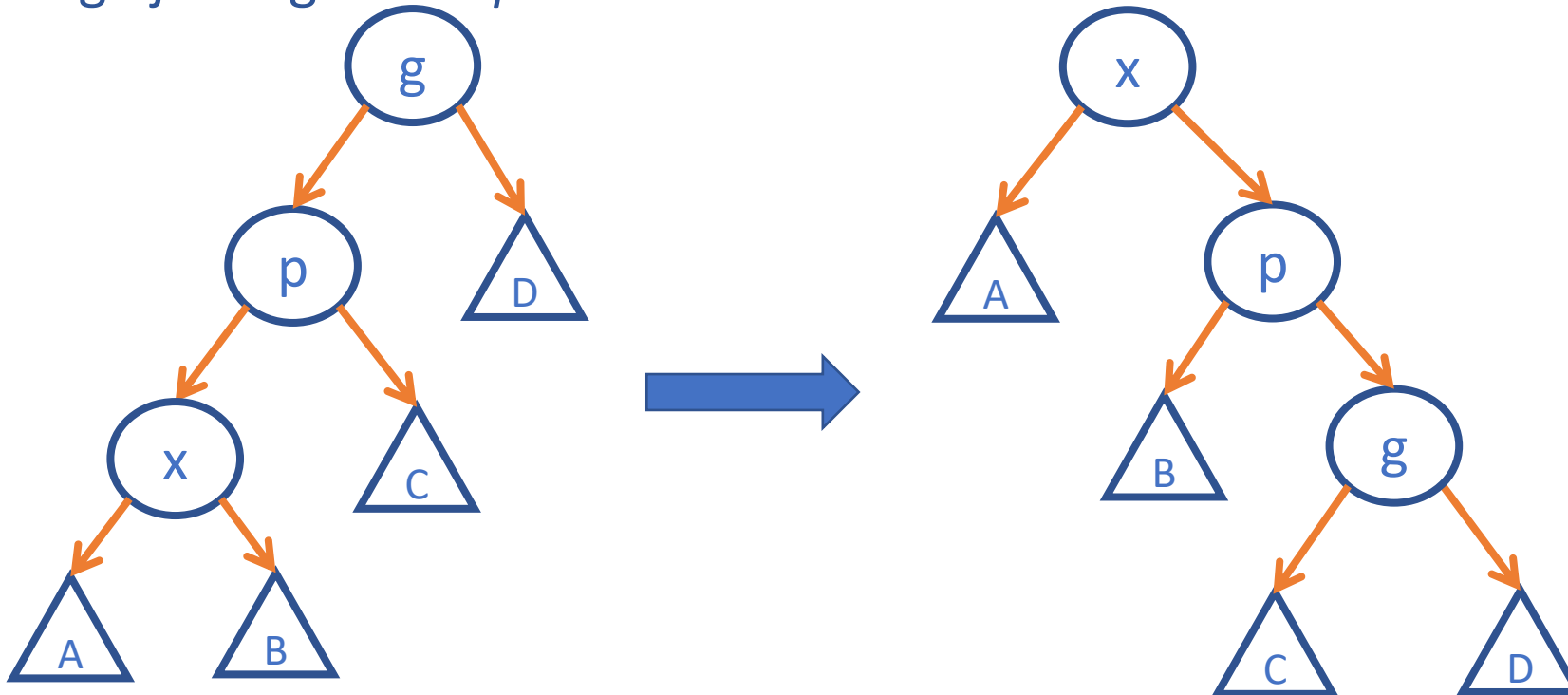
Zig step : this step is done when p is the root. The tree is rotated on the edge between x and p .



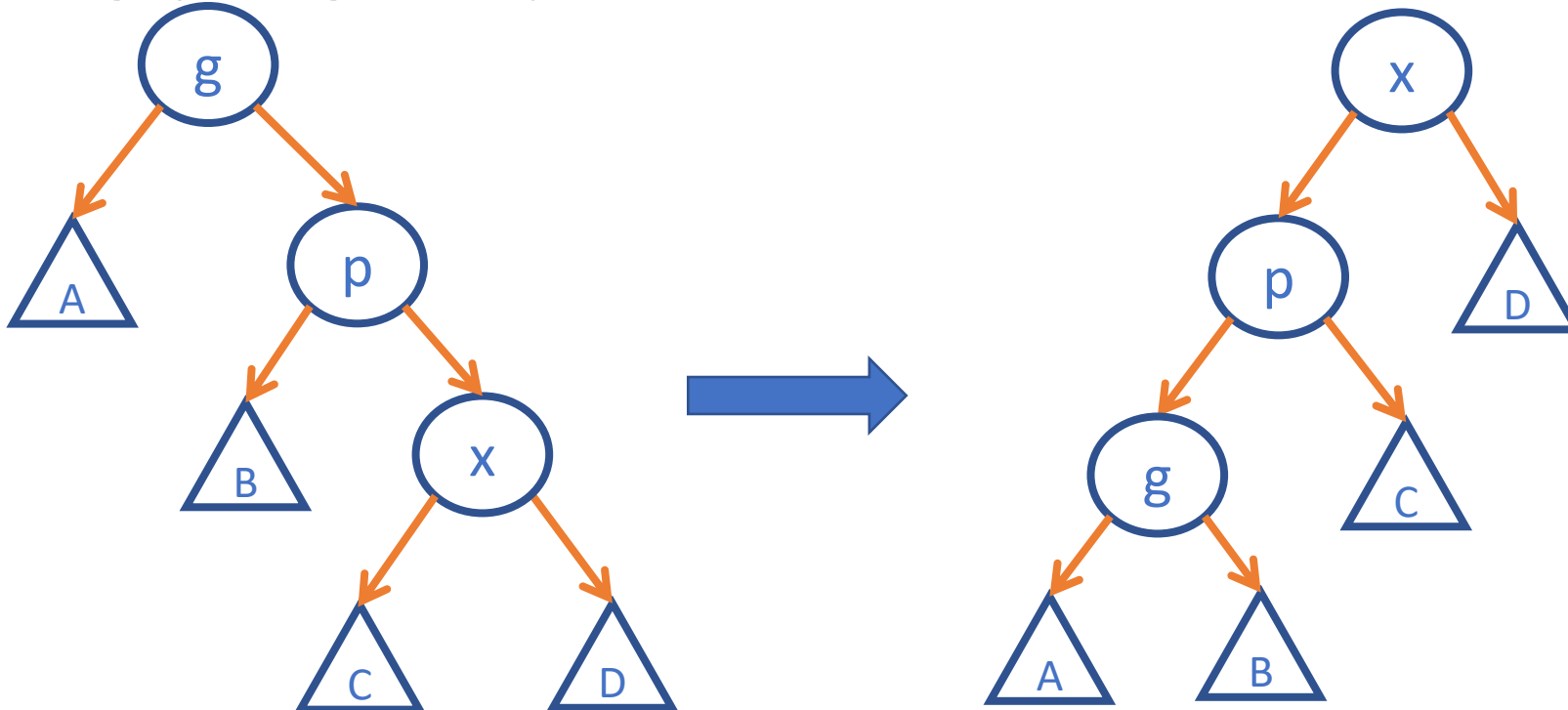
Zig step : this step is done when p is the root. The tree is rotated on the edge between x and p .



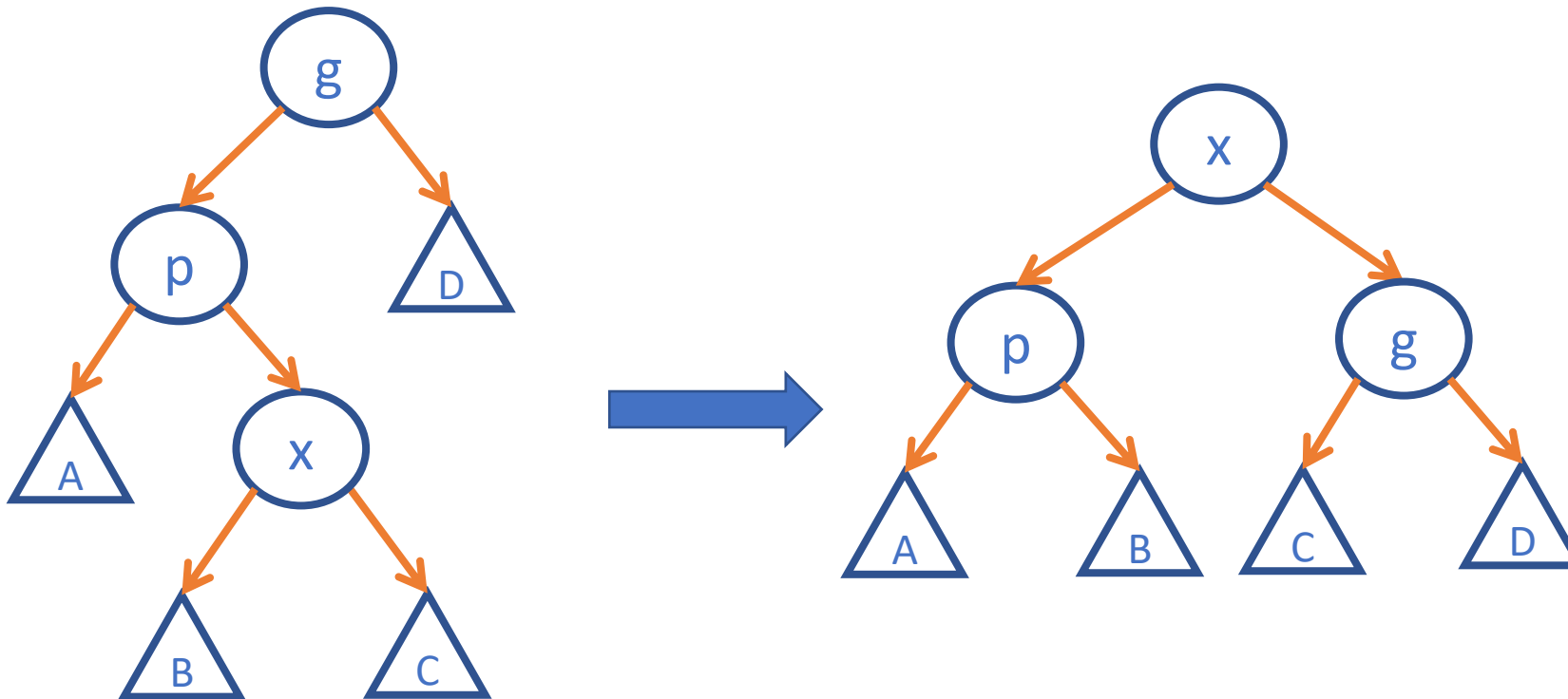
Zig-Zig step : this step is done when p is not the root and x and p are either both right children or are both left children. The picture below shows the case where x and p are both **left** children. The tree is rotated on the edge joining p with *its* parent g , then rotated on the edge joining x with p .



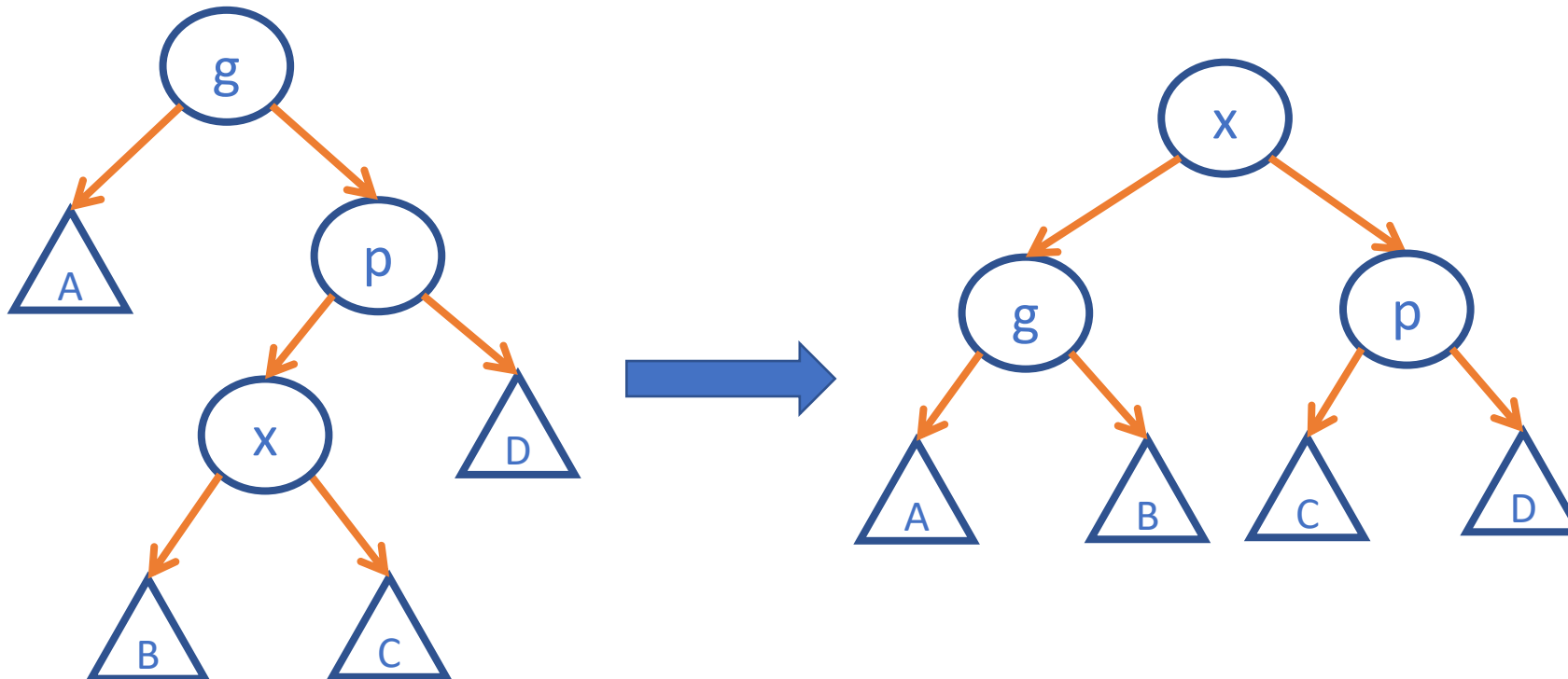
Zig-Zig step : this step is done when p is not the root and x and p are either both right children or are both left children. The picture below shows the case where x and p are both **right** children. The tree is rotated on the edge joining p with *its* parent g , then rotated on the edge joining x with p .



Zig-Zag step : this step is done when p is not the root and x is a right child and p is a left child or vice versa (x is left, p is right). The tree is rotated on the edge between p and x , and then rotated on the resulting edge between x and g .



Zig-Zag step : this step is done when p is not the root and x is a left child and p is a right child or vice versa (x is right, p is left). The tree is rotated on the edge between p and x , and then rotated on the resulting edge between x and g .



Insertion

To insert a value x into a splay tree:

Insert x as with a normal binary search tree.

when an item is inserted, a splay is performed.

As a result, the newly inserted node x becomes the root of the tree.

Deletion

To delete a node x , use the same method as with a binary search tree:

If x has two children:

- Swap its value with that of either the rightmost node of its left sub tree (its in-order predecessor) or the leftmost node of its right subtree (its in-order successor).
- Remove that node instead.

In this way, deletion is reduced to the problem of removing a node with 0 or 1 children.

Unlike a binary search tree, in a splay tree after deletion, we splay the parent of the removed node to the top of the tree.

DATA STRUCTURES AND ITS APPLICATIONS

References

- https://en.wikipedia.org/wiki/Splay_tree
- "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019.





THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree

Kusuma K V

Department of Computer Science & Engineering

Splaying

- There are three types of splay steps, each of which has two symmetric variants: left- and right-handed.
- **Zig step**: this step is done when p is the root.
- **Zig - Zig step**: this step is done when p is not the root and x and p are either both right children or are both left children.
- **Zig - Zag step**: this step is done when p is not the root and x is a right child and p is a left child or vice versa (x is left, p is right).

Insertion

To insert a value x into a splay tree:

Insert x as with a normal binary search tree.

when an item is inserted, a splay is performed.

As a result, the newly inserted node x becomes the root of the tree.

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Insertion

Insert 40, 30, 60, 10, 35, 50

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Insertion

Insert 40, 30, 60, 10, 35, 50

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Insertion

Insert 40, 30, 60, 10, 35, 50

DATA STRUCTURES AND ITS APPLICATIONS

References

- https://en.wikipedia.org/wiki/Splay_tree
- "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019.





THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree

Kusuma K V

Department of Computer Science & Engineering

Splaying

- There are three types of splay steps, each of which has two symmetric variants: left- and right-handed.
- **Zig step**: this step is done when p is the root.
- **Zig - Zig step**: this step is done when p is not the root and x and p are either both right children or are both left children.
- **Zig - Zag step**: this step is done when p is not the root and x is a right child and p is a left child or vice versa (x is left, p is right).

Deletion

To delete a node x , use the same method as with a binary search tree:

If x has two children:

- Swap its value with that of either the rightmost node of its left sub tree (its in-order predecessor) or the leftmost node of its right subtree (its in-order successor).
- Remove that node instead.

In this way, deletion is reduced to the problem of removing a node with 0 or 1 children.

Unlike a binary search tree, in a splay tree after deletion, we splay the parent of the removed node to the top of the tree.

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Deletion

Delete 40, 10, 35, 70

DATA STRUCTURES AND ITS APPLICATIONS

Splay Tree Search

Search 50, 20



DATA STRUCTURES AND ITS APPLICATIONS

References

- https://en.wikipedia.org/wiki/Splay_tree
- "Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019.





THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE19CS202

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Trees

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Tree

//Returns the smallest element in Binary Search Tree

```
int minimum(struct tnode *t)
{
    while(t->left!=NULL)
        t=t->left;
    return(t->data);
}
```



DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Tree

//Returns the largest element in Binary Search Tree

```
int maximum(struct tnode *t)
{
    while(t->right!=NULL)
        t=t->right;
    return(t->data);
}
```



DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Tree

//Computes the height of a Binary Tree

```
int height(struct tnode *t)
{
    if(t==NULL)
        return -1;
    if((t->left==NULL)&&(t->right==NULL))
        return 0;
    return (1+max(height(r->left),height(r->right)));
}
```

DATA STRUCTURES AND ITS APPLICATIONS

Programs on Binary Tree



//Count the number of leaf nodes in a Binary Tree

```
int leafcount(struct tnode *t)
{
    if(t==NULL)
        return 0;
    if((t->left==NULL)&&(t->right==NULL))
        return 1;
    int l=leafcount(t->left);
    int r=leafcount(t->right);
    return(l+r);
}
```



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu

+91 9449867804



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Basic Structure of a Queue , Implementation using an Array

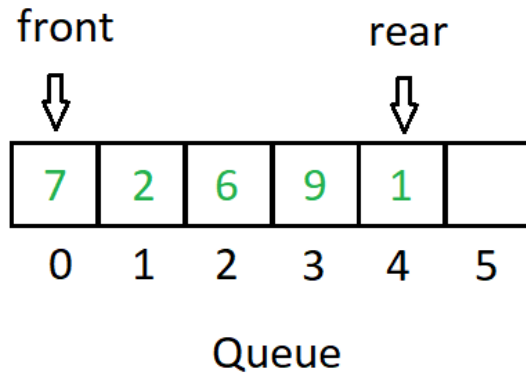
Dinesh Singh

Department of Computer Science & Engineering

Data Structures and its Applications

Queue Data Structure - definition

- A Queue is an ordered collection of items from which items may be deleted at one end (called the front of the queue)and into which items may be inserted at the other end (called the rear of the queue).



- **Different Types of Queues**
 - Simple Queue
 - Circular Queue
 - Priority Queue
 - Dequeue
- **Implementation**
 - Sequential Representation (Arrays)
 - Linked Representation (Linked Lists)

- Three primitive operations can be applied to the queue
- $\text{Insert}(q, x)$: inserts x at the rear of the queue q
- $x = \text{remove}(q)$: deletes front element from the queue and set x to its contents
- $\text{empty}(q)$: returns true or false depending on whether the queue contains any elements.
- Insert operation cannot be performed if the queue has reached the maximum size.
- The result of an illegal attempt to insert an element into a queue which has reached its maximum size is called overflow.
- The remove operation can be applied only if the queue is non empty.
- The result of an illegal attempt to remove an element from an empty queue is called underflow.
- The empty operation is always applicable.

```
#define MAXQUEUE 100
struct queue
{
    int items [MAXQUEUE];
    int front, rear;
};
```

```
struct queue q;
q.rear = q.front = -1;
```

Functions to implement the operations

- insert (x, &q)
- remove (&q)
- empty(&q)

Inserting into a queue

1. Check the queue for overflow condition
2. If the queue is not in overflow condition, increment the rear pointer and insert the element at a location indicated by the rear pointer.
3. If this is the first element in the queue, initialise front to 0.
4. Return 1

Data Structures and its Applications

Simple Queue – Implementation of Insert



```
int insert(int x,struct queue *q)
{
    //check queue overflow
    if(q->rear==MAXQUEUE - 1)
    {
        printf("Queue overflow..\n");
        return -1;
    }
    (q->rear)++;
    q->items[q->rear]=x;
    if(q->front==-1)//if first element
        q->front=0;
    return 1;
}
```

Removing element from the queue

1. Check the queue for underflow condition
2. If the queue is not in underflow condition, remove the element pointed by the front pointer into x.
3. If this was the only element in the queue, initialise front and rear to -1.
4. Return x

Data Structures and its Applications

Simple Queue – Implementation of remove



```
int remove(struct queue *q)
{
    int x;

    if(q->front==-1)
    {
        printf("Queue empty..\n");//underflow
        return -1;
    }
    x=q->items[q->front];
    if(q->front==q->rear)//only one element in queue
        q->front=q->rear=-1;
    else
        (q->front)++;
    return x;
}
```

Data Structures and its Applications

Simple Queue – Implementation of empty and display



```
int empty ( struct queue * q)
{
    if (q->front ==-1)
        return 1;
    return -1;
}

display (struct queue * q)
{
    int i;
    if(q->front==-1)
        printf("Empty queue..\n")
    else
    {
        for ( i = q->front; i<=q->rear;i++)
            printf("%d",q->items[i]);
    }
}
```

Implementation where queue represented as an array , front and rear are separate variables

```
#define MAXQUEUE 100
```

```
int q[MAXQUEUE]
```

```
int front, rear
```

```
front = rear = -1
```

Function calls to implement the operations

- insert(x, q, &front, &rear) ;
- remove (q , &front, &rear);
- empty(q, &front)

Data Structures and its Applications

Simple Queue – Another Implementation



Implementation where queue represented as an array , front and rear are separate variables

```
int insert(int x,int *q,int *f,int *r)
{
    //check queue overflow
    if(*r==MAXQUEUE - 1)
    {
        printf("Queue overflow..\n");
        return -1;
    }
    (*r)++;
    q[*r]=x;
    if(*f==-1)//if first element
        *f=0;
    return 1;
}
```

Data Structures and its Applications

Simple Queue – Another Implementation



Implementation where queue represented as an array , front and rear are separate variables

```
int remove(int *q,int *f,int *r)
{
    int x;
    if(*f==-1)
    {
        printf("Queue empty..\n");
        return -1;
    }
    x=q[*f];
    if(*f==*r)//only one element in queue
        *f=*r=-1;
    else
        (*f)++;
    return x;
}
```

Data Structures and its Applications

Simple Queue – Another Implementation



Implementation where queue represented as an array , front and rear are separate variables

```
display (int *q, int f, int r)
{
    int i;
    if(f==-1)
        printf("Empty queue..\n")
    else
    {
        for ( i = f; i<= r; i++)
            printf("%d",q[i]);
    }
}
```



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Queues – Linked List Implementation

Dinesh Singh

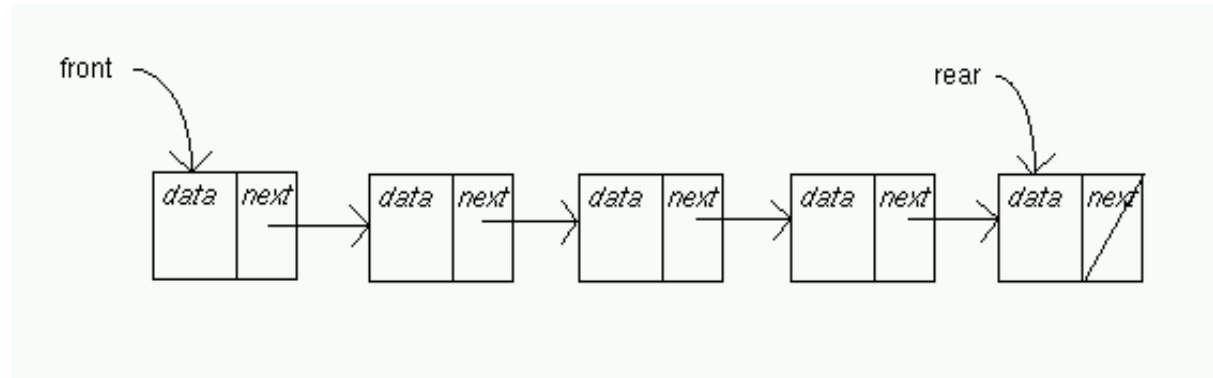
Department of Computer Science & Engineering

Data Structures and its Applications

Queues - Linked list Implementation

In a linked list implementation two pointers are maintained :
front and rear .

- front points to the first item of the queue
- rear points to the last item of the queue



Data Structures and its Applications

Queues - Linked list Implementation



Operations :

- **Insert()** : adds a new node after the rear and moves rear to the next node
- **Remove()** : removes the first node and moves front to the next node
- **Empty()** : Checks if the queue is empty

Data Structures and its Applications

Queues - Linked list Implementation



Structure of queue

```
struct node
{
    int data;
    struct node *next;
};
struct queue
{
    struct node * front;
    struct node *rear;
};
```

```
Struct queue q;
q.front=q.rear = NULL;
```

Insert operation

Insert(q,x)

```
p=getnode();  
initialise the node  
if(q.rear=NULL)  
    q.front=p;  
else  
    next(q.rear) =p;  
q.rear = p;
```

remove operation

remove(q)

```
If(empty(q)  
    print empty queue  
else  
    p=q.front;  
    x=info(p);  
    q.front = next(p);  
    if(q.front =NULL)  
        q,rear=NULL  
    freenode(p);  
    return x;
```

Data Structures and its Applications

Queues - Linked list Implementation – Operations



Insert operation of queue implemented by a linked list

```
void qinsert(struct node * q, int x)
{
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=NULL;

    //if this is the first node
    if(q->front==NULL)
        q->front=q->rear=temp;
    else //insert at the end
    {
        q->rear->next=temp;
        q->rear=temp;
    }
}
```

remove operation of a queue implemented by a linked list

```
int qremove(struct queue * q)
{
    struct node *p;
    int x;
    p=q->front;
    if(p==NULL)
    {
        printf("Empty queue\n");
        return -1;
    }
}
```

```
else
{
    x=q->data;
    if(q->front==q->rear) //only one node
        q->front=q->rear=NULL;
    else
    {
        q->front=q->next; // move front to next node
        return x;
    }
    free(q);
}
```



```
void qdisplay(struct queue q)
{
    struct node * f, *r;
    if(q.front==NULL)
        printf("Queue Empty\n");
    else
    {
        f=q.front; r=q.rear;
        while(f!=r)
        {
            printf("%d-> ",f->data);
            f=f->next;
        }
        printf("%d-> ",f->data); // print the last node
    }
}
```

Data Structures and its Applications

Queues - Linked list Implementation – Operations



Insert operation in an alternate way

```
void qinsert(int x, struct node **f, struct node **r)
//f and r are pointers to variables front and rear of a queue
{
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=NULL;

    //if this is the first node
    if(*f==NULL)
        *f=*r=temp;
    else //insert at the end
    {
        (*r)->next=temp;
        *r=temp;
    }
}
```

Remove operation in an alternate way

```
int qdelete(struct node **f, struct node **r)
{
    struct node *q;
    int x;
    q=*f;
    if(q==NULL)
    {
        printf("Empty queue\n");
        return -1;
    }
    else
    {
        x=q->data;
        if(*f==*r) //only one node
            *f=*r=NULL;
        else
        {
            *f=q->next;
            return x;
        }
        free(q);
    }
}
```

Disadvantages of representing queue by a linked list

- A node in linked list occupies more storage than the corresponding element in an array.
- Two pieces of information per element is necessary in a list node, where as only one piece of information is needed in an array implementation



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

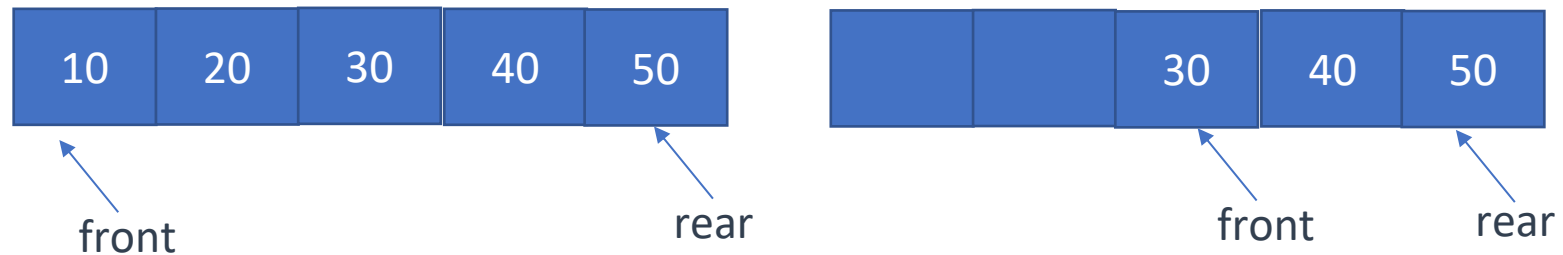
Circular Queue - Implementation

Dinesh Singh

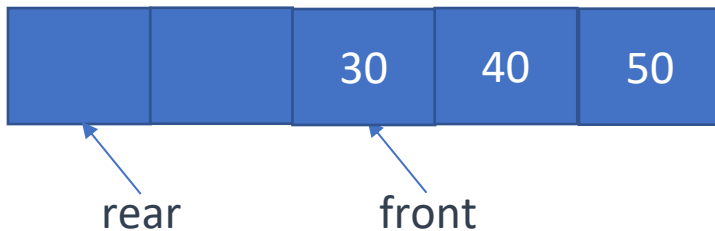
Department of Computer Science & Engineering

- Circular Queue is a linear data structure, which follows the principle of FIFO(First In First Out), but instead of ending the queue at the last position, it again starts from the first position after the last, hence making the queue behave like a circular data structure.
- In a simple queue, once the queue is completely full, it's not possible to insert more elements. Even if we perform remove operation on the queue to remove some of the elements, until the queue is reset, no new elements can be inserted

Structure of the simple queue



Cannot insert even after two elements are removed and
Space available in the front.



It is possible to insert in a circular queue by moving the rear
To the beginning of the queue

- To insert into the queue : Finding the rear index
 $\text{rear} = (\text{rear} + 1) \% \text{size}$
 If(rear=front)
 cannot insert
 else
 insert at rear index
- For eg. If size = 5, front = 2 and rear = 4
- $\text{rear} = (4 + 1) \% 5 = 0,$
- The new element gets inserted at index 0

- For eg. If size = 5, front = 0 and rear = 4
- $\text{rear} = (4 + 1) \% 5 = 0,$
- rear = front , therefore cannot insert

- To remove from the queue :
remove the element pointed by front , move the front
 $\text{front} = (\text{front} + 1) \% \text{size}$

For eg. If size = 5, front = 2 and rear = 4

- $\text{front} = (2 + 1) \% 5 = 3,$
- front moves to index 3 after removal of the element,

- For eg. If size = 5, front = 4 and rear = 2
- $\text{front} = (4 + 1) \% 5 = 0,$
- Front moves to 0 after removal of the element

```
#define MAXQUEUE 100
struct queue
{
    int items [MAXQUEUE];
    int front, rear;
};
```

```
struct queue q;
q.rear = q.front = -1;
```

Functions to implement the operations

- insert (&q,x)
- remove (&q)

Data Structures and its Applications

Implementation of operations - insert

```
int qinsert(struct queue *q, int x)
{

    //check for queue overflow
    if((q->r+1)%MAXQUEUE==q->f)
    {
        printf("Queue Overflow..\n");
        return -1;
    }
    else
    {
        q->rear=(q->rear+1)%size;    //get the rear index
        q->item[q->rear]=x;    //insert at rear index
        if(q->front==-1) //if first element
            q->front=0;    // make front point to 0
        return 1;
    }
}
```

Data Structures and its Applications

Implementation of operations - insert

//ANOTHER WAY TO IMPLEMENT INSERT

```
int qinsert(int *q,int *f, int *r, int size, int x)
{
    // f ,r are pointers to front and rear of the queue, size is the max size of queue
    //check for queue overflow
    if((*r+1)%size==*f)
    {
        printf("Queue Overflow..\n");
        return -1;
    }
    else
    {
        *r=(*r+1)%size;    //get the rear index
        q[*r]=x;    //insert at rear index
        if(*f==-1) //if first element
            *f=0;    // make front point to 0
        return 1;
    }
}
```

Data Structures and its Applications

Implementation of operations - remove

```
int remove(struct queue *q)
{
    int x;
    if(q->front==-1) //check for empty queue
    {
        printf("Queue empty..\n");
        return -1;
    }
    else
    {
        x=q->items[q->front];
        if(q->front==q->rear)//only one element
            q->front=q->rear=-1;
        else
            q->frontf=(q->front+1)%MAXQUEUE; //increment the front
        return x;
    }
}
```

Data Structures and its Applications

Implementation of operations - remove

```
//ANOTHER WAY TO IMPLEMENT REMOVE
```

```
int remove(int *q, int *f, int *r,int size)
```

```
{
```

```
    int x;
```

```
    if(*f==-1) //check for empty queue
```

```
    {
```

```
        printf("Queue empty..\n");
```

```
        return -1;
```

```
    }
```

```
else
```

```
{
```

```
    x=q[*f];
```

```
    if(*f==*r)//only one element
```

```
        *f=*r=-1;
```

```
else
```

```
    *f=(*f+1)%size; //increment the front
```

```
    return x;
```

```
}
```

```
}
```


Data Structures and its Applications

Implementation of operations - display

```
void display(struct queue q)
{
    if(q.front==-1)
        printf("\nQueue empty..\n");
    else
    {
        while(q.front!=q.rear) //increment front till it reaches rear
        {
            printf("%d ",q.items[q.front]);
            q.front=(q.front+1)%MAXQUEUE;
        }
        printf("%d ",q->items[q->front]); // display the last element
    }
}
```

Data Structures and its Applications

Implementation of operations - display

```
void display(int *q, int f, int r, int size)
{
    if(f==-1)
        printf("\nQueue empty..\n");
    else
    {
        while(f!=r) //increment front till it reaches rear
        {
            printf("%d ",q[f]);
            f=(f+1)%size;
        }
        printf("%d ",q[f]); // display the last element
    }
}
```



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Priority Queue - Implementation

Dinesh Singh

Department of Computer Science & Engineering

Priority Queues - definition

- Priority Queue is an extension of queue where every item has a priority associated with it.
- There are two types of priority queue
 - Ascending priority queue
 - Descending Priority queue
- In Ascending Priority queue ,the item with lowest priority is removed
- In descending priority queue, the item with the highest priority is removed.

- Operations in Ascending priority queue
 - `pqinsert(apq,x)` : inserts element x into the priority queue apq.
 - `pqmindelete(apq)` : removes the element with the minimum priority.
- Operations in Descending priority queue
 - `pqinsert(dpq,x)` : inserts element x into the priority queue dpq.
 - `pqmaxdelete(dpq)` : removes the element with the maximum priority.
- The operation `empty(pq)` determines whether the queue is empty or not.

- In alternate definition of a priority queue, the items do not have a priority but the intrinsic ordering of elements determine results of its basic operation.
- In ascending priority queue, items can be inserted arbitrarily, but only the smallest element can be removed.
- In descending priority queue, items can be inserted arbitrarily, but only the largest element can be removed.

- Priority queues can be implemented by
 - Arrays
 - Linked Lists
 - Heap

Data Structures and its Applications

Priority Queues- Array Implementation



```
struct pqueue
{
    int data;
    int pty;
}
struct pqueue pq[10];
```

implementation of descending priority :

Items inserted based on their priority

- **Insert :** The item is inserted based on its priority. The queue is ordered in the descending priority of the items. The item with the highest priority will be at the first location in the array.
- **Remove :** delete always the first element, Shift the other elements to the left after the deletion

Data Structures and its Applications

Priority Queues- Array Implementation



implementation of descending priority :

Items inserted arbitrarily

- **Insert** : The item is inserted at the rear of the queue. The queue is therefore unordered.
- **Delete** : The item with the highest priority is found and removed. The items following the position of the removed element are shifted to the left.

implementation of descending priority where items are inserted according to the priority

```
void pqinsert(int x,int pty,struct pqueue *pq,int *count)
{
    // x is item to be inserted
    // pty is the priority of the item
    // pq is the pointer to the priority queue
    // count is the number of items in the queue

    int j;
    struct pqueue key;
    key.data=x;
    key.ptty=pty;
```

Data Structures and its Applications

Priority Queues- Array Implementation

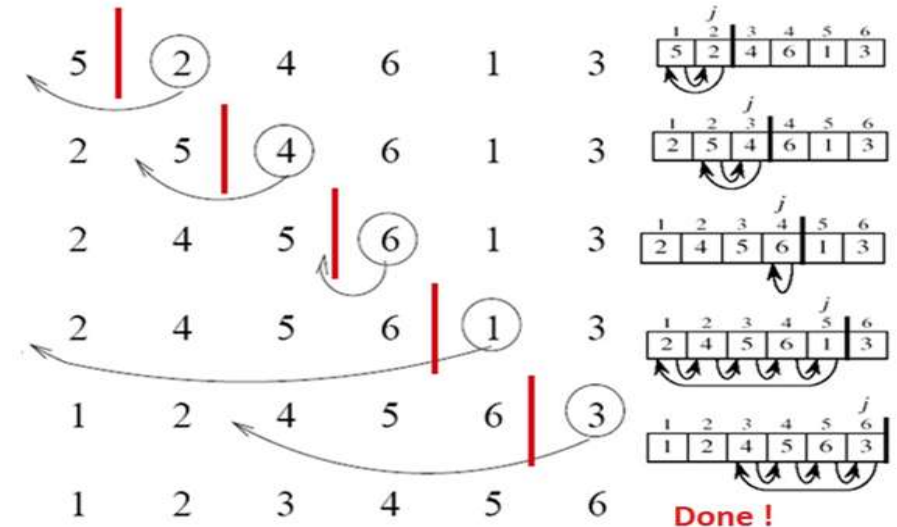
implementation of descending priority where items are inserted according to the priority

$j = *count - 1$; // index of the initial position of the element

//compare the priority of the item being inserted with the
//priority of the items in the queue

// shift the items down while the priority of the item being
//inserted is greater than priority of the item in the queue

```
while((j >= 0) && (pq[j].pty > key.pty))
{
    pq[j+1] = pq[j];
    j--;
}
pq[j+1] = key; // insert the element at its correct location
(*count)++;
}
```



implementation of descending priority where items are inserted according to the priority

```
struct pqueue pqdelete(struct pqueue *pq,int *count)
{
    // pq is a pointer to the priority queue
    // count is the number of elements in the queue

    int i;
    struct pqueue key;
    // if queue is empty, return a structure with priority -1

    if(*count==0)
    {
        key.data=0;
        key.pty=-1;
    }
```

implementation of ascending priority where items are inserted according to the priority

```
//delete the first item
//shift the other items to the left
else
{
    key=pq[0];
    for(i=1;i<=*count-1;i++)
        pq[i-1]=pq[i];
    (*count)--;
}
return key; //return the key with the lowest priority
}
```

Data Structures and its Applications

Priority Queues- Array Implementation



Implement the following

- Ascending Priority queue where item is inserted at the end and item with the lowest priority is deleted.
- Descending priority queue
- Ascending and Descending Priority queue using a linked list



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Deque - Implementation

Dinesh Singh

Department of Computer Science & Engineering

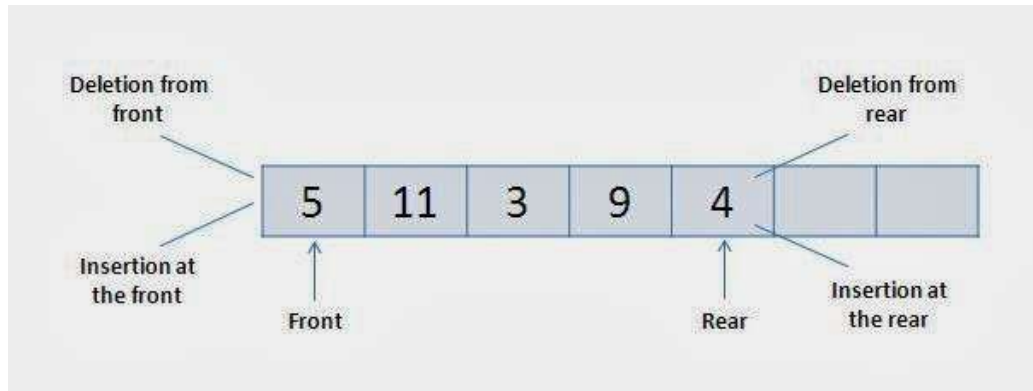
Deque(Double ended Queue) - definition

Double ended queue is a queue that allows insertion and deletion at both ends.



The following four basic operations are performed on dequeue:

- *insertFront()*: Adds an item at the front of Deque.
- *insertRear()*: Adds an item at the rear of Deque.
- *deleteFront()*: Deletes an item from front of Deque.
- *deleteRear()*: Deletes an item from rear of Deque.



Data Structures and its Applications

Deque (Double ended Queue) - Array Implementation

Insert Elements at Rear end :

Check whether the queue is full

If $\text{rear} = \text{size} - 1$

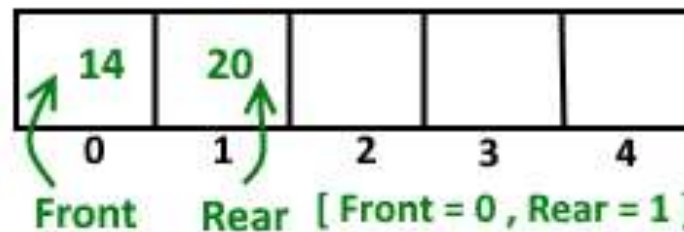
initialise rear to 0.

else

increment rear by 1

insert element at location rear

Insert element at Rear



Insert element front end

Check if the queue is full

If $\text{Front} = 0$

move front to last location ($\text{size} - 1$)

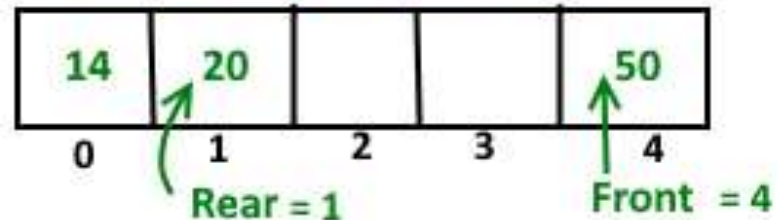
else

decrement front by 1

insert at location front

Insert element at Front end

Now Front points last index



Data Structures and its Applications

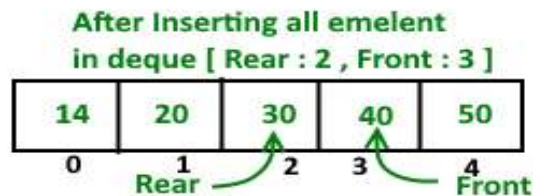
Deque(Double ended Queue) - Array Implementation

Delete element at Rear end

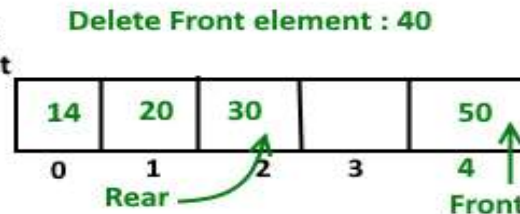
check if the queue is empty
delete the element pointed by rear
If dequeue has one element
 $\text{front} = -1$ $\text{rear} = -1$;
If rear is at first index
 make $\text{rear} = \text{size} - 1$
else
 decrease rear by 1

Delete element at front end

check if the queue is empty
delete the element pointed by front
If dequeue has one element
 $\text{front} = -1$ $\text{rear} = -1$;
If front is at last index
 make $\text{front} = 0$
else
 increase front by 1



Delete Element from
Front end , New front



Structure of Dequeue

```
struct dequeue
{
    struct node * front;
    struct node * rear;
};
struct node
{
    int data;
    struct node * prev, *next;
};
struct dequeue dq;
```

```
dq.front=dq.rear = NULL
```


Data Structures and its Applications

Deque(Double ended Queue) - - Doubly Linked list Implementation



```
//insert in front of the queue
void qinsert_head(int x,struct dequeue *dq)
{
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->prev=temp->next=NULL;

    if(dq->front==NULL) // first element
        dq->front=dq->rear=temp;
    else
    {
        temp->next=dq->front; // insert in front
        dq->front->prev=temp;
        temp->prev=NULL;
        dq->front=temp;
    }
}
```

Data Structures and its Applications

Deque (Double ended Queue) - - Doubly Linked list Implementation



//insert at the rear of the queue

```
void qinsert_tail(int x, struct dequeue* dq)
{
    struct node *temp;

    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->prev=temp->next=NULL;

    if(dq->front==NULL)
        dq->front=dq->rear=temp;
    else
    {
        dq->rear->next=temp;
        temp->prev=dq->rear;
        dq->rear=temp;
    }
}
```

Data Structures and its Applications

Deque (Double ended Queue) - - Doubly Linked list Implementation



```
//delete at the front of the queue
int qdelete_head(struct dequeue* dq)
{
    struct node *q;
    int x;
    if(dq->front==NULL)
        return -1;

    q=dq->front;
    x=q->data;
    if(dq->front==dq->rear)//only one node
        dq->front=dq->rear=NULL;
    else
    {
        dq->front=dq->front->next;
        dq->front->prev=NULL;
    }
    free(q);
    return x;
}
```

Data Structures and its Applications

Deque (Double ended Queue) - - Doubly Linked list Implementation



```
//delete at the rear of the queue
int qdelete_tail(struct dequeue* dq)
{
    struct node *q;
    int x;
    if(dq->front==NULL)
        return -1;
    q=dq->rear;
    x=q->data;
    if(dq->front==dq->rear)//only one node
        dq->front=dq->rear=NULL;
    else
    {
        dq->rear=dq->rear->prev;
        dq->rear->next=NULL;
    }
    free(q);
    return x;
}
```



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Kundhavai K R

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Queues – Implementation of Josephus Problem

Kundhavai K R

Department of Computer Science & Engineering

Data Structures and its Applications

Queues – Implementation of Josephus Problem



- **Josephus Problem** : Postulates a group of soldiers surrounded by an overwhelming enemy force. There is no hope of victory without reinforcements. There is one horse available for escape
- The soldiers agree to a pact to determine which of them is to escape and seek help. The soldiers form a circle and a number n is picked from a hat. One of the names is also picked from the hat.
- Beginning with the soldier whose name is picked , they begin to count clockwise around the circle. when the count reaches n , that soldier is removed from the circle and the count begins with the next soldier.
- The process continues so that each time the count reaches n , another soldier is removed from the circle. Any soldier removed from the circle is no longer counted. The last soldier remaining is to take the horse and escape.

Data Structures and its Applications

Queues – Implementation of Josephus Problem



- The input to the program is the number n and list of names, which is the clockwise ordering of the circle, beginning with the soldier from whom the count is to start.
- The program should print the names in the order that they are eliminated and the name of soldier who escapes.
- For example if $n=3$ and that there are five soldiers named A,B,C,D and E. We count three soldiers starting at A, so that C is eliminated first.
- We then begin at D and count D E and back to A. A is eliminated. Then we count B D and E, E is eliminated. And finally B D and B is eliminated.
- D is the one who escapes

Data Structures and its Applications

Queues – Implementation of Josephus Problem



- Data structure used is a circular list where each node represents one soldier
- To represent the removal of a soldier from the circle, a node is deleted from the circular list.
- Finally one node remains on the list and the result is determined

Pseudo code of implementation using circular list

```
read(n)
read(name)
while(all the names are read)
{
    insert name on the circular list
    read(name)
}
while(there is more than one node on the list)
{
    count through n-1 nodes on the list
    print name in the nth node
    delete the nth node
}
print the name of the only node on the list
```

Code of implementation using circular list

```
int survivor(struct node **head, int n)
{
    // head is pointer to first node

    struct node *p, *q;
    int i;
    q = p = *head;
    while (p->next != p)
    {
        for (i = 0; i < n - 1; i++)
        {
            q = p;
            p = p->next;
        }
        q->next = p->next;
        printf("%d has been killed.\n", p->num);
        free(p);
        p = q->next;
    }
    *head = p;
    return (p->num);
}
```

Pseudo code of implementation using circular queue

Enter n

while(all the names are read)

{

 insert name into the queue

 read(name)

}

while(q has one element)

{

 dequeue n-1 names from the queue and enqueue it.

 dequeue the n^{th} name

 print the n^{th} name

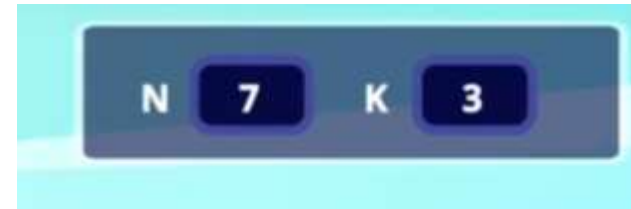
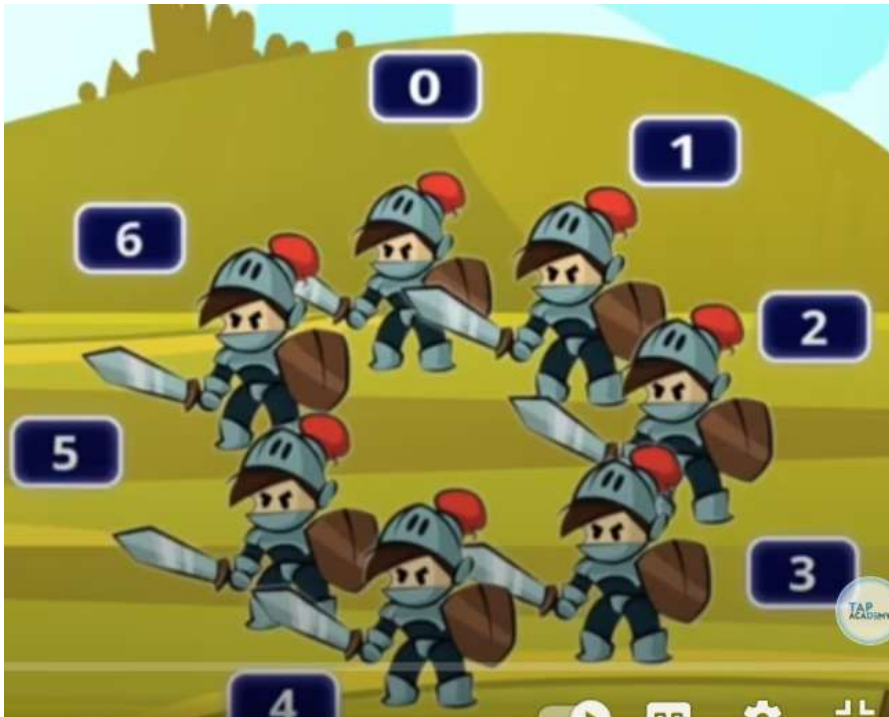
}

dequeue the only name of the queue

print the name

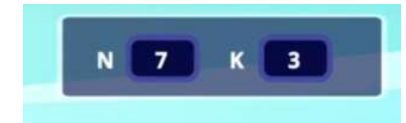
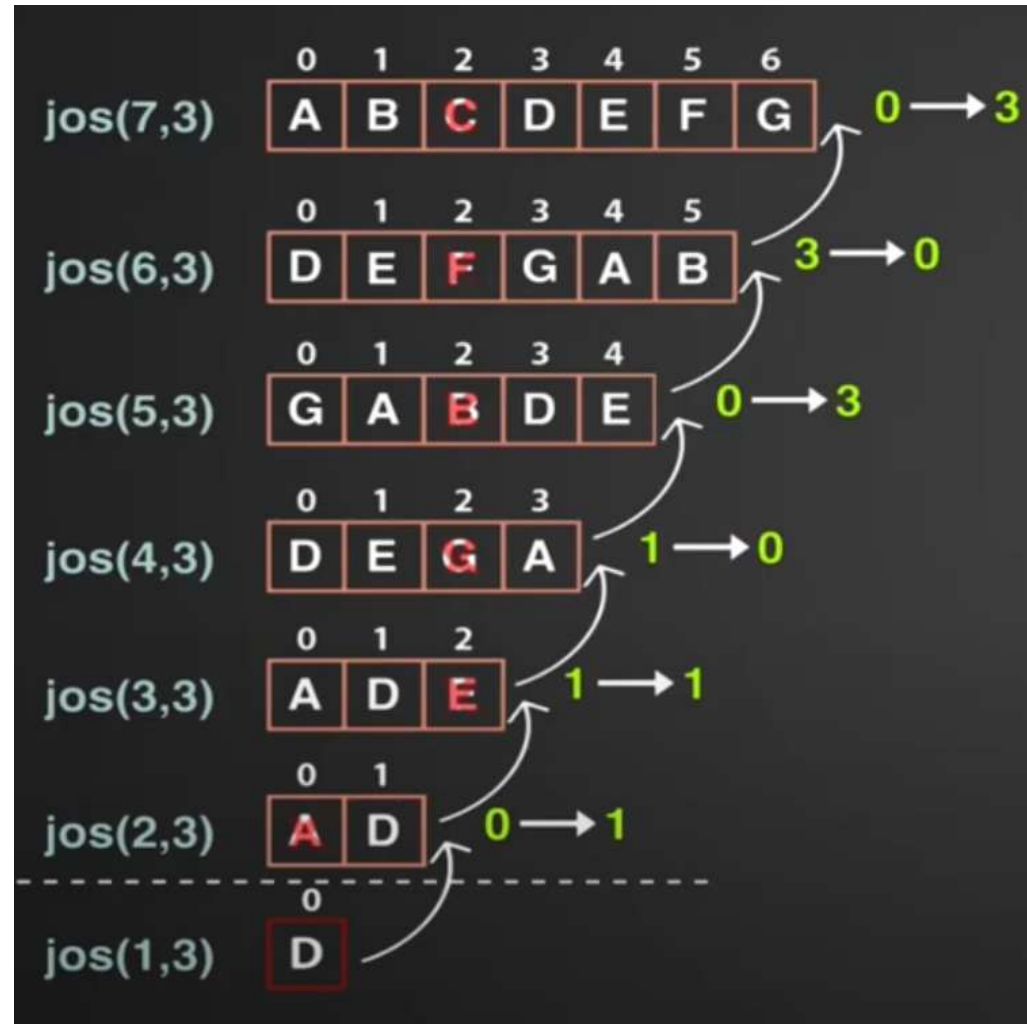
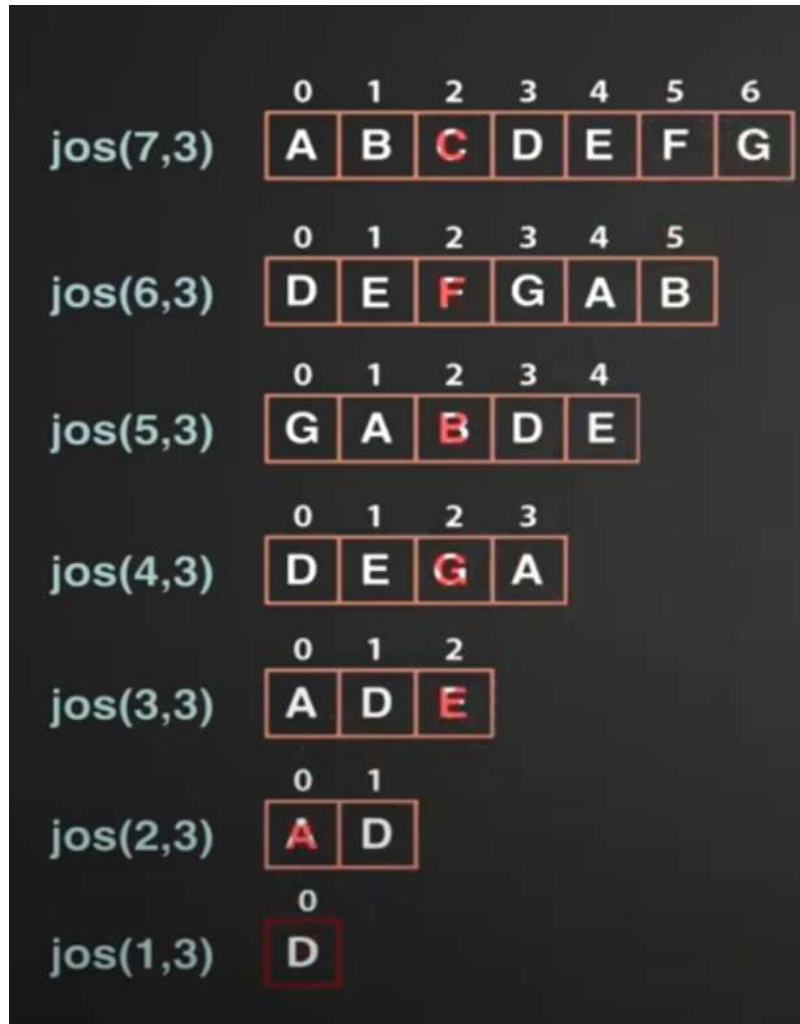
Data Structures and its Applications

Queues – Implementation of Josephus Problem Recursively



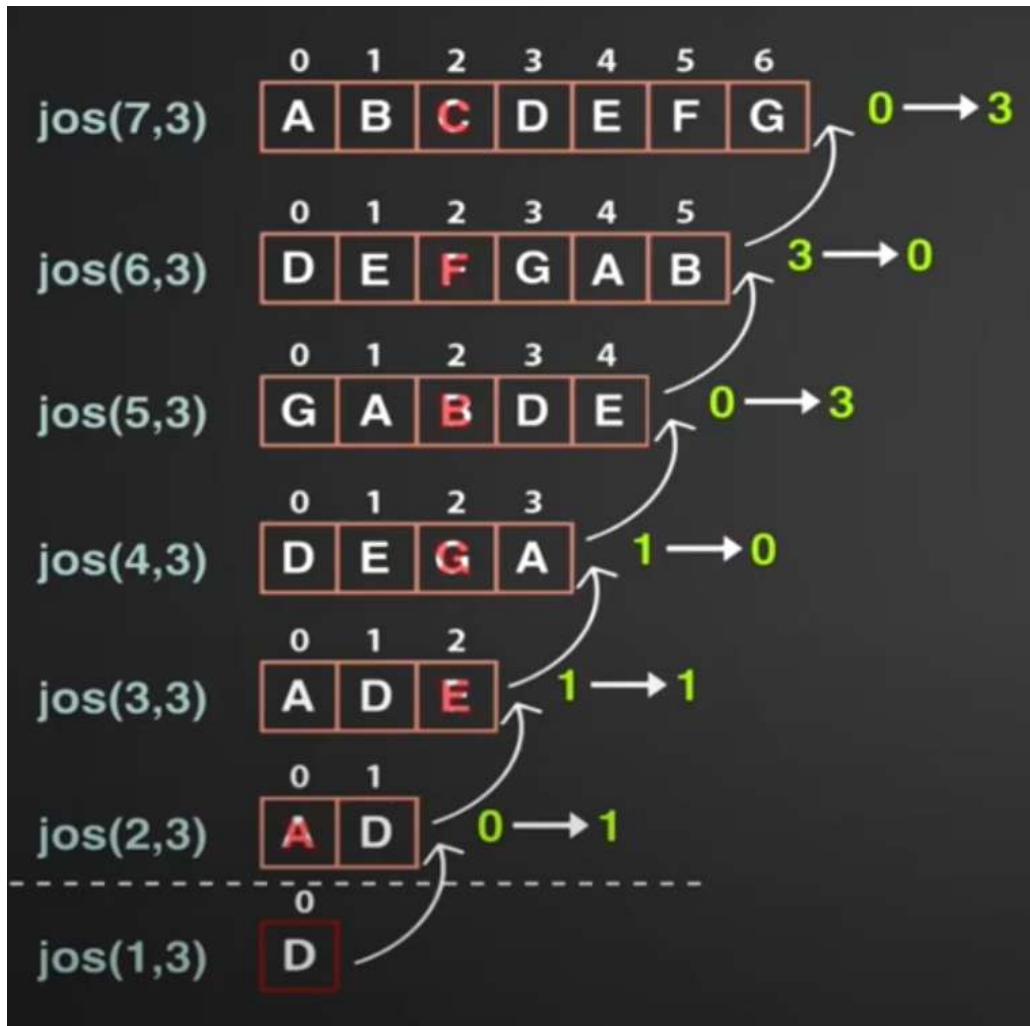
Data Structures and its Applications

Queues – Implementation of Josephus Problem



Data Structures and its Applications

Queues – Implementation of Josephus Problem



N 7 K 3	
0	3
1	4
2	5
3	6
4	0
5	1

N 7 K 3	
$(0+3)\%7$	3
$(1+3)\%7$	4
$(2+3)\%7$	5
$(3+3)\%7$	6
$(4+3)\%7$	0
$(5+3)\%7$	1

$$(\text{jos}(6,3)+k) \% n$$
$$\text{jos}(n,k) = (\text{jos}(n-1,k) + k) \% n$$

Data Structures and its Applications

Queues – Implementation of Josephus Problem



Assignment :

Implement the Josephus by using circular queue

Implement Josephus Problem by using recursion



THANK YOU

Kundhavai K R

Department of Computer Science & Engineering



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

CPU scheduling using Queue

Dinesh Singh

Department of Computer Science & Engineering

Different Scheduling Algorithms:

First Come First Serve CPU Scheduling:

- Simplest scheduling algorithm that schedules according to arrival times of processes.
- First come first serve scheduling algorithm states that the process that requests the CPU first is allocated the CPU.
- It is implemented by using the simple queue. When a process enters the ready queue, its PCB is linked onto the rear of the queue.
- When the CPU is free, it is allocated to the process at the front of the queue.
- The running process is then removed from the queue.

Data Structures and its Applications

CPU scheduling using queue



Shortest Job First(Preemptive):

- In Preemptive Shortest Job First Scheduling, jobs are put into the ready queue as they arrive
- As a process with short burst time arrives, the existing process is preempted or removed from execution, and the shorter job is executed first

Shortest Job First(Non-Preemptive):

- In Non-Preemptive Shortest Job First, a process which has the shortest burst time is scheduled first.
- If two processes have the same burst time then FCFS is used to break the tie

Data Structures and its Applications

CPU scheduling using queue



Longest Job First(Preemptive):

It is similar to an Shortest Job First scheduling(SJF) algorithm.

In this scheduling algorithm, priority is given to the process having the largest burst time remaining.

Longest Job First(Non-Preemptive):

- It is similar to an SJF scheduling algorithm. But, in this scheduling algorithm, priority is given to the process having the longest burst time.
- This is non-preemptive in nature i.e., when any process starts executing, can't be interrupted before complete execution.

Data Structures and its Applications

CPU scheduling using queue



Round Robin Scheduling:

- To implement Round Robin scheduling, The processes are kept in the queue of processes.
- New processes are added to the rear of the simple queue. The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1-time quantum, and dispatches the process.
- The process may have a CPU burst of less than 1-time quantum. In this case, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue.
- Otherwise, if the CPU burst of the currently running process is longer than 1-time quantum, the timer will go off and will cause an interrupt to the operating system.
- A context switch will be executed, and the process is put at the rear of the ready queue. The CPU scheduler will then select the next process in the ready

Data Structures and its Applications

CPU scheduling using queue



Preemptive Priority Based Scheduling:

- In Preemptive Priority Scheduling, at the time of arrival of a process in the ready queue, its priority is compared with the priority of the other processes present in the ready queue as well as with the one which is being executed by the CPU at that point of time.
- The One with the highest priority among all the available processes will be given the CPU next.

Priority Based(Non-Preemptive) Scheduling:

- In the Non Preemptive Priority scheduling, The Processes are scheduled according to the priority number assigned to them.
- Once the process gets scheduled, it will run till the completion



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



DATA STRUCTURES AND ITS APPLICATIONS

UE23CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Basic Concept and Definitions: Trees

Shylaja S S

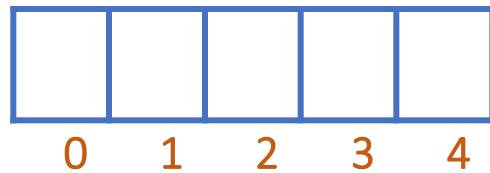
Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Trees



Linear Data Structures



List as an Array

Disadvantage:

- Fixed Size
 - Expansion ✗
 - Shrink ✗
- Random Insertion & Deletion is Time Consuming



List as a Linked List

Disadvantage:

- Random Access is Time consuming

DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Trees



Linear organization of
data doesn't help in
quick retrieval of
elements randomly



Go for Non Linear
Organization !!!

DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Trees



Example: To improve the probability of purchase of Women's Formal Wear in Less Time

Name: abc Gender: M Age: 25 email id: abc@xyz.com	Name: def Gender: F Age: 21 email id: def@xyz.com	Name: ghi Gender: F Age: 10 email id: ghi@xyz.com	...	Name: pqr Gender: F Age: 60 email id: pqr@xyz.com
---	---	---	-----	---

 0  1  2  ...  9999

	...	...		...
--	-----	-----	--	-----

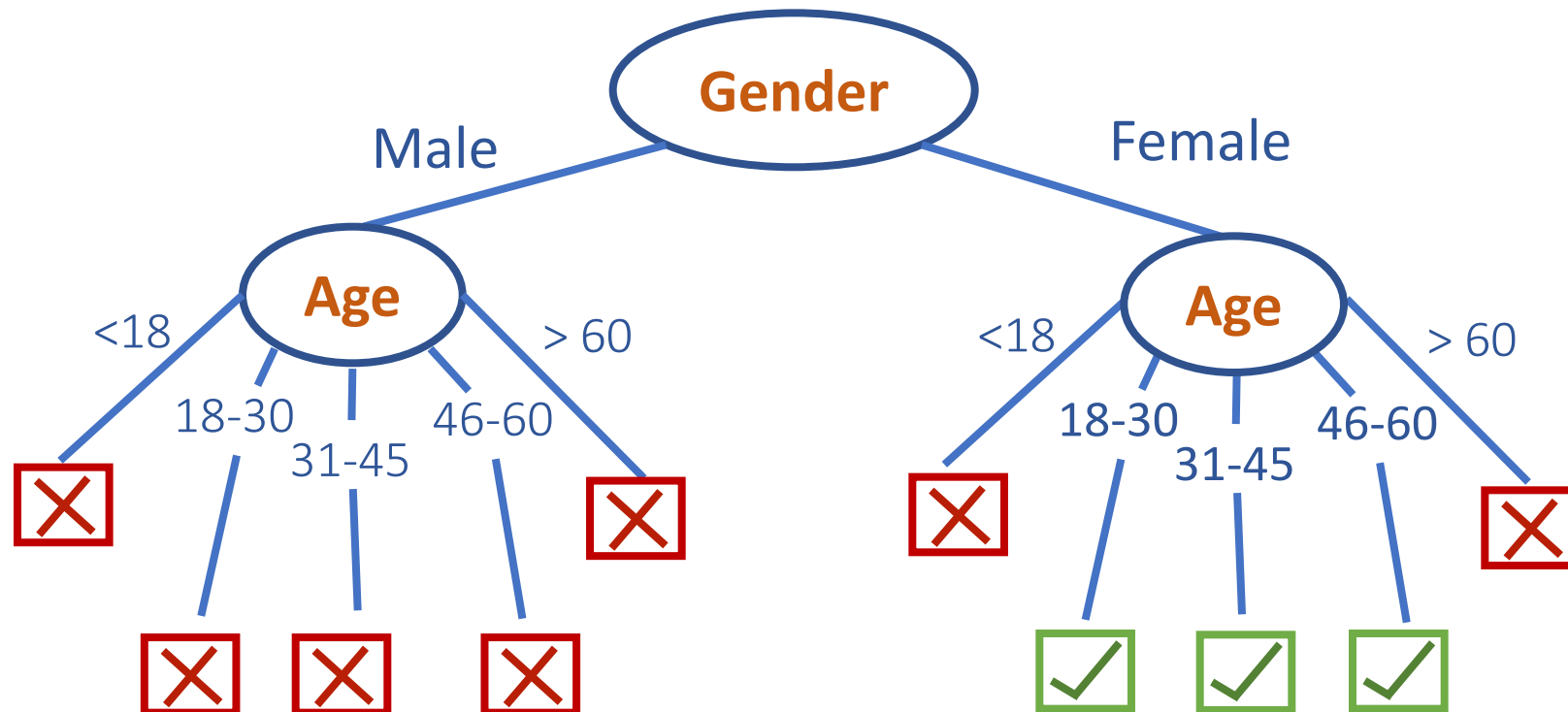
0 1 2 ... 9999

DATA STRUCTURES AND ITS APPLICATIONS

Introduction to Trees



Example: To improve the probability of purchase of Women's Formal Wear in Less Time



Search Not Matched

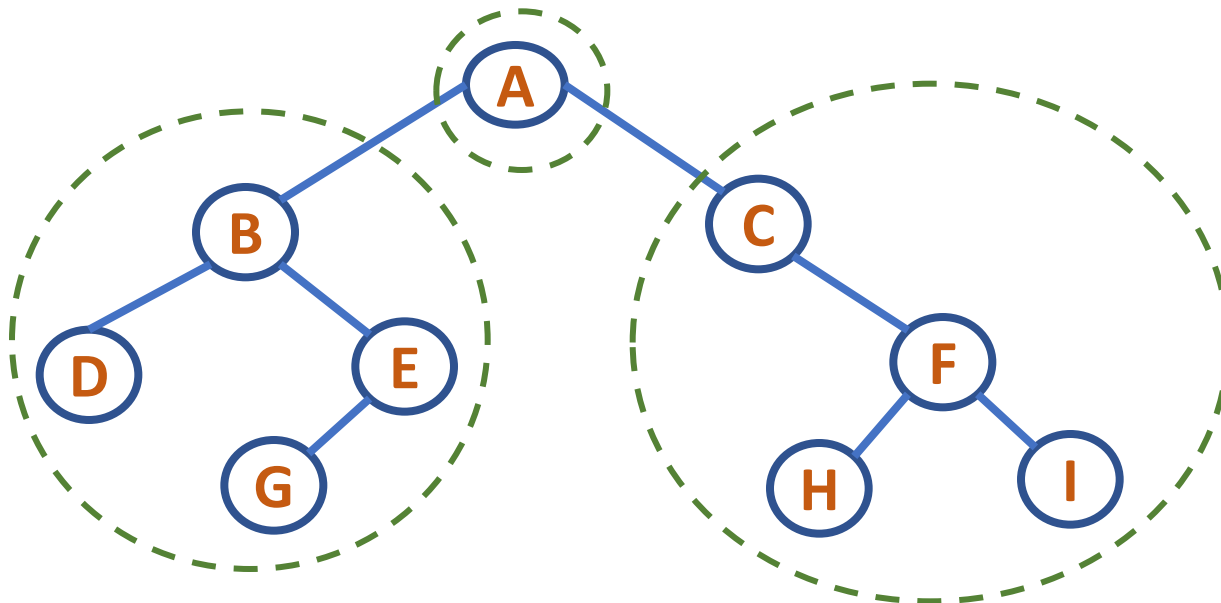


Search Matched

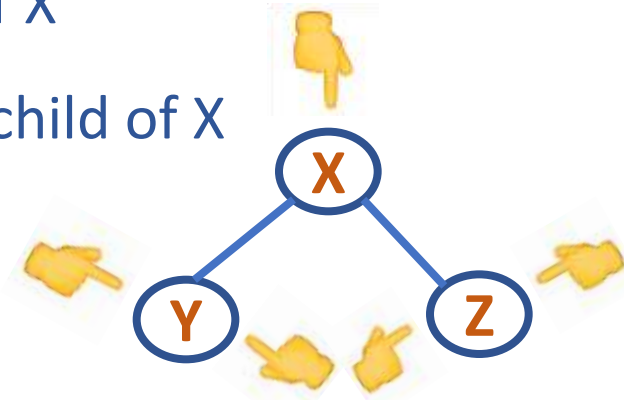
DATA STRUCTURES AND ITS APPLICATIONS

Binary Trees

- Non Linear Data Structure
- Finite set of elements that is either empty or is partitioned into three subsets
- First subset: is a single element, called the root
- Second subset: is a binary tree, called the left binary tree
- Third subset: is a binary tree, called the right binary tree



- Each element of a binary tree is called a node of the tree
- Left node Y of X is called left child of X
- Right node Z of X is called the right child of X
- X is called the parent of Y and Z
- Y and Z are called siblings
- A node which has no children is called leaf node/external node
- A node which has a child is called the non leaf node/internal node



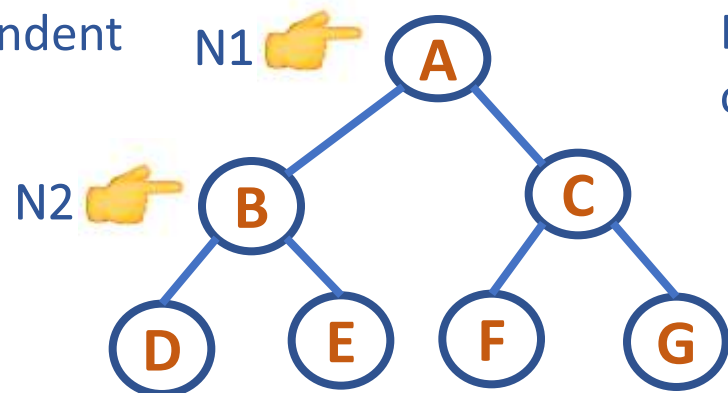
DATA STRUCTURES AND ITS APPLICATIONS

Binary Trees: Terminologies

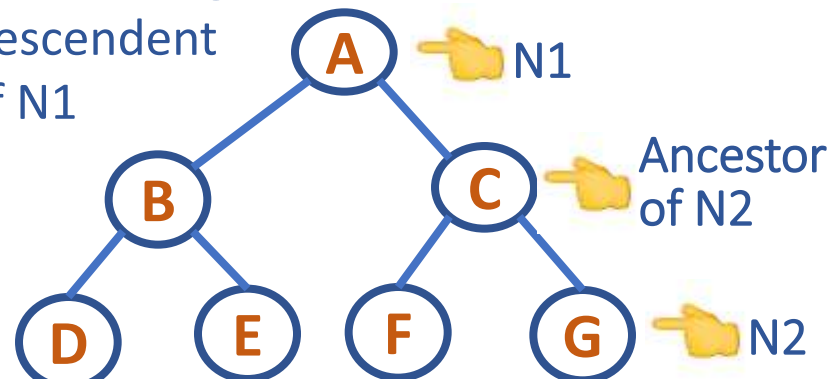


- A node N1 is called the ancestor of a node N2 if
 - N1 is either the parent of N2 or
 - N1 is the parent of some ancestor of N2
- A node N2 becomes the descendent of node N1
- Descendent can be either the left descendent or the right descendent

N2 is the Left
Descendent
of N1



N2 is the Right
Descendent
of N1



DATA STRUCTURES AND ITS APPLICATIONS

Binary Trees: Terminologies



- Level of a node

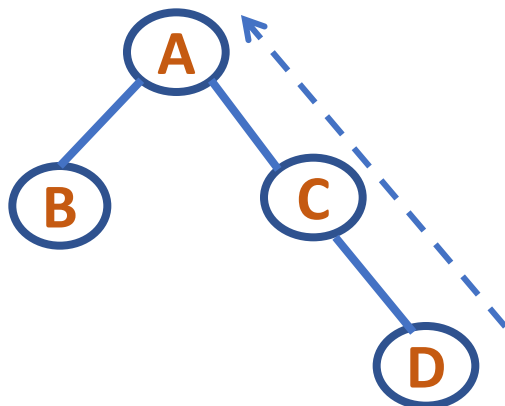
Root has level 0; level of any other node is one more than its parent

- Depth of a tree

Maximum level of any leaf in the tree (path length from the deepest leaf to the root)

- Depth of a node

Path length from the node to the root



Level of node A – 0

Level of node B – 1

Level of node C – 1

Level of node D – 2

Depth of tree: 2

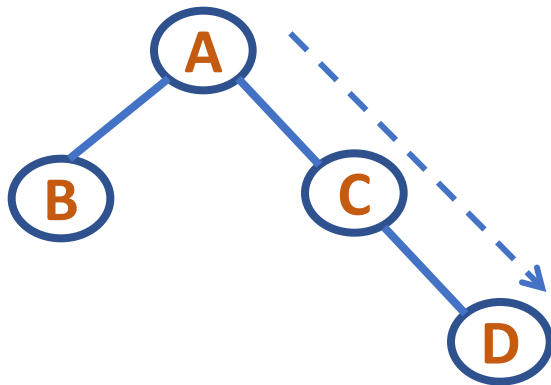
Depth of node A: 0

Depth of node B: 1

Depth of node C: 1

Depth of node D: 2

- Height of a tree: Path length from the root node to the deepest leaf
- Height of a node: Path length from the node to the deepest leaf



Height of Tree: 2

Height of Node A : 2

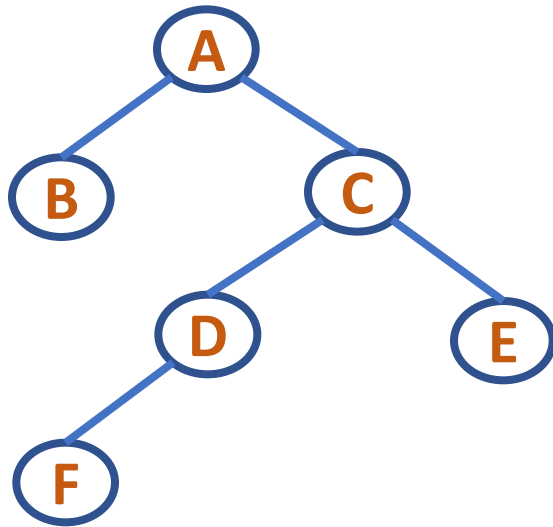
Height of Node B : 0

Height of Node C : 1

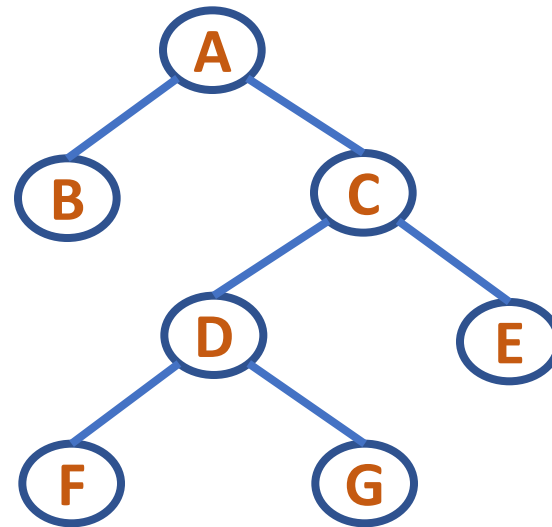
Height of Node D : 0

Strictly Binary Tree

A Binary tree where every node has either zero/two children



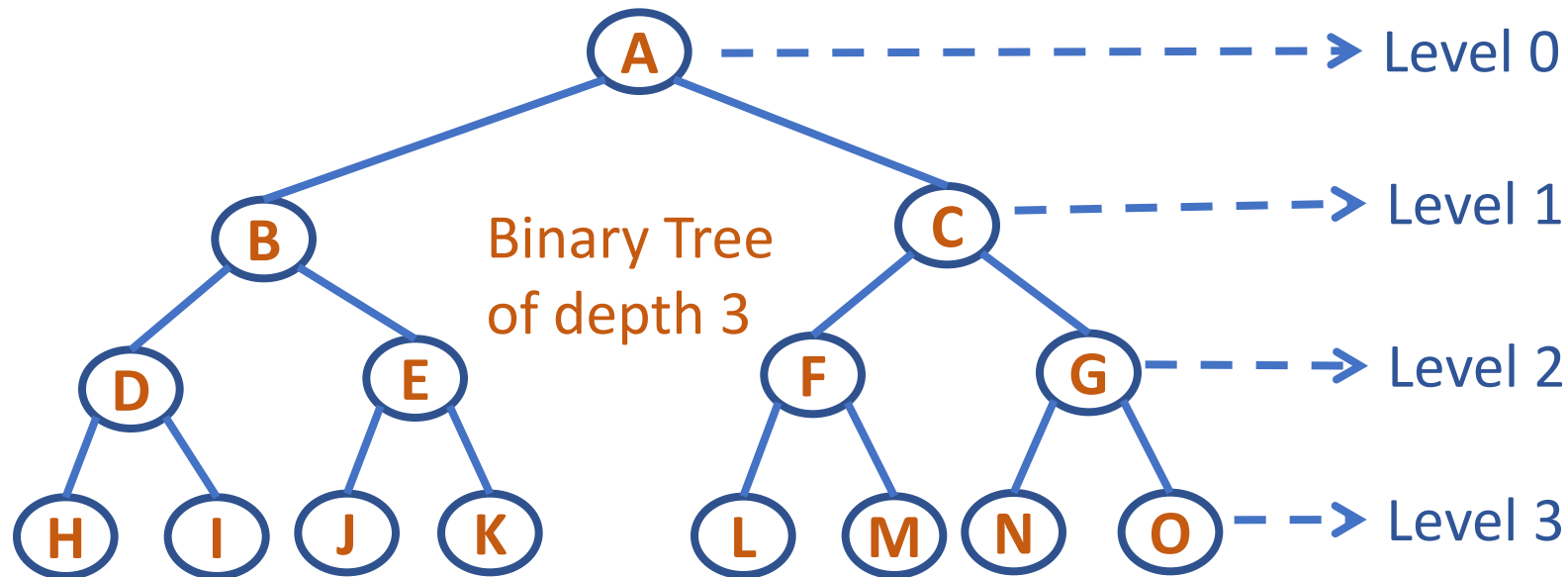
Not a Strictly Binary Tree



Strictly Binary Tree

Fully Binary Tree

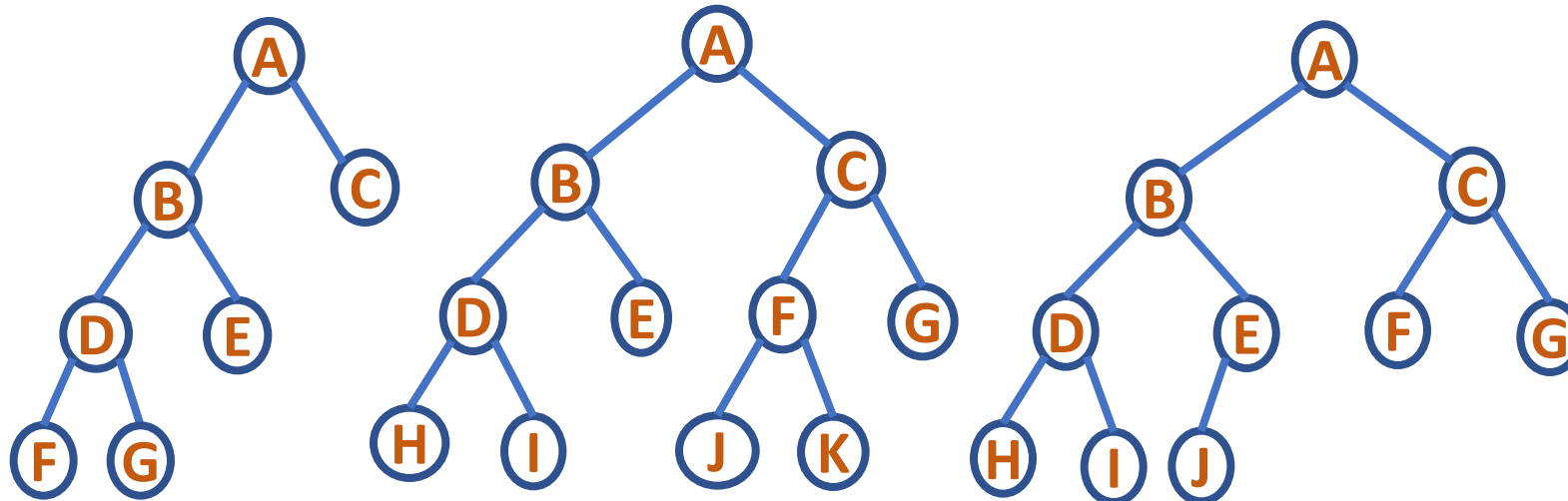
- A binary tree with all the leaves at the same level
- If the binary tree has depth d , then there are 0 to d levels
- Total no. of nodes = $2^0 + 2^1 + \dots + 2^d = 2^{(d+1)} - 1$



Complete Binary Tree

For a Complete Binary Tree with n nodes and depth d :

- Any node n_d at level less than $d-1$ has two children
- For any node n_d of the tree with a right descendent at level d , n_d must have a left child and every left descendent of n_d is either a leaf at level d or has two children



Not Complete Binary Trees

Complete Binary Tree

Binary Tree Properties

- Every node except the root has exactly one parent
- A tree with n nodes has $n-1$ edges (every node except the root has an edge to its parent)
- A tree consisting of only root node has height of zero
- The total number of nodes in a full binary tree of depth d is $2^{(d+1)} - 1$, $d \geq 0$
- For any non-empty binary tree, if n_0 is the number of leaf nodes and n_2 the nodes of degree 2, then $n_0 = n_2 + 1$



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE23CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversals



Important operation: Traversal

Traversal: Moving through all the nodes in a binary tree and visiting each one in turn

Trees: There are many orders possible since it is a nonlinear DS

Tasks: 1. Visiting a node denoted by V

2. Traversing the left subtree denoted by L

3. Traversing the right subtree denoted by R

Six ways to arrange them: VLR, LVR, LRV, VRL, RVL, RLV

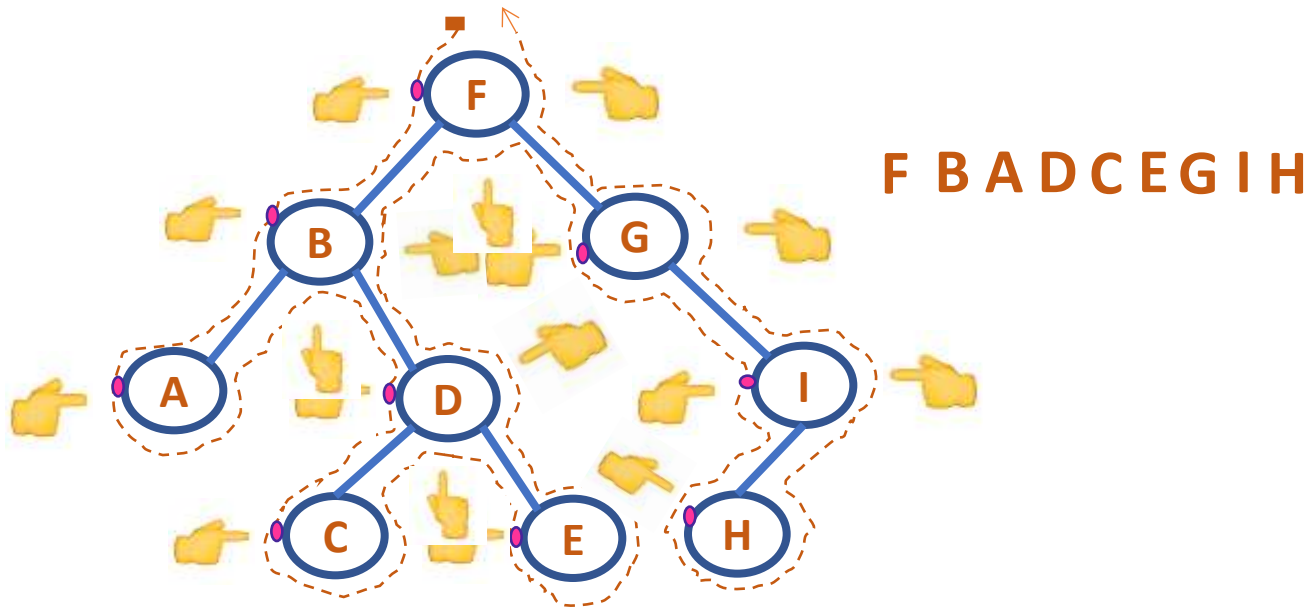
Standard Traversals include: VLR-Preorder, LVR-Inorder,
LRV-Postorder

DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal: Preorder

Steps:

- Root Node is visited before the subtrees
- Left subtree is traversed in preorder
- Right subtree is traversed in preorder

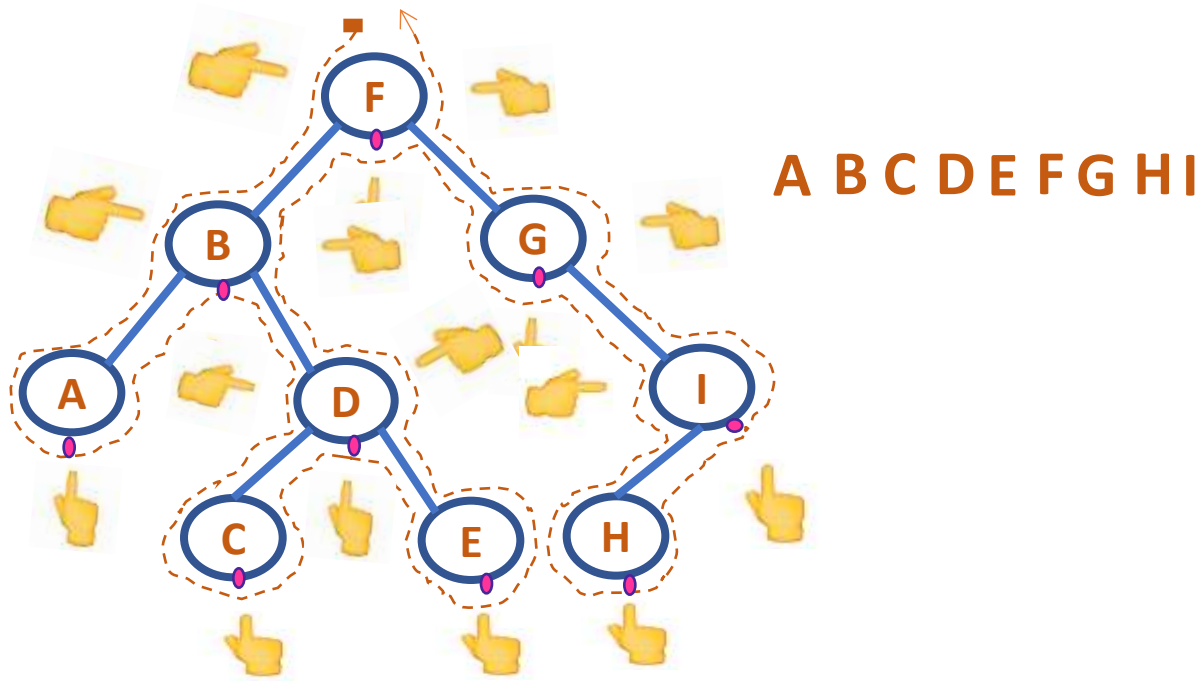


DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal: Inorder

Steps:

- Left subtree is traversed in Inorder
- Root Node is visited
- Right subtree is traversed in Inorder

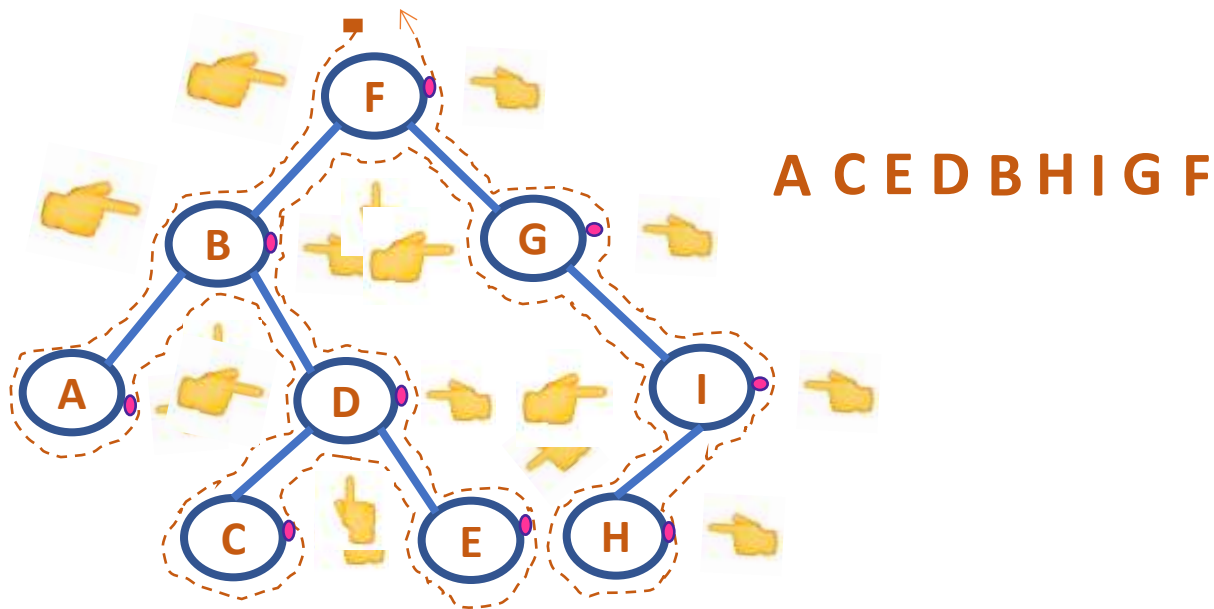


DATA STRUCTURES AND ITS APPLICATIONS

Binary Tree Traversal: Postorder

Steps:

- Left subtree is traversed in postorder
- Right subtree is traversed in postorder
- Root Node is visited





THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE23CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Basic Concept and Definitions: Trees

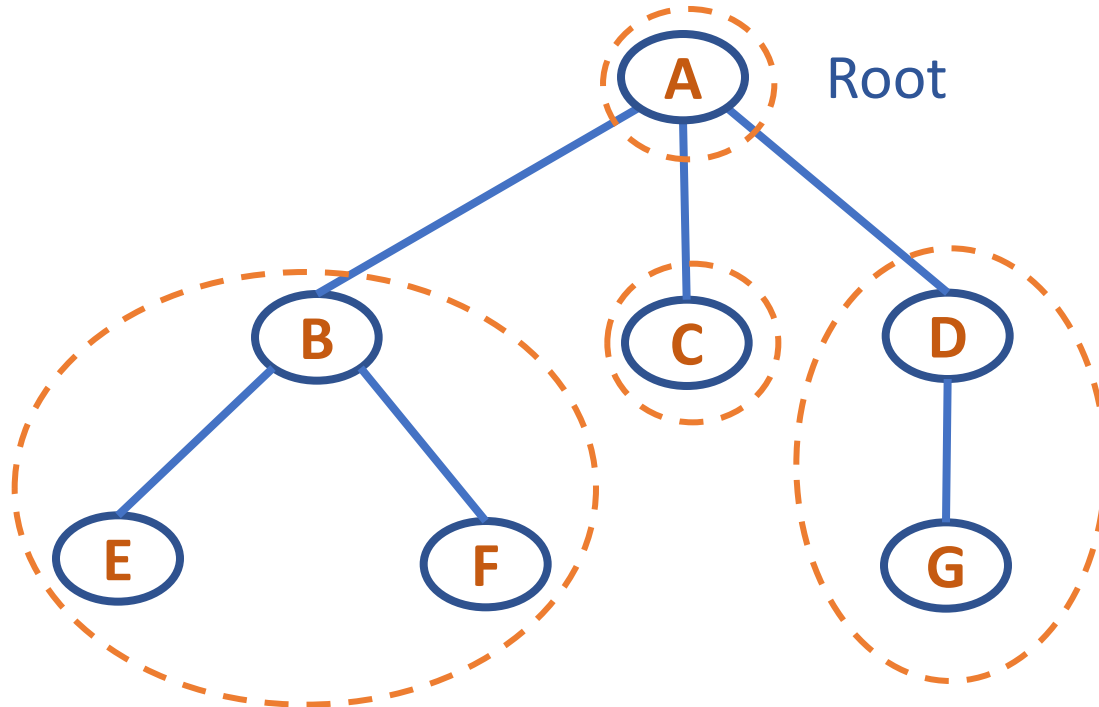
Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Trees

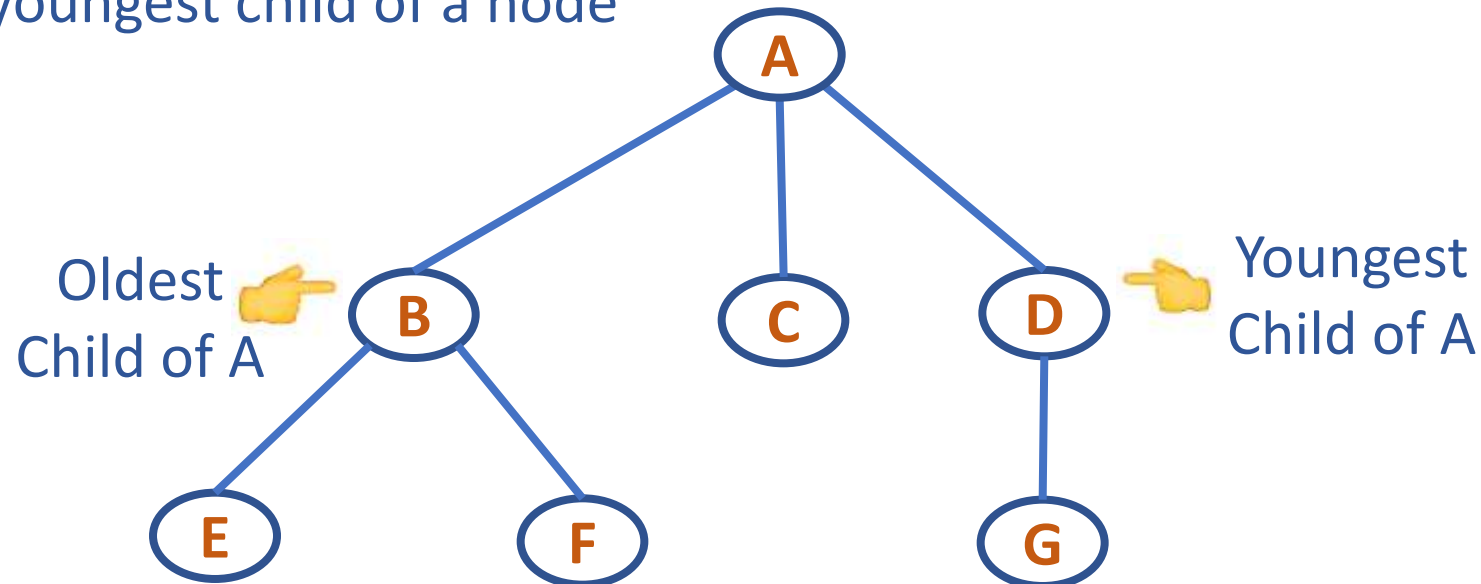
- Non Linear Data Structure
- Finite nonempty set of elements
 - One element is the root
 - Remaining elements are partitioned into $m \geq 0$ disjoint subsets each of which is itself a tree



DATA STRUCTURES AND ITS APPLICATIONS

Trees

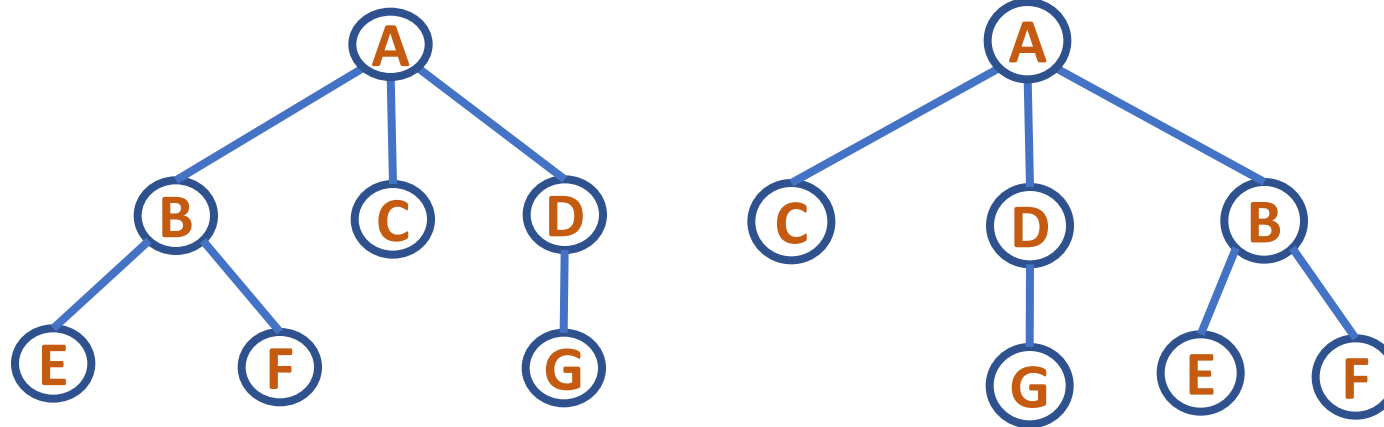
- Ordered Tree: a tree in which subtrees of each node forms an ordered set
- In such a tree we define first, second, ..., Last child of a particular node
- First child is called the oldest child and the last child the youngest child of a node



DATA STRUCTURES AND ITS APPLICATIONS

Trees

As unordered trees the below figures are equivalent but as ordered trees, they are different

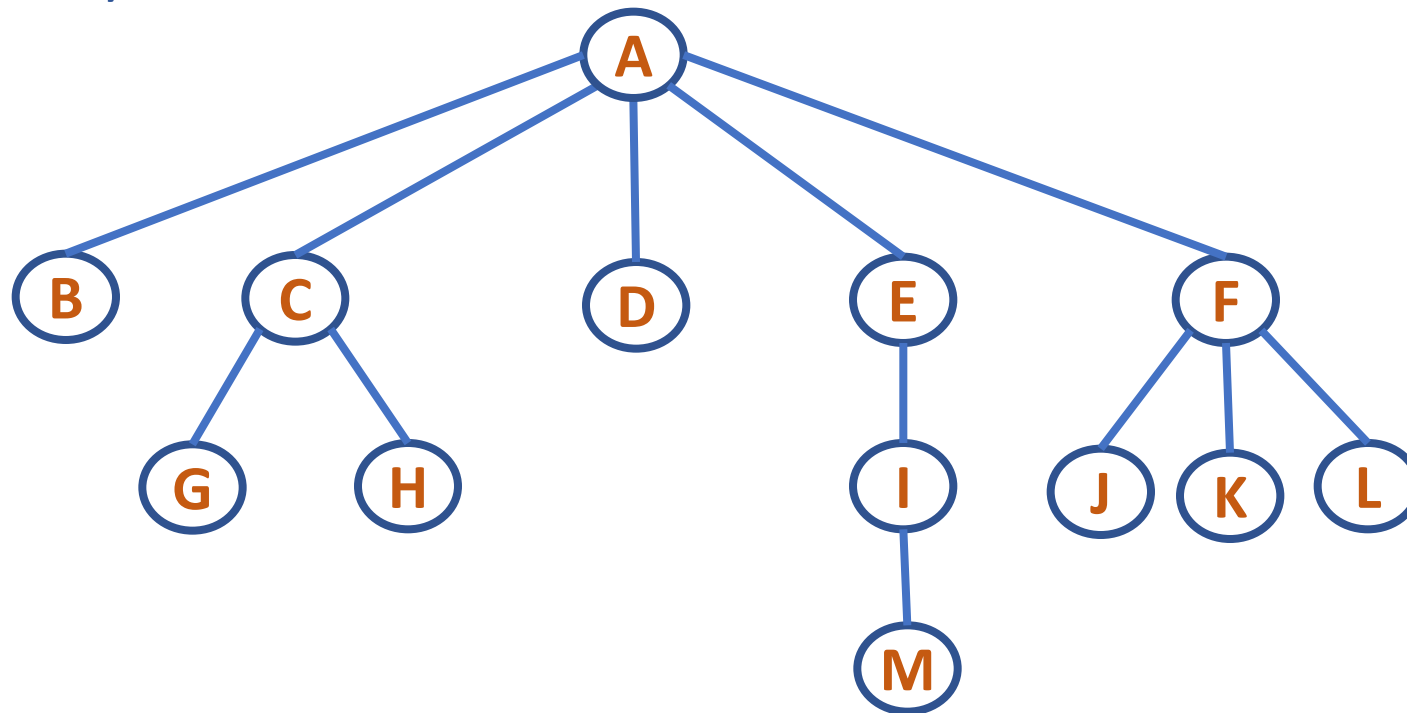


DATA STRUCTURES AND ITS APPLICATIONS

N-ary Tree & Forest

n-ary tree: A rooted tree in which each node has no more than **n** children

- A binary tree is an n-ary tree with $n=2$
- n-ary tree with $n=5$



- Forest: is an ordered set of ordered trees

DATA STRUCTURES AND ITS APPLICATIONS

Tree



Representation of trees:

Tree node options:

```
struct treenode{  
    int info;  
    struct treenode *child[MAX];  
};
```

where MAX is a constant

Restrictions with the above implementation:

- A node cannot have more than MAX children. Therefore cannot expand the tree

DATA STRUCTURES AND ITS APPLICATIONS

Tree



2nd implementation:

- All the children of a given node are linked and only the oldest child is linked to the parent
- A node has link to first child and a link to immediate sibling

```
struct treenode{  
    int info;  
    struct treenode *child;  
    struct treenode *sibling;  
};
```

```
struct treenode {  
    int info;           // Stores the data (integer) in the node  
    struct treenode *child; // Pointer to the first (leftmost) child node  
    struct treenode *sibling; // Pointer to the next sibling node (right sibling)  
};
```

DATA STRUCTURES AND ITS APPLICATIONS

Tree

```
struct treenode {  
    int info;           // Stores the data (integer) in the node  
    struct treenode *child; // Pointer to the first (leftmost) child node  
    struct treenode *sibling; // Pointer to the next sibling node (right sibling)  
};
```

Key Parts

1. **info**: This holds the data of the current node (in this case, an integer).
2. **child**: This points to the **first (leftmost) child** of the current node. If the node has no children, this pointer will be **NULL**.
3. **sibling**: This points to the **next sibling** of the current node (i.e., the next node at the same level). If there are no more siblings, this pointer will be **NULL**.

How This Structure Works

Instead of using a fixed-size array or a linked list for children, this approach models each node as having:

- A **single child** (the first or leftmost child).
- A **single sibling** (the next sibling of the current node).

This effectively turns the general tree into a binary-like structure:

- The **child pointer** connects downward (to the children).
- The **sibling pointer** connects horizontally (to the siblings).

DATA STRUCTURES AND ITS APPLICATIONS

Tree



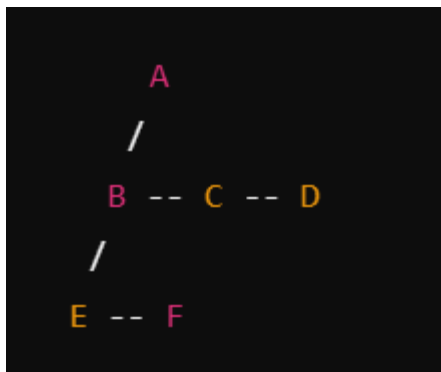
Consider this structure



In this tree:

- Node **A** has three children: **B**, **C**, and **D**.
- Node **B** has two children: **E** and **F**.
- Nodes **C**, **D**, **E**, and **F** have no children.

In the memory the above tree would look like:



- Node **A**:
 - **child** → **B** (first child)
 - **sibling** → **NULL** (no sibling, since **A** is the root)
- Node **B**:
 - **child** → **E** (first child)
 - **sibling** → **C** (sibling)
- Node **C**:
 - **child** → **NULL** (no child)
 - **sibling** → **D** (sibling)
- Node **D**:
 - **child** → **NULL** (no child)
 - **sibling** → **NULL** (no sibling)
- Node **E**:
 - **child** → **NULL** (no child)
 - **sibling** → **F** (sibling)
- Node **F**:
 - **child** → **NULL** (no child)
 - **sibling** → **NULL** (no sibling)



DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree



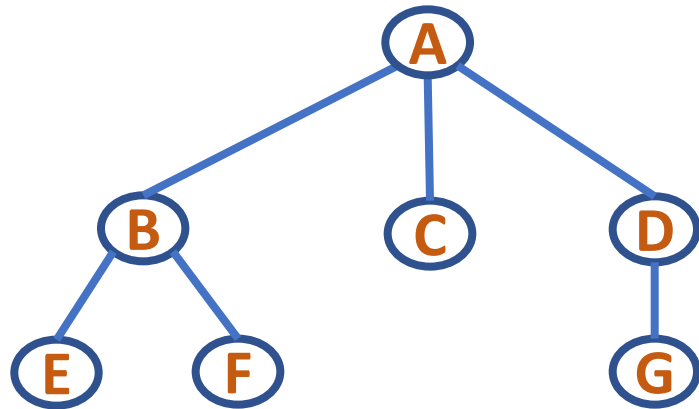
Left Child – Right Sibling Representation

- Link all the siblings of a node
- Delete all links from a node to its children except for the link to its leftmost child
- The left child in binary tree is the node which is the oldest child of the given node in an n-ary tree, and the right child is the node to the immediate right of the given node on the same horizontal line. Such a binary tree will not have a right sub tree
- The node structure corresponds to that of

Data	
Left Child	Right Sibling

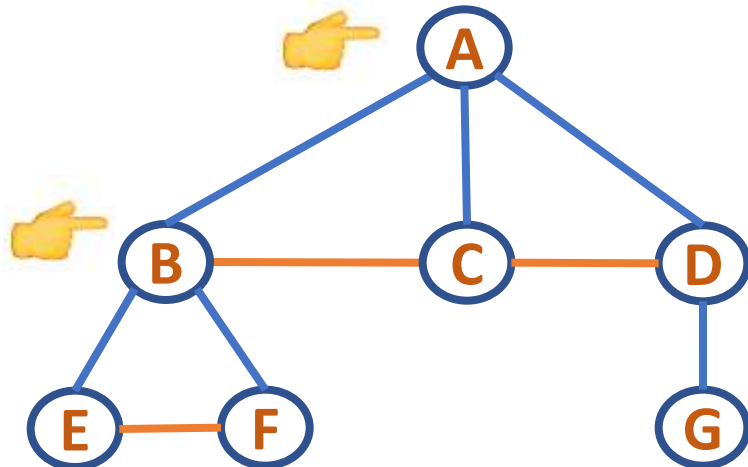
DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree



3-ary tree

Link all siblings of a Node



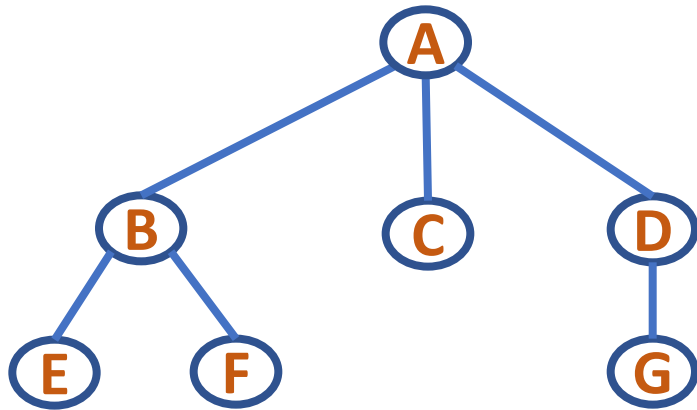
Delete all links from a Node to its children except for the link to its leftmost child

Left child – Right sibling

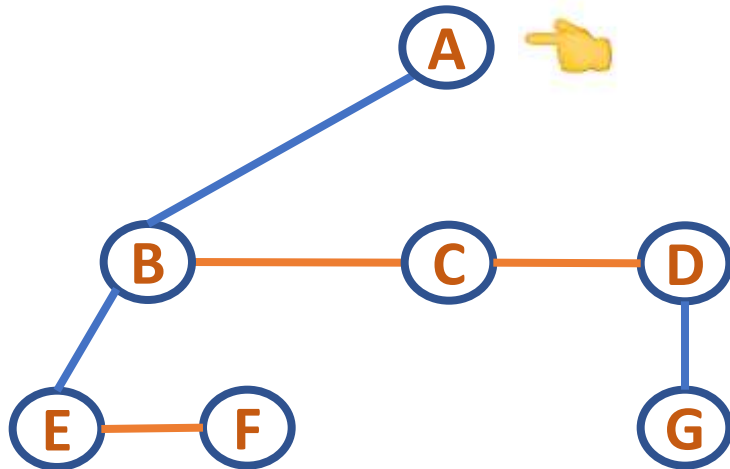
Representation of the above 3-ary tree

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

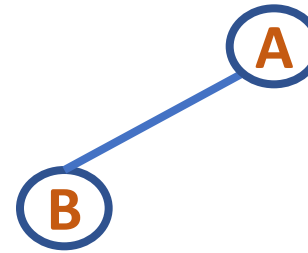


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree



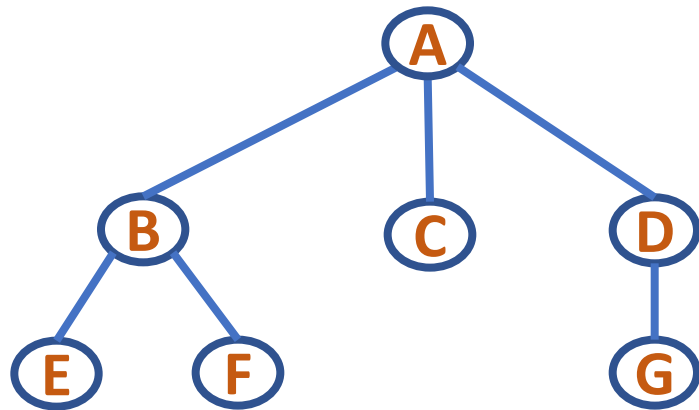
Node Below is B – Left child

No Right sibling so – no Right child

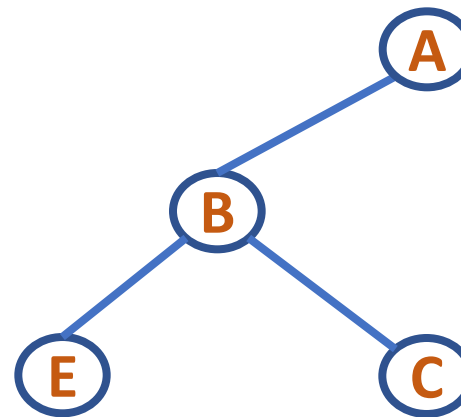
Corresponding
binary tree

DATA STRUCTURES AND ITS APPLICATIONS

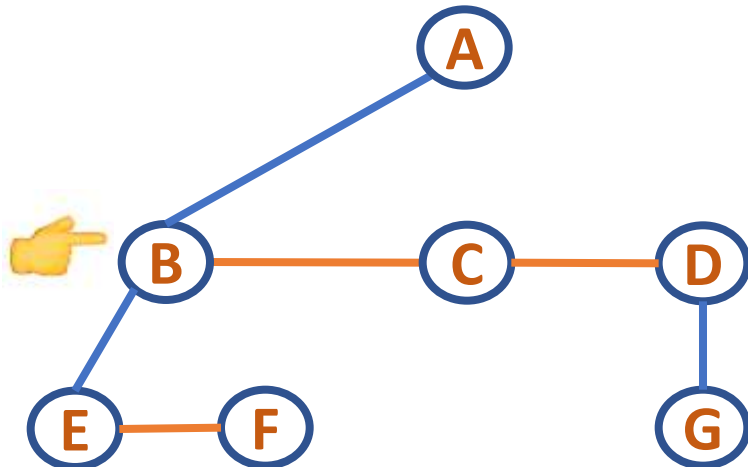
Conversion of an n-ary Tree to a Binary Tree



3-ary tree



Node Below is E – Left child
Next Right sibling is C – Right child



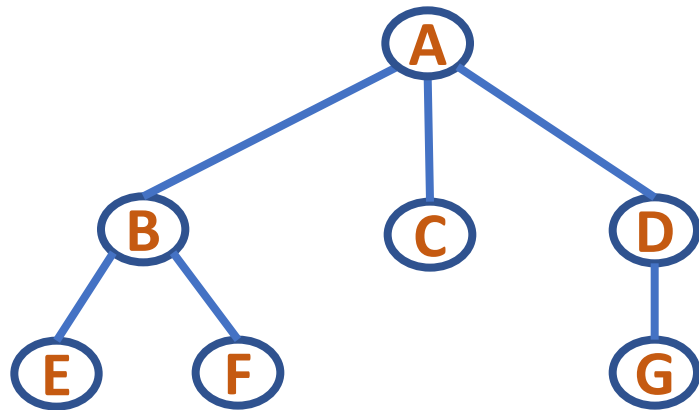
Left child – Right sibling

Representation of the above 3-ary tree

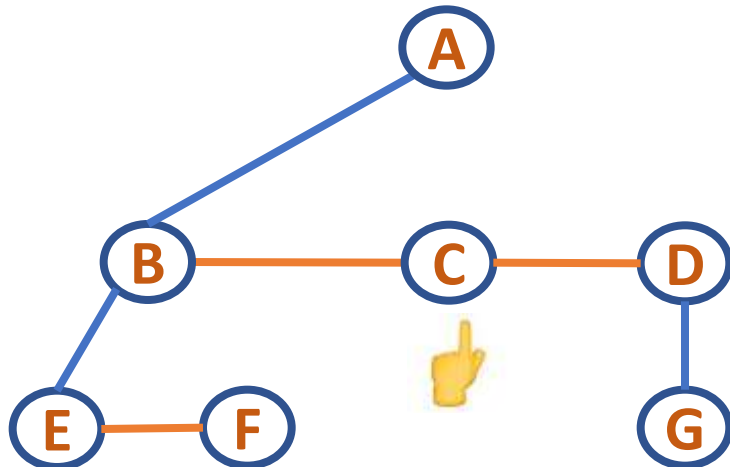
Corresponding
binary tree

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

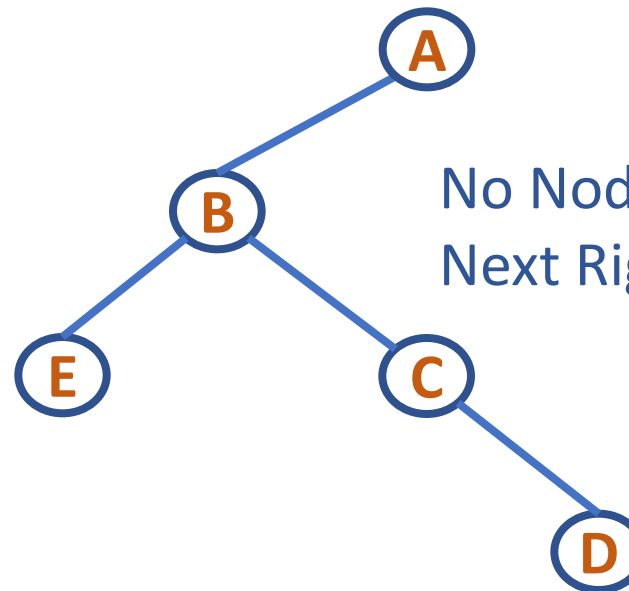


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree

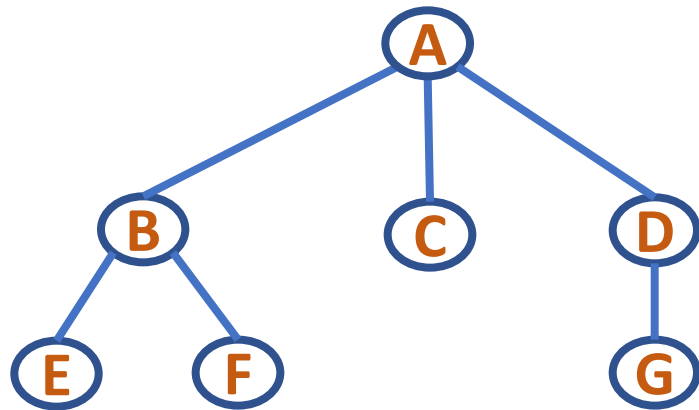


Corresponding
binary tree

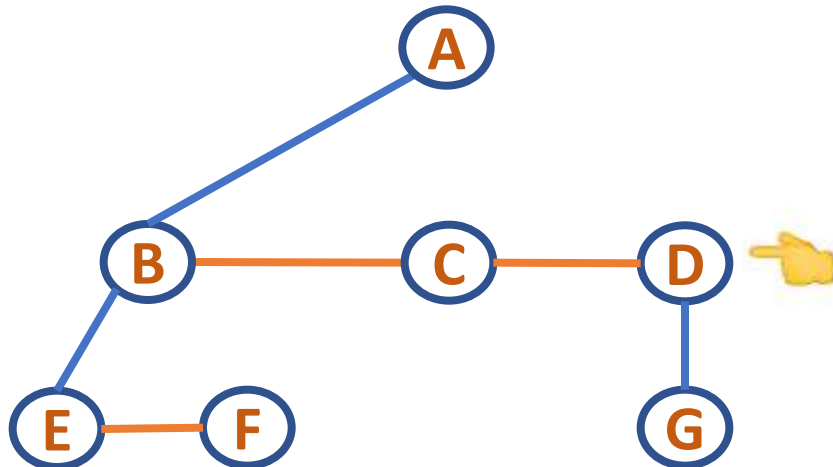
No Node Below so – no Left child
Next Right sibling is D – Right child

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

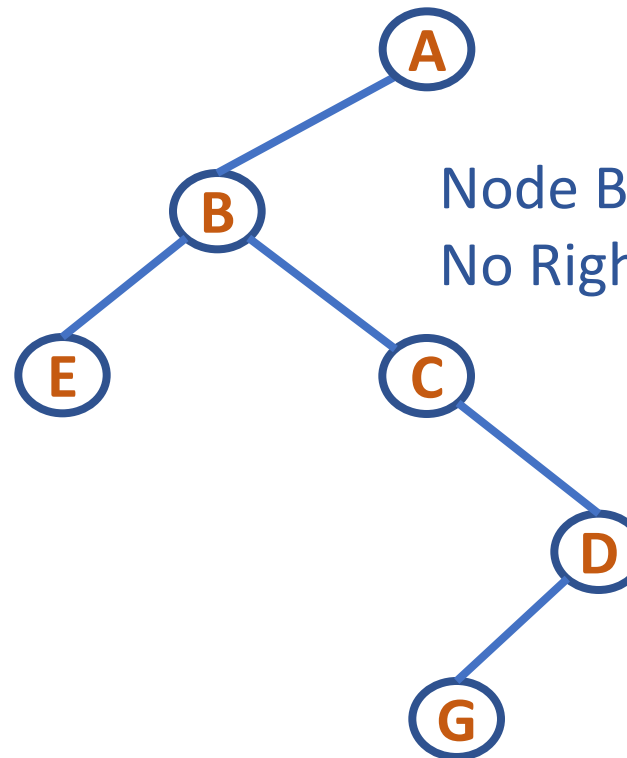


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree

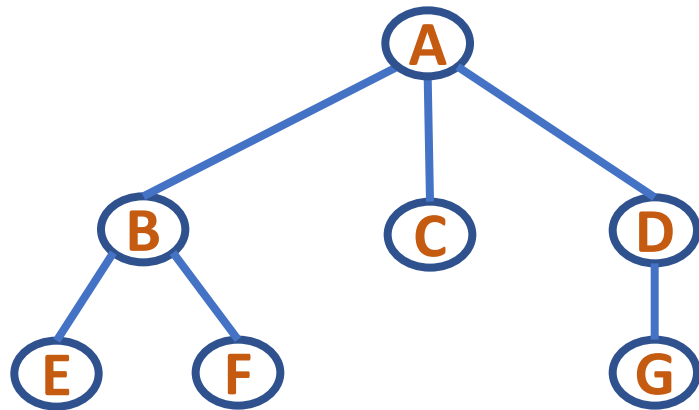


Corresponding
binary tree

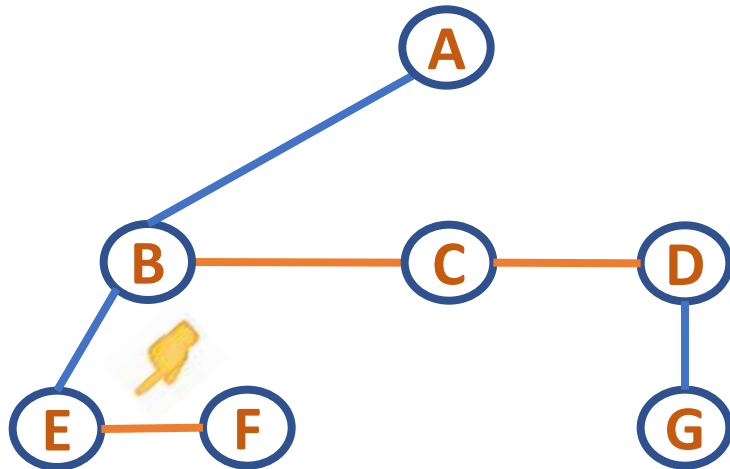
Node Below is G – Left child
No Right sibling so – no Right child

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

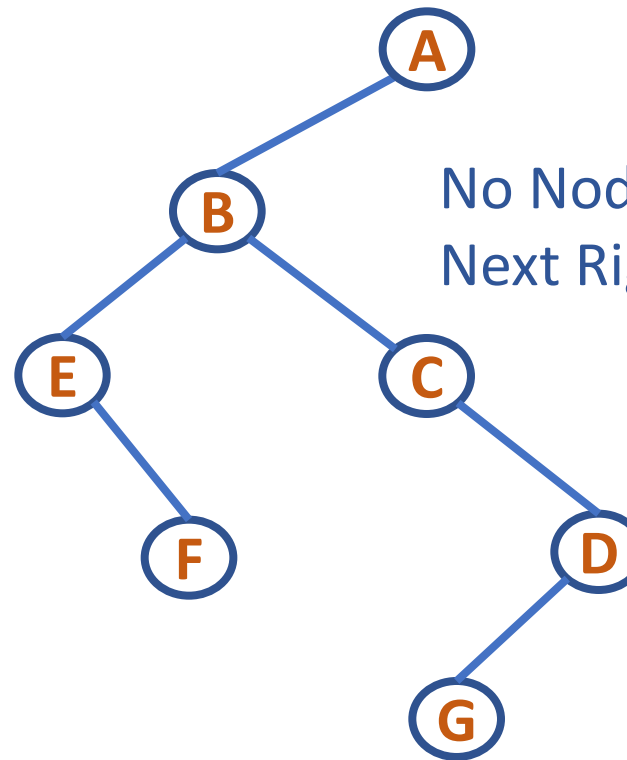


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree

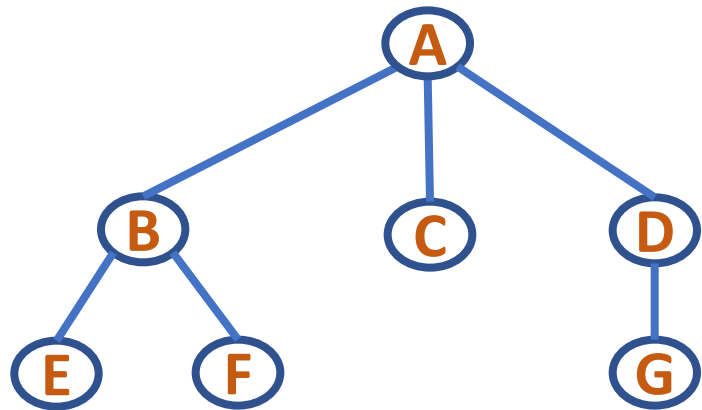


Corresponding
binary tree

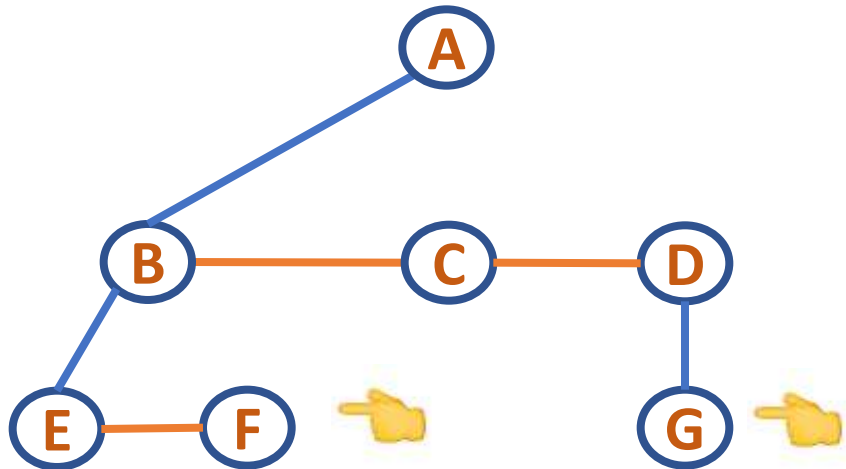
No Node Below so – no Left child
Next Right sibling is F – Right child

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of an n-ary Tree to a Binary Tree

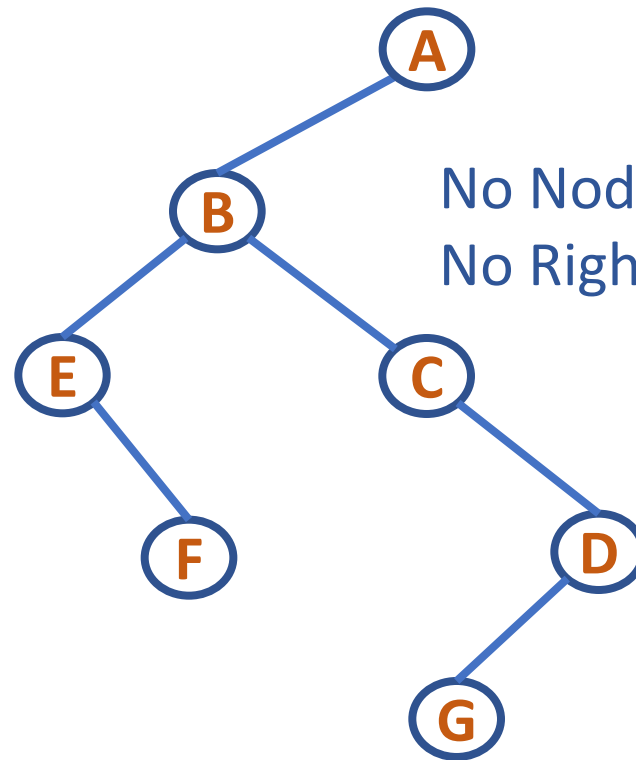


3-ary tree



Left child – Right sibling

Representation of the above 3-ary tree



Corresponding
binary tree

No Node Below so – no Left child
No Right sibling so – no Right child

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of a Forest to a Binary Tree



- Right Child of the root node of every resulting binary tree will be empty. This is because the root of the tree we are transforming has no siblings.
- On the other hand, if we have a forest then these can all be transformed into a single binary tree as follows:
 - First obtain the binary tree representation of each of the trees in the forest
 - Link all the binary trees together through the right sibling field of the root nodes

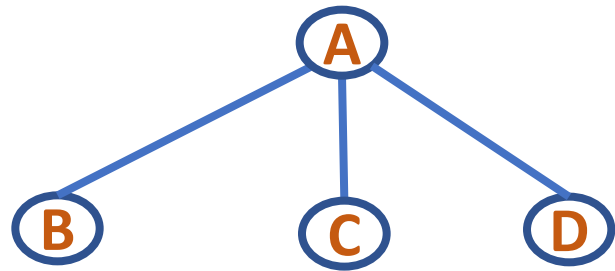
Conversion of a Forest to a Binary Tree can be formally defined as follows:

- If $T_1, \dots, T_n$ is a forest of n trees, then the binary tree corresponding to this forest, denoted by $B(T_1, \dots, T_n)$:
 - is empty if $n = 0$
 - has root equal to root (T_1)
 - has left subtree equal to $B(T_{11}, T_{12}, \dots, T_{1m})$
where $T_{11}, \dots, T_{1m}$ are the subtrees of root(T_1)
 - has right subtree $B(T_2, \dots, T_n)$

DATA STRUCTURES AND ITS APPLICATIONS

Conversion of a Forest to a Binary Tree

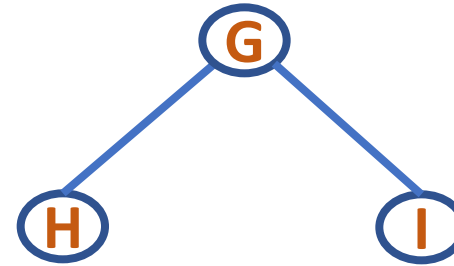
Consider the following Forest with three Trees



T1

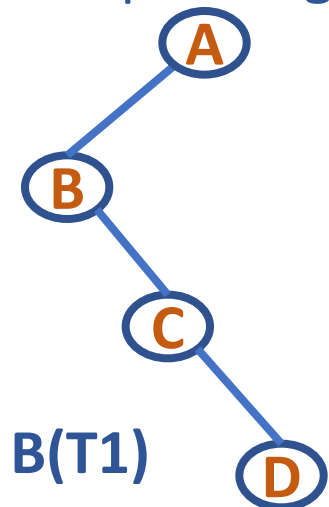


T2

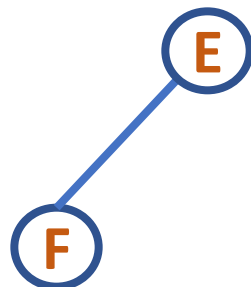


T3

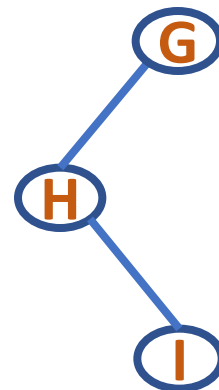
Corresponding Binary Trees



B(T1)



B(T2)

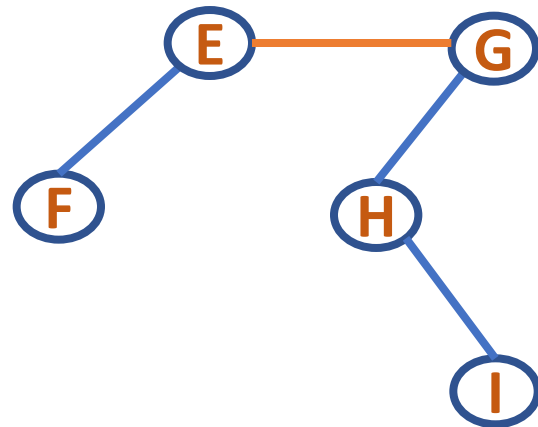


B(T3)

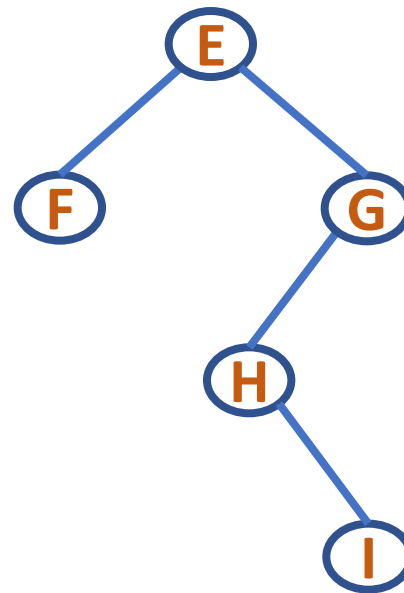
DATA STRUCTURES AND ITS APPLICATIONS

Conversion of a Forest to a Binary Tree

Link B(T2) and B(T3)



G becomes Right Child of E

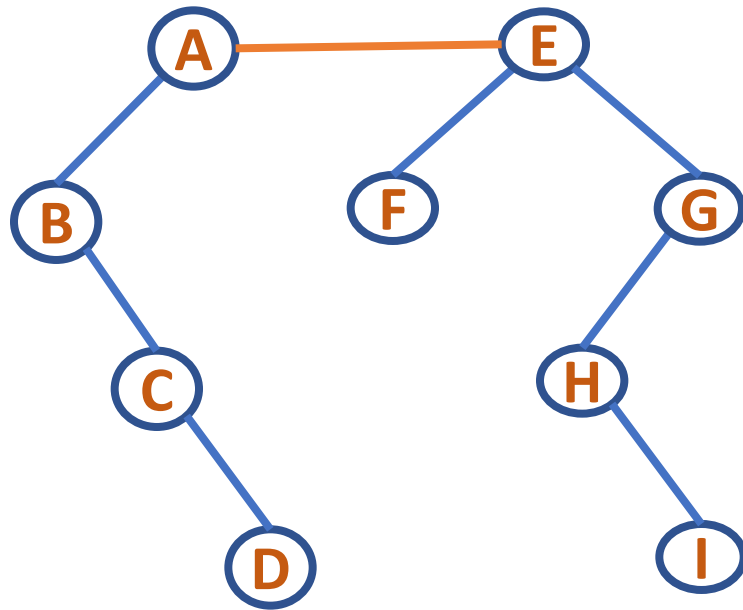


Corresponding Binary Tree
B(T2, T3)

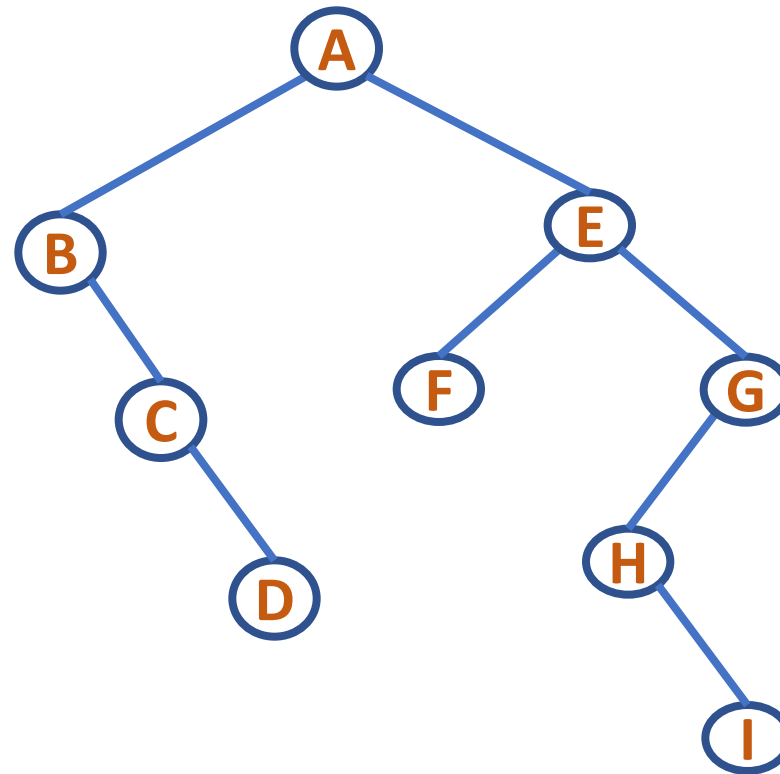
DATA STRUCTURES AND ITS APPLICATIONS

Conversion of a Forest to a Binary Tree

Link $B(T_1)$ and $B(T_2, T_3)$



E becomes Right Child of A



Corresponding Binary Tree $B(T_1, T_2, T_3)$



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu



DATA STRUCTURES AND ITS APPLICATIONS

UE23CS252A

Shylaja S S & Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES AND ITS APPLICATIONS

n-ary Tree Traversal

Shylaja S S

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Tree Traversal



Structure of a treenode revisited

```
struct treenode{  
    int info;  
    struct treenode *child;  
    struct treenode *sibling;  
};
```

With the treenode implemented as having pointers to first child and immediate sibling, the traversal preorder, inorder and postorder for a tree are defined as follows:

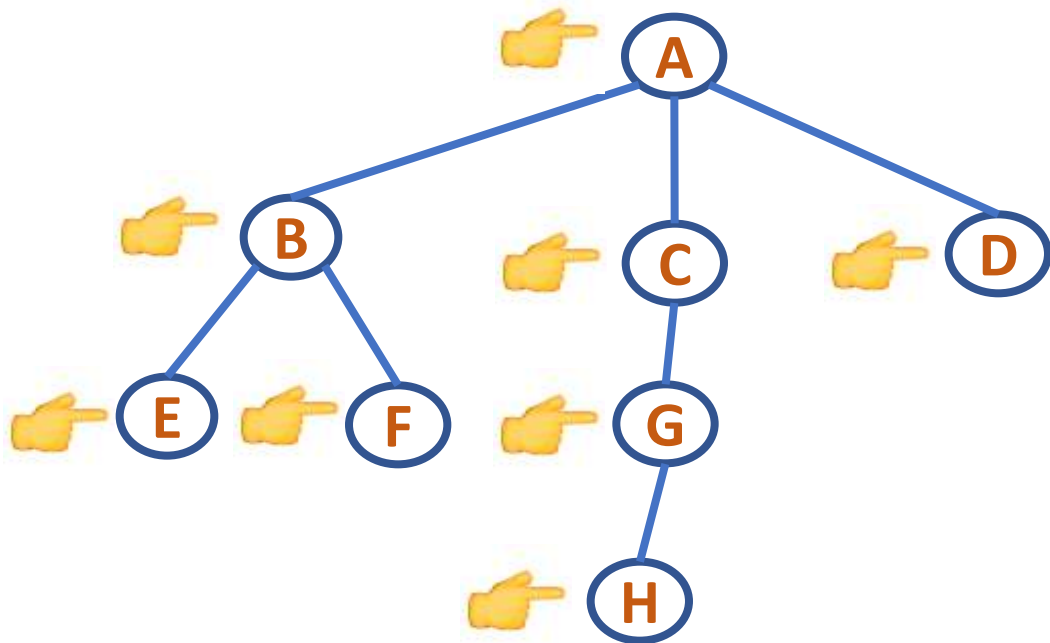
Preorder:

1. Visit the root of the first tree in the forest
2. Traverse in preorder the forest formed by the subtrees of the first tree, if any
3. Traverse in preorder the forest formed by the remaining trees in the forest, if any

DATA STRUCTURES AND ITS APPLICATIONS

Tree Traversal

Preorder Tree Traversal



ABEFCGHD

DATA STRUCTURES AND ITS APPLICATIONS

Tree Traversal



```
void preorder(TREE *root)
{
    if(root!=NULL)
    {
        printf(" %d ",root->info);
        preorder(root->child);
        preorder(root->sibling);
    }
}
```

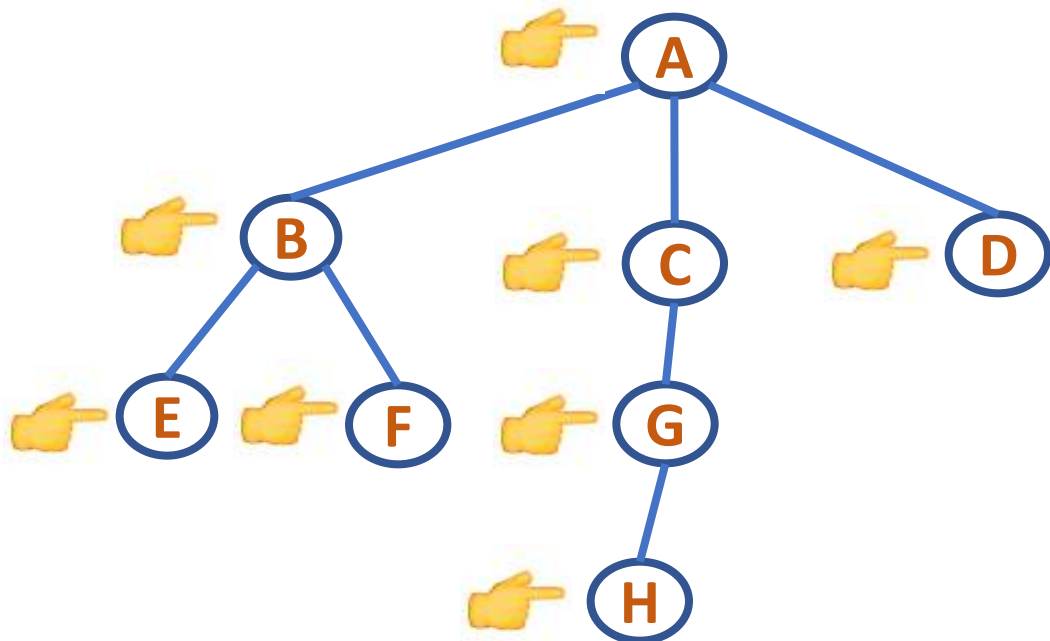
Inorder

1. Traverse in inorder the forest formed by the subtrees of the first tree, if any
2. Visit the root of the first tree in the forest
3. Traverse in inorder the forest formed by the remaining trees in the forest, if any

DATA STRUCTURES AND ITS APPLICATIONS

Tree Traversal

Inorder Tree Traversal



E F B H G C D A

DATA STRUCTURES AND ITS APPLICATIONS

Tree Traversal



```
void inorder(TREE *root)
{
    if(root!=NULL)
    {
        inorder(root->child);
        printf(" %d ",root->info);
        inorder(root->sibling);
    }
}
```

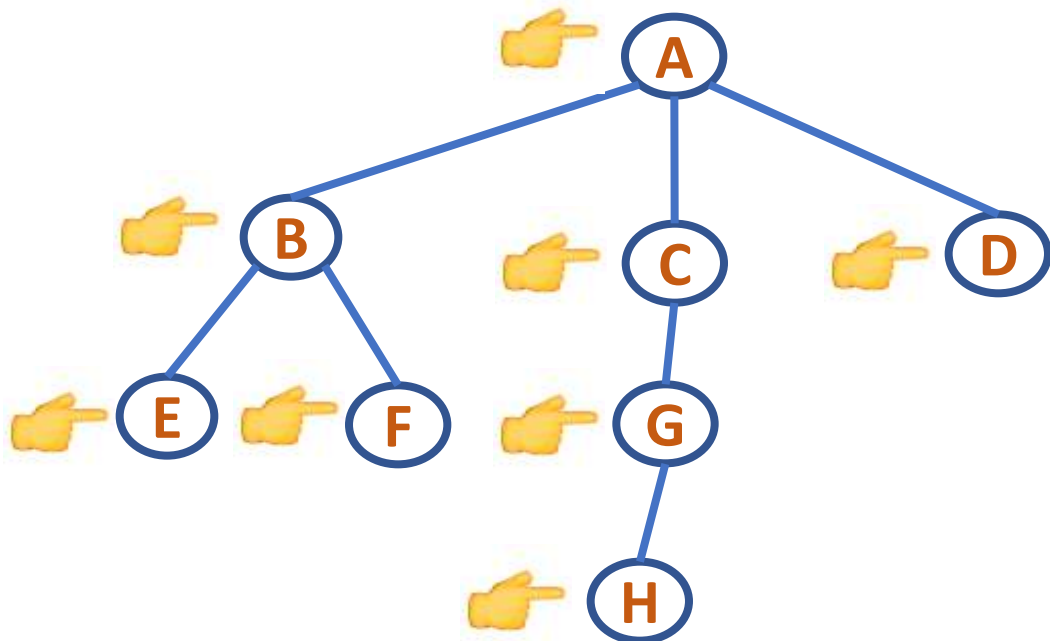
Postorder

1. Traverse in postorder the forest formed by the subtrees of the first tree, if any
2. Traverse in postorder the forest formed by the remaining trees in the forest, if any
3. Visit the root of the first tree in the forest

DATA STRUCTURES AND ITS APPLICATIONS

Tree Traversal

Postorder Tree Traversal



F E H G D C B A

DATA STRUCTURES AND ITS APPLICATIONS

Tree Traversal



```
void postorder(TREE *root)
{
    if(root!=NULL)
    {
        postorder(root->child);
        postorder(root->sibling);
        printf(" %d ", root->info);
    }
}
```



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu



DATA STRUCTURES & ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES & ITS APPLICATIONS

UNIT 1: Skip List

Kusuma K V

Department of Computer Science & Engineering

- `Size()`:
 - $O(1)$ if size is stored explicitly, else $O(n)$
- `IsEmpty()`:
 - $O(1)$
- `FindElement(k)`:
 - $O(n)$
- `InsertItem(k, e)`:
 - $O(1)$ (assumes item already found)
- `Remove(k)`:
 - $O(1)$ (assumes item already found)

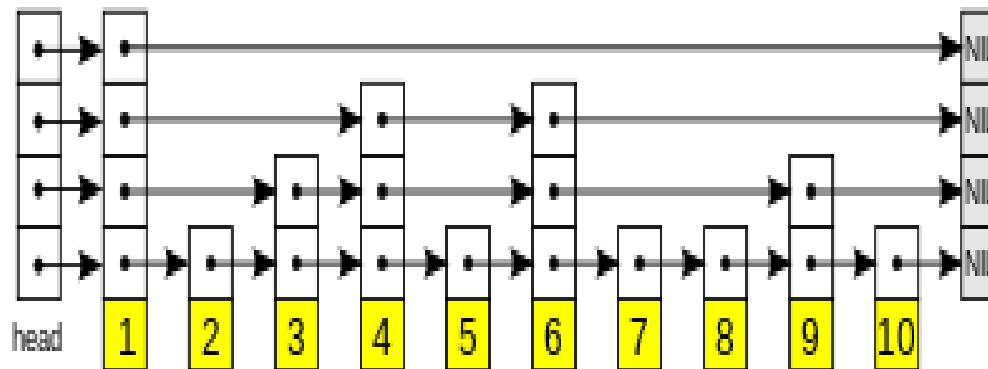
- Skip Lists support $O(\log n)$
 - Insertion
 - Deletion
 - Search
- A relatively recent data structure
 - W. Pugh in 1989
- “A probabilistic alternative to balanced trees”

Skip List

- A skip list is a probabilistic data structure that allows $O(\log n)$ search complexity as well as $O(\log n)$ insertion complexity within an ordered sequence of n elements
- Thus it can get the best features of a sorted array (for searching) while maintaining a linked list-like structure that allows insertion, which is not possible in an array

Skip List

- Fast search is made possible by maintaining a linked hierarchy of sub sequences, with each successive subsequence skipping over fewer elements than the previous one
- Searching starts in the sparsest subsequence until two consecutive elements have been found, one smaller and one larger than or equal to the element searched for
- Via the linked hierarchy, these two elements link to elements of the next sparsest subsequence, where searching is continued until finally we are searching in the full sequence

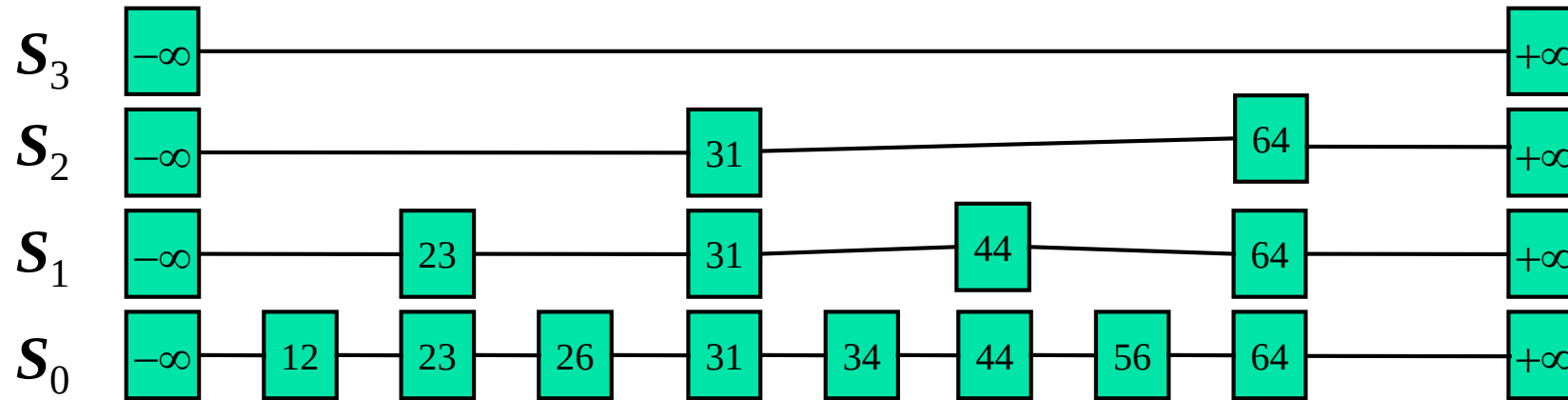


- A skip list is a collection of lists at different levels
- The lowest level (0) is a sorted, singly linked list of all nodes
- The first level (1) links alternate nodes
- The second level (2) links every fourth node
- In general, level i links every 2^i th node
- In total, $\lceil \log_2 n \rceil$ levels (i.e. $O(\log_2 n)$ levels)
- Each level has half the nodes of the one below it

DATA STRUCTURES & ITS APPLICATIONS

Perfect Skip List

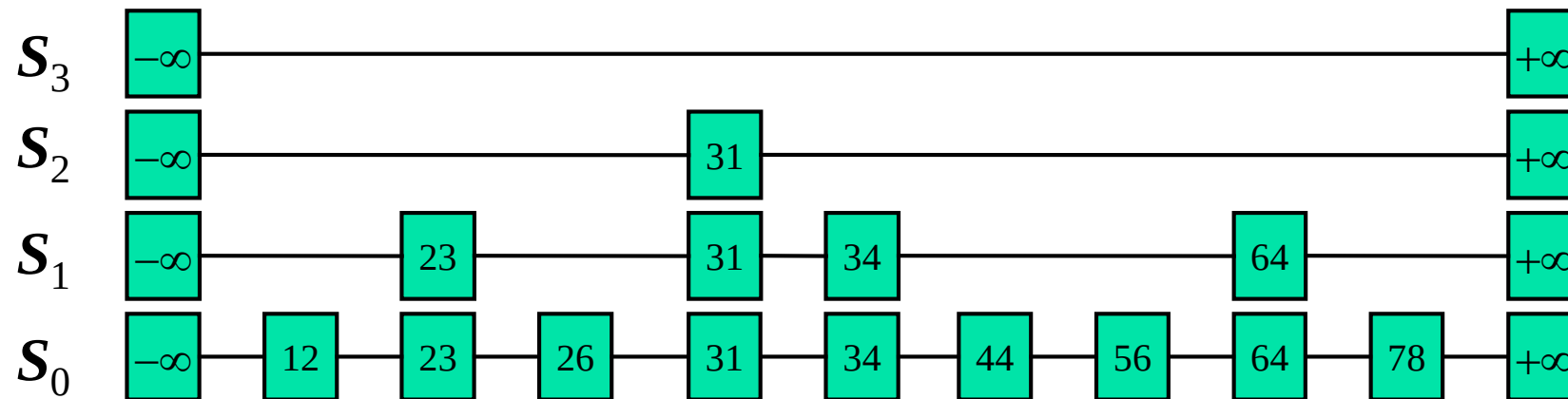
- Example of a Perfect Skip List



When we add a new node, our beautifully precise structure might become invalid

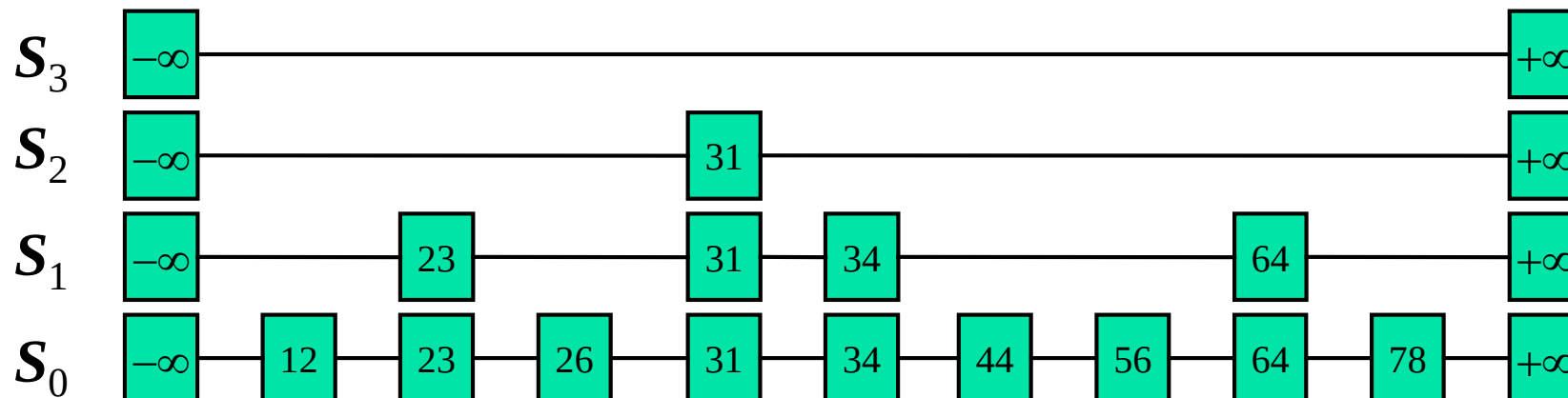
- We may have to change the level of every node
- One option is to move all the elements around
- But it takes $O(n)$ time, which is back where we began
- Is it possible to achieve a net gain?

Example of a Randomized Skip List



Skip List definition

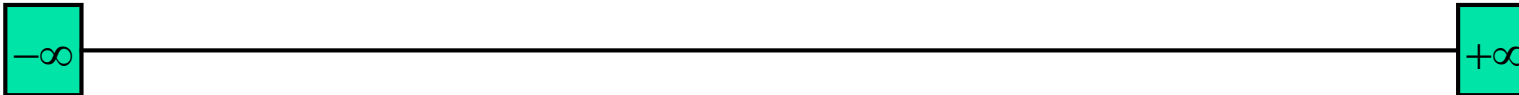
- A skip list for a set S of distinct (key, element) items is a series of lists $S_0, S_1, \dots, S_h$ such that:
 - Each list S_i contains the special keys $+\infty$ and $-\infty$
 - List S_0 contains the keys of S in non decreasing order
 - Each list is a subsequence of the previous one, i.e.,
$$S_0 \supseteq S_1 \supseteq \dots \supseteq S_h$$
 - List S_h contains only the two special keys



Initialization

A new list is initialized as follows:

- A node NIL ($+\infty$) is created and its key is set to a value greater than the greatest key that could possibly be used in the list
- Another node NIL ($-\infty$) is created, value set to lowest key that could be used



DATA STRUCTURES & ITS APPLICATIONS

References

- Presentation Slides: Advanced Data Structures
Dr. RamaMoorthy Srinath, Professor, CSE, PESU





THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



DATA STRUCTURES & ITS APPLICATIONS

UE21CS252A

Kusuma K V

Department of Computer Science
& Engineering

DATA STRUCTURES & ITS APPLICATIONS

UNIT 1: Skip List

Kusuma K V

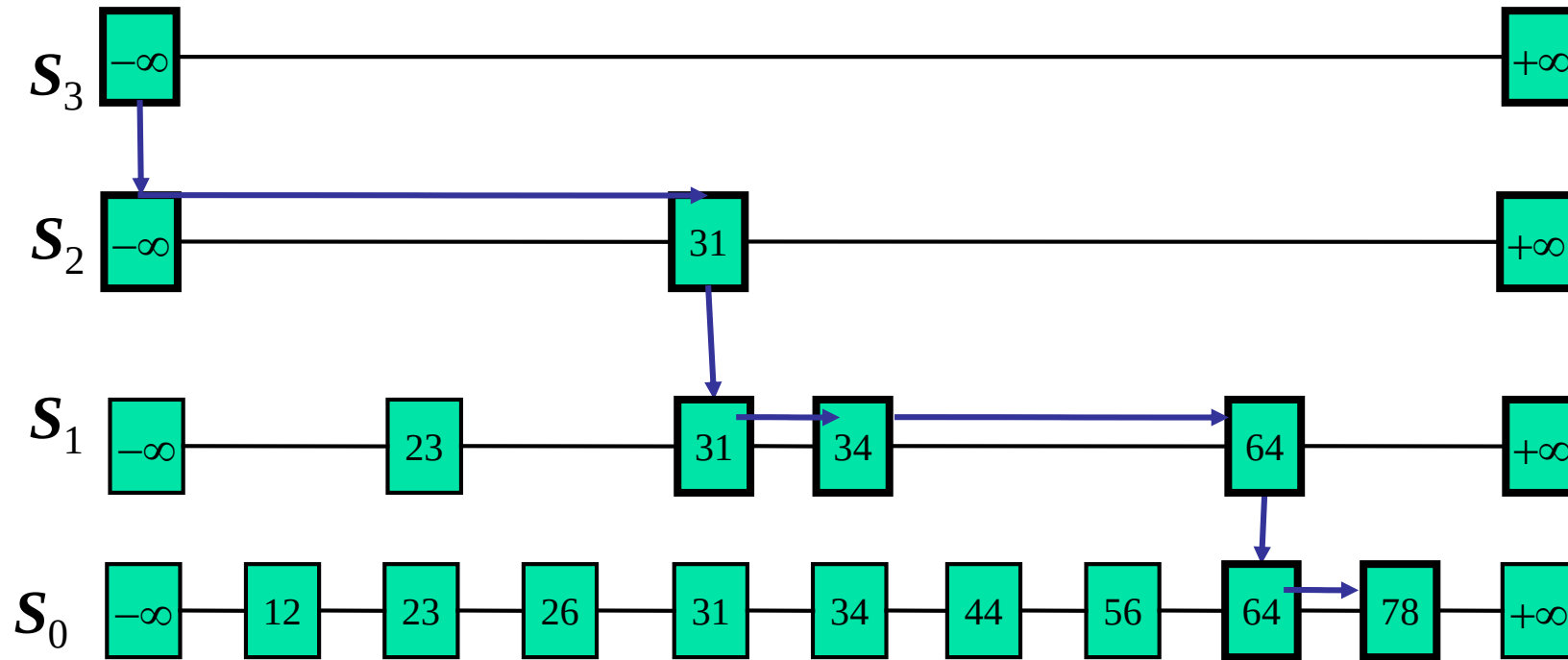
Department of Computer Science & Engineering

Search

We search for a key x in a skip list as follows:

- We start at the first position of the top list
- At the current position p , we compare x with y i.e., $\text{key}(\text{after}(p))$
 - $x = y$: we return $\text{element}(\text{after}(p))$
 - $x > y$: we “scan forward”
 - $x < y$: we “drop down”
- If we try to drop down past the bottom list, we return `NO_SUCH_KEY`
- Example: search for 78

Searching in Skip List Example



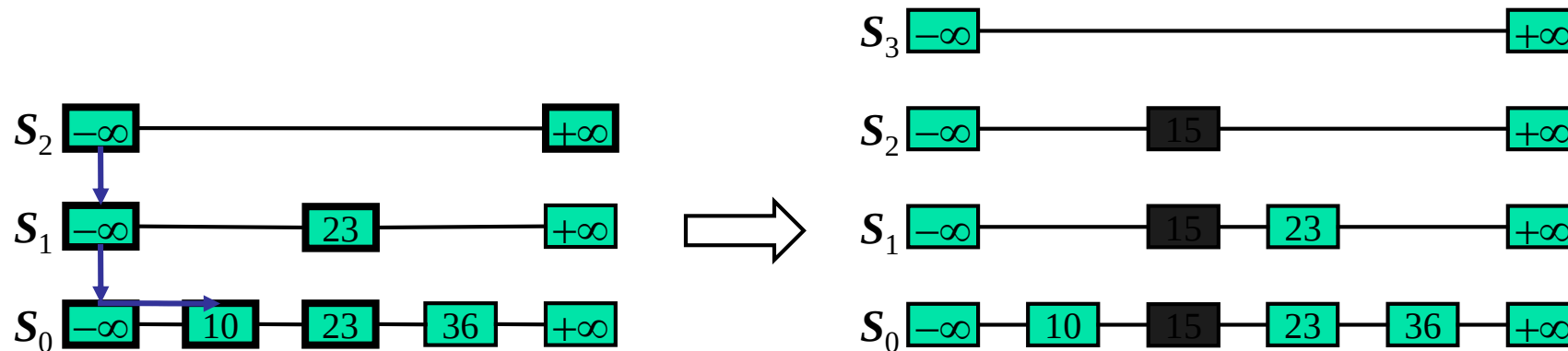
Insertion

- The insertion algorithm for skip lists uses **randomization** to decide how many references to the new item should be added to the skip list
- We then insert the new item in this bottom-level list at its appropriate position. After inserting the new item at this level we “flip a coin”
 - If the flip comes up tails, then we stop right there.
 - If the flip comes up heads, we move to next higher level and insert in this level at the appropriate position

- A randomized algorithm performs coin tosses (i.e., uses random bits) to control its execution
- Its running time depends on the outcomes of the coin tosses
- We analyze the expected running time of a randomized algorithm under the following assumptions
 - the coins are unbiased, and
 - the coin tosses are independent
- The worst-case running time of a randomized algorithm is large but has very low probability (e.g., it occurs when all the coin tosses give “heads”)

Insertion in Skip List Example

- Suppose we want to insert 15
- Do a search, and find the spot between 10 and 23
- Suppose the coin comes up “head” two times

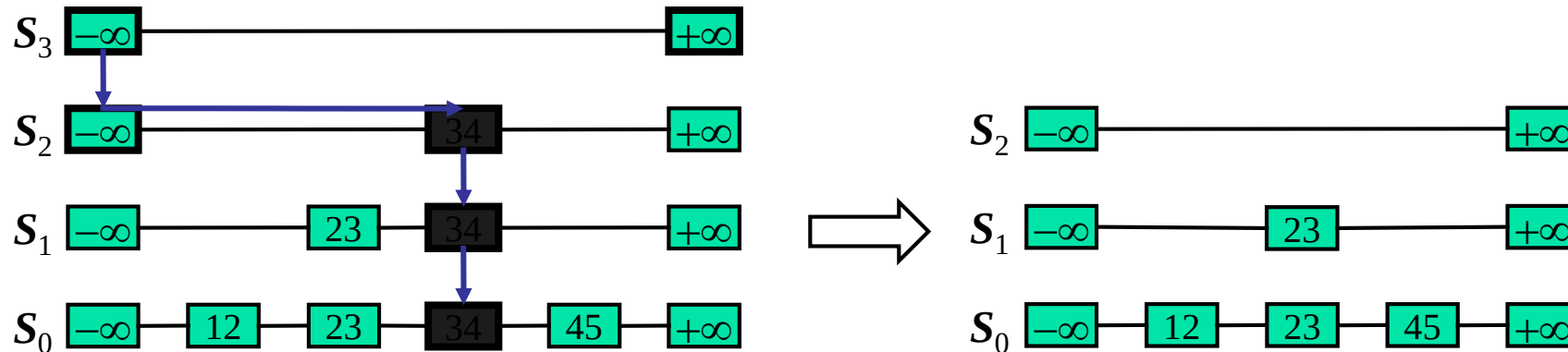


Deletion

- We begin by performing a search for the given key k .
- If a position p with key k is not found, then we return the NO_SUCH_KEY element
- Otherwise, if a position p with key k is found (it would be found on the bottom level), then we remove all the position above p
- If more than one upper level is empty, remove it

Deletion in Skip List Example

- Suppose we want to delete 34
- Do a search, find the spot between 23 and 45
- Remove all the position above p



Skip List

Starting with an empty skip list, insert these keys, with these “randomly generated” levels

- 5, with level 1
- 26, with level 1
- 25, with level 4
- 6, with level 3
- 21, with level 0
- 3, with level 2
- 22, with level 2

Note that the ordering of the keys in the skip list is determined by the value of the keys only; the levels of the nodes are determined by the random number generator only

DATA STRUCTURES & ITS APPLICATIONS

Skip List

5 with level 1, 26 with level 1, 25 with level 4, 6 with level 3
21 with level 0, 3 with level 2, 22 with level 2



DATA STRUCTURES & ITS APPLICATIONS

References

- Presentation Slides: Advanced Data Structures
Dr. RamaMoorthy Srinath, Professor, CSE, PESU





THANK YOU

Kusuma K V

Department of Computer Science
& Engineering

kusumakv@pes.edu



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Stacks

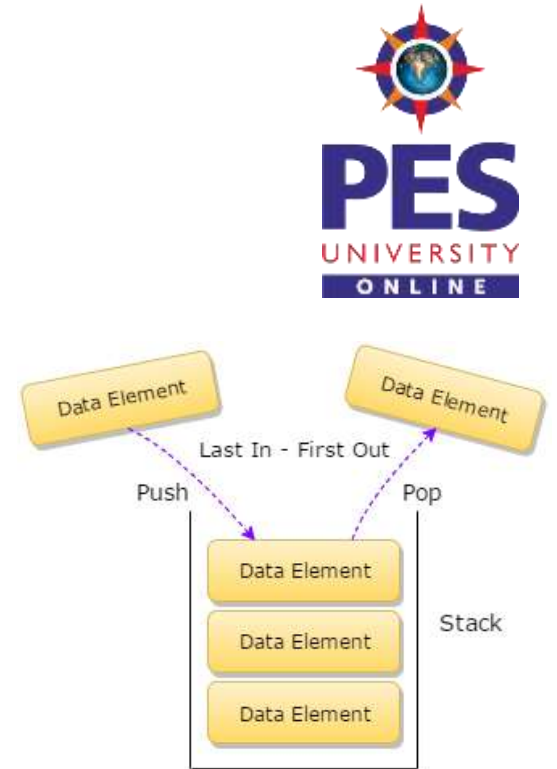
Dinesh Singh

Department of Computer Science & Engineering

Data Structures and its Applications

Stacks - Definition

- A Stack is a data Structure in which all the insertions and deletions of entries are at one end. This end is called the TOP of the stack.
- When an item is added to a stack it is called push into the stack
- When an item is removed it is called pop from the stack.
- The Last item pushed onto a stack is always the first that will be popped from the stack.
- This property is called the *last in, first out* or LIFO for short



Data Structures and its Applications

Stacks – Representation in C



A stack in C is declared as a structure containing two objects :

- An array to hold the elements of the stack
- An Integer to indicate the position of the current stack top within the array
- Stack of integers can be done by the following declaration

```
#define STACKSIZE 100
```

```
struct stack
```

```
{
```

```
    int top;
```

```
    int items[STACKSIZE]
```

```
};
```

Once this is done, actual stack can be declared by

```
struct stack s;
```

Data Structures and its Applications

Stacks – Representation in C



Items need not be restricted to integers, items can be of any type.

A stack can contain items of different types by using C unions.

```
#define STACKSIZE 100
```

```
#define INT 1
```

```
#define FLOAT 2
```

```
#define STRING 3
```

```
struct stackelement {
```

```
    int etype;
```

```
    union{
```

```
        int ival;
```

```
        float fval;
```

```
        char *pavl; //pointer to string
```

```
    } element;
```

```
};
```

```
struct stack
```

```
{  
    int top;  
    struct stackelement items[STACKSIZE];  
};
```

- The above declaration defines a stack whose items can either be integers, floating point numbers or string depending on the value of etype (previous slide).

Data Structures and its Applications

Stacks – Implementation of operations of stack



Operations on stack

- Inserting an element on to the stack : push
- Deleting an element from the stack : pop
- Checking the top element : peep
- Checking if the stack is empty : empty
- Checking if the stack is full : overflow

Representation of stack will be as follows

```
#define STACKSIZE 100
```

```
struct stack
```

```
{
```

```
    int top;
```

```
    int items[STACKSIZE]
```

```
};
```


Data Structures and its Applications

Stacks – Implementation of operations of stack



```
void push(struct stack *ps, int x)
```

```
/*ps is pointer to the structure representing stack, x is integer to be inserted
```

```
top is integer that indicates the position of the current stack top within the  
array, items is an integer array that represents stack, STACK_SIZE is the  
maximum size of the stack */
```

```
{
```

```
    if (ps->top == STACKSIZE -1) //check if the stack is full
```

```
        printf("STACK FULL Cannot insert..");
```

```
else
```

```
{
```

```
    ++(ps->top); //increment top
```

```
    ps->items[ps->top]=x; //insert the element at a location top
```

```
}
```

```
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack



```
int pop(struct stack *ps )
```

```
/*ps is pointer to the structure representing stack, top is integer that indicates  
the position of the current stack top within the array , items is an integer  
array that represents stack, STACK_SIZE is the maximum size of the stack */
```

```
{
```

```
if (ps->top == -1) // check if the stack is the empty
```

```
    printf("STACK EMPTY Cannot DELETE..");
```

```
else
```

```
{
```

```
    x=ps->items[ps->top]; //delete the element
```

```
    --(ps->top); //decrement top
```

```
    return x;
```

```
}
```

```
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack



```
int display(struct stack *ps )
```

```
/*ps is pointer to the structure representing stack, top is integer that indicates  
the position of the current stack top within the array , items is an integer  
array that represents stack, STACK_SIZE is the maximum size of the stack */
```

```
{
```

```
if (ps->top == -1) // check if the stack is the empty
```

```
    printf("STACK EMPTY ");
```

```
else
```

```
{
```

```
    for (i=ps->top;i>=0;i--) // displays the elements from top
```

```
        printf("%d",ps->items[i]);
```

```
}
```

```
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack



```
int peep(struct stack *ps )
{
    if (ps->top == -1)
        printf("STACK EMPTY ..");
    else
    {
        x=ps->items[ps->top]; //get the element
        return x;
    }
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack



```
int empty(struct stack *ps )
{
    if (ps->top == -1)
        return 1;
    return 0;
}

int overflow(struct stack *ps)
{
    if (ps->top==STACKSIZE-1)
        return 1;
    return 0;
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack



implementation of stack operations where the items array and top are separate variables (Not part of structure)

```
void push(int *s, int *top, int x)
{
    if(*top==STACKSIZE-1)//check if the stack is full
    {
        printf("stack overflow..cannot insert");
        return 0;
    }
    else
    {
        ++*top; //increment the top
        s[*top]=x; //insert the element
    }
    return 1;
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack



- Implementation of stack operations where the items and the top are separate variables (Not part of structure)

```
int pop(int *s, int *top)
{
    if(*top==-1)//check if the stack is empty
    {
        printf("Stack empty .. Cannot delete");
        return -1;
    }
    else
    {
        x=s[*top]; //insert the element
        --*top; //decrement top
        return x; // return the deleted element
    }
}
```

Data Structures and its Applications

Stacks – Implementation of operations of stack



- Implementation of stack operations where the items and the top are separate variables (Not part of structure)

```
display(int *s, int *top)
{
    if(*top==-1)
        printf("Empty stack");
    else
    {
        for(i=*top;i>=0;i--) // display the elements from the top
            printf("%d",s[i]);
    }
}
```


Data Structures and its Applications

Stacks – Application of Stack



- Write an algorithm to determine if an input character string is of the form $x C y$ where x is a string consisting of the letters 'A' and 'B' and where y is the reverse of x . At each point you may read only the character of the string

```
int check(t)
```

```
//the function returns 1 if string t is of the form x C y, else returns 0
```

```
//uses stack s and its operations push and pop
```

```
{
```

```
    i=0;
```

```
    while(t[i]!='C') //push all the characters of the string into the stack
```

```
until C is encountered
```

```
{
```

```
    push(&s, t[i]);
```

```
    i=i+1;
```

```
}
```

Data Structures and its Applications

Stacks – Application of Stack



```
//pop the contents of the stack and compare with t[i] until the end of the string
while(t[i]!='\0')
{
    x=pop(&s);
    if(t[i]!=x) //if the character popped out is not equal to the character read from the string
        return 0; // not of the form xCy
    i=i+1;
}
return 1; // string of the form
}
```



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Stacks – Linked List Implementation

Dinesh Singh

Department of Computer Science & Engineering

Data Structures and its Applications

Stacks – linked list implementation

- A stack can be easily implemented through the linked list. In the Implementation by a linked list, the stack contains a top pointer. which is “head” of the stack. The pushing and popping of items happens at the head of the list.

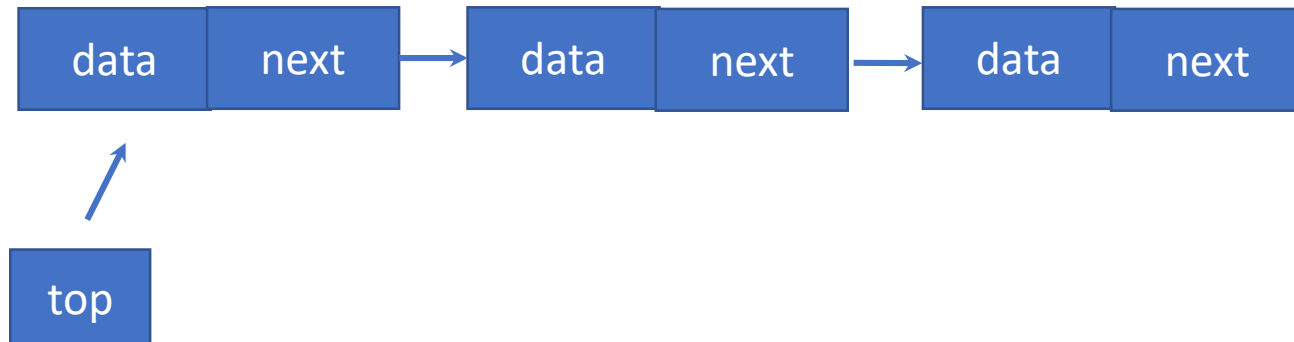
Structure of the stack

struct node

```
{  
    int data;  
    struct node *next;  
}
```

struct node *top

top=NULL;



Data Structures and its Applications

Stacks – linked list implementation



Note :

- Items of the stack represented as the linked list.
- Each item is a node
- Top is a pointer that points to the first node (top of the stack)
- Top is initially NULL (Empty stack)
- Insertion and deletion happens at the front of the list
- stack size is not limited.

Operations on the stack

- Push : Inserting an element at the front of the list
- Pop : delete an element from the front of the list
- Display : displaying the list

//implements the push operation

```
void push(int x, struct node **top)
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->data=x;
    temp->next=*top; // insert in front of the list
    *top=temp; // make top points to the new top node
}
```


Data Structures and its Applications

Stacks – linked list implementation



```
//implements the pop operation
//returns top element, -1 if stack is empty
int pop(struct node **top)
{
    int x;
    struct node *q;

    if(*top==NULL)
    {
        printf("Empty Stack\n");
        return -1;
    }
}
```

//implements the pop operation

else

```
{  
    q=*top;  
    x=q->data; // get the top element  
    *top=q->next; // make top point to the next top  
    free(q); // free the memory of the node  
    return(x);  
}  
}
```

Data Structures and its Applications

Stacks – linked list implementation

//implements the display operation

```
void display(struct node *top)
{
    if(top==NULL)
        printf("Empty Stack\n");
    else
    {
        while(top!=NULL)
        {
            printf("%d->",top->data);
            top=top->next;
        }
    }
}
```

Data Structures and its Applications

Stacks – linked list implementation



Another representation of structure of stack

struct node

```
{  
    int data;  
    struct node *next;  
};
```

struct stack

```
{  
    struct node *top;  
}
```

struct stack s;

s.top=NULL;

//implements the push operation

void push(int x, struct stack * s)

//s is pointer to structure stack

{

struct node *temp;

temp=(struct node*)malloc(sizeof(struct node));

temp->data=x;

temp->next=s->top; // insert in front of the list

s->top=temp; // make top points to the new top node

}

Data Structures and its Applications

Stacks – linked list implementation



```
//implements the pop operation
//returns top element, -1 if stack is empty
int pop(struct stack *s)
{
    int x;
    struct node *q;

    if(s->top==NULL)
    {
        printf("Empty Stack\n");
        return -1;
    }
}
```

//implements the pop operation

else

```
{  
    q=s->top;  
    x=q->data; // get the top element  
    s->top=q->next; // make top point to the next top  
    free(q); // free the memory of the node  
    return(x);  
}  
}
```

Data Structures and its Applications

Stacks – linked list implementation



//implements the display operation

```
void display(struct stack *s)
{
    struct node *q;
    if(s->top==NULL)
        printf("Empty Stack\n");
    else
    {
        q=s->top;
        while(q!=NULL)
        {
            printf("%d->",q->data);
            q=q->next;
        }
    }
}
```


Data Structures and its Applications

Stacks – Application



Write an Algorithm to print a string in the reverse order

//prints the text in a reverse order

reverse(t)

```
{  
    i=0;  
    //push all the characters on to the stack  
    while(t[i]!='\0')  
    {  
        push(s,t[i]);  
        i=i+1;  
    }
```

pop all the characters from the stack until the stack is empty

```
while(!empty(s))
```

```
{
```

```
    x= pop(s);
```

```
    print(x)
```

```
}
```

```
}
```



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Application of stacks– Functions, nested functions and Recursion

Dinesh Singh

Department of Computer Science & Engineering

Data Structures and its Applications

Application of stacks – Functions, nested functions



Activation record :

- When functions are called, The system (or the program) must remember the place where the call was made, so that it can return there after the function is complete.
- It must also remember all the local variables, processor registers, and the like, so that information will not be lost while the function is working.
- This information is considered as large data structure . This structure is sometimes called the invocation record or the activation record for the function call.

Data Structures and its Applications

Application of stacks – Functions, nested functions and Recursion



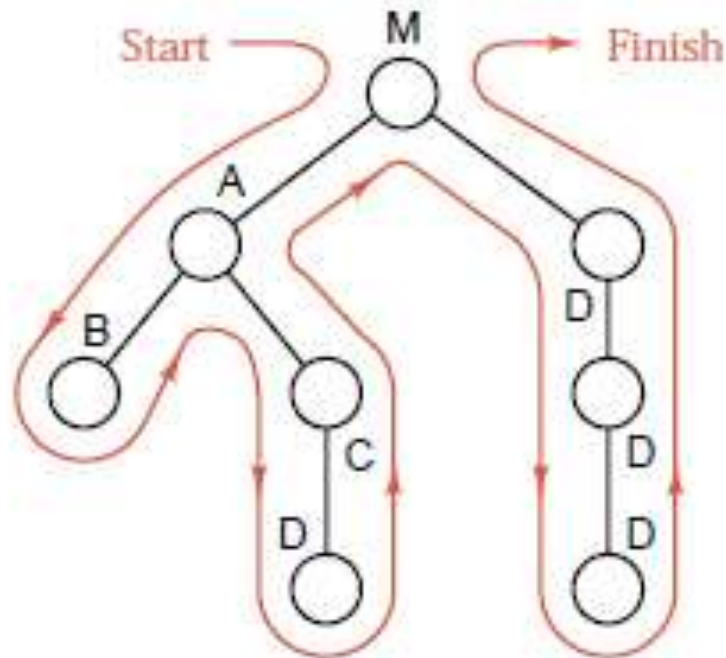
- Suppose that there are three functions called A,B and C. M is the main function.
- Suppose that A invokes B and B invokes C. Then B will not have finished its work until C has finished and returned. Similarly, A is the first to start work, but it is the last to be finished, not until sometime after B has finished and returned.
- Thus the sequence by which function activity proceeds is summed up as the property last in, first out.
- The machine's task of assigning temporary storage area (activation records) used by functions would be in same order (LIFO).
- Since LIFO property is used, the machine allocates these records in the stack
- Hence a stack plays a key role in invoking functions in a computer system.

Data Structures and its Applications

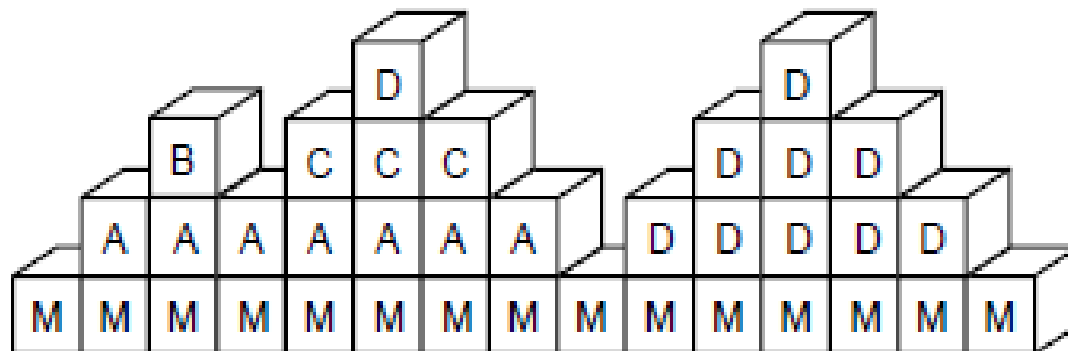
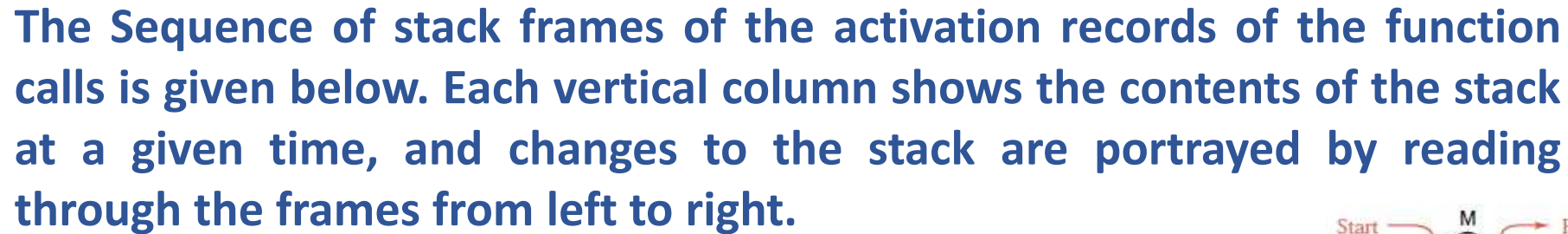
Application of stacks – Functions, nested functions

Example :

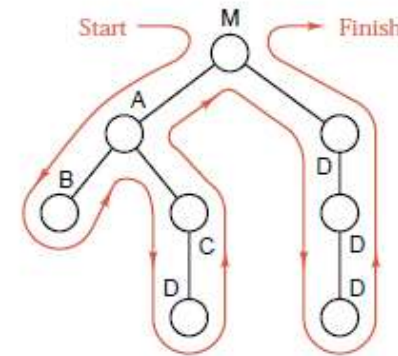
Consider the following showing the order in which the functions are invoked



Application of stacks – Functions, nested functions



Time $\longrightarrow$



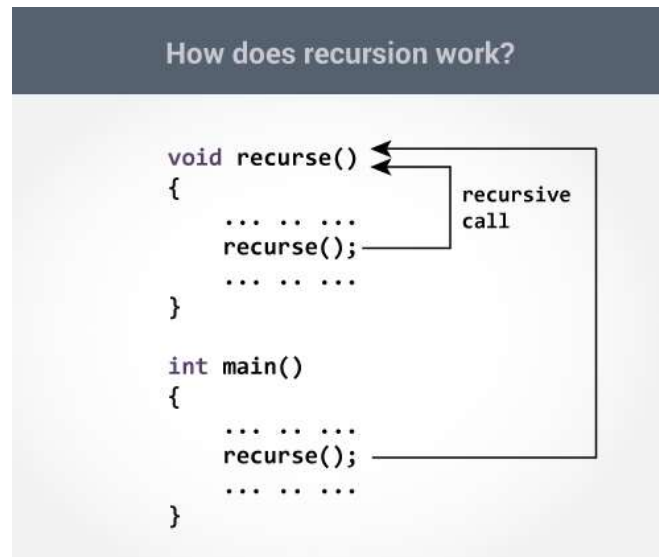
↑ Stack space for data

Data Structures and its Applications

Application of stacks – Recursion

Recursion :

- Recursion is a computer programming technique involving the use of a procedure, subroutine or a function.
- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function.



How a particular problem is solved using recursion?

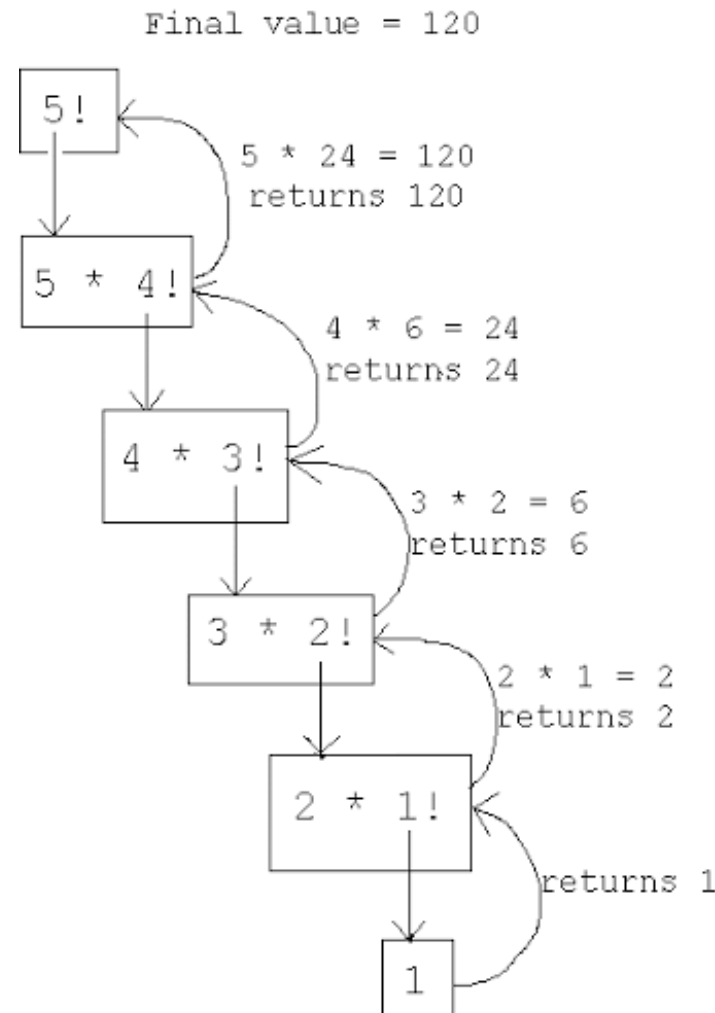
- The idea is to represent a problem in terms of one or more smaller problems.
- A way is found to solve these smaller problems and then build up a solution to the entire problem.
- The sub problems are in turn broken down into smaller sub problems until the solution to the smallest sub problem is known.
- The solution to the smallest sub problem is called the base case.
- In the recursive program, the solution to the base case is provided

Example 1: Factorial of a Number n

Recursive definition :

$n! = 1$ if $n=1$

$n! = n * (n-1)!$ If $n > 1$

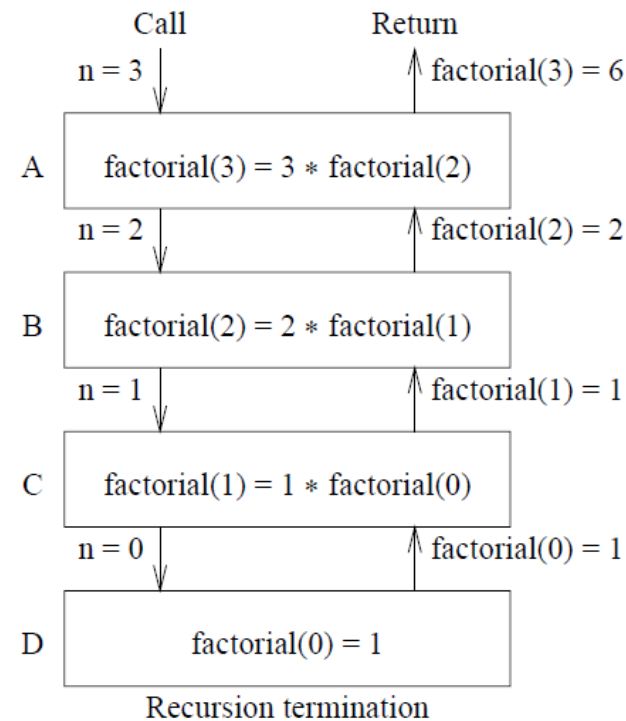


Data Structures and its Applications

Application of stacks – Recursion

Recursive function to compute factorial of n

```
factorial(n)
{
    int f;
    if(n==0)
        return 1;
    f=n*factorial(n-1);
    return f;
}
```



(a)

Activation record for A
Activation record for B
Activation record for C
Activation record for D

(b)

Data Structures and its Applications

Application of stacks – Recursion



Recursive function to find the product of $a*b$

$a*b = a$ if $b=1$;

$a*b = a*(b-1)+a$ if $b > 1$;

To evaluate $6 * 3$

$6*3 = 6*2 + 6 = 6*1 + 6 + 6 = 6 + 6 + 6 = 18$

```
multiply(int a, int b)
```

```
{
```

```
    int p;
```

```
    if (b==1)
```

```
        return a
```

```
    p= multiply(a,b-1) + a;
```

```
    return p;
```

```
}
```

Fibonacci Sequence

The fibonacci sequence is the sequence of integers
1, 1, 2, 3, 5, 8, 13, 21, 34...

Each element is the sum of two preceding elements

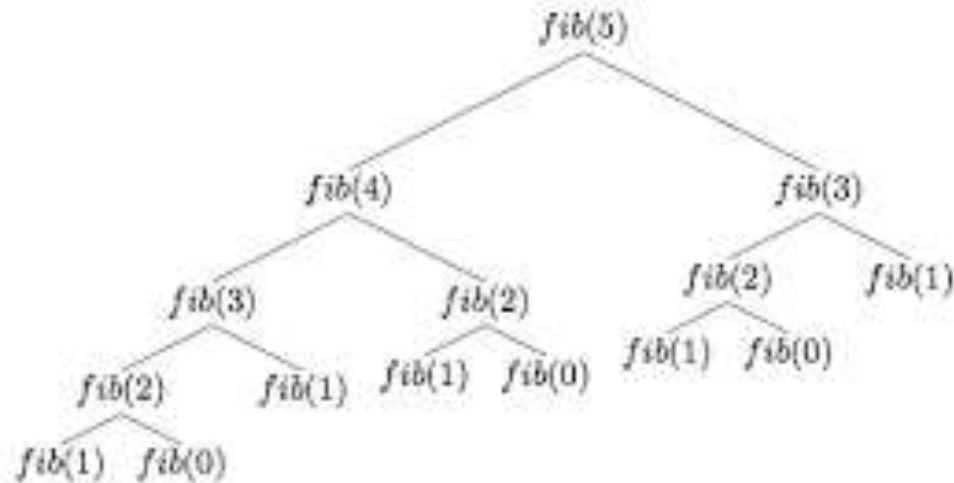
If we consider $\text{fib}(0) = 1$ and $\text{fib}(1)=1$
then recursive definition to compute the n th element in the sequence is

$\text{fib}(n) = n$ if $n=0$ or $n=1$

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ for $n \geq 2$

Recursive function to compute the nth element in the Fibonacci Sequence

```
fib(int n)
{
    int x,y;
    if ( (n==0) || (n==1) )
        return n;
    x= fib(n-1) ;
    y=fib(n-2);
    return x+y;
}
```



Data Structures and its Applications

Application of stacks – Recursion



Recursive function to find the sum of elements of an array

```
int sum(int *a, int n)
```

```
//a is pointer to the array, n is the index of the last element of the array
```

```
{
```

```
    int s;
```

```
    if(n==0) // base condition
```

```
        return a[0];
```

```
    s= sum(a,n-1) + a[n]; // compute sum of n-1 elements and add the nth element
```

```
    return s;
```

```
}
```

Data Structures and its Applications

Application of stacks – Recursion

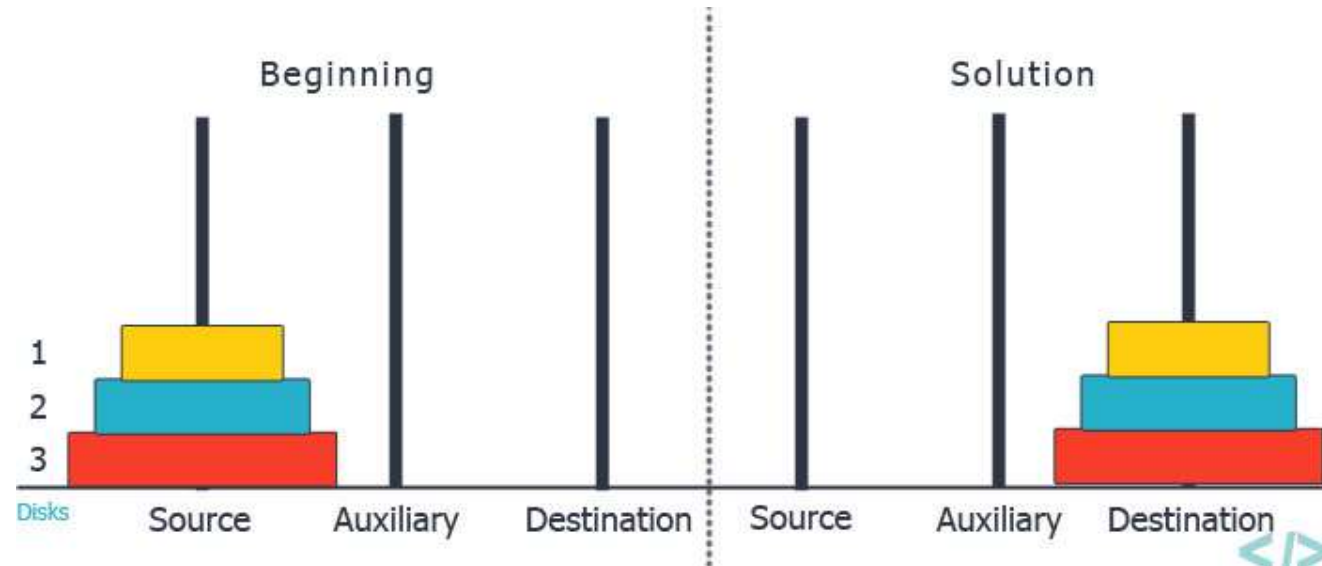


Recursive function to display the elements of the linked list in the reverse order

```
int display(struct node *p)
{
    if(p->next!=NULL)
        display(p->next)
    printf("%d ",p->data);
}
```

Tower of Hanoi

Three Pegs A, B and C exists. N disks of differing diameters are placed on peg A. The Larger disk is always below a smaller disk. The objective is to move the N disks from Peg A to Peg C using Peg B as the auxillary peg



Tower of Hanoi – recursive solution

If a solution to $n-1$ disks is found, then the problem would be solved. Because in the trivial case of one disk, the solution would be to move the single disk from Peg A to Peg C.

To move n disks from A to C , the recursive solution would be as follows

1. If $n=1$ move the single disk from A to C and stop
2. Move the top $n-1$ disks from A to B using C as auxillary
3. Move the remaining disk from A to C
4. Move $n-1$ disks from B to C using A as the auxillary

Recursive function for Tower of Hanoi

```
void tower(int n,char src,char tmp,char dst)
{
    if(n==1)
    {
        printf("\nMove disk %d from %c to %c",n,src,dst);
        return;
    }
    tower(n-1,src,dst,tmp);
    printf("\nMove disk %d form %c to %c",n,src,dst);
    tower(n-1,tmp,src,dst);
    return;
}
```

Recursive function for Tower of Hanoi

Solution for 4 disks

Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 3 from peg A to peg B
Move disk 1 from peg C to peg A
Move disk 2 from peg C to peg B
Move disk 1 from peg A to peg B
Move disk 4 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 2 from peg B to peg A
Move disk 1 from peg C to peg A
Move disk 3 from peg B to peg C
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C

Solution for moving 3 disks from A to B
Move 4th disk from A to C
Solution for moving 3 disks from B to C



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Infix to Postfix and Prefix Expressions – Implementation

Dinesh Singh

Department of Computer Science & Engineering

Data Structures and its Applications

Infix , Postfix and Prefix Expressions



- Consider the sum of A and B expressed as $A + B$
- This representation is called infix.
- There are two alternate notations for expressing sum of A and B using the symbols A , B and + , these are
 - Prefix : $+ A B$
 - Postfix : $A B +$
- The prefixes “Pre” , “post” and “in” refers to the relative position of the operator with respect to the two operands.
- In prefix, operators precedes the two operands
- In postfix, the operator follows the two operands
- In infix, the operator is in between the two operands

Data Structures and its Applications

Infix , Postfix and Prefix Expressions



Conversion of Infix to Postfix

- For Example : consider the expression $A + B * C$
- The evaluation of the above expression requires the knowledge of precedence of operators
- $A + B * C$ can be expressed as $A + (B * C)$ as multiplication takes precedence over addition
- Applying the rules of precedence the above infix expression can be converted to postfix as follows

$$\begin{aligned} A + B * C &= A + (B * C) \\ &= A + (BC *) \quad \text{convert the multiplication} \\ &= A (B C *) + \quad \text{convert the addition} \\ &= A B C * + \end{aligned}$$

Data Structures and its Applications

Infix , Postfix and Prefix Expressions



Conversion of Infix to Prefix

- For Example : consider the expression $A + B * C$
- Applying the rules of precedence the above infix expression can be converted to prefix as follows

$$\begin{aligned} A + B * C &= A + (B * C) \\ &= A + (* B C) \quad \text{convert the multiplication} \\ &= + A (* B C) + \quad \text{convert the addition} \\ &= + A * B C \end{aligned}$$

Data Structures and its Applications

Infix , Postfix and Prefix Expressions



Applying the rules of precedence the table shows the conversion of Infix to Postfix and Prefix Expression

Infix	Postfix	Prefix
$A + B * C + D$	$A B C * + D +$	$++ A * B C D$
$(A + B) * (C + D)$	$A B + C D + *$	$* + A B + C D$
$A * B + C * D$	$A B * C D * +$	$+ * A B * C D$
$A + B + C + D$	$A B + C + D +$	$+++ A B C D$
$A \$ B * C - D + E / F / (G + H)$	$A B \$ C * D - E F / G H + / +$	$+ - * \$ A B C D / / E F + G H$
$((A + B) * C - (D - E)) \$ (F + G)$	$A B + C * D E - - F G + \$$	$\$ - * + A B C - D E + F G$
$A - B / (C * D \$ E)$	$A B C D E \$ * / -$	$- A / B * C \$ D E$

$\$$ is the exponentiation operator and its precedence is from right to left

For example $A \$ B \$ C = A \$ (B \$ C)$

Data Structures and its Applications

Infix to postfix conversion - algorithm



opstk is the empty stack

while(not end of input)

{

 symb = next input character

 // if the input symbol is an operand , add it to the postfix string

 if(symb is an operand)

 add symb to postfix string

else

{

 // pop the contents of the stack while the precedence of

 // top of the stack is greater than the precedence of the symbol scanned

 //prcd function returns true in this case

 while(!empty(opstk) and (prcd(stacktop(opstk),symb))

 {

 topsymb=pop(opstk)

 add topsymb to postfix string

 }

Data Structures and its Applications

Infix to postfix conversion - algorithm



```
/* if prcd function returns false, then
if the stack is empty and symbol is not ')', push symb on to the stack*/
    if( empty(opstk) || symb!=')')
        push(opstk,symb)
    else
        topsymb=pop(opstk); // if symb is ')', pop '(' from the stack
}
while(!empty(opstk)
{
    topsymb=pop(opstk)
    add topsymb to postfix string
}
}
```

Data Structures and its Applications

Infix to postfix conversion - algorithm



- $\text{prcd}(\text{OP1}, \text{OP2})$ is a function that compares the precedence of the top of the stack (OP1) and the input symbol (OP2) and returns TRUE if the precedence is greater else returns FALSE.
- Some of the return values of prcd function
 - $\text{prcd}(' * ', ' + ')$ returns TRUE : $\text{prcd}(' * ', ' / ')$ returns TRUE
 - $\text{prcd}(' + ', ' * ')$ returns FALSE : $\text{prcd}(' / ', ' * ')$ returns TRUE
 - $\text{prcd}(' + ', ' + ')$ returns TRUE :
 - $\text{prcd}(' + ', ' - ')$ returns TRUE
 - $\text{prcd}(' - ', ' - ')$ returns TRUE
 - $\text{prcd}(' \$ ', ' \$ ')$ returns FALSE : exponentiation operator, associatively from right to left
 - $\text{prcd}(' (', \text{op})$, $\text{prcd}(\text{op} ' (')$ returns FALSE for any operator
 - $\text{prcd}(\text{op} , ') ')$ returns TRUE for any operator
 - $\text{Prcd}(' (', ') ')$ returns FALSE

Data Structures and its Applications

Infix to postfix conversion - algorithm



- The motivation behind the conversion algorithm is the desire to output the operators in the order in which they are to be executed.
- If an incoming operator is of greater precedence than the one on top of the stack, this new operator is pushed on to the stack.
- If on the other hand , the precedence of the new operator is less than the one on the top of the stack, the operator on the top of the stack should be executed first.
- Therefore the top of the stack is popped out and added to the postfix and the incoming symbol is compared with the new top and so on.
- Parenthesis in the input string override the order of operations
- When a left parenthesis is scanned, it is pushed on to the stack
- When its associated right parenthesis is found, all the operators between the two parenthesis are placed on the output string. Because they are to be executed before any operators appearing after the parenthesis

Data Structures and its Applications

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $A + B * C$

symb	postfix string	opstk	Remarks
A	A	Empty	symb is operand , add 'A'on to the postfix string
+	A	+	symb is operator, and stack empty, push + on to the stack
B	AB	+	symb is operand , Add 'B' to the postfix string
*	AB	+*	symb is operator, precedence of + (stack top) is < than precedence of *, therefore push * on to the stack

Data Structures and its Applications

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $A + B * C$

symb	postfix string	opstk	Remarks
C	ABC	+ *	symb is operand , add C on to the postfix string
End of input	ABC*	+	Pop * from the stack and add to the postfix string
-	ABC*+	Empty	Pop + from the stack and add to the postfix string

Data Structures and its Applications

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $(A + B) * C$

symb	postfix string	opstk	Remarks
(	-	Empty	Stack empty, push '(' on to the stack
A	A	(	Symb is an operand, add 'A' to the postfix string
+	A	(+	Precedence of top of the stack '(' is less than '+', push '+' on to the stack
B	AB	(+	Symb is an operand, add B to the postfix string
)	AB+	(	Precedence of stack top '+' is > than ')' Pop '+' from the stack and add to the postfix string
	AB+	empty	Precedence of stack top stack top '(' is not greater than ') and symb is ')', therefore pop '(' from the stack

Data Structures and its Applications

Infix to postfix conversion – Trace of the algorithm

Trace of the algorithm for $(A + B) * C$

symb	postfix string	opstk	Remarks
*	AB+	*	Stack empty, push '*' on to the stack
C	AB+C	*	Symb is an operand, add 'C' to the postfix string
End of string	AB+C*	Empty	End of input, pop * from the stack and add to the postfix.

Data Structures and its Applications

Infix to postfix conversion – another algorithm



```
void convert_postfix(char *infix,char*postfix)
{
    i=0;
    char ch;
    j=0;
    push(s,&top,'#');
    while(infix[i]!='\0')
    {
        ch=infix[i];
        //while the precedence of top of stack is greater than the
        //precedence of the input symbol, pop and add to the postfix
        while(stack_prec(peep(s,top))>input_prec(ch))
            postfix[j++]=pop(s,&top);
```

Data Structures and its Applications

Infix to postfix conversion – another algorithm



```
if(input_prec(ch)!=stack_prec(peep(s,top)))
    push(s,&top,ch);
else
    pop(s,&top);
i++;
}
while(peep(s,top)!='#')
    postfix[j++]=pop(s,&top);
postfix[j]='\0';
}
```

Data Structures and its Applications

Infix to postfix conversion – another algorithm

Precedence table

Operator	Input Precedence	Stack Precedence
+, -	1	2
*, /	3	4
\$	6	5
Operands	7	8
)	0	- : Never pushed on to stack
(	9	0
#	-	-1

Data Structures and its Applications

Infix to prefix conversion – algorithm



Example 1:

Input : $A * B + C / D$

Output : $+ * A B / C D$

Example 2 :

Input : $(A - B/C) * (D/K-L)$

Output : $*-A/BC-/DKL$

1: Reverse the infix expression i.e $A+B*C$ will become $C*B+A$.

Note while reversing each '(' will become ')' and each ')' becomes '('.

2: Obtain the postfix expression of the modified expression i.e $CB*A+$.

3: Reverse the postfix expression. Hence in our example prefix is $+A*BC$.



THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402



Data Structures and its Applications

Dinesh Singh

Department of Computer Science & Engineering

DATA STRUCTURES AND ITS APPLICATIONS

Evaluation of Postfix expression and Parenthesis matching

Dinesh Singh

Department of Computer Science & Engineering

- Each operator in a postfix string refers to the previous two operands.
- Each time an operand is read , it is pushed on to the stack
- When an operator is reached, its operands will be the top two elements on to the stack.
- The two elements are popped out, the indicated operation is performed on them and result is pushed on the stack so that it will be available for use as an operand of the next operator.

Data Structures and its Applications

Evaluation of Postfix Expression - Algorithm



```
opndstk is the empty stack
while(not end of the input) // scan the input string
{
    symb=next input character;
    if (symb is an operand)
        push(opndstk,symb)
    else
    {
        opnd2=pop(opndstk);
        opnd1=pop(opndstk);
        value = result of applying symb to opnd1 and opn2;
        push(opndstk, value);
    }
    return(pop(opndstk));
}
```

Data Structures and its Applications

Evaluation of Postfix Expression – Trace of the Algorithm



Infix : $3 + 5 * 4$

Postfix expression : $3\ 5\ 4\ *\ +$

Symb	Opnd1	Opnd2	Value	opndstk
3	-			3
5				3, 5
4				3, 5, 4
*	5	4	20	3, 20
+	3	20	60	60

```
int postfix_eval(char* postfix)
{
    int i,top,r;
    int s[10]; //stack
    top=-1;
    i=0;

    while(postfix[i]!='\0')
    {
        char ch=postfix[i];
        if(isoper(ch))
        {
            int op1=pop(s,&top);
            int op2=pop(s,&top);
```



```
switch(ch)
{
    case '+':r=op1+op2;
        push(s,&top,r);
        break;
    case '-':r=op2-op1;
        push(s,&top,r);
        break;
    case '*':r=op1*op2;
        push(s,&top,r);
        break;
    case '/':r=op2/op1;
        push(s,&top,r);
        break;
} //end switch
} //end if
```

Data Structures and its Applications

Evaluation of Postfix Expression - implementation



```
else
    push(s,&top,ch-'0');//convert charcter to
integer and push
    i++;
} //end while
return(pop(s,&top));
}
```

Examples

1. `(( ))` : Valid Expression
2. `(( ( ))` : Invalid Expression (Extra opening parenthesis)
3. `(( )) )` : Invalid Expression (Extra closing parenthesis)
4. `(( } )` : Invalid Expression (Parenthesis mismatch)
5. `(( ) ]` : Invalid Expression (Parenthesis mismatch)

1. Read the input symbol from the input expression
2. If the input symbol is one of the open parenthesis (' (' , ' { ' or ' ['), it is pushed on to the stack
3. If the input symbol is of closing parenthesis, stack is popped and the popped parenthesis is compared with the input symbol, if there is a mismatch in the type of the parenthesis, return 0 (Mismatch of parenthesis)
4. If there is a match in the parenthesis , the next input symbol is read.
5. If during this process, if the stack becomes empty, return 0 (Extra closing parenthesis)
6. If at the end of the expression, if the stack is not empty, return 0 (Extra opening parenthesis)
7. If at the end of the input expression, if the stack is empty, return 1. (Parenthesis are matching)

Data Structures and its Applications

Parenthesis Matching - Implementation

```
int match(char *expr)
{
    int i,top;
    char s[10],ch,x;//stack
    i=0;
    top=-1;

    while(expr[i]!='\0')
    {
        ch=expr[i];
        switch(ch)
        {
            case '(':
            case '{':
            case '[':push(s,&top,ch);
                    break;
```

```
case ')':if(!isempty(top))
{
    x=pop(s,&top);
    if(x=='(')
        break;
    else
        return 0;//mismatch of parenthesis
}
else
    return 0;//extra closing parenthesis
```

```
case '}':if(!isempty(top))
{
    x=pop(s,&top);
    if(x=='{')
        break;
    else
        return 0;//mismatch of parenthesis
}
else
    return 0;//extra closing parenthesis
```

```
case ']':if(!isempty(top))
    {
        x=pop(s,&top);
        if(x=='[')
            break;
        else
            return 0;//mismatch of parenthesis
    }
else
    return 0;//extra closing parenthesis

} //end switch
i++;
} //end while
if(isempty(top))
    return 1;
return 0;//extra opening parenthesis
}
```




THANK YOU

Dinesh Singh

Department of Computer Science & Engineering

dineshs@pes.edu

+91 8088654402